



Universidad Politécnica
de Madrid



**Escuela Técnica Superior de
Ingenieros Informáticos**

Grado en Ingeniería Informática

Trabajo Fin de Grado

**Desarrollo de una Aplicación Web para
Facilitar los Servicios de un Restaurante
con sus Clientes**

Autor: Keyu Lin Chen

Tutor(a): Jose Antonio Calvo-Manzano Villalón

Madrid, Junio de 2024

Este Trabajo Fin de Grado se ha depositado en la ETSI Informáticos de la Universidad Politécnica de Madrid para su defensa.

Trabajo Fin de Grado

Grado en Ingeniería Informática

Título: Desarrollo de una Aplicación Web para Facilitar los Servicios de un
Restaurante con sus Clientes

Junio 2024

Autor: Keyu Lin Chen

Tutor: Jose Antonio Calvo-Manzano Villalón

Lenguajes y Sistemas Informáticos e Ingeniería de Software

ETSI Informáticos

Universidad Politécnica de Madrid

Agradecimientos

Quiero expresar mi más profundo agradecimiento a todas las personas que han hecho posible la realización de este trabajo.

A mi familia,

por acompañarme en cada etapa de mi vida y motivarme a seguir siempre adelante.

A mi tutor,

por haber aceptado mi propuesta de Trabajo de Fin de Grado y por su constante apoyo a lo largo de todo este proyecto.

Resumen

A pesar del aumento de la digitalización en diversos sectores de la economía española, sigue habiendo numerosos establecimientos de la restauración que aún no se han digitalizado, lo que supone una mayor carga de trabajo tanto para los propietarios como para los empleados del restaurante, quienes deben manejar las tareas manualmente, como atender a las llamadas telefónicas, gestionar las reservas de mesas o manejar los pedidos de comida de los clientes. Además, debido a la preferencia actual de los clientes de interactuar de manera digital, ya sea para realizar pedidos o para escribir reseñas, la falta de digitalización supondrá una pérdida de clientela y una reducción de oportunidades para mejorar y destacar en el mercado.

Por otro lado, aquellos establecimientos que sí han adoptado tecnologías digitales a menudo dependen de otras plataformas externas, lo que limita su capacidad para gestionar de manera autónoma aspectos como las reseñas o el diseño personalizado del sitio web. Asimismo, esta dependencia puede generar costos adicionales debido a las comisiones cobradas por estas plataformas y puede aumentar indirectamente la competencia entre los diferentes restaurantes registrados en la aplicación, dificultando así la posibilidad de destacar y atraer a nuevos clientes.

Como solución a estos problemas se ha propuesto el presente Trabajo de Fin de Grado (TFG), que tiene como objetivo principal el desarrollo de una aplicación web integral personalizada para un restaurante en concreto, con el fin de facilitar los diversos servicios que el establecimiento ofrece a sus clientes. De esta manera, además de dejar de depender de plataformas de terceros, también permitirá a los propietarios tener un mayor control administrativo del restaurante. Sumado a lo anterior, desde el punto de vista del cliente, la aplicación ofrece una experiencia más sencilla y conveniente, permitiéndoles realizar múltiples acciones en una única plataforma, como reservar, hacer pedidos o dejar reseñas, sin la necesidad de acceder a otras aplicaciones.

El proyecto se estructura en varios capítulos, comenzando con la importancia de la digitalización en la hostelería y la problemática actual hasta el propio desarrollo de la aplicación, que pasará por las diferentes fases de ciclo de vida de desarrollo de software. De esta manera, la aplicación web resultante contará con un sistema de autenticación e identificación, un sistema de gestión de reservas y pedidos, y un sistema de reseñas y valoraciones.

Para la construcción de esta aplicación web integral, se ha seguido la metodología Scrum, lo cual permitió una gestión ágil y eficiente del proyecto mediante Sprints y entregas incrementales. Durante su desarrollo, se mantuvo una interacción constante con los clientes de la aplicación para asegurar que las entregas satisficieran sus necesidades y expectativas. En cuanto al entorno de desarrollo, se seleccionaron tecnologías como Angular 17 para el Front-End, Spring Boot 3 para el Back-End y PostgreSQL 16 para la gestión de la base de datos, siguiendo la arquitectura de software de Modelo-Vista-Controlador (MVC).

Abstract

Despite the increase in digitalization in various sectors of the Spanish economy, there are still numerous restaurant establishments that have not yet been digitalized, which means a greater workload for both restaurant owners and employees, who must handle tasks manually, such as answering phone calls, managing table reservations, or managing customer food orders. Furthermore, due to customers' current preference to interact digitally, whether to place orders or write reviews, the lack of digitalization will mean a loss of clientele and a reduction in opportunities to improve and stand out in the market.

On the other hand, those establishments that have adopted digital technologies often depend on other external platforms, which limits their ability to autonomously manage aspects such as reviews or personalized website design. Likewise, this dependency can generate additional costs due to the commissions charged by these platforms and can indirectly increase competition between the different restaurants registered in the application, thus making it difficult to stand out and attract new customers.

As a solution to these problems, this Final Degree Project (TFG) has been proposed, whose main objective is the development of a comprehensive personalized web application for a specific restaurant, in order to facilitate the various services offered by the establishment to his clients. In this way, in addition to stopping depending on third-party platforms, it will also allow the owners to have greater administrative control of the restaurant. In addition, from the customer's point of view, the application offers a simpler and more convenient experience, allowing them to perform multiple actions on a single platform, such as booking, placing orders or leaving reviews, without the need to access other applications.

The project is structured into several chapters, starting with the importance of digitalization in the hospitality industry and the current problems to the development of the application itself, which will go through the different phases of the software development life cycle. In this way, the resulting web application will have an authentication and identification system, a reservation and order management system, and a reviews and ratings system.

For the construction of this comprehensive web application, the Scrum methodology has been followed, which allowed agile and efficient management of the project through Sprints and incremental deliveries. During its development, constant interaction was maintained with the application's customers to ensure that deliveries met their needs and expectations. Regarding the development environment, technologies such as Angular 17 for the Front-End, Spring Boot 3 for the Back-End and PostgreSQL 16 for database management are selected, following the Model-View-Controller software architecture (MVC).

Índice

1	Introducción.....	1
1.1	Importancia de la digitalización y la hostelería española.....	1
1.2	Problemática	1
1.3	Aproximación a la resolución	5
1.4	Lista de objetivos.....	6
1.5	Estructura del documento.....	7
2	Estado de arte.....	9
2.1	Aplicaciones existentes en el mercado actual	9
2.2	Desventajas de las aplicaciones existentes	12
2.3	Necesidad de una aplicación web construida con frameworks	15
3	Entorno de desarrollo	16
3.1	Historia del desarrollo web	16
3.2	Tecnología para Front-End	17
3.2.1	Tecnologías disponibles para Front-End en la actualidad	17
3.2.2	Tecnologías seleccionadas para Front-End.....	18
3.3	Tecnología para Back-End.....	19
3.3.1	Tecnologías disponibles para Back-End en la actualidad	19
3.3.2	Tecnología seleccionada para Back-End	22
3.4	Tecnología para bases de datos	23
3.4.1	Tecnologías disponibles para bases de datos en la actualidad.....	23
3.4.2	Tecnología seleccionada para la base de datos	25
3.5	Tecnología para el control de versiones	26
3.5.1	Tecnologías actuales disponibles para el control de versiones	26
3.5.2	Tecnología seleccionada para el control de versiones	28
4	Metodología de desarrollo de software	29
4.1	Ciclo de vida del desarrollo de software	29
4.2	Modelos SDLC.....	30
4.3	Ciclo de vida y metodología seleccionados	31
4.4	Origen de la metodología Scrum.....	31
4.5	Funcionamiento de la metodología Scrum	32
4.6	Cómo se aplicará Scrum en el proyecto	36
5	Desarrollo de la aplicación	38
5.1	Organización del capítulo	38
5.2	Refinamiento del Product Backlog	38
5.3	Arquitectura del software	52
5.4	Planificación de Sprints.....	56
5.5	Sprint 1	56

5.5.1	Planificación del Sprint	56
5.5.2	Diseño	58
5.5.2.1	Diseño Front-End	58
5.5.2.2	Diseño Back-End	63
5.5.3	Pruebas de sistema.....	67
5.5.4	Revisión y Retrospectiva del Sprint	75
5.6	Sprint 2.....	77
5.6.1	Planificación del Sprint	77
5.6.2	Diseño	78
5.6.2.1	Diseño Front-End	78
5.6.2.2	Diseño Back-End	81
5.6.3	Pruebas de sistema.....	85
5.6.4	Revisión y Retrospectiva del Sprint	91
5.7	Sprint 3.....	92
5.7.1	Planificación del Sprint	92
5.7.2	Diseño	93
5.7.2.1	Diseño Front-End	93
5.7.2.2	Diseño Back-End	95
5.7.3	Pruebas de sistema.....	99
5.7.4	Revisión y Retrospectiva del Sprint	104
5.8	Sprint 4.....	106
5.8.1	Planificación del Sprint	106
5.8.2	Diseño	107
5.8.2.1	Diseño Front-End	107
5.8.2.2	Diseño Back-End	111
5.8.3	Prueba de sistema	117
5.8.4	Revisión y Retrospectiva del Sprint	129
5.9	Sprint 5.....	130
5.9.1	Planificación del Sprint	130
5.9.2	Diseño	131
5.9.2.1	Diseño Front-End	131
5.9.2.2	Diseño Back-End	134
5.9.3	Pruebas de sistema.....	138
5.9.4	Revisión y Retrospectiva del Sprint	147
6	Dragon Eats	149
6.1	Breve explicación del capítulo	149
6.2	Vistas de todos los usuarios antes de autenticarse.....	149
6.3	Vistas de los clientes	152
6.4	Vistas del administrador	158

7	Resultados y conclusiones	163
7.1	Repaso general y resultados obtenidos	163
7.2	Líneas futuras	164
7.3	Análisis de impacto	164
7.4	Conclusión personal.....	166
8	Bibliografía	167

1 Introducción

1.1 Importancia de la digitalización y la hostelería española

Actualmente, en un mundo en el que el 66,2% [1] de la población mundial tiene acceso a Internet (un 95,4% en España [2]) y el 97,8% [1] dispone de un dispositivo telefónico (un 99,9% en España [2]), las herramientas tecnológicas han dejado de ser una mera apuesta para el futuro para convertirse en una necesidad del presente. Esta nueva realidad, debido en gran medida a la pandemia del coronavirus, ha acelerado la transformación digital en diversos sectores, revolucionando así la forma en que las empresas operan y se relacionan con sus clientes.

En España, la industria de la hostelería, muy ligada al turismo, juega un papel fundamental tanto en la economía como en la cultura del país, llegando a aportar el 6,5% del PIB en 2019, que corresponde a 131.209 millones de euros de facturación. Además de esto, había cerca de 1,7 millones de trabajadores, que supone el 8,7% del empleo total de España, convirtiéndose en la tercera industria nacional, y también había unos 320.000 establecimientos por todo el país, entre restaurantes, bares y cafeterías. Cabe destacar que la mayor parte del empleo correspondía al subsector de restauración [3].

Sin embargo, en 2020, debido a la pandemia del COVID-19, los restaurantes, bares y otros establecimientos tuvieron que enfrentarse a numerosos retos para evitar la propagación del virus, como el distanciamiento social y la adopción de medidas de higiene. Ante esta crisis sanitaria, los números bajaron significativamente, llegando a aportar un 3,3% menos del PIB y perdiendo unos 300.000 empleados y unos 85.000 establecimientos [3].

Para poder adaptarse a esta nueva normalidad, la gran mayoría de los locales empezaron a digitalizarse y modernizarse poco a poco, integrando nuevas tecnologías para ofrecer servicios más seguros y eficientes. Algunos ejemplos son la digitalización de menús a través de códigos QR, sistemas TPV para comandas o la contratación de sistemas de entrega de pedidos en línea [4].

Este proceso de digitalización no solo ha sido una respuesta a las circunstancias impuestas por la pandemia, sino que también ha marcado el inicio de una nueva era digital en la que la tecnología juega un papel crucial en la interacción entre los establecimientos hosteleros y su clientela. En este escenario, por primera vez en 2023, se han recuperado ya prácticamente los datos prepandemia, superando en la primera mitad del año las cifras alcanzadas en 2019 en casi la totalidad de los indicadores [5]. Este resurgimiento no solo refleja la resiliencia del sector, sino también confirma la efectividad de la digitalización como requisito esencial en el nuevo paradigma de negocio, consolidando el futuro de la hostelería.

1.2 Problemática

A pesar de que la pandemia haya acelerado la transformación digital hostelera y esta ha demostrado su efectividad en la actual era digital, según el informe “Estudio de Digitalización de la Hostelería” realizado por EY en 2021 [6], el 63% de los encuestados desconoce completamente o sólo tiene alguna idea de las herramientas tecnológicas existentes, pero no se han informado en profundidad (véase Ilustración 1). Sumado a esto, más de la mitad de los hosteleros todavía

no han decidido sobre la previsión de inversión en tecnología en corto plazo (véase Ilustración 2) [6].

¿Conoces las diferentes herramientas/iniciativas que hay disponibles para digitalizar tu negocio?



Ilustración 1: Conocimiento de las posibles herramientas digitales disponibles.

¿Cuál es tu previsión de inversión en tecnología a corto plazo?

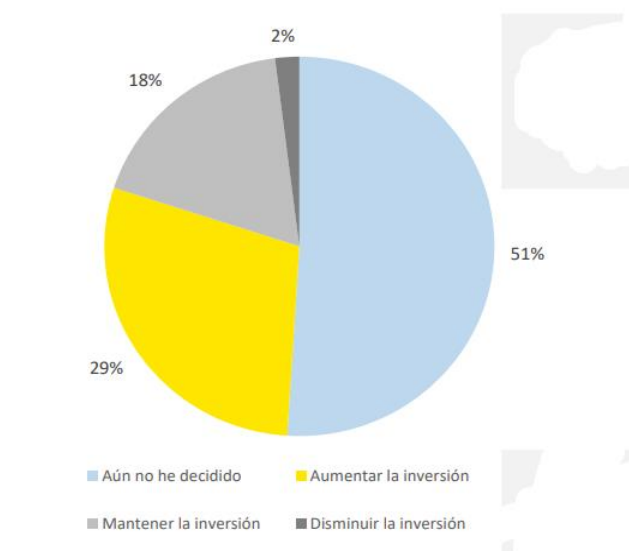


Ilustración 2: Previsión de inversión en tecnología a corto plazo.

Debido a esta indecisión y la falta de conocimiento sobre la digitalización, según el informe “El estado de digitalización de la hostelería en España” realizado por BBC Innovation y Delectatech en 2022 [7], solo el 15,93% de los establecimientos están altamente digitalizados y el 32,41% van a estarlo. Más concretamente, como se observa en la Ilustración 3, se ha detectado que un

2,8% del total de establecimientos son innovators o entusiastas, es decir, los primeros en adoptar la innovación; 13,13% son early adopters o visionarios, dicho de otra manera, son la referencia para el resto de consumidores; 32,41% son early majority o pragmáticos, en otras palabras, adoptan la innovación cuando ya ha sido aprobada por los early adopters; 47,96% son late majority o conservadores, es decir, esperan a adoptar la innovación cuando la utiliza gran parte del mercado; y, por último, el 3,7% son laggards o rezagados, dicho de otro modo, son los últimos en probar la innovación [7].

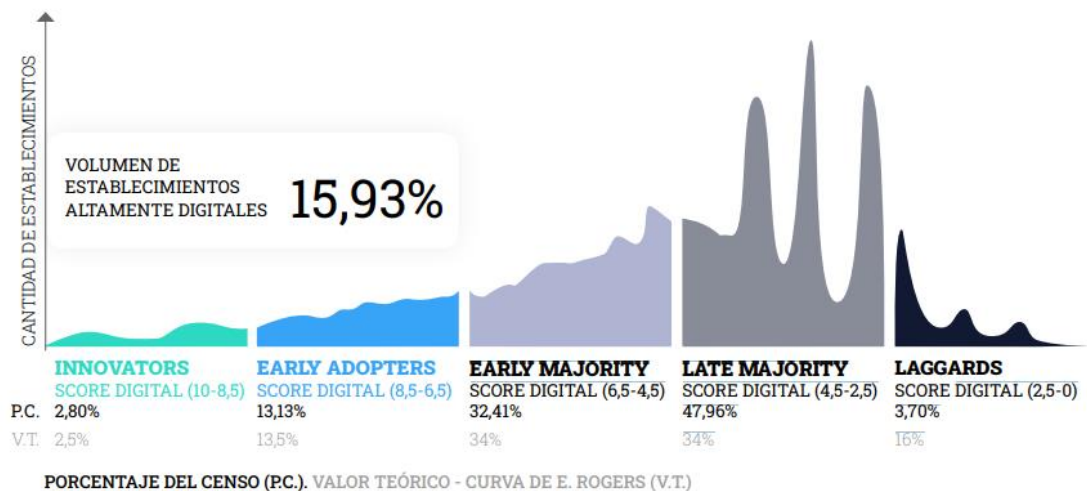


Ilustración 3: Estado de digitalización de los establecimientos.

Además de esto, este informe ha analizado la presencia de marca a través de la digitalización y ha revelado que el 42,8% de los establecimientos en España tiene página web. En cuanto a la gestión y comunicación con el cliente, se ha detectado que el 33,6% de los locales que están en Google o Tripadvisor no tienen el control de sus datos o no los actualizan, mientras que los establecimientos que sí tienen el control de sus datos tienen tres veces más opiniones en estos directorios online. En referencia a las reservas digitales, se ha constatado que solo el 7,3% usa sistema de reserva digital y el 13,8% posee carta digital. Por otro lado, del 34,3% de los locales que hacen delivery & take away, el 76,4% de estos poseen agregador de delivery, como Glovo, Just Eat o UberEats, y el 23,6% con delivery propio o no digital. Cabe destacar que más de la mitad de aquellos con agregador usan más de un agregador al mismo tiempo.

A pesar de que la digitalización ha avanzado mucho estos últimos años, estos estudios muestran que sigue habiendo un largo camino por recorrer. A continuación, se va a describir una serie de problemas tanto para establecimientos no digitalizados como digitalizados.

Por un lado, los establecimientos no digitalizados tienen que enfrentarse a problemas como:

- Ineficiencia en la gestión de pedidos en línea. Este proceso se inicia cuando el cliente realiza una llamada telefónica al local y el empleado que está a cargo deberá anotar su pedido y, tras esto, tendrá que pasar el pedido al cocinero. Todo este proceso puede propiciar errores o malentendidos por parte tanto del cliente como de los empleados. Además de esto, la falta de seguimiento automatizado de los pedidos

puede resultar retrasos en la cocina, afectando negativamente a la velocidad de servicio.

- Ineficiencia en la gestión de reservas. Este proceso se inicia cuando el cliente va presencialmente al establecimiento para realizar la reserva o cuando realiza una llamada telefónica al local dentro de su horario laboral, generando a veces demoras de tiempo. Por su parte, el empleado que está a cargo deberá saber las horas y las mesas disponibles para ese día determinado que quiere reservar el cliente, lo que puede conducir a errores humanos si no se ha gestionado manualmente todas las reservas en un papel de manera correcta.
- Falta de interacción con el cliente. Ante la falta de presencia en línea, los restaurantes pueden perder oportunidades para interactuar con los clientes fuera del establecimiento. Esto podría limitar la capacidad de establecer conexiones y fidelizar a la clientela.
- Dificultad en la recopilación de reseñas y valoraciones. Relacionado con el problema anterior, la falta de presencia digital hace que la única manera de recopilar las opiniones de los clientes es por medio del habla o usando aplicaciones externas como Google Maps o TripAdvisor. Sin embargo, como ya se ha visto anteriormente, hay un 33,6% de los locales que no tienen control sobre ello, lo que dificulta la retroalimentación por parte del restaurante, limitando su capacidad de mejora y de adaptación a las preferencias del cliente.
- Pérdida de competitividad. A pesar de que sólo el 16% de los establecimientos están altamente digitalizados, aquellos que se están digitalizándose tarde o directamente no lo hacen, pueden perder competitividad. Esto es debido a que, en la actual era digital, la mayoría de los consumidores buscan comida en línea y la falta de visibilidad digital puede resultar en la pérdida de clientes potenciales.

Por otro lado, algunos establecimientos digitalizados tienen que enfrentarse a posibles problemas como:

- Dependencia de múltiples tecnologías y costes asociados. El uso de aplicaciones de terceros como Just Eat, Glovo, Uber Eats, The Fork, entre otras, puede generar una dependencia tecnológica significativa para los diferentes locales. La necesidad de adaptarse y mantenerse en varias plataformas puede resultar costosa tanto en términos de tarifas asociadas como en la complejidad operativa.
- Falta de personalización en plataformas externas. Relacionado con el problema anterior, la uniformidad en el diseño y la estructura de estas plataformas puede limitar la capacidad de los restaurantes para personalizar su contenido, lo que podría afectar la presentación de la identidad única del establecimiento.
- Falta de control de las opiniones online. La presencia de plataformas como Google Maps o TripAdvisor implica que los clientes pueden dejar reseñas y opiniones sobre el restaurante. Sin embargo, estas opiniones pueden ser difíciles de gestionar y, en ocasiones, pueden no reflejar con precisión la calidad del servicio o la experiencia gastronómica. Los restaurantes se enfrentan al desafío de lidiar con críticas negativas que pueden afectar su reputación, incluso si son sin justificación. La falta de control sobre estas opiniones online puede ser un obstáculo para mantener una imagen positiva.

- Intensa competitividad. La participación en plataformas digitales compartidas, como Just Eat o Glovo, hace que los restaurantes estén en una constante competencia con numerosos establecimientos registrados en estas plataformas. Esto hace que la elección por parte de los clientes esté influenciada por factores como las reseñas o la clasificación. Además de esto, la competencia puede generar presión para ofrecer descuentos o promociones, afectando potencialmente los márgenes de ganancia de los restaurantes.

1.3 Aproximación a la resolución

Como respuesta a la necesidad de aumentar la digitalización en el sector hostelero y de resolver los problemas identificados en el apartado anterior, se propone el diseño y el desarrollo de una aplicación web integral para facilitar los diferentes servicios que ofrece un restaurante a sus clientes mediante una interfaz gráfica intuitiva e interactiva. El propósito de esta aplicación es mejorar la gestión administrativa interna de los restaurantes y proporcionar una experiencia más completa y satisfactoria tanto para los propietarios como para sus clientes, pudiendo adaptarse a sus preferencias y necesidades.

Para ello, la aplicación propuesta contará con diversas funcionalidades.

- En primer lugar, la aplicación contará con un sistema de gestión de pedidos en línea, lo que reducirá los posibles errores humanos y agilizará el proceso global de un pedido, permitiendo tanto a los empleados como a los clientes conocer el estado actual de un pedido.
- En segundo lugar, la aplicación contará con un sistema de reservas en línea, lo que simplificará la administración de las mesas por parte de los propietarios y se eliminará la necesidad de realizar reservas presenciales o mediante llamadas telefónicas, proporcionando a los clientes un método más cómodo para planificar sus visitas.
- En tercer lugar, la plataforma incorporará un sistema interactivo de reseñas y valoraciones de los diferentes platos disponibles en la web, facilitando la retroalimentación directa de los clientes y ofreciendo a los propietarios una herramienta valiosa para mejorar la calidad de sus servicios.
- Por último, la plataforma contará con un sistema de identificación y autenticación para salvaguardar la privacidad y seguridad de los datos de los clientes. Esto no solo garantizará la confidencialidad de la información personal y financiera, sino que también facilitará a los propietarios gestionar de manera efectiva los pedidos y reservas de sus clientes.
- Como funcionalidad extra, se quiere incorporar un sistema de recomendación de platos basado en inteligencia artificial dependiendo de las valoraciones que ha realizado el cliente.

Con estas funcionalidades, esta aplicación podrá resolver los problemas identificados para los establecimientos no digitalizados, además de incrementar la competitividad por la posibilidad de interacción con los clientes fuera del local.

A parte de esto, la plataforma es capaz de resolver los problemas identificados para los establecimientos digitalizados con dependencia tecnológica, porque, al disponer de una página web propia, se elimina la necesidad de depender de múltiples plataformas externas como Just Eat, Glovo o Uber Eats. Esto no solo reduce la complejidad operativa y los costos asociados, como las comisiones por cada pedido, sino que también proporciona a los propietarios un mayor control

sobre la presentación de su establecimiento y la personalización de sus servicios, superando la uniformidad impuesta por dichas plataformas.

Además, al contar con un sistema de reseñas y valoraciones integrado, los propietarios pueden gestionar de manera más efectiva las opiniones online, respondiendo y gestionando las críticas de manera más directa. Esto hará que sea más fácil controlar la reputación en línea del restaurante.

Desde la perspectiva del cliente, la aplicación integral proporciona una experiencia más sencilla y conveniente, ya que los usuarios pueden realizar pedidos, reservas y dejar reseñas, todo desde una única plataforma, eliminando la necesidad de buscar en múltiples páginas o aplicaciones para interactuar con su restaurante favorito.

En resumen, esta aplicación web propuesta representará una solución integral que impulsará la transformación digital en el sector hostelero, afrontando los desafíos tanto de los establecimientos no digitalizados como de aquellos con dependencia tecnológica. Al proporcionar una gestión eficiente, reducir costos y brindar una experiencia personalizada, la plataforma no solo satisfará la demanda actual de digitalización, sino que también impulsará la competitividad de los restaurantes en un entorno tecnológico en constante evolución.

1.4 Lista de objetivos

El **objetivo principal** del Trabajo de Fin de Grado (TFG) es la documentación y el desarrollo de una aplicación web integral para facilitar los servicios de un restaurante con sus clientes. Para lograr el objetivo principal, se necesita conseguir los siguientes **subobjetivos**:

1. Análisis del problema y aplicaciones existentes:
 - a. Investigar los posibles problemas y proponer una solución aproximada.
 - b. Investigar y comparar aplicaciones semejantes existentes en el mercado.
2. Elección del entorno de tecnológico:
 - a. Investigar, estudiar y justificar las tecnologías y herramientas adecuadas para el desarrollo de la aplicación.
3. Definición de requisitos:
 - a. Identificar y documentar requisitos funcionales y no funcionales.
4. Diseño e implementación del Front-End de la aplicación:
 - a. Desarrollar una interfaz gráfica intuitiva e interactiva para facilitar la interacción de los clientes con la aplicación.
5. Diseño e implementación del Back-End de la aplicación:
 - a. Diseñar bases de datos e implementar la lógica del negocio para gestionar eficientemente los pedidos, reservas y reseñas.
6. Desarrollo de un sistema de identificación y autenticación:
 - a. Permitir a los clientes registrarse e iniciar sesión en la página web.
 - b. Permitir al administrador registrarse e iniciar sesión en la página web.
7. Desarrollo de un sistema de gestión de pedidos:
 - a. Permitir a los clientes realizar pedidos de comida a través de la aplicación.
 - b. Permitir al administrador visualizar y gestionar los pedidos y el tipo de pago seleccionado.
8. Desarrollo de un sistema de gestión de reservas:

- a. Permitir a los clientes reservar mesas en el restaurante a través de la aplicación.
- b. Permitir al administrador visualizar y gestionar las reservas de mesas de los clientes.
- 9. Desarrollo de un sistema de reseñas y valoración de los platos:
 - a. Permitir a los clientes realizar reseñas y valoraciones de los diferentes platos a través de la aplicación.
 - b. Permitir al administrador visualizar las reseñas y las valoraciones realizadas por los clientes.
- 10. Realización de pruebas y validaciones:
 - a. Realizar pruebas exhaustivas para garantizar la fiabilidad y el correcto funcionamiento de la aplicación.
 - b. Validar la aplicación con usuarios para recopilar retroalimentación y realizar mejoras.
- 11. Elaboración de la memoria:
 - a. Documentar todo el proceso de desarrollo, desde la idea hasta su implementación.
 - b. Presentar resultados, análisis y conclusiones en una memoria final.
- 12. Adquisición de nuevos conocimientos:
 - a. Obtener conocimientos en desarrollo web, tanto en el Front-End como en el Back-End.
 - b. Obtener conocimientos en la gestión de un proyecto software.

Como **objetivo opcional** es el desarrollo de un sistema de recomendación mediante inteligencia artificial, destinado a proporcionar a los clientes sugerencias de platos adaptadas a sus preferencias, basándose en las valoraciones que hayan realizado anteriormente.

1.5 Estructura del documento

El presente documento está estructurado de la siguiente manera para brindar una comprensión detallada del proyecto:

- 1. Capítulo 1: Introducción.** En este apartado se ha explicado la importancia de la hostelería española y de la digitalización en la era actual, se ha presentado los problemas existentes y una aproximación a la solución de estos problemas, se han detallado los objetivos a alcanzar y, por último, se ha proporcionado una visión general de la estructura de este documento.
- 2. Capítulo 2: Estado de arte.** En este apartado se realizará un análisis comparativo de las aplicaciones existentes en el mercado, concretamente aquellas relacionadas con la gestión de restaurantes. También se llevará a cabo una evaluación rigurosa de las características y funcionalidades presentes en estas aplicaciones.
- 3. Capítulo 3: Entorno de desarrollo.** En este apartado se explicarán las herramientas tecnológicas que se van a emplear para el diseño y el desarrollo de la aplicación. Además de esto, se justificarán las tecnologías seleccionadas y su importancia dentro del proyecto.

- 4. Capítulo 4: Metodología de desarrollo.** En este apartado se explicará la metodología que se va a seguir para la creación de la aplicación, describiendo paso a paso cómo se llevará a cabo su desarrollo.
- 5. Capítulo 5: Desarrollo de la aplicación.** En este apartado se especificará los requisitos funcionales de la aplicación, se elaborará los diseños de alto y bajo nivel de la aplicación, se documentará todo el proceso de desarrollo tanto del Front-End como del Back-End de la aplicación y, por último, se realizará una serie de pruebas para garantizar su correcto funcionamiento.
- 6. Capítulo 6: Dragon Eats.** En este apartado se presentará todas las vistas de la aplicación final.
- 7. Capítulo 7: Resultados y conclusiones.** En este apartado se realizará un resumen de los logros alcanzados durante el proyecto, una reflexión sobre los desafíos enfrentados y lecciones aprendidas y, por último, una serie de propuestas para las futuras mejoras y expansiones de la aplicación.
- 8. Capítulo 8: Bibliografía.** En este apartado se listarán todas las fuentes consultadas y referenciadas durante el desarrollo del proyecto.

2 Estado de arte

2.1 Aplicaciones existentes en el mercado actual

En la actualidad, la industria de la restauración se encuentra inmersa en un continuo proceso de transformación digital, donde la eficiencia operativa y la satisfacción del cliente se han convertido en dos pilares fundamentales para alcanzar el éxito de un establecimiento. Este escenario ha propiciado la aparición de diversas aplicaciones destinadas a agilizar y mejorar las diferentes gestiones internas que tiene un restaurante, que van desde la supervisión de los empleados hasta la administración de reservas o pedidos, aumentando así la productividad del local. A continuación, se explorarán detalladamente algunas de las soluciones digitales más destacadas que han redefinido la interacción entre los clientes y sus restaurantes preferidos, contribuyendo así a una experiencia culinaria más moderna y adaptada a las expectativas contemporáneas:

1. Aplicaciones de entrega de comida. Son aquellas aplicaciones que permiten a los usuarios realizar pedidos de comida y pagos en línea, accediendo así a una amplia variedad de opciones gastronómicas desde la comodidad de su hogar. Entre ellas destacan las siguientes:

a. Just Eat. Es la plataforma líder en España de entrega de comida que pone en contacto a los amantes de la comida con los restaurantes que quieren expandir su oferta gastronómica más allá de las puertas de su local. Su objetivo principal es permitir al usuario explorar una amplia variedad de menús de diferentes tipos de cocina, desde opciones tradicionales hasta propuestas culinarias innovadoras, a través de una interfaz intuitiva que facilita la búsqueda de restaurantes y la selección de platos, brindando a los usuarios una experiencia de pedido sin complicaciones [8].

b. Uber Eats. Es la extensión de la conocida plataforma de transporte Uber y se presenta como una opción de entrega de comida a domicilio. Además de ofrecer una amplia gama de restaurantes, Uber Eats destaca por su enfoque innovador, que incluye la entrega rápida y la posibilidad de rastrear en tiempo real el estado del pedido. La integración con la aplicación principal de Uber también agrega un nivel adicional de comodidad para los usuarios habituales [9].

c. Glovo. A diferencia de otras aplicaciones, Glovo va más allá de ser una simple aplicación de entrega de comida, ya que también ofrece servicios de entrega de diversos productos, permitiendo a los usuarios realizar pedidos que van desde comidas de restaurantes y alimentos de supermercados hasta productos de farmacia, deporte o moda. Esta amplitud de opciones convierte a Glovo en una aplicación versátil y completa, capaz de satisfacer diversas necesidades del cliente de manera conveniente [10].

2. Aplicaciones de reservas de mesas. Son aquellas aplicaciones que optimizan la gestión de mesas para los restaurantes, permitiendo a los usuarios realizar reservas y planificar sus experiencias gastronómicas de

manera conveniente, directamente desde sus dispositivos electrónicos. Entre ellas destacan:

- a. **OpenTable.** Es la aplicación líder en reservas de restaurantes, permitiendo a los usuarios explorar una amplia variedad de restaurantes según su ubicación o su tipo de cocina y reservar en tiempo real con disponibilidad actualizada. Además, la aplicación permite a los usuarios visualizar los menús y acceder a opiniones de otros comensales. También puede aumentar la lealtad de los clientes con programas de fidelidad que ofrecen acumulación de puntos para luego canjearlos como descuentos. Por último, ofrece una interfaz intuitiva a los restaurantes para gestionar las reservas [11].
- b. **TheFork.** Es una aplicación muy similar a la anterior, con las funcionalidades básicas para el usuario, como la reserva de mesas y la visualización de menús y opiniones, pero presenta dos características adicionales. En primer lugar, ofrece la visualización en el mapa de restaurantes con sus respectivos porcentajes de descuento, proporcionando a los usuarios una visión rápida de las ofertas disponibles en su área. En segundo lugar, incorpora la conveniencia del pago móvil a través de TheFork Pay, mejorando la experiencia del usuario [12].

3. Aplicaciones de reseñas y opiniones. Son aquellas aplicaciones que ofrecen a los usuarios nuevos una valiosa fuente de opiniones y experiencias redactadas por otros usuarios acerca de diversos establecimientos gastronómicos. Entre ellas destacan:

- a. **TripAdvisor.** Es una aplicación que no solo abarca restaurantes, sino también hoteles y atracciones turísticas, ofreciendo una amplia variedad de reseñas y calificaciones, lo que proporciona a los usuarios una visión integral de la calidad de un establecimiento. A parte de esto, la comunidad global de viajeros que comparte experiencias en todo el mundo añade un componente enriquecedor a esta plataforma, convirtiéndola en una guía confiable para explorar nuevos lugares [13].
- b. **Google Maps.** Además de ser reconocido como una herramienta de navegación, Google Maps es una aplicación que permite a los usuarios evaluar todo tipo de establecimientos, desde restaurantes hasta otros lugares de interés. Además de esto, mejora la interacción entre usuarios al proporcionar un espacio para compartir comentarios y acceder a opiniones de otros, enriqueciendo así la experiencia de explorar y seleccionar lugares. Por último, al igual que la aplicación anterior, cabe destacar que Google Maps facilita la información de contacto de los diferentes lugares de interés como el número de teléfono o su página web oficial [14].

4. Sistemas de gestión de relación con el cliente (CRM, Customer Relationship Management). Son herramientas diseñadas para centralizar y gestionar todas las relaciones e interacciones con los clientes, ya sean existentes o potenciales. En el contexto de los

restaurantes u otras empresas, estos sistemas permiten la administración de contactos, el análisis de datos y la mejora continua de las relaciones con los clientes [15]. Entre ellas destacan:

- a. **Salesforce.** Es la plataforma líder de gestión de las relaciones con los clientes (CRM) en la nube que proporciona a las empresas las capacidades necesarias para administrar eficazmente todas las fases del ciclo de vida del cliente, fortaleciendo así las relaciones comerciales. Se destaca por su enfoque altamente personalizable, ofreciendo herramientas potentes de marketing, automatización de ventas y servicios al cliente [16].
- b. **HubSpot.** Al igual que la anterior, es un CRM especializado en el marketing, ventas y servicios al cliente. Sin embargo, Hubspot ofrece herramientas con menor personalización, lo que significa que es más fácil de usar, simplificando así el proceso de marketing y ventas. Por último, sigue la metodología del inbound marketing [17] y marketing de contenido que consiste en atraer clientes de manera orgánica y natural [18].

5. Sistemas de gestión de contenidos (CMS, Content Management Systems). Son plataformas que ofrecen una interfaz intuitiva que simplifica el proceso de creación, edición y gestión de contenido digital, permitiendo a los usuarios construir y mantener sitios web sin requerir habilidades de programación avanzadas [19]. Entre ellas destacan:

- a. **Wordpress.** Es la plataforma líder de gestión de contenidos (CMS) que proporciona una gran cantidad de funciones, especialmente en la gestión de blogs, desde programar publicaciones hasta administrar comentarios. Su editor es compatible con HTML [20] y Markdown [21], y viene equipado con Jetpack [22], que proporciona estadísticas del sitio web, SEO básico [23], optimización de velocidad y uso compartido en redes sociales. Además de blogs, ofrece una enorme cantidad de plugins como WooCommerce [24] para el comercio electrónico. Además de esto, Wordpress permite a los usuarios elegir cientos de miles de plantillas para crear sus sitios webs con casi total libertad de personalización. Por último, cabe destacar que cuenta con la comunidad más extensa entre los CMS, lo que proporciona un amplio respaldo para los usuarios [25] [26].
- b. **Wix.** Es una plataforma que tiene componentes y aplicaciones que funcionan como parte de un sistema de gestión de contenidos (CMS), pero no es un CMS puro [27]. Es conocido por su enfoque amigable para principiantes, ofreciendo una amplia variedad de plantillas, widges y herramientas de diseño visual para la creación de páginas webs. También presenta herramientas analíticas a los usuarios para proporcionarles información sobre el tráfico de su sitio web o el comportamiento de sus visitantes. Por último, ofrece soluciones para blogs y tiendas en línea, reproductores de música personalizables, galerías de medios y herramientas básicas de marketing por correo electrónico [25] [28].

- c. Squarespace.** A diferencia de Wix, que se basa en complementos para desarrollar su funcionalidad básica, Squarespace es un sistema de gestión de contenidos (CMS) que ofrece una rica variedad de funciones desde el principio. Además de contar con más de 100 plantillas personalizables, todas ellas diseñadas profesionalmente, proporciona un conjunto de funciones para el comercio electrónico como herramientas para la programación, la gestión de reservas o la capacidad de crear áreas para miembros. Por último, dispone de poderosas herramientas de SEO (Search Engine Optimization) y una variedad de herramientas de análisis de sitios web para ayudar al usuario en el crecimiento de su página web [25] [29].

2.2 Desventajas de las aplicaciones existentes

A pesar de la popularidad de estas aplicaciones o herramientas tecnológicas disponibles en el mercado actual, existen varias razones por las cuales estas soluciones pueden no ser la mejor elección para todos los restaurantes. A continuación, se va a explicar las desventajas más importantes que presentan estas soluciones, agrupadas en tres categorías:

1. Desventajas de las aplicaciones de terceros:

- a. Comisiones elevadas.** Las aplicaciones de entrega de comida suelen imponer a los restaurantes una tarifa de activación al unirse a la plataforma, una tarifa de servicio y recurrentes comisiones que oscilan entre el 25% y el 40% por cada pedido realizado [30]. De manera similar, las aplicaciones de reservas también aplican cargos por cada reserva realizada a través de sus plataformas. Además de esto, estas aplicaciones pueden cobrar tarifas adicionales por destacar un restaurante en sus sitios web. Como consecuencia, estos costos acumulativos pueden afectar negativamente los márgenes de beneficio de los restaurantes. En contraste, el uso de una aplicación web integral propia brinda la oportunidad de reducir estos costos al eliminar intermediarios.
- b. Falta de control.** Al adoptar aplicaciones de terceros, los restaurantes se ven obligados a depender en gran medida de estas plataformas para gestionar sus operaciones, lo que puede traducirse en una pérdida de control sobre algunos aspectos del negocio, como la gestión de pedidos, reservas o relaciones con los clientes. Esto puede dar lugar a situaciones imprevistas, como errores en la entrega de comida sin que el restaurante tenga conocimiento inmediato, errores en la confirmación de reservas o publicaciones de críticas poco constructivas imborrable por el propietario. Como consecuencia, esto perjudicará la reputación del establecimiento de manera injusta. Además de esto, estas plataformas pueden realizar cambios repentinos en las condiciones de servicio o en las tarifas, pudiendo afectar negativamente en la eficiencia y rentabilidad del restaurante. En contraposición, una aplicación web integral proporciona al restaurante un mayor control sobre sus operaciones, permitiendo una gestión más adaptada a las necesidades específicas del negocio.

c. Falta de personalización. Estas plataformas suelen ofrecer interfaces estandarizadas que limitan la capacidad del restaurante para diseñar y adaptar su contenido, lo que puede afectar negativamente su identidad de marca única. Además de esto, la falta de personalización también se refleja en la incapacidad de ajustar completamente las funcionalidades de la aplicación a las necesidades específicas del restaurante. Por ejemplo, las aplicaciones de entrega de comida pueden carecer de la funcionalidad de reservas, mientras que las aplicaciones de reservas o reseñas no cuenta con la opción de realizar pedidos en línea. Esta limitación también se traduce en la pérdida de oportunidades para implementar programas de fidelización personalizados. Por último, estas plataformas actúan como intermediarios entre el restaurante y el cliente, limitando la interacción directa entre ellos. Por el contrario, una aplicación web integral propia supera estas limitaciones al brindar al restaurante la libertad total para personalizar cada aspecto de la plataforma, desde el diseño visual hasta las funciones específicas, adaptándose completamente a las necesidades del restaurante. De esta manera, no solo conserva la autenticidad de la marca, sino que también asegura una experiencia única para los clientes.

d. Intensa competitividad. En un entorno donde miles de restaurantes compiten por la atención en estas plataformas, destacarse individualmente se convierte en un gran desafío. La falta de opciones de personalización, como se mencionó anteriormente, limita significativamente las posibilidades de diferenciación para cada restaurante. Esto se traduce en que las elecciones de los clientes estén más influenciadas por las reseñas y calificaciones existentes en estas aplicaciones que en las características únicas de cada establecimiento. En contraste, una aplicación web integral propia brinda la oportunidad de construir una identidad distintiva y destacar entre la competencia, permitiendo una conexión más personalizada con los clientes.

2. Desventajas de los sistemas de gestión de relaciones con el cliente (CRM):

a. Complejidad y alto coste. La mayoría de los CRM están diseñados con un enfoque más empresarial, por lo que puede disponer de funciones que no sean necesariamente relevantes para un pequeño restaurante local, que solo busca atraer a más clientes y aumentar sus ventas. Además, la adopción de un sistema CRM puede ser compleja y requerir una inversión significativa de tiempo y recursos para su implementación y su mantenimiento constante con el fin de garantizar su correcto funcionamiento. Cabe destacar también que el alto precio de los algunos programas CRM puede representar una barrera financiera para muchos restaurantes pequeños. Por último, la formación de nuevos empleados dedicados específicamente al uso del CRM se convierte en una necesidad, lo que puede agregar una carga adicional a la gestión de personal del restaurante. En contraste, una aplicación web integral propia ofrece una solución más adaptada a las necesidades específicas de un restaurante, sin

la complejidad y el alto coste asociados con muchos sistemas CRM empresariales [31].

- b. Dependencia de la calidad de datos.** Para que un CRM funcione correctamente, es necesario proporcionarle datos precisos y actualizados, como detalles de contacto de clientes, historial de compras y preferencias, ya que cualquier error puede hacer que el sistema genere resultados inexactos, afectando negativamente a la toma de decisiones y las estrategias de marketing para la empresa [31]. Para evitar esto, es necesario mantener la calidad de los datos a lo largo del tiempo, lo que puede requerir esfuerzos constantes de limpieza y actualización, convirtiéndose en una carga adicional en la gestión operativa del restaurante. En contraposición, una aplicación web integral propia puede simplificar este aspecto al ofrecer a los clientes la capacidad de gestionar y actualizar su información de manera conveniente a través de su cuenta personal.

3. Desventajas de los sistemas de gestión de contenidos (CMS):

- a. Falta de personalización.** A pesar de que los CMS proporcionan diversas plantillas, ya sea de pago o gratuitas, diseñadas para adaptarse a cualquier sitio web (blog, tienda online, etc.), estas vienen predefinidas y los cambios que se pueden realizar en ellas son mínimos, afectando así a la imagen de marca y la singularidad de la página, ya que existe el riesgo de que otra empresa utilice la misma plantilla. Además de esto, abordar completamente las necesidades específicas de un restaurante puede ser costosa y complicada, porque será necesario adaptarse a los módulos existentes (plugins) de la plataforma o buscar uno adecuado o desarrollar un nuevo módulo desde cero. Por último, existe el riesgo de que el diseño no se vea bien en todos los navegadores debido a la falta de pruebas y evaluación por parte de los desarrolladores de la plantilla. En contraste, optar por una aplicación web integral propia brinda una solución superior, permitiendo una personalización total desde el diseño hasta las funciones específicas de cada restaurante. De esta manera, es posible hacer que la web sea visualmente atractivo y fácil de usar y genere de manera efectiva una experiencia satisfactoria para el cliente [32] [33] [34] [35].
- b. Problemas de escalabilidad.** Relacionado con el punto anterior, debido a la falta de personalización, la plataforma puede tener dificultades en términos de escalabilidad, ya que carece de la flexibilidad necesaria para adaptarse eficientemente a un aumento significativo de la cantidad de contenido, usuarios o funciones. Esta incapacidad para escalar adecuadamente podría resultar en problemas como la ralentización de la plataforma, la imposibilidad de la expansión posterior o la necesidad de inversiones con el fin de actualizar su infraestructura. En contraste, una aplicación web integral propia, al ser diseñada específicamente para las necesidades de cada restaurante, ofrece una solución escalable y sin complicaciones para acompañar el desarrollo y evolución del restaurante [33].

- c. Problemas de rendimiento y SEO.** La utilización de CMS estándar a menudo conlleva desafíos significativos en términos de rendimiento del sitio web y optimización para motores de búsqueda (SEO). Esto se debe a que los CMS, al tener una estructura más compleja y estar cargados con características genéricas, generan un exceso de código innecesario que impacta negativamente el rendimiento del sitio. Esta situación se traduce en una carga más lenta de la página, afectando directamente la experiencia del usuario y provocando potencialmente una disminución en las clasificaciones de los motores de búsqueda. Por el contrario, una aplicación web integral desarrollada específicamente para las necesidades de un restaurante, con un código limpio y eficiente, puede garantizar un rendimiento óptimo y una implementación eficaz de estrategias de SEO adaptadas, contribuyendo a una presencia en línea más efectiva y exitosa [33].
- d. Dependencia de extensiones y problemas de seguridad.** La seguridad de los CMS se ve comprometida al depender de extensiones y plugins desarrollados por una empresa específica, ya que la protección del sistema queda supeditada a las actualizaciones de seguridad que implemente dicha empresa. A pesar de que los CMS desarrollados bajo licencias libres suelen incorporar medidas de seguridad avanzadas, su uso generalizado y extendido los convierte en blancos frecuentes para hackers que buscan vulnerabilidades. Una vez encontrado, estos hackers suelen divulgar esta información permitiendo que personas menos familiarizadas con la programación puedan llevar a cabo ataques exitosos contra el sitio web. Por el contrario, al utilizar bibliotecas de terceros que pueden ser personalizadas internamente, una aplicación web integral desarrollada específicamente para las necesidades de un restaurante posee una seguridad más robusta frente a los ataques de los hackers. Además, al contar con todos los niveles de seguridad necesarios, se protege tanto el contenido como los datos del cliente de manera más efectiva [33] [34].

2.3 Necesidad de una aplicación web construida con frameworks

Ante las desventajas explicadas anteriormente, surge la necesidad de crear una aplicación web integral personalizada para cada restaurante mediante el uso de frameworks. De esta manera, se eliminará la dependencia de aplicaciones de terceros, se reducirá los costes asociados a comisiones y se proporcionará un mayor control sobre la gestión de pedidos, reservas y relaciones con los clientes. Además de esto, la capacidad de personalización total asegura que la identidad de marca se mantenga intacta, permitiendo así una diferenciación efectiva en un mercado altamente competitivo. Por último, también se solucionaría tanto el problema de la complejidad y del alto coste de implementación de los CRM como los problemas de personalización, escalabilidad, rendimiento y seguridad de los CMS. De esta manera, esta aplicación web construida con frameworks no solo simplificará la gestión operativa interna de los restaurantes, sino que también mejorará la experiencia del cliente y reducirá los costos a largo plazo.

3 Entorno de desarrollo

3.1 Historia del desarrollo web

Antes de abordar las diferentes tecnologías que existen en la actualidad, se quiere realizar un breve repaso histórico sobre el desarrollo web para comprender la evolución de esta disciplina a lo largo del tiempo y entender las bases sobre las cuales se sustenta el panorama actual de la programación web.

La historia comienza a finales de la década de 1980, cuando Tim Berners-Lee, un científico informático británico que trabajaba en el CERN, tuvo la idea de crear un lugar donde se pudiera compartir información entre investigadores de todo el mundo, lo que dio lugar al nacimiento de lo que se conoce hoy en día como la World Wide Web. Después de esto, en 1990, propuso formalmente el concepto de la web y desarrolló el primer navegador web junto con el software de servidor web correspondiente. Por último, en 1991, introdujo el HTML (Hypertext Markup Language), un lenguaje fundamental para las páginas web, ya que permitía la estructuración de documentos con hipervínculos, lanzando así en 1993 el primer sitio web que explicaba la World Wide Web [36].

Tras un período de intensa competencia entre distintos navegadores, destacando especialmente Netscape Navigator e Internet Explorer, en 1995, los desarrolladores comenzaron a experimentar con JavaScript para añadir contenido dinámico e interactivo a las páginas web. Posteriormente, en 1996, el World Wide Web Consortium (W3C) introdujo CSS (Cascading Style Sheets), que permitió a los desarrolladores separar el contenido de la presentación, haciendo que el diseño web fuera más flexible y estandarizado [36].

A principios de la década de 2000, se introdujo la Web 2.0, que enfatizaba el contenido generado por el usuario, las redes sociales y tecnologías como AJAX (Asynchronous JavaScript and XML) que permitían la creación de aplicaciones web dinámicas. Más tarde, en 2007, con la llegada del iPhone, los dispositivos móviles se convirtieron en un gran desafío para el desarrollo web debido a los diferentes tamaños de pantalla y orientaciones que tenían estos dispositivos. Como solución, surgió el concepto de diseño adaptativo, también conocido como “Responsive Design”, que permitía a los sitios web adaptarse de manera fluida a diversos dispositivos, garantizando una experiencia de usuario óptima en cualquier contexto [36].

A finales de la década de 2000 hasta el presente, la popularidad de frameworks y bibliotecas como jQuery, Angular, React y Vue.js ha crecido exponencialmente, facilitando el desarrollo y promoviendo las mejores prácticas para la creación de aplicaciones web interactivas. Además de esto, el surgimiento del desarrollo Full-Stack ha permitido a los desarrolladores dominar tanto el Front-End (lado del cliente) como el Back-End (lado del servidor), mientras que las prácticas de DevOps han acortado la brecha entre el desarrollo y las operaciones de TI. Por último, también han cobrado gran importancia la accesibilidad web y las aplicaciones web progresivas, junto con el avance de tecnologías como la inteligencia artificial, la realidad virtual o la realidad aumentada [36].

En resumen, el desarrollo web ha experimentado una notable transformación desde los inicios de la World Wide Web hasta su estado actual y, con el ritmo acelerado de los avances tecnológicos, los desarrolladores web seguirán dando forma al futuro de la web, abordando nuevos desafíos y explorando nuevas posibilidades para ofrecer experiencias digitales cada vez más avanzadas.

3.2 Tecnología para Front-End

3.2.1 Tecnologías disponibles para Front-End en la actualidad

El Front-End, también conocido como el lado del cliente, es la parte de una aplicación web con la que los usuarios pueden ver e interactuar directamente: menús de navegación, colores y estilos de texto, botones, imágenes, etc. Por ello, el desarrollador Front-End tiene como objetivo diseñar y crear una interfaz de usuario atractiva e intuitiva para brindar una experiencia única al usuario. Además de esto, debe asegurarse de que el sitio web tenga una capacidad de respuesta óptima y se adapte correctamente a los diferentes dispositivos que existen en la actualidad [37]. A continuación, se presentan las principales tecnologías para el desarrollo del Front-End:

- **HTML (Hypertext Markup Language):** Es el lenguaje estándar para la creación y el diseño de las distintas páginas web. Consiste en una combinación entre el hipertexto, que define los enlaces entre las páginas web, y el lenguaje de marcado, que define la estructura del documento de texto mediante una serie de elementos que indican al navegador cómo mostrar el contenido. Estos elementos etiquetan diferentes partes del contenido, como encabezados, párrafos, enlaces, etc [20].
- **CSS (Cascading Style Sheets):** Es un lenguaje de hojas de estilo utilizado para controlar el diseño y la presentación de las páginas webs. Para ello, añade estilos a los documentos HTML, describiendo cómo deben mostrarse los distintos elementos HTML en la página web. Por último, cabe destacar que la palabra "cascading" significa que un estilo aplicado a un elemento padre también se aplicará a todos los elementos hijos dentro del padre [38].
- **JavaScript:** Es un lenguaje de programación de alto nivel, interpretado, multiplataforma y orientado a objetos, utilizado principalmente para crear contenido interactivo y dinámico en las páginas web. Para ello, permite a los desarrolladores agregar diversas funcionalidades, como la creación de animaciones, la validación de formularios, la manipulación del DOM (Document Object Model) o la comunicación con servidores a través del AJAX. Actualmente, es un componente esencial en el desarrollo web moderno y se ejecuta tanto en el navegador del cliente (Front-End) como en aplicaciones de servidor (Back-End) con tecnologías como Node.js [39].
 - **jQuery:** Es una biblioteca de JavaScript rápida, ligera y rica en funciones desarrollada por John Resign en 2006 con el objetivo de simplificar la creación de las páginas web dinámicas e interactivas, permitiendo a los desarrolladores manipular fácilmente el DOM, controlar los eventos, crear las animaciones y realizar las llamadas AJAX en JavaScript. Con jQuery, los desarrolladores pueden escribir menos código para lograr efectos similares en comparación con JavaScript puro [40].
 - **React:** Al igual que el anterior, React, desarrollada por Facebook, es una de las bibliotecas de JavaScript de código abierto más populares, eficientes y potentes para construir interfaces de usuario dinámicas e interactivas, especialmente para aplicaciones de una sola página (SPA). Se destaca por su arquitectura basada en componentes y el DOM virtual, lo que lo hace altamente eficiente en la representación de interfaces de usuario, asegurando así un rendimiento óptimo. Por último, su capacidad de utilizar

componentes reutilizables lo convierte en una herramienta valiosa para el desarrollo eficiente y el mantenimiento de aplicaciones a gran escala [41].

- **Angular:** Es una plataforma de desarrollo creada por Google y construida sobre TypeScript de Microsoft [42], un superset de JavaScript que agrega tipado estático opcional, lo que permite definir tipos de datos para las variables con el fin de detectar errores en tiempo de compilación y mejorar la capacidad de mantenimiento y escalabilidad de los proyectos de software. Además de esto, Angular ofrece un framework basado en componentes para construir aplicaciones web escalables, junto con un conjunto de herramientas para desarrolladores y una colección de bibliotecas bien integradas que abarcan una amplia variedad de funcionalidades, como el enrutamiento, la gestión de formularios y la comunicación cliente-servidor [43].
- **Vue.js:** Es un framework de JavaScript progresivo de código abierto creado por Evan You para construir fácilmente interfaces de usuario y aplicaciones de una sola página (SPA). Se destaca por su enfoque basado en componentes reutilizables y modulares, su sistema reactivo que permite que sus componentes de la interfaz de usuario reaccionen automáticamente a los cambios de estado, y su uso de directivas para enlazar datos y manipular el DOM de manera declarativa. Por último, cabe mencionar que utiliza la sintaxis de plantillas HTML, lo que lo hace fácilmente entendible para los desarrolladores familiarizados con HTML [44].

3.2.2 Tecnologías seleccionadas para Front-End

Para el desarrollo Front-End de este proyecto, se va a utilizar de manera indispensable **HTML**, ya que proporciona la estructura fundamental de la página web, definiendo claramente la funcionalidad de cada elemento mediante sus etiquetas semánticas. De esta manera, no solo ayudará a los motores de búsqueda a comprender el contenido, sino que también mejorará la accesibilidad para todo tipo de usuarios. Por último, al ser un estándar ampliamente reconocido y compatible con todos los navegadores modernos, garantiza que la página web sea accesible para la mayoría de los dispositivos y plataformas actuales.

Además de HTML, también se va a usar de manera indispensable **CSS**, ya que proporciona la capacidad para controlar la presentación y el estilo visual de los documentos HTML, brindando una experiencia única a los usuarios. Además, al optimizar el código CSS mediante técnicas como la combinación de reglas, se puede mejorar el rendimiento de la página, reduciendo el tiempo de carga y ofreciendo una experiencia de usuario más rápida y fluida.

Junto a HTML y CSS, se ha decidido emplear **Angular** como framework para el desarrollo Front-End de este proyecto. A continuación, se detallan los principales motivos que respaldan esta elección sobre los demás frameworks:

1. El principal motivo es la integración de Angular con TypeScript, ya que este lenguaje de programación permite escribir un código JavaScript más comprensible y seguro que el propio JavaScript al introducir el tipado estático y otras características adicionales de programación orientada a objetos, como clases, interfaces y herencia de tipos. Estas características hacen que el código sea más estructurado y mantenible, lo cual es especialmente beneficioso en proyectos de gran escala.

2. El segundo motivo es la estructura modular que ofrece Angular gracias a su arquitectura basada en el patrón MVC (Modelo-Vista-Controlador), proporcionando una organización clara para las aplicaciones web, de manera que cada módulo puede contener uno o varios componentes, y cada uno de estos puede utilizar diferentes servicios. De esta manera, el módulo actúa como punto de entrada de una aplicación Angular, donde se declaran las dependencias y se configura la aplicación; los componentes son los responsables de mostrar información al usuario y gestionar sus interacciones con la aplicación; y los servicios son los encargados de proporcionar datos y funcionalidades a través de sus métodos específicos. Esta división de responsabilidades entre componentes y servicios permite una mayor modularidad y reutilización de código, lo que resulta en aplicaciones más flexibles, mantenibles y escalables a lo largo del tiempo.
3. El tercer y último motivo es el desarrollo eficiente de código gracias al conjunto de herramientas y características que proporciona Angular. Entre estas características se incluyen el enlace de datos bidireccional automático, que sincroniza automáticamente el modelo de datos con la vista de la aplicación, eliminando la necesidad de escribir código adicional para gestionar cambios en los datos; el uso de las directivas personalizadas, que permite a los desarrolladores crear sus propias etiquetas o atributos HTML, simplificando la manipulación del DOM; y el uso de un sistema de inyección de dependencias entre los distintos componentes de la aplicación, que permite una mejor modularidad y reutilización del código, tal y como se explicó en el anterior punto. Estas características, combinadas con la amplia documentación, la activa comunidad de desarrolladores de Angular y su carácter multiplataforma, hacen que sea una opción sólida para el desarrollo de aplicaciones web modernas y de alto rendimiento, reduciendo significativamente tiempo y esfuerzo para implementar funcionalidades complejas.

3.3 Tecnología para Back-End

3.3.1 Tecnologías disponibles para Back-End en la actualidad

El Back-End, también conocido como el lado del servidor, es la parte de una aplicación web que no es visible ni interactúa directamente con los usuarios, sino que es la porción de software que se encarga de gestionar la lógica de negocio de la aplicación. Para ello, es el encargado de procesar solicitudes del cliente, realizar operaciones en la base de datos, autenticar a los usuarios, gestionar sus sesiones y garantizar la seguridad de los datos. Además de esto, se encarga de optimizar el rendimiento del sitio web, manejar los posibles errores y excepciones de manera adecuada, y crear APIs (Application Programming Interfaces) para permitir la integración con otras aplicaciones y servicios. A pesar de que no es visible para los usuarios, el backend desempeña un papel fundamental en la operación de un sitio web, ya que proporciona la funcionalidad necesaria para que el Front-End pueda mostrar la interfaz de usuario y comunicarse con el servidor [37]. A continuación, se presentan las principales tecnologías para el desarrollo del Back-End:

- **Node.js:** Es un entorno de tiempo de ejecución de JavaScript de código abierto y multiplataforma creado por Ryan Dahl en 2009 para permitir a los desarrolladores realizar casi cualquier tipo de proyecto, tanto Front-End como Back-End. Está basado en el motor V8 de Google Chrome para ejecutar JavaScript de manera rápida y eficiente; y utiliza un modelo de

entrada/salida asíncrona sin bloqueo y orientado a eventos, lo que lo hace eficiente y adecuado para aplicaciones en tiempo real que manejan una gran cantidad de solicitudes concurrentes. Entre sus usos, destacan el desarrollo de servidores web, aplicaciones web y aplicaciones escalables de red. Por último, cabe destacar que una aplicación construida con Node.js ejecuta solamente un proceso simple, sin necesidad de crear un nuevo hilo para cada solicitud [45].

- **Express.js:** Es un framework minimalista y flexible para Node.js que proporciona características y herramientas robustas para el desarrollo Back-End de las aplicaciones web y móviles, la creación rápida de rutas, el enrutamiento, el manejo de solicitudes y respuestas HTTP y el uso de middleware para el procesamiento de solicitudes intermedias. Su enfoque ligero y modular lo hace ideal para construir aplicaciones web desde simples APIs REST hasta aplicaciones web completas y escalables [46].
- **Python:** Es un lenguaje de programación interpretado, orientado a objetos y de alto nivel, ampliamente utilizado y apreciado por su sintaxis clara y legible, lo que lo hace accesible tanto para principiantes como para desarrolladores experimentados. Con su enfoque en la simplicidad y la versatilidad, Python se adapta bien a una gran variedad de aplicaciones, desde el desarrollo de aplicaciones web y scripts de automatización hasta el análisis de datos y la inteligencia artificial. Además de esto, Python admite módulos y paquetes, lo que fomenta la modularidad del programa y la reutilización de código [47].
 - **Django:** Es un framework de alto nivel para web escrito en Python que fomenta el desarrollo rápido y un diseño limpio y pragmático al proporcionar un conjunto completo de herramientas y componentes preconstruidos, permitiendo a los desarrolladores concentrarse más en la lógica de la aplicación que en los detalles complejos de una página web. Algunas de las características de Django es el uso de un ORM potente, un sistema de plantillas robusto para crear vistas HTML y un sistema de enrutamiento intuitivo. Por último, su enfoque en la seguridad, la escalabilidad y la sencillez lo convierte en una opción muy popular tanto para aplicaciones pequeñas como para grandes proyectos web [48].
- **Java:** Es un lenguaje de programación orientado a objetos, centrado en la red y de alto nivel, creado por Sun Microsystems en 1995 para permitir a los desarrolladores realizar cualquier tipo de proyecto, desde aplicaciones web o móviles y software empresarial hasta aplicaciones de macrodatos y tecnologías del servidor. Además, por su fiabilidad, seguridad y robustez, ha sido una de las opciones más populares entre los desarrolladores durante más de dos décadas, con millones de aplicaciones Java en uso en la actualidad. Por último, Su característica más destacada es la capacidad de escribir código una vez y ejecutarlo en cualquier dispositivo que tenga una máquina virtual Java (JVM) instalada, lo que lo hace ideal para el desarrollo de aplicaciones multiplataforma. [49].
 - **Spring:** Es un framework de código abierto para el desarrollo de aplicaciones Java empresariales. Para ello, ofrece una gran variedad de funcionalidades que facilitan el desarrollo, la configuración y la gestión de las aplicaciones mediante técnicas como la programación orientada a aspectos (AOP), la inversión de control (IoC) o la inyección de dependencias (DI). Además de esto, Spring promueve la escalabilidad y modularidad de los sistemas,

permitiendo a los desarrolladores construir aplicaciones robustas y mantenibles a lo largo del tiempo. Por último, cabe destacar que Spring puede ofrecer soluciones desde aplicaciones web construidas con microservicios hasta aplicaciones sin servidor en la nube, convirtiéndolo en una herramienta muy versátil para todos los desarrolladores del mundo [50].

- **C#:** Es un lenguaje de programación moderno, innovador, multiplataforma, orientado a objetos y de código abierto, desarrollado por Microsoft para permitir a los desarrolladores crear una gran variedad de aplicaciones, desde aplicaciones de escritorio hasta aplicaciones web y móviles. Es conocido por su sencillez, su seguridad y su capacidad para trabajar con tipos de datos complejos, lo que lo convierte en una opción popular tanto para principiantes como para desarrolladores experimentados. Además de esto, su sintaxis viene derivada de C y C++ y utiliza el modelo de objetos de la plataforma .NET, muy parecido al de Java, aunque incluye mejoras propias de otros lenguajes. Por último, como curiosidad, la letra C proviene de la nota musical do y el símbolo # significa sostenido, lo que indica que es un semitono más alto, sugiriendo que C# es superior a C y C++ [51].
 - **.NET:** Es un framework de desarrollo de software creado por Microsoft que ofrece un entorno de ejecución y bibliotecas para la creación y ejecución de aplicaciones en diversos tipos de dispositivos y sistemas operativos. Utilizado principalmente con el lenguaje de programación C#, .NET proporciona un conjunto de herramientas robustas y escalables para muchos tipos de aplicaciones, desde aplicaciones de escritorio y web hasta aplicaciones móviles, juegos o servicios en la nube. Por último, aunque .NET se usa principalmente con C#, también es compatible con otros lenguajes de programación, lo que permite a los desarrolladores aprovechar sus habilidades existentes y trabajar en un entorno familiar y productivo [52].
- **PHP:** Es el acrónimo recursivo de Hypertext Preprocessor y se trata de un lenguaje de programación de código abierto muy popular para el desarrollo web. Su principal característica es su capacidad para ser incrustado dentro de los documentos HTML, permitiendo la creación de páginas web dinámicas. Además de esto, su compatibilidad con los diferentes sistemas operativos y servidores lo convierte en una opción bastante versátil que, junto a su capacidad para interactuar con las bases de datos, facilita la gestión de información dinámica en las aplicaciones web. Por otra parte, es conocido por su facilidad de uso, ofreciendo tanto características básicas como funciones avanzadas para desarrolladores de todos los niveles. Por último, además de su enfoque en el desarrollo web, también se utiliza en otros contextos, como el procesamiento de datos o la automatización de tareas del lado del servidor [53].
 - **Symfony:** Es un framework PHP de código abierto que ofrece una estructura sólida y modular para desarrollar aplicaciones web profesionales de forma rápida y segura. Su licencia MIT permite un uso gratuito incluso en aplicaciones comerciales, y su documentación extensa y gratuita lo hace accesible para desarrolladores de todos los niveles. Symfony se compone de varios componentes independientes que pueden utilizarse juntos o por separado, lo que brinda flexibilidad y facilidad de personalización. Por último, por su enfoque en la seguridad, su

comunidad activa y su historial probado de más de dos décadas, Symfony es considerado el mejor framework PHP para la creación de las aplicaciones web [54].

3.3.2 Tecnología seleccionada para Back-End

Para el desarrollo Back-End de este proyecto, se utilizará el lenguaje de programación **Java** junto con el framework **Spring Boot**, una extensión de Spring Framework que facilita la configuración y el desarrollo de las aplicaciones Java de manera rápida y sencilla. A continuación, se detallan los principales motivos de esta elección:

1. El principal motivo es la familiaridad con el lenguaje de programación Java, siendo el lenguaje más utilizado durante toda la carrera, lo que reducirá la curva de aprendizaje y acelerará el proceso de desarrollo, centrándose sobre todo en la lógica de negocio de la aplicación.
2. El segundo motivo es la integración de Spring Boot con un servidor web, lo que permite a los desarrolladores crear y desplegar fácilmente aplicaciones web sin necesidad de instalar y configurar un servidor web externo.
3. El tercer motivo radica en que Spring Boot utiliza convenciones sobre la configuración, lo que permite a los desarrolladores comenzar a trabajar rápidamente con una configuración mínima requerida. Además de esto, proporciona una configuración automática para casos de uso comunes, como la conectividad de bases de datos o la seguridad, reduciendo la cantidad de código y de configuraciones repetitivas. Por último, proporciona un sistema POM (Project Object Model) inicial para facilitar la gestión de las dependencias y una herramienta CLI para generar rápidamente código repetitivo e iniciar nuevos proyectos.
4. El cuarto motivo es la integración de Spring Boot con una serie de herramientas de compilación, de manera que las aplicaciones Spring Boot pueden ser construidas mediante herramientas como Maven o Gradle, simplificando el proceso de compilación y empaquetado, lo que facilita el ciclo de vida del proyecto y garantiza una mayor consistencia en el proceso de construcción de la aplicación.
5. El quinto motivo radica en el soporte que brinda Spring Boot para la construcción de microservicios. Su diseño liviano, servidor integrado y compatibilidad con patrones de microservicios populares, como Circuit Breaker y API Gateway, lo convierten en una opción ideal para arquitecturas distribuidas y escalables.
6. El sexto motivo es el amplio soporte que ofrece Spring Boot para las pruebas, integrándose con marcos de pruebas populares como JUnit y Mockito, lo que facilita la realización de pruebas unitarias y de integración para el correcto funcionamiento de la aplicación.
7. El séptimo motivo consiste en la integración de Spring Boot con herramientas para el monitoreo de métricas y la realización de comprobaciones de salud del servidor, facilitando el desarrollo y la operación de aplicaciones en entornos de producción.
8. Por último, el octavo motivo radica en la extensa comunidad que respalda a Spring Boot, brindando una gran variedad de recursos en línea y documentación.

3.4 Tecnología para bases de datos

3.4.1 Tecnologías disponibles para bases de datos en la actualidad

A pesar de que el manejo de las bases de datos realmente forma parte del Back-End, se ha decidido crear una nueva sección para abordar las diferentes tecnologías disponibles en este ámbito. En términos generales, una base de datos es un conjunto organizado de datos almacenados electrónicamente dentro de un sistema informático. Para poder almacenar, recuperar, actualizar, mantener y gestionar la información dentro de una base de datos es necesario un programa de software llamado sistema de gestión de bases de datos (DBMS), que sirve como interfaz entre la base de datos y sus aplicaciones o usuarios finales. Además de administrar de manera eficiente una base de datos, también proporciona herramientas para garantizar la seguridad de los datos, controlar el acceso de los usuarios, realizar copias de seguridad y restauración, y optimizar el rendimiento del sistema [55]. Actualmente, existen dos enfoques principales en el mundo de las bases de datos:

- **Bases de datos SQL (Structured Query Language):** Es una base de datos relacional que almacena y organiza los datos en tablas que contienen filas y columnas, de manera que cada fila representa un registro y cada columna representa un atributo de ese registro. Además de esto, existen relaciones entre tablas mediante el uso de claves foráneas, asegurando así la integridad de los datos y reduciendo las posibles redundancias. Por último, al utilizar SQL para seleccionar, insertar, actualizar y eliminar los datos de una base de datos, se siguen las propiedades ACID (Atomicidad, Consistencia, Aislamiento, Durabilidad) [56]. Algunos DBMS conocidos para bases de datos SQL, más conocido como sistemas de gestión de bases de datos relacionales (RDBMS), son los siguientes:
 - **PostgreSQL:** Es un RDBMS de código abierto y potente creado en la Universidad de California en Berkeley, que utiliza el lenguaje SQL combinado con una serie de características para almacenar y escalar de forma segura las cargas de trabajo de datos más complicados. Con más de 35 años de desarrollo, es conocido por su arquitectura comprobada, confiabilidad, integridad de datos, conjunto de funciones robustas y la dedicación de la comunidad de código abierto para ofrecer soluciones innovadoras y de alto rendimiento. Por último, cabe destacar que PostgreSQL se ejecuta en todos los principales sistemas operativos, compatible con ACID desde 2001, cumple al menos con 170 de los 179 requisitos obligatorios para la conformidad central de SQL:2023, y cuenta con tipos de datos y funciones personalizadas y extensiones de bases de datos geoespaciales como PostGIS. [57].
 - **MySQL:** Es un RDBMS de código abierto desarrollado inicialmente por MySQL AB, pero luego fue adquirido por Sun Microsystems y esta, a su vez, fue comprado por Oracle Corporation. Su característica principal es su modelo cliente-servidor, de manera que el servidor es el lugar donde realmente residen los datos y el cliente debe lanzar una petición a dicho servidor para solicitar aquellos datos que desea consultar, insertar, actualizar o eliminar. Esta arquitectura cliente-servidor permite a los usuarios acceder a los datos de manera fácil, eficiente, rápida y segura. Además de esto, MySQL ofrece una gran cantidad de características que incluyen soporte para múltiples plataformas, tipos de datos y

lenguajes de programación. Por último, MySQL se caracteriza por su alto rendimiento y su amplia flexibilidad y escalabilidad, siendo capaz de gestionar desde pequeñas bases de datos de proyectos personales hasta bases de datos de grandes empresas con millones de registros, convirtiéndose en el DBMS estándar de la industria [58].

- **SQLite:** Es una biblioteca en lenguaje C que implementa un motor de base de datos SQL pequeño, rápido, transaccional, autocontenido, de alta confiabilidad y con todas las funciones integradas. Además de esto, es de código abierto, por tanto, es libre para cualquier propósito, comercial o privado. A diferencia de otros DBMS, SQLite no tiene un servidor independiente, sino que lee y escribe directamente en los archivos de disco ordinarios, es decir, se integra directamente en la aplicación que lo utiliza, lo que lo hace ideal para aplicaciones de escritorio y móviles. Por otro lado, el formato de archivo de SQLite es estable, multiplataforma y compatible con versiones anteriores, y los desarrolladores se comprometen a mantenerlo así hasta el año 2050. Por último, a pesar de estar bastante compacto, SQLite es muy fiable gracias a las múltiples pruebas exhaustivas que se realizan antes de cada lanzamiento, convirtiéndolo en una gran opción para aquellas aplicaciones que requieren una base de datos simple, pero muy completo [59].
- **Bases de datos NoSQL (Not Only SQL):** Es una base de datos no relacional diseñada principalmente para manejar grandes volúmenes de datos no estructurados o semi-estructurados de manera flexible y escalable. A diferencia de las bases de datos relacionales, que siguen el modelo de tablas y relaciones, las bases de datos NoSQL pueden ser de diferentes tipos según su modelo de datos. Las principales son los siguientes [56]:
 - **Bases de datos clave-valor:** Almacenan datos en forma de pares clave-valor, de manera que los datos se recuperan mediante el uso de clave. Algunos ejemplos son Redis y Amazon DynamoDB [56].
 - **Bases de datos de documentos:** Almacenan datos en forma de documentos JSON, XML u otro formato similar, donde cada documento puede contener diferentes campos y estructuras. Algunos ejemplos son MongoDB, Cloud Firestore, CouchDB [56].
 - **Bases de datos de grafos:** Almacenan datos en forma de nodos y relaciones, lo que permite representar y consultar datos complejos. Algunos ejemplos son Neo4j y JanusGraph [56].
 - **Bases de datos de columnas:** Almacenan datos en forma de columnas en lugar de filas, lo que permite consultas eficientes sobre conjuntos de columnas específicos. Algunos ejemplos son Apache Cassandra y HBase [56].

A continuación, se quiere profundizar en algunos de los DBMS más conocidos para bases de datos NoSQL:

- **MongoDB:** Es un DBMS NoSQL orientado a documentos, de código abierto y diseñado para almacenar y gestionar grandes volúmenes de datos de forma flexible y escalable. Además, no guarda los datos en tablas, sino en estructuras de datos BSON (similar a JSON) con un esquema dinámico, por lo que los campos pueden variar entre documentos y la estructura de datos puede cambiarse a lo largo del tiempo. Este modelo de documento se mapeará directamente a los objetos en el código de la aplicación,

lo que facilita el trabajo con los datos. Además de esto, ofrece consultas ad hoc, indexación y agregación en tiempo real para acceder y analizar los datos de manera eficiente y segura. Por último, cabe destacar que MongoDB es una base de datos distribuida con numerosas herramientas de automatización, monitoreo y respaldo, caracterizada por una alta disponibilidad con replicación y respaldo integrados, y una escalabilidad horizontal con fragmentación nativa [60].

- **Cloud Firestore:** Es una base de datos NoSQL flexible y escalable en la nube, creada en la infraestructura de Google Cloud, con el objetivo de almacenar y sincronizar los datos en tiempo real para el desarrollo de aplicaciones web, móviles y de servidor de una manera eficiente y sencilla. En primer lugar, con su modelo de datos flexible y jerárquico, permite a los desarrolladores almacenar datos en forma de documentos, organizados en colecciones, con soporte para estructuras de datos complejas y subcolecciones. Además de esto, permite a los desarrolladores realizar consultas expresivas para recuperar documentos individuales específicos o todos los documentos de una colección que coincidan con los parámetros de consulta. Por otro lado, al igual que Firebase Realtime Database, Cloud Firestore utiliza la sincronización de datos para actualizar la información en tiempo real en cualquier dispositivo conectado, permitiendo a la aplicación escribir, manipular y consultar los datos incluso en modo offline al almacenar activamente los datos en su caché, y sincronizando los cambios automáticamente una vez que el dispositivo recupera la conexión. Por último, al estar construido bajo la robusta infraestructura de Google Cloud, se puede escalar eficientemente, ofreciendo la replicación automática de datos multirregión, las garantías de coherencia sólida, las operaciones atómicas por lotes y el soporte integral para las transacciones [61].
- **Redis:** Es el acrónimo de Remote Dictionary Server y se trata de una base de datos NoSQL que almacena datos en forma de pares clave-valor en memoria de manera rápida y de código abierto. Redis permite almacenar prácticamente cualquier tipo de datos, desde simples cadenas hasta hashes que almacenan una lista de campos y valores, e HyperLogLogs que cuentan los elementos únicos de un conjunto de datos. Por otro lado, su velocidad excepcional se debe a que todos los datos residen en la memoria principal del servidor, evitando los retrasos asociados con el acceso a discos. Además de esto, ofrece características como el uso de una arquitectura de replicación maestro-esclavo, la persistencia de datos mediante snapshots y la creación de archivos de solo anexos (AOF), junto con una amplia compatibilidad con numerosos lenguajes de programación. Por último, con casos de uso que van desde el almacenamiento en caché hasta la administración de sesiones y sistemas de chat y mensajería, Redis se destaca como una opción popular gracias a su alta velocidad, versatilidad y facilidad de uso [62].

3.4.2 Tecnología seleccionada para la base de datos

Para este proyecto se quería utilizar un DBMS distinto a MySQL, ya que es el que se ha utilizado principalmente durante toda la carrera, por lo que se optó

por emplear **PostgreSQL**, considerado actualmente como el DBMS más popular entre todos los profesionales según la encuesta anual para desarrolladores en 2023 realizado por Stack Overflow [63]. A continuación, se detallan otros motivos que influyeron en esta elección:

1. El primer motivo radica en que PostgreSQL puede ofrecer una gran variedad de características avanzadas, como vistas materializadas, disparadores INSTEAD OF o procedimientos almacenados escritos en lenguajes distintos de SQL. Además, también admite una gran variedad de tipos de datos, como enteros, booleanos, caracteres, arrays, JSON, XML o geometría espaciales. Esta diversidad de características y tipos de datos proporciona a los usuarios la flexibilidad necesaria para modelar y almacenar datos de manera eficiente y precisa.
2. El segundo motivo radica en que PostgreSQL ofrece las características ACID en todas sus configuraciones, a diferencia de otros RDBMS, como MySQL, que solo cuentan con estas características al utilizar motores de almacenamiento como InnoDB o NDB. Estas propiedades de la base de datos son fundamentales, ya que aseguran que la base de datos permanezca en un estado válido incluso después de un error inesperado, algo que los DBMS NoSQL no proporcionan en su totalidad. Además de esto, PostgreSQL ofrece una característica avanzada de control de concurrencia multiversión (MVCC), lo que permite a los usuarios leer y modificar los mismos datos de manera simultánea sin comprometer la integridad de los datos.
3. El tercer motivo es el alto rendimiento y escalabilidad que puede ofrecer PostgreSQL para el futuro de este proyecto a medida que aumenten los datos, asegurando una respuesta eficiente tanto de lectura como de escritura ante grandes cargas de trabajo, destacando especialmente en cargas de trabajo OLTP (OnLine Transaction Processing) y OLAP (OnLine Analytical Processing), pudiendo servir para el sistema de recomendaciones opcional de platos en el proyecto.
4. El cuarto motivo radica en que PostgreSQL usa el registro de escritura anticipada (WAL), un mecanismo que registra los cambios de datos antes de que se produzcan las modificaciones reales, lo que protege contra la pérdida de datos o la corrupción de la base de datos en caso de accidentes o cortes de energía. Además de esto, PostgreSQL ofrece medidas avanzadas de seguridad, como la autenticación basada en roles, la encriptación nativa de datos en reposo con el módulo pgcrypto o el uso de políticas de acceso a filas con RLS y MASK.
5. El quinto motivo es el modelo objeto relacional que usa PostgreSQL, lo que significa que los datos pueden adquirir características similares a los objetos de la programación orientada a objetos, proporcionando una excelente flexibilidad al modelar los datos del mundo real.
6. El sexto motivo radica en la gran comunidad de usuarios de PostgreSQL, ya que garantiza una resolución rápida de problemas con numerosos recursos y documentación en línea. Además, por ser de código abierto y compatible con múltiples plataformas, lo convierte en una opción flexible para los desarrolladores, además de evitar costos de licencia.

3.5 Tecnología para el control de versiones

3.5.1 Tecnologías actuales disponibles para el control de versiones

Los sistemas de control de versiones (VCS) son herramientas software diseñadas para gestionar y rastrear los cambios producidos en el código fuente y otros

archivos del proyecto software a lo largo del tiempo. Estas herramientas facilitan el trabajo en equipo en cualquier proyecto software, ya que, al proporcionar un historial detallado de cada modificación realizada en los archivos del proyecto, permitirán a los desarrolladores realizar un seguimiento de quién realizó cada cambio, cuándo lo realizó y qué cambios específicos implementó. Además de facilitar la comparación entre diferentes versiones de un archivo, como el código de un proyecto, una aplicación o un componente software normalmente se organiza en una estructura de carpetas o “árbol de archivos”, cada desarrollador podrá generar una nueva versión del código de un proyecto, de manera que cuando implemente una nueva funcionalidad, no bloqueará el avance de otro desarrollador. Tras terminar cada desarrollador con sus tareas, estas herramientas les permitirán fusionar sus cambios, creando así una versión que puede salir a producción después de las pruebas necesarias. Por último, gracias a la existencia de múltiples versiones, siempre se podrá volver a una versión anterior si se produjera algún error, pudiendo resolverse así dicho error. Esto no solo mejora la colaboración entre equipos de desarrollo, sino que también ayuda a mantener la integridad y la coherencia del código a lo largo del tiempo [64]. A continuación, se presentan los VCS más conocidos:

- **Git:** Es un sistema de control de versiones distribuido, gratuito y de código abierto, creado por Linus Torvalds en 2005 con el objetivo de permitir a los desarrolladores gestionar eficientemente el código fuente de sus proyectos. Una de sus características más destacadas es su capacidad para mantener un completo historial de cambios a largo plazo de todos los archivos, lo que incluye la creación, eliminación y modificación de estos, junto con la información del autor, la fecha y las notas descriptivas de cada cambio. Además de esto, Git permite a los desarrolladores crear ramas propias, generando así su propio repositorio local donde puede trabajar individualmente sin bloquear el avance de otros. Una vez terminado con sus implementaciones, Git permite operaciones de fusión de ramas en rama principal donde se encuentran los cambios de los demás. También existe la operación de reversión si se produjera algún fallo en el proyecto y se quiere restaurar una versión anterior. Por otro lado, su capacidad para gestionar flujos de trabajo no lineales, su uso de algoritmos eficientes para el almacenamiento y manipulación de los datos y su compatibilidad con una gran variedad de sistemas y protocolos hacen que sea una herramienta flexible y de buen rendimiento para proyectos de cualquier tamaño. También destaca por su capacidad para conservar la integridad del código fuente mediante el uso de hash criptográficos, lo que proporciona una garantía de que el historial de cambios es totalmente trazable y seguro frente a modificaciones accidentales o maliciosas. Por último, aunque puede tener una curva de aprendizaje pronunciada para algunos, Git ofrece una serie de ventajas que lo convierten en una herramienta muy importante para el desarrollo de software moderno [65] [66].
- **Apache Subversion (SVN):** Es un sistema de control de versiones menos utilizado que Git que, a diferencia de este último, sigue un modelo centralizado en el que todos los archivos y su historial están almacenados en un único servidor. Lanzado en el año 2000 por CollabNet, pero actualmente bajo la licencia de Apache, SVN permite a los equipos de desarrollo hacer un seguimiento de los diferentes cambios realizados en los archivos a lo largo del tiempo. A pesar de esto y de ser uno de los sistemas más populares en las décadas anteriores, su popularidad ha ido disminuyendo con el tiempo debido a las limitaciones en la escalabilidad

y en el rendimiento, así como las carencias de características avanzadas de ramificación y fusión que posee Git. A pesar de sus desventajas, SVN sigue siendo utilizado por algunas organizaciones debido a su bajo costo y a la resistencia a cambiar hacia sistemas alternativos. Además de esto, SVN es más fácil de entender para usuarios nuevos con el control de versiones y ofrece una interfaz de usuario más amigable que Git. Por tanto, SVN sigue siendo una opción válida para proyectos que requieren un control estricto sobre las versiones y los permisos de acceso, aunque esto podría hacerle menos flexible en entornos de desarrollo altamente dinámicos y colaborativos [67] [68].

3.5.2 Tecnología seleccionada para el control de versiones

Para el control de versiones de este proyecto, se ha seleccionado **Git** como principal tecnología en base a los siguientes motivos fundamentales:

1. El primer motivo es la gestión eficiente de las versiones de los proyectos, manteniendo un historial detallado de todos los cambios realizados en los diferentes archivos del proyecto a lo largo del tiempo.
2. El segundo motivo es la flexibilidad que proporciona Git en el flujo de trabajo, de manera que, si en un futuro este proyecto contara con más desarrolladores, Git permitirá a los desarrolladores crear nuevas ramas independientes para trabajar en nuevas funcionalidades o en correcciones de errores sin afectar al código principal del proyecto, ya que, al ser un VCS distribuido, cada desarrollador posee su propio repositorio local para poder trabajar de forma offline. Además de esto, Git permitirá la fusión de las diferentes ramas con sus respectivos cambios, la reversión de los cambios no deseados o errores, y la reversión completa del repositorio a una versión anterior de manera rápida y sencilla. Además, Git es compatible con diferentes sistemas operativos y plataformas de desarrollo, lo que facilita la colaboración entre desarrolladores con diferentes configuraciones de desarrollo.
3. El tercer motivo es la integración de Git con diferentes herramientas de desarrollo, como IDEs (Entornos de Desarrollo Integrado) y sistemas de integración continua, lo que facilita el flujo de trabajo de desarrollo. Por ejemplo, tanto Visual Studio Code como IntelliJ IDEA tienen integración con Git.
4. El cuarto y último motivo radica en la gran comunidad que respalda el uso de Git, proporcionando una gran cantidad de recursos, documentación y soporte en línea para ayudar a los desarrolladores a resolver los problemas y a mejorar su flujo de trabajo.

4 Metodología de desarrollo de software

4.1 Ciclo de vida del desarrollo de software

Antes de abordar las diferentes metodologías de desarrollo de software, es importante comprender qué es el ciclo de vida del desarrollo de software (SDLC, Software Development Life Cycle). Este ciclo representa un proceso organizado y estructurado que los equipos de desarrollo utilizan para planificar, diseñar, crear, probar y mantener un producto software de primer nivel en términos de calidad, rentabilidad y eficiencia de tiempo. Su objetivo principal es minimizar los riesgos del proyecto mediante una planificación anticipada que permita que el software cumpla con las expectativas del cliente tanto durante como después de la fase de producción. Esta planificación, además de guiar el desarrollo del producto, divide el proceso de desarrollo de software en diferentes fases con distintas tareas que se pueden asignar, completar y medir [69]. Las fases más comunes son las siguientes:

1. **Análisis.** En esta fase se establecen los objetivos del proyecto, se realiza un estudio de viabilidad del producto software, se analizan los costos y los riesgos que puede tener el proyecto, se identifican los recursos necesarios y se desarrolla un plan detallado que guiará todo el proceso de desarrollo. Además, durante esta etapa, el equipo de desarrollo recopila y analiza los requisitos de las partes interesadas, como clientes, expertos internos y externos, y directivos para crear un documento de especificaciones de requisitos del software que representan las diferentes funcionalidades que va a poseer el sistema.
2. **Diseño.** En esta etapa se crea el diseño del sistema software en función de los requisitos recopilados en la fase de análisis. Puede ser el diseño de la arquitectura del software, la estructura de datos, la lógica del flujo de datos y/o la interfaz de usuario.
3. **Implementación.** Durante esta fase, el equipo de desarrollo empieza a programar el producto software, basándose en el diseño elaborado previamente para cubrir los diferentes requisitos del cliente.
4. **Pruebas.** En esta etapa se llevan a cabo tanto pruebas automáticas como manuales para comprobar el correcto funcionamiento del software y corregir posibles errores. Puede ser pruebas unitarias, de integración, de sistema y/o de aceptación del usuario.
5. **Despliegue y mantenimiento.** Después de las múltiples pruebas y de ser aprobado por las partes interesadas, el equipo de desarrollo despliega el software en el entorno de producción para su lanzamiento ante los usuarios. Tras esto, comienza el mantenimiento del software, donde el equipo se dedica a corregir los errores y a resolver los problemas de los usuarios en un entorno diferente al de producción, llamado entorno de pruebas, que es invisible para los usuarios finales. Asimismo, se monitorea de cerca el rendimiento del sistema, la seguridad y la experiencia del usuario, identificando oportunidades para mejorar continuamente el software existente.

4.2 Modelos SDLC

Un modelo de SLDC es una representación conceptual y organizada de cómo llevar a cabo el proceso de desarrollo de software, con el fin de guiar a las diferentes organizaciones. Estos modelos organizan las fases del SDLC en diferentes secuencias cronológicas para optimizar el ciclo de desarrollo [69]. Los más conocidos son los siguientes:

1. **Modelo en cascada.** Este modelo organiza todas las fases del SLDC de forma secuencial, es decir, cada fase depende de la finalización de la fase anterior. Este modelo es adecuado para proyectos pequeños al ser simple y fácil de usar, ya que todo está bien organizado y las fases no pueden mezclarse entre sí. Sin embargo, una vez que se considera que una fase está completa, es difícil realizar cambios de las fases anteriores, lo cual no solo afecta al cronograma, al presupuesto y a la calidad del software, sino que también puede generar repercusiones considerables en el desarrollo general del proyecto [70].
2. **Modelo incremental.** Este modelo comienza con el desarrollo de un subconjunto de requisitos para luego incrementar de forma iterativa las diferentes funcionalidades del software hasta que el software esté listo para la producción. De esta manera, en cada iteración, los incrementos atraviesan todas las fases del SDLC y al concluir, se genera una versión actualizada del software. Este modelo facilita el proyecto al dividir el problema en incrementos más pequeños con entregas regulares, pudiendo modificar de manera más sencilla el software y gestionar los posibles riesgos. Sin embargo, debido a que los ciclos en el modelo incremental tienden a ser prolongados, existe la posibilidad de que se produzcan cambios en los objetivos iniciales del proyecto y en la asignación de recursos [71].
3. **Modelo en espiral.** Este modelo combina los ciclos repetidos del modelo incremental con el flujo secuencial y lineal del modelo en cascada para dar prioridad al análisis de riesgos, permitiendo identificar y mitigar los riesgos en cada ciclo. Por ello, este modelo se organiza en cuatro fases principales: establecimiento de objetivos, análisis de riesgos, desarrollo y verificación y planificación del siguiente ciclo. Este modelo es adecuado para grandes proyectos, permitiendo adaptarse a cambios frecuentes. Sin embargo, su complejidad de gestión y la necesidad de experiencia en la evaluación de riesgos pueden aumentar los costos de su implementación [72].
4. **Modelo ágil.** Este modelo se caracteriza por organizar las fases del SDLC en varios ciclos cortos de desarrollo, donde el equipo itera rápidamente a través de las fases y realiza pequeñas mejoras incrementales en el software en cada ciclo. Como se trata de un modelo centrado en la interacción con el cliente, los equipos realizan entregas y revisiones frecuentes, se adaptan a los cambios tras los comentarios del cliente y mejoran el software continuamente. A pesar de que este modelo fomenta la colaboración estrecha con los clientes para mejorar constantemente el producto software, depender en exceso de la opinión de los clientes puede hacer que los objetivos cambien drásticamente o dejar el proyecto a medias [73].

4.3 Ciclo de vida y metodología seleccionados

Tras analizar detenidamente los diversos ciclos de vida presentados anteriormente, se ha optado por el enfoque ágil, específicamente la metodología Scrum, que es un marco de gestión de proyectos basado en el modelo ágil y diseñado para ayudar a los equipos a estructurar y gestionar el trabajo mediante un conjunto de valores, principios y prácticas, entregando de forma iterativa incrementos de producto con alto valor para el cliente, a través de ciclos cortos de trabajo llamados “Sprints”. De esta manera, esta metodología permite a los equipos adaptarse rápidamente a los cambios, aprender de sus experiencias y de autoorganizarse para mejorar continuamente el producto con el fin de satisfacer las necesidades del cliente de manera efectiva. Aunque principalmente es utilizado por los equipos de desarrollo de software, actualmente está siendo adoptado por una gran cantidad de industrias, como el marketing, el sector financiero, la educación, entre otras, debido a su capacidad para aumentar la productividad y la eficiencia de los equipos [74].

A pesar de que la metodología Scrum está orientado a pequeños equipos multidisciplinarios, a continuación, se explica los motivos de la elección de Scrum como metodología de desarrollo de software para este proyecto individual:

1. El principal motivo radica en la posibilidad de interactuar estrechamente con los clientes, que en este caso son los dueños del restaurante. De esta manera, al realizar entregas parciales y regulares de incrementos de producto, esta metodología facilita la obtención de retroalimentación temprana de ellos, permitiendo ajustes y mejoras continuas del software para satisfacer sus necesidades y expectativas, lo que se traduce en una mayor satisfacción.
2. El segundo motivo es la capacidad de Scrum de optimizar el tiempo en un proyecto individual, ya que, al dividir el trabajo en incrementos manejables con plazos claros para cada tarea durante los Sprints, se puede concentrar en las actividades más importantes y obtener resultados de manera más eficiente, lo que puede aumentar la productividad y mantener un progreso constante hacia los objetivos.
3. El tercer y último motivo radica en la capacidad de Scrum para promover la motivación personal, la autoevaluación y la mejora continua del proyecto, ya que, al disponer de una lista de tareas a realizar en cada Sprint, se puede observar el progreso del trabajo, lo que no solo genera una satisfacción personal al ver el avance, sino que también genera un sentido de logro al acabar las diferentes tareas.

4.4 Origen de la metodología Scrum

Antes de abordar el funcionamiento de la metodología Scrum, es importante comprender su historia y origen. El concepto de “Scrum” se introdujo por primera vez en un artículo titulado “The New New Product Development Game” de la revista Harvard Business Review en 1986, escrito por Hirotaka Takeuchi e Ikujiro Nonaka con el fin de presentar un enfoque innovador para el desarrollo de nuevos productos. Para ello, en el artículo, tomaron el término “Scrum” (o “Melé” de origen francés) del rugby, una formación en la que los jugadores de ambos equipos se agrupan y se empujan unos contra otros, explicando que “como en el rugby, la pelota se pasa dentro del equipo, a medida que se mueve como una unidad por el campo”, lo cual hace similitud con los miembros de un

equipo de trabajo para desarrollar un nuevo producto, apoyándose mutuamente y avanzando de manera conjunta en el proyecto [74] [75] [76] [77].

Más tarde, en 1993, Jeff Sutherland, John Scumniotales y Jeff McKenna aplicaron por primera vez la idea de Scrum en el área de la tecnología e información, creando así el primer proceso de desarrollo de software con esta metodología. Sin embargo, no fue hasta 1995 cuando Scrum fue presentado públicamente en el artículo “The Scrum development process” por Ken Schwaber en la Conferencia Object-Oriented Programming, Systems, Languages & Applications (OOPSLA), en la cual expone las ventajas de este marco de trabajo [74] [75] [76] [77].

En 2001, tras observarse la creciente necesidad de una alternativa a los procesos de desarrollo de software pesados y basados en documentación, Ken Schwaber, junto a Jeff Sutherland y otras quince personas muy influyentes crearon y firmaron El Manifiesto Ágil, que consta de cuatro valores y doce principios, convirtiéndose en la base de todo desarrollo de software ágil hasta el día de hoy. Por último, en 2010, Ken Schwaber y Jeff Sutherland publican la primera Guía Scrum, un documento dinámico que actualizan de forma regular (2011, 2013, 2016, 2017 y 2020) en el que se explica detalladamente qué es Scrum y cómo ponerlo en práctica, proporcionando un conjunto de principios, roles, eventos y artefactos que forman la base de Scrum [74] [75] [76] [77].

4.5 Funcionamiento de la metodología Scrum

Para comprender correctamente el funcionamiento de la metodología Scrum es necesario explicar los siguientes tres pilares sobre las que se apoya la metodología [74] [78] [79]:

1. **Roles.** Un equipo de trabajo de Scrum suele ser un equipo pequeño y ágil de unas diez personas dedicados a ofrecer incrementos de productos en los diferentes sprints que se han establecido al principio. Para ello, se necesita los siguientes tres roles específicos:
 - a. **Product Owner.** Es la persona que más conoce el producto, entendiendo los requisitos empresariales, de los clientes y del mercado, es decir, los requisitos establecidos por los Stakeholders, para luego comunicarse eficientemente con el equipo de desarrollo con el fin de maximizar el valor del producto a entregar. Sus responsabilidades incluyen la creación y gestión del backlog del producto, asociarse estrechamente con el negocio y el equipo para garantizar el entendimiento de los diferentes elementos de trabajo del backlog, proporcionar al equipo una clara orientación sobre qué funcionalidades se deben entregar a continuación y decidir cuándo y con qué frecuencia se realizan las entregas. Por último, cabe destacar que es esencial que el product owner sea una única persona para evitar directrices cruzadas.
 - b. **Scrum Master.** Es la persona especialista en la metodología Scrum dentro del equipo, cuyo objetivo principal es dirigir los distintos eventos de Scrum y garantizar que se sigan los valores y principios de este enfoque ágil para aumentar la productividad y efectividad del equipo. Para lograr este objetivo, proporciona formación al equipo, al product owner y a la empresa para mejorar los procesos de Scrum y optimizar la entrega, planifica los recursos necesarios para cada Sprint, facilita las reuniones diarias de actualización, organiza las reuniones de planificación, revisión y análisis retrospectivo del sprint y se comunica con grupos

externos para resolver cualquier reto al que se enfrente el equipo en su conjunto. A través de estas acciones, fomenta la autoorganización y la mejora continua del equipo, así como la resolución de las dificultades que pueden aparecer durante todo el proceso de Scrum.

- c. **Equipo de desarrollo de Scrum.** Este equipo de desarrollo está formado por una gran variedad de profesionales, que van desde testers, diseñadores y especialistas en UX/UI (User eXperience/User Interface) hasta ingenieros de operaciones y desarrolladores, quienes colaboran estrechamente para cumplir con las tareas asignadas durante el Sprint y alcanzar los objetivos del proyecto en general. Para lograr esto, se autoorganizan de manera efectiva, se ayudan entre sí, promueven prácticas de desarrollo sostenible y estiman cuánto trabajo pueden finalizar cada día para completar con éxito cada Sprint basándose en su historial de entregas anteriores. La diversidad de habilidades dentro del equipo garantiza que cada miembro pueda complementar a los demás, evitando así que alguien se convierta en un cuello de botella para la entrega del trabajo. Por último, para que el equipo funcione correctamente, se necesita entre cinco y siete miembros, siguiendo la famosa “regla de las dos pizzas” de Jeff Bezos, fundador de Amazon, que sugiere que el equipo debe ser lo suficientemente pequeño como para compartir dos pizzas.

- 2. **Artefactos.** Son herramientas imprescindibles que ayudan a los equipos de trabajo de Scrum a gestionar el trabajo de manera efectiva y a mantener un entendimiento compartido de los objetivos del proyecto, proporcionando información sobre la planificación y las tareas a realizar, tanto para los miembros del equipo como para las partes interesadas. Estos son los tres principales artefactos para Scrum:

- a. **Product Backlog.** Es una lista dinámica de todas las funciones, requisitos, características, mejoras y correcciones que actúa como la base del Sprint Backlog, proporcionando una visión general de las tareas que deben abordarse para el producto en ese momento. Esta lista es dinámica porque el Product Owner está constantemente revisando, clasificando, reordenando y actualizando las prioridades del producto a medida que se obtiene nueva información del mercado, de los clientes o del equipo de proyecto, de manera que elimina elementos de la lista que dejan de ser relevantes o añade otros nuevos de mayor importancia.
- b. **Sprint Backlog.** Es la lista de elementos que el equipo de desarrollo se ha comprometido a implementar antes de que finalice el Sprint actual. Para ello, antes del comienzo de cada Sprint, durante la reunión de planificación de este, el equipo selecciona los elementos en los que trabajará durante dicho periodo a partir del Product Backlog. Por último, aunque el Sprint Backlog es flexible y puede evolucionar durante el Sprint, permitiendo la adición o eliminación de elementos según sea necesario, es importante no comprometer el objetivo fundamental del Sprint, que consiste en acabar todas las tareas definidas en el Sprint Backlog.
- c. **Incremento.** Es el producto final utilizable que se entrega al final de cada Sprint, durante la revisión de este, como muestra de lo

que el equipo ha logrado completar durante ese periodo. Este incremento del producto representa el objetivo o hito que ha establecido el equipo al inicio del Sprint y puede consistir en diferentes elementos, como un nuevo producto, una función adicional, una mejora o una corrección de errores, dependiendo de la forma en que cada equipo define su concepto de "Finalizado" y sus objetivos del Sprint.

3. Eventos: Son las prácticas, los protocolos y las reuniones secuenciales que los equipos de Scrum llevan a cabo periódicamente para facilitar la planificación, seguimiento y mejora continua del trabajo. Estos son algunos de los eventos típicos de la metodología Scrum:

- a. Product Backlog Refinement:** Durante este evento, también conocido como “Limpieza del Backlog” o “Refinamiento del Backlog”, el Product Owner asume la responsabilidad de dirigir el desarrollo del producto con la visión establecida y de mantenerse informado sobre las necesidades del mercado y de los clientes. Para ello, mediante los comentarios de los usuarios y del equipo de desarrollo, revisa regularmente la lista del Product Backlog con el fin de aclarar cualquier duda que tenga el equipo de desarrollo y de mantener la lista limpia para poder trabajar en ella en cualquier momento. Este proceso asegura que el Product Backlog esté siempre actualizado, priorizado y alineado con los objetivos del producto y las necesidades del cliente, lo que facilita la selección de los elementos más importantes para tener en cuenta en futuros Sprints.
- b. Sprint Planning:** Durante esta reunión, dirigida por el Scrum Master, todo el equipo de desarrollo colabora en la planificación y la estimación del trabajo que se realizará durante el Sprint actual, definiendo los objetivos que se quiere lograr al final de este. Para ello, se seleccionan de manera consensuada los elementos más prioritarios del Product Backlog que ayudan a alcanzar dichos objetivos, de forma que, al finalizar esta reunión de planificación, cada miembro del equipo debe tener claro qué tareas se llevará a cabo en el Sprint y cómo se entregará este incremento del producto.
- c. Sprint:** Es el periodo durante el cual el equipo de Scrum trabaja conjuntamente para completar un incremento del producto según lo establecido en la reunión de planificación. Su duración puede variar entre una y cuatro semanas, siendo dos semanas la más común. Siguiendo la sugerencia de Dave West, CEO y Product Owner de Scrum.org, cuanto más complejo sea el trabajo y más incertidumbre haya, más corto debería ser el Sprint. Sin embargo, esto dependerá siempre de las necesidades del proyecto y de lo que mejor funcione para cada equipo. Durante este periodo, es esencial mantener una comunicación abierta entre todos los miembros del equipo, así como con el Product Owner, para lograr los objetivos establecidos, pudiendo realizar ajustes de tiempo si es necesario. A pesar de esto, es importante mantener la consistencia en la duración del Sprint durante todo el desarrollo del proyecto para que el equipo pueda aprender de las experiencias pasadas y aplicar ese conocimiento en futuros Sprints.
- d. Daily Scrum Meeting:** Es una reunión corta que se lleva a cabo todos los días en el mismo lugar y a la misma hora, generalmente

por las mañanas, con una duración de unos quince minutos, aunque esto puede variar según las necesidades del equipo. Durante esta reunión, cada miembro del equipo comparte su progreso desde la Daily anterior, explica cualquier dificultad que haya encontrado y planifica las tareas que realizará durante el día de hoy. El objetivo principal de esta reunión es mantener a todos los miembros del equipo alineados con el objetivo del Sprint, facilitar la resolución rápida de los obstáculos y asegurar una planificación efectiva para las próximas 24 horas. Esta reunión es fundamental para fomentar el trabajo en equipo, ya que todos los miembros del equipo tienen la oportunidad de compartir información y apoyarse mutuamente para alcanzar los objetivos del Sprint.

- e. **Sprint Review:** Al final del Sprint, el equipo se reúne en una sesión informal para presentar y demostrar el incremento desarrollado durante este periodo, mostrando los elementos marcados como "Terminado" del Sprint Backlog a las partes interesadas y a los compañeros de equipo para recibir comentarios y sugerencias. Tras esto, el Product Owner puede decidir publicar o no publicar el incremento, aunque normalmente sí lo publica. Por último, cabe destacar que esta reunión suele tener una duración máxima de cuatro horas para Sprints de un mes.
- f. **Sprint Retrospective:** Durante este evento, el equipo se reúne para reflexionar sobre el Sprint que acaba de finalizar, documentando y analizando qué aspectos funcionaron bien y cuáles necesitan mejorar. Estos aspectos pueden ser el propio proyecto, las relaciones interpersonales, las herramientas utilizadas o los procesos establecidos. El objetivo final de esta reunión es identificar las oportunidades de mejora para futuros Sprints, centrándose en fortalecer lo positivo y abordar los aspectos que requieren ajustes.

Una vez entendido los tres pilares fundamentales de la metodología Scrum (roles, artefactos y eventos), así como sus interacciones y su importancia en el ciclo de vida del proyecto, se proporciona a continuación una representación visual de todo este proceso integral de Scrum (véase Ilustración 4). En ella, se puede observar que el flujo comienza con el Product Owner creando el Product Backlog. Luego, en la reunión de Sprint Planning, el equipo selecciona los elementos del Product Backlog para trabajar durante el Sprint y los convierte en el Sprint Backlog. Durante este periodo de trabajo, el Scrum Master realizará las reuniones diarias de Daily Scrum para sincronizar a todo el equipo. Finalmente, cuando se acaba el Sprint, se realiza tanto una Sprint Review para demostrar el trabajo completado como una Sprint Retrospective para reflexionar sobre el Sprint y buscar mejoras. Después de esto, se inicia un nuevo ciclo con la creación de un nuevo Sprint Backlog en la siguiente reunión de Sprint Planning, repitiendo este proceso a lo largo del proyecto.

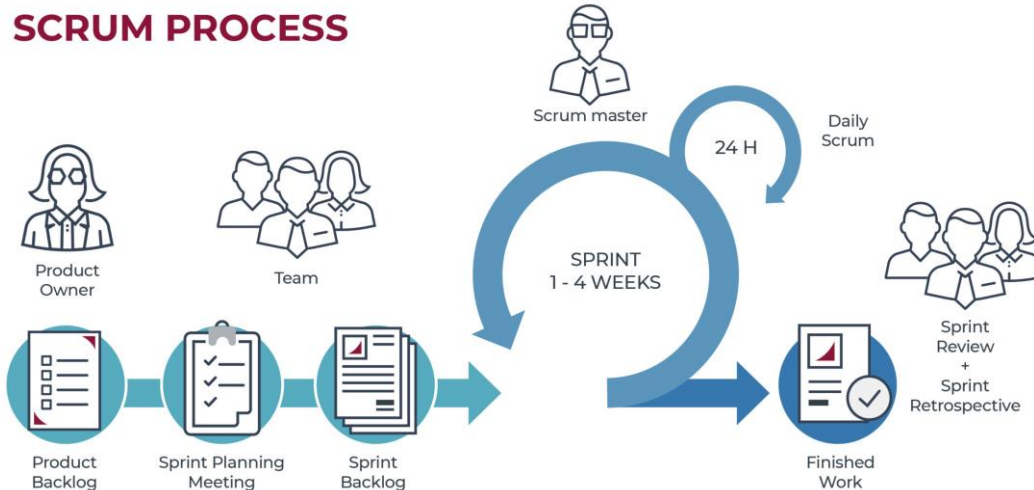


Ilustración 4: Metodología Scrum.

4.6 Cómo se aplicará Scrum en el proyecto

Aunque la aplicación de la metodología Scrum sobre este proyecto individual tendrá lugar en el Capítulo 5, en este apartado se quiere explicar cómo se llevará a cabo esta metodología.

En primer lugar, con respecto a los roles, se va a asumir los tres roles anteriormente descritos, mientras que el rol de cliente lo va a asumir los dueños del restaurante, con los que se tiene una comunicación directa.

En segundo lugar, con respecto a los artefactos, se va a crear un Product Backlog que estará formado por los siguientes tipos de elementos:

- **Epic (Épica):** Representa una unidad grande de trabajo compuesta por diversas funcionalidades o características, conocidas como Features. Las épicas sirven para proporcionar una visión general y de alto nivel del objetivo que se quiere alcanzar con el proyecto.
- **Feature (Característica):** Representa una funcionalidad completa y significativa de una aplicación, que puede descomponerse en varias funcionalidades más específicas, conocidas como historias de usuario.
- **User Story (Historia de usuario):** Representa un requisito o una funcionalidad específica del sistema, escrita desde la perspectiva del usuario final y derivada de una Feature. Además de esto, sigue los principios **INVEST**:
 - **Independiente (Independent):** Cada historia de usuario debe ser completada de forma independiente.
 - **Negociable (Negociable):** Cada historia de usuario debe ser negociable para poder reemplazarla por otra de diferente prioridad.
 - **Valiosa (Valuable):** Cada historia de usuario debe aportar valor al cliente o usuario final.
 - **Estimable (Estimable):** Cada historia de usuario debe ser estimada fácilmente, permitiendo al equipo de desarrollo evaluar el esfuerzo necesario para implementarla.
 - **Pequeña (Small):** Cada historia de usuario debe ser lo suficientemente pequeña para poder ser entregada en un Sprint.

- **Comprobable (Testable):** Cada historia de usuario debe ser probado.

Por otro lado, cada historia de usuario se compone de tres elementos principales:

- **Título:** Declaración de la funcionalidad.
- **Descripción:** Proporciona información adicional de la funcionalidad que se quiere implementar. Normalmente sigue el siguiente formato: **“Como [tipo de usuario], quiero [realizar alguna acción], para [lograr algún objetivo]”**. A continuación, se proporciona una explicación más detallada de cada parte:
 - Como [tipo de usuario]: Describe el rol del usuario para el cual se está desarrollando esta funcionalidad.
 - Quiero [realizar alguna acción]: Describe la acción que el usuario desea llevar a cabo utilizando esta funcionalidad.
 - Para [lograr algún objetivo]: Describe el objetivo que el usuario espera alcanzar después de realizar dicha acción.

Un ejemplo sería el siguiente: “Como usuario registrado, quiero realizar una reserva en el restaurante para asegurarme de que habrá una mesa disponible cuando llegue al restaurante”.

- **Criterios de aceptación:** Son condiciones que deben cumplirse para que una historia de usuario se considere completada. Normalmente sigue el siguiente formato: **“Dado [condiciones iniciales], cuando [acción realizada], entonces [resultado esperado]”**. A continuación, se proporciona una explicación más detallada de cada parte:
 - Dado [condiciones iniciales]: Describe el estado inicial antes de que ocurra la acción.
 - Cuando [acción realizada]: Describe la acción o el evento que se está probando.
 - Entonces [resultado esperado]: Describe el resultado esperado después de que se haya realizado la acción.

Un ejemplo sería el siguiente: “Dado que el cliente necesita reservar una mesa, cuando inicia sesión, entonces debería encontrar un botón que le permita hacer reservas.”

- **Work Item:** Representa cualquier otro elemento de trabajo que debe ser completado como parte del proyecto. Por ejemplo: “Refactorizar”.
- **Bug:** Representa cualquier anomalía, error o problema identificado dentro del proyecto que afecta a su correcto funcionamiento.

En tercer lugar, con respecto a los eventos, se llevará a cabo el proceso de Product Backlog Refinement siempre que se realice algún cambio en el Product Backlog. Además de esto, al principio de cada Sprint, se llevará a cabo el Sprint Planning para crear el Sprint Backlog con el que se trabajará durante dicho Sprint. Además, todos los días, se llevará a cabo una Daily individual para revisar el progreso del trabajo. Por otro lado, al final de cada Sprint, se llevará a cabo el Sprint Review para demostrar el trabajo completado a los clientes, y el Sprint Retrospective para reflexionar sobre lo que funcionó bien, lo que no funcionó tan bien y cómo mejorar en el próximo Sprint. Por último, durante cada Sprint, se llevará a cabo un proceso de diseño, implementación y realización de pruebas según el Sprint Backlog definido. Para implementar la metodología Scrum, se va a utilizar **Jira** [80] como herramienta de gestión y seguimiento de este proyecto, ya que ofrece diversas funcionalidades como la creación de tableros personalizados para visualizar y gestionar el trabajo en Sprints o la gestión del Product Backlog y Sprint Backlog.

5 Desarrollo de la aplicación

5.1 Organización del capítulo

En este capítulo, se abordarán las diferentes etapas que se han llevado a cabo para el desarrollo de la aplicación, desde la concepción inicial de sus diversas funcionalidades hasta su implementación final. Para ello, se comenzará con la creación del Product Backlog, luego se proporcionará un calendario con los Sprints planificados y, finalmente, se profundizará en el desarrollo de cada uno de los Sprints, que incluirá un proceso de planificación, diseño, implementación, pruebas y revisión con el fin de garantizar el correcto funcionamiento de la aplicación.

5.2 Refinamiento del Product Backlog

Para comenzar con el desarrollo de la aplicación, se ha llevado a cabo una primera reunión con los clientes (dueños del restaurante) con el objetivo de establecer el alcance del proyecto, definiendo las funcionalidades claves que formarán parte del Product Backlog. Este último, al ser una lista dinámica, podrá ir cambiando según las necesidades y requerimientos que surjan a medida que avance el desarrollo del proyecto. En este contexto, se emplearán principalmente **User Stories (US)** para representar los requisitos funcionales del sistema, que describen las funcionalidades deseadas desde la perspectiva del usuario. Por otro lado, para representar los requisitos no funcionales, que describen las características del sistema relacionadas con la calidad, la seguridad, el rendimiento y otros aspectos, se hará uso de los criterios de aceptación de los US. Además, para estimar la carga o la complejidad del trabajo de los US se utilizarán los **Story Points (SP)**. Por último, cabe destacar que los US se agruparán dentro de una **Feature (FE)**, y estas, a su vez, se organizarán bajo una **Epic (EP)**. Además de esto, el Product Backlog también puede incluir otros elementos como **Work Items (WI)** y **Bugs (BU)** cuando sea necesario. A continuación, se presenta el Product Backlog con el que se trabajará:

EP1: Desarrollo de una plataforma web integral para optimizar los servicios de un restaurante.	
FE1: Gestión de usuario	Administra la información de los usuarios en la aplicación (US1, US2, US3, US4 y US5).
FE2: Gestión de información del restaurante	Administra la información general de un restaurante (US6 y US7).
FE3: Gestión de reservas de mesas	Administra las reservas de los clientes (US8, US9, US10 y US11)
FE4: Gestión de platos de comida	Administra la variedad de platos existentes en un restaurante (US12, US14, US15 y US16).
FE5: Gestión de pedidos de comida	Administra los pedidos realizados por los clientes (US17, US18, US19, US20, US21 y US22).
FE6: Gestión de reseñas	Administra los comentarios dejados por los clientes (US23, US24, US25, US26, US27 y US28).

Tabla 1: Epic y Features de la aplicación.

US1: Registrar cuenta	
Prioridad	Muy alta
Complejidad	3
Descripción	Como cliente no registrado, quiero poder registrarme en la aplicación para tener una cuenta personal.
Criterios de aceptación	• Dado que el cliente está en la página de inicio de sesión, cuando da al botón "Crear cuenta", entonces el sistema le muestra el formulario de registro. • Dado que se necesita la información básica del cliente, cuando el cliente accede al formulario de registro, entonces debe aparecer los campos marcados como obligatorios. • Dado que el cliente puede cometer errores durante el registro, cuando rellena con datos incorrectos, entonces el sistema le muestra un mensaje de error. • Dado que el cliente introduce un correo electrónico y un número de teléfono, cuando es un correo y número ya existentes, entonces el sistema le muestra un mensaje de que ya está en uso. • Dado que el cliente completa el formulario de registro, cuando crea la cuenta, entonces el sistema debe mostrar un mensaje de confirmación. • Dado que el cliente registra correctamente, cuando inicia sesión por primera vez, entonces el sistema le permite acceder a su cuenta utilizando la información proporcionada en el registro. • Dado que la velocidad de respuesta es importante para que el cliente tenga una buena experiencia, cuando interactúa con el formulario de registro, entonces el sistema debe procesar rápido.

Tabla 2: US1 - Registrar cuenta.

US2: Iniciar sesión	
Prioridad	Alta
Complejidad	5
Descripción	Como cliente registrado o administrador, quiero iniciar sesión en mi cuenta para utilizar las diferentes funcionalidades de la aplicación.
Criterios de aceptación	• Dado que el usuario (cliente o administrador) va a acceder a su cuenta, cuando visita la página principal, entonces debería encontrar el botón que dirige al formulario de inicio de sesión. • Dado que se requiere autenticación, cuando el usuario (cliente o administrador) entra al formulario de inicio de sesión, entonces debe poder ingresar su correo electrónico y su contraseña.

	• Dado que la seguridad es importante, cuando el usuario (cliente o administrador) ingresa su información, entonces el sistema debe validar si sus datos son correctos y permitirle el acceso. • Dado que el usuario (cliente o administrador) puede cometer errores al ingresar la información, cuando intenta iniciar sesión con datos incorrectos, entonces el sistema debe mostrar un mensaje de error. • Dado que el usuario (cliente o administrador) va a iniciar sesión, cuando termina de completar correctamente el formulario, entonces el sistema debe redirigirlo a la página principal y debe mostrarle un mensaje de que se ha iniciado sesión con éxito. • Dado que la seguridad de la cuenta es crucial, cuando el usuario inicia sesión, entonces el sistema debe asegurar de que la sesión se maneje de forma segura utilizando técnicas como el cifrado de contraseñas.
--	--

Tabla 3: US2 – Iniciar sesión.

US3: Cerrar sesión	
Prioridad	Alta
Complejidad	2
Descripción	Como cliente registrado o administrador, quiero cerrar sesión para que mi cuenta esté segura cuando termino de usar la aplicación.
Criterios de aceptación	• Dado que el usuario (cliente o administrador) necesita cerrar sesión, cuando está navegando por la aplicación, entonces debería encontrar un botón de cierre de sesión en el perfil del usuario. • Dado que el usuario (cliente o administrador) va a cerrar sesión, cuando pulsa el botón de cerrar sesión, entonces el sistema debe redirigir al usuario a la página de inicio de sesión.

Tabla 4: US3 – Cerrar sesión.

US4: Editar perfil de usuario	
Prioridad	Media
Complejidad	3
Descripción	Como cliente registrado o administrador, quiero modificar mi perfil de usuario para mantener mis datos actualizados.
Criterios de aceptación	• Dado que el usuario (cliente o administrador) quiere modificar su información, cuando accede a su perfil, entonces debería poder editar sus datos personales. • Dado que el usuario (cliente o administrador) puede cometer errores, cuando realiza cambios en su perfil,

	entonces el sistema debe validar que los datos modificados cumplan con los requisitos. • Dado que el usuario (cliente o administrador) edita su perfil de usuario, cuando termina de realizar los cambios en su perfil, entonces debería recibir una confirmación de que los datos han sido guardado con éxito. • Dado que el usuario (cliente o administrador) necesita una confirmación visual, cuando guarda los cambios realizados, entonces estos datos actualizados deben reflejarse en su perfil.
--	--

Tabla 5: US4 – Editar perfil de usuario.

US5: Cambiar de contraseña	
Prioridad	Media
Complejidad	3
Descripción	Como cliente registrado o administrador, quiero cambiar mi contraseña para mantener la seguridad de mi cuenta.
Criterios de aceptación	• Dado que el usuario (cliente o administrador) quiere cambiar su contraseña, cuando accede al formulario de cambio de contraseña en su perfil, entonces debería poder proporcionar su contraseña actual e introducir una nueva contraseña. • Dado que el usuario (cliente o administrador) se encuentra en el formulario de cambio de contraseña, cuando ingresa su nueva contraseña, entonces el sistema debe validar que la contraseña cumpla con los requisitos de seguridad. • Dado que el usuario (cliente o administrador) quiere ver si se ha guardado la nueva contraseña, cuando termina de cambiar correctamente la contraseña, entonces el sistema debe mostrar un mensaje de que se ha realizado el cambio de contraseña con éxito. • Dado que el usuario (cliente o administrador) ya ha cambiado su contraseña, cuando inicia sesión, entonces el sistema debe permitir al usuario ingresar a la aplicación con su nueva contraseña.

Tabla 6: US5 – Cambiar de contraseña.

US6: Actualizar información del restaurante	
Prioridad	Alta
Complejidad	3
Descripción	Como administrador, quiero actualizar la información del restaurante para mantenerla actualizada.
Criterios de aceptación	• Dado que es necesario mantener la información del restaurante actualizada, cuando el administrador inicia

	sesión en la aplicación web, entonces debería poder acceder al botón de “Restaurante”. • Dado que el administrador necesita actualizar los detalles del restaurante, cuando accede al formulario de edición, entonces debería poder modificar los campos que quiere. • Dado que es importante asegurar el formato de la información, cuando el administrador edita la información del restaurante, entonces el sistema debe validar los datos ingresados. • Dado que los cambios deben ser reflejados, cuando el administrador guarda las modificaciones, entonces el sistema debe actualizar la información en la base de datos, mostrar un mensaje de que sea guardado con éxito y actualizarlo en la página inicial.
--	--

Tabla 7: US6 – Actualizar información del restaurante.

US7: Ver información del restaurante	
Prioridad	Alta
Complejidad	2
Descripción	Como cliente registrado o no registrado, quiero ver la información actualizada del restaurante para tomar decisiones informadas.
Criterios de aceptación	• Dado que el cliente necesita información para tomar decisiones informadas, cuando accede a la página inicial, entonces debería encontrar una sección dedicada para ver los datos actualizados del restaurante. • Dado que la velocidad es importante para que el cliente tenga una buena experiencia, cuando accede a la página inicial, entonces la información del restaurante debe procesar con rapidez.

Tabla 8: US7 – Ver información del restaurante.

US8: Crear reservas de mesa	
Prioridad	Muy alta
Complejidad	5
Descripción	Como cliente registrado, quiero realizar reservas en el restaurante para asegurarme de que habrá una mesa disponible cuando llegue al restaurante.
Criterios de aceptación	• Dado que el cliente necesita reservar una mesa, cuando inicia sesión en la aplicación, entonces debería encontrar un botón que le permita hacer reservas. • Dado que el cliente quiere realizar una reserva, cuando accede al formulario de reservas, entonces debería

	poder seleccionar la fecha, la cantidad de personas y la hora para la reserva. • Dado que el cliente necesita una confirmación, cuando completa el formulario de reserva, entonces el sistema debe mostrar un mensaje de que se ha reservado con éxito.
--	--

Tabla 9: US8 – Crear reservas de mesa.

US9: Ver reservas de mesa (cliente)	
Prioridad	Alta
Complejidad	2
Descripción	Como cliente registrado, quiero ver mis reservas para saber el estado de confirmación de la reserva y recordarme de mis visitas.
Criterios de aceptación	• Dado que el cliente necesita acceder a sus reservas, cuando inicia sesión en la aplicación, entonces debería encontrar una sección con su lista de reservas. • Dado que el cliente quiere ver la información de la reserva, cuando accede a la lista de reservas, entonces debería poder ver detalles como la fecha, la hora, el número de personas y el estado de confirmación. • Dado que el cliente quiere consultar sus reservas pasadas, cuando accede a su lista de reservas, entonces debería poder ver todas las reservas anteriores.

Tabla 10: US9 – Ver reservas de mesa (cliente).

US10: Ver reservas de mesa (administrador)	
Prioridad	Alta
Complejidad	2
Descripción	Como administrador, quiero ver todas las reservas realizadas por mis clientes para gestionar la capacidad del restaurante.
Criterios de aceptación	• Dado que el administrador necesita gestionar las reservas, cuando accede a la página principal, entonces debería encontrar la sección dedicada a las reservas. • Dado que el administrador necesita ver todas las reservas, cuando accede a la sección de reservas, entonces debería poder ver una lista de todas las reservas realizadas por los clientes y sus detalles.

Tabla 11: US10 – Ver reservas de mesa (administrador).

US11: Responder a las reservas de mesa de mis clientes	
Prioridad	Media
Complejidad	2
Descripción	Como administrador, quiero responder a las reservas de mesa de mis clientes, ya sea confirmando o denegando las solicitudes, para comunicar la disponibilidad o falta de disponibilidad de las mesas en el restaurante.
Criterios de aceptación	• Dado que el administrador tiene que revisar las reservas, cuando accede al panel de reservas, entonces debería poder administrar las solicitudes. • Dado que el administrador necesita responder a las reservas, cuando revisa las solicitudes y la disponibilidad que hay en el restaurante, entonces debería poder confirmar o rechazar las solicitudes. • Dado que el cliente necesita una respuesta, cuando el administrador confirma o rechaza una reserva, entonces el sistema debe actualizar el panel de reservas y enviar un mensaje a través del correo al cliente con la respuesta correspondiente. • Dado que los clientes esperan una respuesta rápida, cuando el administrador confirma o rechaza una solicitud de reserva, entonces el sistema debería actualizar el panel de reservas y enviar el correo al cliente de manera rápida.

Tabla 12: US11 – Responder a las reservas de mesa de mis clientes.

US12: Crear platos de comida	
Prioridad	Muy alta
Complejidad	5
Descripción	Como administrador, quiero añadir nuevos platos al menú para ofrecer una gran variedad de opciones a los clientes y permitirles explorar nuevas experiencias culinarias.
Criterios de aceptación	• Dado que el administrador necesita añadir nuevos platos, cuando accede al panel de menú, entonces debería encontrar un botón para añadir más platos. • Dado que el administrador necesita rellenar los detalles del plato, cuando selecciona añadir un nuevo plato, entonces debería tener un formulario donde pueda ingresar los datos del plato. • Dado que la información debe ser correcta, cuando el administrador rellena los datos del nuevo plato, entonces el sistema debe validar los datos ingresados para asegurar que sean adecuados y completos. • Dado que se añaden nuevos platos al menú, cuando el administrador completa el proceso de añadir un plato, entonces el sistema debe actualizar el menú para que los clientes lo vean.

Tabla 13: US12 – Crear platos de comida.

US13: Ver platos de comida (administrador)	
Prioridad	Alta
Complejidad	2
Descripción	Como administrador, quiero ver la lista completa de platos disponibles en el menú para realizar ajustes cuando sea necesario.
Criterios de aceptación	Dado que el administrador necesita ver la lista de platos disponibles, cuando accede a la sección de menú, entonces debería encontrar una lista completa de platos y su información.

Tabla 14: US13 – Ver platos de comida (administrados).

US14: Actualizar platos de comida	
Prioridad	Media
Complejidad	3
Descripción	Como administrador, quiero actualizar los detalles de un plato existente en el menú para corregir errores o realizar cambios.
Criterios de aceptación	- Dado que el administrador necesita actualizar los datos de un plato, cuando accede a la sección de menú, entonces debería de tener una opción para editar la información de los platos. - Dado que es importante revisar los cambios, cuando el administrador actualiza los detalles de un plato, entonces el sistema debe validar todos los datos ingresados. - Dado que se realizan actualizaciones, cuando el administrador guarda los cambios, entonces el sistema debe mostrar un mensaje de que los cambios han sido guardados correctamente y actualizar el menú para que los clientes lo vean.

Tabla 15: US14 – Actualizar platos de comida.

US15: Eliminar platos de comida	
Prioridad	Media
Complejidad	3
Descripción	Como administrador, quiero tener la capacidad de eliminar los platos del menú para retirar los platos menos populares entre los clientes.
Criterios de aceptación	Dado que el administrador necesita gestionar el menú, cuando accede al panel de administración del menú, entonces debería encontrar un botón para eliminar platos existentes.

	• Dado que el administrador puede eliminar como error un plato, cuando presiona el botón de borrar, entonces debe aparecer un mensaje de confirmación. • Dado que el administrador necesita ver una confirmación, cuando elimina un plato, entonces el sistema debe mostrar un mensaje de que se ha eliminado correctamente y actualizar el menú para que los clientes lo vean.
--	--

Tabla 16: US15 – Eliminar platos de comida.

US16: Ver platos de comida (cliente)	
Prioridad	Alta
Complejidad	2
Descripción	Como cliente registrado, quiero explorar los platos de comida disponibles en el menú del restaurante para conocer las opciones que ofrece el restaurante y tomar decisiones informadas sobre mi pedido.
Criterios de aceptación	• Dado que el cliente necesita explorar las opciones de comida, cuando accede a la aplicación, entonces debería encontrar una sección al menú del restaurante. • Dado que el cliente necesita los detalles de los platos, cuando navega por el menú, entonces debería poder ver toda la información de los platos y su imagen. • Dado que el cliente quiere tener una experiencia positiva, cuando navega en la sección de menú del restaurante, entonces el sistema debe cargar con fluidez todas las imágenes y los detalles de los platos.

Tabla 17: US16 – Ver platos de comida (cliente).

US17: Agregar platos de comida al carrito	
Prioridad	Muy alta
Complejidad	5
Descripción	Como cliente registrado, quiero agregar platos del menú al carrito para realizar un pedido en línea.
Criterios de aceptación	• Dado que el cliente necesita seleccionar los platos, cuando visualiza un plato en el menú, entonces debe haber un botón para agregar ese plato al carrito. • Dado que el cliente necesita una confirmación, cuando agrega un plato al carrito, entonces el sistema debe mostrar un mensaje de que el plato ha sido añadido con éxito. • Dado que el cliente puede navegar por diferentes partes de la aplicación, cuando agrega platos al carrito, entonces el carrito debe conservar los platos seleccionados incluso si navega en otras páginas.

Tabla 18: US17 – Agregar platos de comida al carrito.

US18: Ajustar la cantidad de un plato de mi carrito	
Prioridad	Media
Complejidad	3
Descripción	Como cliente registrado, quiero ajustar (aumentar o disminuir) la cantidad de un plato de mi carrito o eliminar un plato de mi carrito para reflejar con precisión mi pedido deseado.
Criterios de aceptación	• Dado que el cliente necesita modificar su pedido, cuando visualiza su carrito, entonces debería haber opciones para ajustar la cantidad de cada plato o eliminar un plato. • Dado que el cliente necesita ver el precio actualizado, cuando ajusta la cantidad de un plato, entonces el sistema debe actualizar el total del pedido reflejando los cambios. • Dado que es importante que el cliente tenga una buena experiencia de usuario, cuando ajusta la cantidad de un plato en el carrito, entonces la interfaz debe ser intuitiva.

Tabla 19: US18 – Ajustar la cantidad de un plato de mi carrito.

US19: Realizar pedidos de comida	
Prioridad	Alta
Complejidad	3
Descripción	Como cliente registrado, quiero finalizar mi pedido y proporcionar detalles de entrega para recibir la comida en una ubicación deseada.
Criterios de aceptación	• Dado que el cliente necesita completar su pedido, cuando finaliza su pedido, entonces debería haber un botón que le permita finalizar el pedido. • Dado que es necesario especificar la dirección de la entrega, cuando el cliente está en la página de finalización de pedido, entonces debería haber un formulario donde pueda elegir entre comida a domicilio o recoger en el restaurante. • Dado que el cliente quiere pedir comida a domicilio, cuando elige la opción de comida a domicilio, entonces debería poder ingresar la dirección de entrega. • Dado que el cliente quiere una confirmación de su pedido, cuando finaliza el proceso de pedido, entonces el sistema debe mostrar un mensaje de que el pedido se ha realizado con éxito y también le notifica por correo. • Dado que la seguridad y la privacidad son importantes, cuando el cliente proporciona datos de la entrega, entonces el sistema debe garantizar que toda la información personal está guardada de forma segura.

Tabla 20: US19 – Realizar pedidos de comida.

US20: Ver pedidos de comida (cliente)	
Prioridad	Alta
Complejidad	2
Descripción	Como cliente registrado, quiero ver una lista de todos mis pedidos anteriores para hacer un seguimiento de mis compras pasadas.
Criterios de aceptación	• Dado que el cliente quiere revisar sus compras anteriores, cuando accede a la página principal, entonces debería haber un apartado de pedidos, donde se muestra los pedidos realizados anteriormente. • Dado que el cliente necesita los detalles de sus pedidos pasados, cuando accede a la sección de pedidos, entonces debería poder ver los detalles de todos los pedidos, incluyendo fechas, platos pedidos, cantidad de platos y precio total.

Tabla 21: US20 – Ver pedidos de comida (cliente).

US21: Ver pedidos de comida (administrador)	
Prioridad	Alta
Complejidad	2
Descripción	Como administrador, quiero ver una lista de todos los pedidos realizados por mis clientes para gestionar la operación del restaurante.
Criterios de aceptación	• Dado que el administrador necesita gestionar los pedidos, cuando accede a la página principal, entonces debe haber una opción para ver la lista completa de todos los pedidos realizados por los clientes. • Dado que el administrador necesita revisar los detalles de los pedidos pasados, cuando accede a la sección de pedidos, entonces debería poder ver los detalles de todos los pedidos. • Dado que la información de los pedidos es confidencial, cuando el administrador accede a la lista de pedidos, entonces el sistema debe garantizar que solo los usuarios con privilegios administrativos pueden gestionar esta información.

Tabla 22: US21 – Ver pedidos de comida (administrador).

US22: Cambiar estado del pedido	
Prioridad	Media
Complejidad	2
Descripción	Como administrador, quiero cambiar el estado de un pedido para mantener a los clientes informados sobre el progreso de su pedido.

Criterios de aceptación	• Dado que el administrador necesita controlar el progreso de los pedidos, cuando pulsa botón de cambiar el estado de un pedido, entonces debería poder modificar su estado como “en proceso”, “en camino” o “entregado”. • Dado que el cliente necesita una respuesta, cuando el administrador modifica el estado de los pedidos, entonces el sistema debe actualizar el panel de pedidos y enviar al cliente un correo electrónico con la respuesta correspondiente.
--------------------------------	---

Tabla 23: US22 – Cambiar estado del pedido.

US23: Dejar reseñas	
Prioridad	Muy alta
Complejidad	5
Descripción	Como cliente registrado, quiero dejar reseñas para compartir mi experiencia en el restaurante y mis opiniones sobre los platos que he probado.
Criterios de aceptación	• Dado que el cliente quiere compartir su experiencia, cuando inicia sesión en su cuenta, entonces debe haber un apartado de reseñas. • Dado que el cliente quiere dejar una reseña, cuando está en la sección de sus reseñas, entonces debe haber un botón que le permita escribir sus opiniones sobre el restaurante o sobre los platos. • Dado que el cliente quiere escribir una valoración, cuando está escribiendo la reseña, entonces debería poder calificar el restaurante o el plato y subir una imagen. • Dado que el cliente necesita una confirmación, cuando finaliza con la reseña, entonces el sistema debe mostrar un mensaje de que la reseña se ha guardado con éxito.

Tabla 24: US23 – Dejar reseña.

US24: Modificar reseñas	
Prioridad	Media
Complejidad	3
Descripción	Como cliente registrado, quiero editar las reseñas que he dejado previamente para actualizar mis opiniones pasadas.
Criterios de aceptación	• Dado que el cliente quiere editar las reseñas, cuando está en el apartado de sus reseñas, entonces debe haber un botón que le permita editar la reseña anterior. • Dado que es importante mantener las reseñas actualizadas, cuando el cliente termina de editar una reseña, entonces el sistema debe modificar la reseña anterior a la nueva, reflejarlo en la sección de reseñas

	y mostrar un mensaje de que se ha actualizado con éxito.
--	--

Tabla 25: US24 – Modificar reseñas.

US25: Eliminar reseñas (cliente)	
Prioridad	Media
Complejidad	3
Descripción	Como cliente registrado, quiero eliminar las reseñas que he dejado previamente para retirar mis opiniones pasadas.
Criterios de aceptación	• Dado que el cliente quiere eliminar sus comentarios, cuando está en el apartado de sus reseñas, entonces debe haber un botón que le permita eliminar reseñas. • Dado que el cliente necesita saber si la reseña ha sido eliminada correctamente, cuando pulsa el botón de borrar, entonces el sistema debe mostrar un mensaje de que la reseña se ha eliminado con éxito y actualizar la página con los cambios realizados.

Tabla 26: US25 – Eliminar reseñas.

US26: Ver reseñas (cliente)	
Prioridad	Alta
Complejidad	2
Descripción	Como cliente registrado, quiero ver las reseñas dejadas por otros clientes para tomar decisiones informadas sobre qué platos pedir.
Criterios de aceptación	• Dado que el cliente quiere tomar decisiones informadas sobre qué platos pedir, cuando navega por la página principal del restaurante, entonces debe haber una sección que le permita ver las reseñas dejadas por otros clientes. • Dado que es importante que el cliente tenga una buena experiencia, cuando accede a la sección de reseñas, entonces el sistema debe cargar las reseñas con rapidez.

Tabla 27: US26 – Ver reseñas (cliente).

US27: Ver reseñas (administrador)	
Prioridad	Alta
Complejidad	2
Descripción	Como administrador, quiero ver todas las reseñas dejadas por los clientes para comprender las experiencias de los clientes e identificar áreas de mejora en el restaurante.

Criterios de aceptación	Dado que el administrador quiere ver las reseñas dejadas por los clientes, cuando navega por la sección de reseñas, entonces debe poder ver una lista de reseñas con el nombre del cliente y el contenido.
--------------------------------	--

Tabla 28: US27 – Ver reseñas (administrador).

US28: Eliminar reseñas (administrador)	
Prioridad	Media
Complejidad	3
Descripción	Como administrador, quiero eliminar las reseñas dejadas por los clientes para asegurarme de que no sea visible un contenido inapropiado o irrespetuoso.
Criterios de aceptación	- Dado que el administrador quiere eliminar los comentarios inapropiados o irrespetuosos de los clientes, cuando se encuentra en el apartado de reseñas, entonces debe haber un botón que le permita eliminar reseñas. - Dado que el administrador necesita saber si la reseña ha sido eliminada, cuando pulsa el botón de borrar, entonces el sistema debe mostrar un mensaje de que la reseña se ha eliminado con éxito y actualizar la página con los cambios realizados.

Tabla 29: US28 – Eliminar reseñas (administrador).

Por último, cabe señalar que el Product Backlog ha sido registrado en Jira para facilitar la gestión del proyecto, permitiendo un seguimiento detallado de las Features y User Stories mencionadas anteriormente (véase Ilustración 5).

Issue	Summary	Category	Priority	Status	Progress
TFG-7	Registrar cuenta	GESTIÓN DE USUARIO	TO DO	3	100%
TFG-8	Iniciar sesión	GESTIÓN DE USUARIO	TO DO	5	100%
TFG-9	Cerrar sesión	GESTIÓN DE USUARIO	TO DO	2	100%
TFG-10	Editar perfil de usuario	GESTIÓN DE USUARIO	TO DO	3	100%
TFG-11	Cambiar de contraseña	GESTIÓN DE USUARIO	TO DO	3	100%
TFG-12	Actualizar información del restaurante	GESTIÓN DE INFORMACIÓN	TO DO	3	100%
TFG-13	Ver información del restaurante	GESTIÓN DE INFORMACIÓN	TO DO	2	100%
TFG-14	Crear reservas de mesa	GESTIÓN DE RESERVAS	TO DO	5	100%
TFG-15	Ver reservas de mesa (cliente)	GESTIÓN DE RESERVAS	TO DO	2	100%
TFG-16	Ver reservas de mesa (administrador)	GESTIÓN DE RESERVAS	TO DO	2	100%
TFG-17	Responder a las reservas de mesa	GESTIÓN DE RESERVAS	TO DO	2	100%
TFG-18	Crear platos de comida	GESTIÓN DE PLATOS D.	TO DO	5	100%
TFG-19	Ver platos de comida (administrador)	GESTIÓN DE PLATOS D.	TO DO	2	100%
TFG-20	Actualizar platos de comida	GESTIÓN DE PLATOS D.	TO DO	3	100%
TFG-21	Eliminar platos de comida	GESTIÓN DE PLATOS D.	TO DO	3	100%
TFG-22	Ver platos de comida (cliente)	GESTIÓN DE PLATOS D.	TO DO	2	100%
TFG-23	Agregar platos de comida al carrito	GESTIÓN DE PEDIDOS	TO DO	5	100%
TFG-24	Ajustar la comida de un plato en el carrito	GESTIÓN DE PEDIDOS	TO DO	3	100%
TFG-25	Realizar pedidos de comida	GESTIÓN DE PEDIDOS	TO DO	3	100%
TFG-26	Ver pedidos de comida (cliente)	GESTIÓN DE PEDIDOS	TO DO	2	100%
TFG-27	Ver pedidos de comida (administrador)	GESTIÓN DE PEDIDOS	TO DO	2	100%
TFG-28	Cambiar estado del pedido	GESTIÓN DE PEDIDOS	TO DO	2	100%
TFG-29	Dejar reseñas	GESTIÓN DE RESEÑAS	TO DO	5	100%
TFG-30	Modificar reseñas	GESTIÓN DE RESEÑAS	TO DO	3	100%
TFG-31	Eliminar reseñas (cliente)	GESTIÓN DE RESEÑAS	TO DO	3	100%
TFG-32	Ver reseñas (cliente)	GESTIÓN DE RESEÑAS	TO DO	2	100%
TFG-33	Ver reseñas (administrador)	GESTIÓN DE RESEÑAS	TO DO	2	100%
TFG-34	Eliminar reseñas (administrador)	GESTIÓN DE RESEÑAS	TO DO	3	100%

Ilustración 5: Product Backlog.

5.3 Arquitectura del software

Antes de comenzar con la arquitectura del software, se explicará primero los componentes que se van a utilizar tanto de Angular como de Spring Boot para este proyecto.

- **Angular**

- Guard (Guardián): Servicio que se utiliza para controlar el acceso a las rutas en una aplicación Angular. Determina si el usuario está autenticado antes de permitir el acceso a una ruta protegida. Por lo tanto, si un cliente intenta acceder a una ruta reservada para un administrador, entonces el guard no le permitirá el acceso, ya que no tiene los permisos necesarios. En caso contrario pasaría lo mismo.
- Template (Plantilla): Archivo HTML que define la estructura de la interfaz de usuario de un componente en una aplicación Angular. Vincula los datos y eventos a la vista y crea vistas dinámicas e interactivas. Para ello, utiliza algunas sintaxis especiales de Angular como la interpolación, las directivas, y los enlaces de eventos y propiedades.
- Component (Componente): Clase TypeScript que define la lógica y el comportamiento de una parte específica de la interfaz de usuario. Además, está compuesto por un archivo HTML, que define el template y un archivo CSS, que define los estilos asociados al componente.
- Service (Servicio): Clase TypeScript que se usa para compartir datos, lógica de la aplicación y funcionalidades entre diferentes componentes. Se utilizan principalmente para realizar llamadas a APIs y se inyectan en los componentes.
- Interceptor (Interceptor): Tipo de middleware que se utiliza para interceptar y modificar las solicitudes HTTP y las respuestas en una aplicación Angular.

- **Spring Boot**

- Controller (Controlador): Maneja las solicitudes HTTP entrantes y devuelve las respuestas HTTP. Se encarga de recibir las peticiones del cliente, los procesa y devuelve una respuesta adecuada.
- Data Transfer Object (DTO): Sirve para transferir datos entre diferentes capas de la aplicación.
- Service (Servicio): Procesa las solicitudes recibidas desde los controladores, interactúa con los repositorios y realiza una operación de negocio.
- Repository (Repositorio): Accede a la base de datos y realiza operaciones de Crear, Leer, Actualizar y Eliminar (CRUD, Create, Read, Update y Delete). Utiliza JPA para interactuar con la base de datos a través de las entidades definidas.
- Entity (Entidad): Mapea objetos Java a tablas en la base de datos, de manera que cada entidad tiene una serie de campos que corresponden con las columnas de la tabla.
- Security (Seguridad): Para la seguridad del servidor, se utiliza principalmente Spring Security, que sirve para configurar la autenticación y autorización de la aplicación, la gestión de los diferentes usuarios y sus roles, y la protección de las APIs de la aplicación.

Por último, se utilizará JSON Web Token (JWT) para establecer la autenticación y autorización en la aplicación, permitiendo a los usuarios acceder a distintas secciones de la aplicación dependiendo de su token JWT.

Una vez explicado los diferentes componentes que se va a emplear en este proyecto, con el fin de garantizar la solidez de la aplicación, es esencial establecer primero la arquitectura del software, ya que se trata de una estructura formal que define los componentes de un sistema de software, sus relaciones, cómo interactúan entre sí y cómo se organizan. De esta manera, la arquitectura puede proporcionar una visión global del sistema, lo que permite comprender mejor cómo funciona internamente una aplicación. Además, una arquitectura bien definida mejora la eficiencia del desarrollo, optimiza el rendimiento del software y facilita el mantenimiento y la actualización del mismo, ya que, al tener una arquitectura clara, es más sencillo detectar y arreglar los posibles errores que van surgiendo, reduciendo así los costes a largo plazo [81].

En la actualidad, a pesar de los numerosos patrones y principios de arquitectura de software, se ha optado finalmente por la arquitectura de Modelo-Vista-Controlador (MVC, Model-View-Controller) para la realización de este proyecto (véase Ilustración 6). Se trata de un patrón de arquitectura muy utilizado en las aplicaciones web, el cual separa la aplicación en tres componentes: el modelo, la vista y el controlador. Cada uno de estos componentes tienen unas funciones específicas, por lo que facilita la comprensión y el mantenimiento del código, además de que permite actualizar uno de los componentes sin afectar a los otros. También favorece el escalado de la aplicación, dado que las responsabilidades están bien definidas. Para comprender mejor el funcionamiento de MVC se explicará a continuación sus tres componentes [82]:

- **Modelo:** Es la capa donde se gestiona los datos, las reglas de negocio y la lógica de la aplicación, por tanto, interactúa principalmente con la base de datos e informa a la vista sobre los cambios en los datos. En este proyecto, el modelo estará compuesto por los siguientes componentes de Spring Boot: servicio, repositorio, entidad y DTO; junto a la base de datos PostgreSQL con el fin de realizar las diferentes operaciones CRUD.
- **Vista:** Es el responsable de la visualización de la interfaz de usuario. La vista muestra los datos provenientes del modelo en forma de tablas, formularios, gráficos, etc. Además de esto, recibe las interacciones del usuario y las transmite al controlador para su posterior procesamiento. En este proyecto, se utilizará principalmente las plantillas HTML y los estilos CSS. Cabe destacar que no se considerará el componente TypeScript de Angular como vista, dado que maneja cierta lógica de la aplicación.
- **Controlador:** Actúa como intermediario entre el modelo y la vista, manejando las solicitudes de entrada del usuario y notificando al modelo las operaciones que debe realizar y a la vista sobre qué debe mostrar. En este proyecto, se hará uso de los servicios y componentes TypeScript de Angular, y los controladores de Spring Boot.

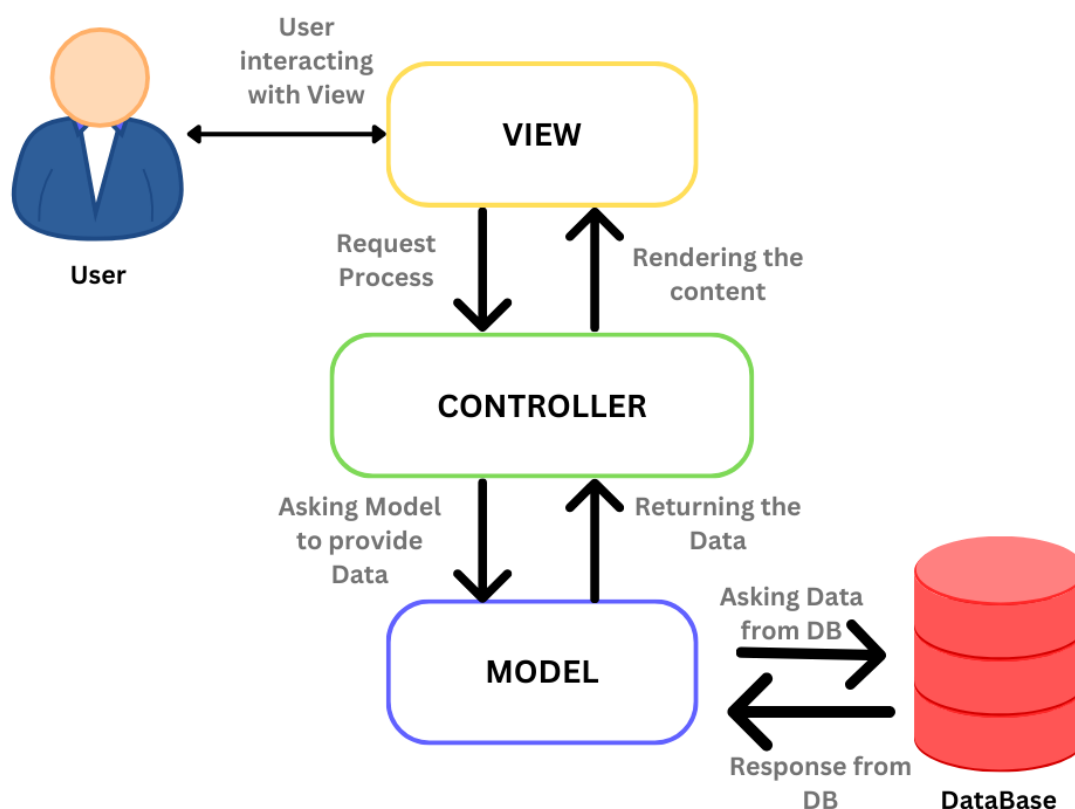


Ilustración 6: MVC

Para comprender el flujo de la aplicación desde que un usuario realiza una petición hasta que recibe la respuesta, se ha desarrollado el siguiente diagrama de la arquitectura de la aplicación (véase Ilustración 7). Este diagrama destaca tanto los componentes del Front-End como los del Back-End, y muestra las diversas interacciones entre ellos. El flujo de la aplicación se desarrolla de la siguiente manera:

1. En primer lugar, antes de que el usuario acceda completamente a la aplicación, Angular invoca el guard asociado a esa ruta con el fin de verificar si el usuario está autenticado y posee los permisos necesarios (roles) para acceder a la ruta solicitada. Si el usuario no está autorizado, el guard redirige al usuario a una página de inicio de sesión o muestra un mensaje de error. En caso contrario, si el usuario está autorizado, el guard le permitirá acceder a la ruta solicitada. Este paso es crucial para asegurar que solo los usuarios autenticados y autorizados puedan acceder a ciertas partes de la aplicación, protegiendo así la información y funcionalidad sensible.
2. En segundo lugar, tras permitir el acceso al usuario, Angular cargará el componente correspondiente a esa ruta, mostrando su template asociado. Además, el componente podrá manejar y mostrar los diferentes errores de formulario.
3. En tercer lugar, cuando el componente necesita presentar los datos del Back-End, como una lista de reservas, o cuando el usuario realiza alguna acción que requiera la comunicación con el Back-End, por ejemplo, al pulsar un botón, entonces el componente llamará al servicio

correspondiente, con el fin de realizar llamadas HTTP al Back-End. Si la solicitud requiere autenticación y autorización, el Interceptor de Angular agregará el token JWT al encabezado de la solicitud utilizando el estándar de seguridad "Bearer token". Tras esto, se envía la solicitud HTTP al servidor Back-End.

4. Por último, el controlador de Spring Boot manejará la solicitud HTTP, validará cualquier parámetro y autenticará al usuario si es necesario. Luego delegará la lógica de negocio al servicio correspondiente, que a su vez puede interactuar con un repositorio para acceder a la base de datos, con el fin de realizar las operaciones CRUD necesarias y devolver los resultados al servicio. Tras esto, el servicio transforma los datos en un DTO antes de devolverlos al controlador que le había llamado en un principio. Y finalmente, el controlador construye la respuesta HTTP correspondiente y la envía de vuelta al servicio de Angular, pasando por el interceptor, que puede modificar la respuesta entrante, hasta llegar al componente que le había llamado en un primer momento. Tras saber la respuesta, actualizará la vista con los nuevos datos, utilizando de nuevo el template para reflejar cualquier cambio en la interfaz de usuario.

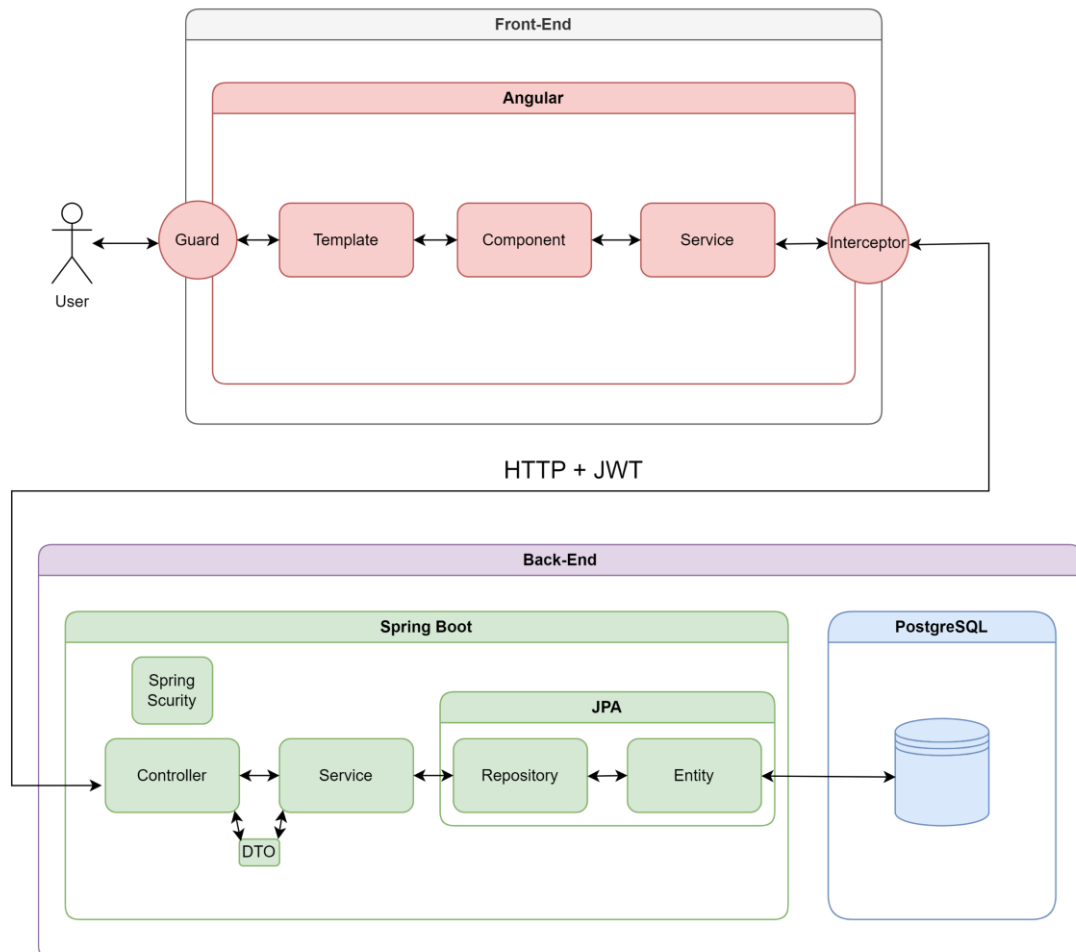


Ilustración 7: Diagrama de la arquitectura de la aplicación.

5.4 Planificación de Sprints

Una vez definido el Product Backlog, se ha creado en Jira un total de 6 Sprints para llevar a cabo el desarrollo de la aplicación (véase Ilustración 8). Es importante resaltar que cada Sprint tendrá una duración de una semana, con una dedicación de aproximadamente 40 horas de trabajo.

	APR							MAY					MAY					MAY																	
	22	23	24	25	26	27	28	29	30	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26
Sprints	TFG Sprint 1							TFG Sprint 2							TFG Sprint 3					TFG Sprint 4					TFG Sprint 5										
Releases																																			
▼ <u>TFG-1 Gestión de Usuario</u>																																			
TFG-7 Registrar cuenta	TO DO																																		
TFG-8 Iniciar sesión	TO DO																																		
TFG-9 Cerrar sesión	TO DO																																		
TFG-10 Editar perfil de usuario	TO DO																																		
TFG-11 Cambiar de contraseña	TO DO																																		
▼ <u>TFG-2 Gestión de Información del Restaurante</u>																																			
TFG-12 Actualizar información del restau...	TO DO																																		
TFG-13 Ver información del restaurante	TO DO																																		
▼ <u>TFG-3 Gestión de Reservas de Mesas</u>																																			
TFG-14 Crear reservas de mesa	TO DO																																		
TFG-15 Ver reservas de mesa (cliente)	TO DO																																		
TFG-16 Ver reservas de mesa (administr...	TO DO																																		
TFG-17 Responder a las reservas de me...	TO DO																																		
▼ <u>TFG-4 Gestión de Platos de Comida</u>																																			
TFG-18 Crear platos de comida	TO DO																																		
TFG-19 Ver platos de comida (administra...	TO DO																																		
TFG-20 Actualizar platos de comida	TO DO																																		
TFG-21 Eliminar platos de comida	TO DO																																		
TFG-22 Ver platos de comida (cliente)	TO DO																																		
▼ <u>TFG-5 Gestión de Pedidos de Comida</u>																																			
TFG-23 Agregar platos de comida al carr...	TO DO																																		
TFG-24 Ajustar la comida de un plato en...	TO DO																																		
TFG-25 Realizar pedidos de comida	TO DO																																		
TFG-26 Ver pedidos de comida (cliente)	TO DO																																		
TFG-27 Ver pedidos de comida (administ...	TO DO																																		
TFG-28 Cambiar estado del pedido	TO DO																																		
▼ <u>TFG-6 Gestión de Reseñas</u>																																			
TFG-29 Dejar reseñas	TO DO																																		
TFG-30 Modificar reseñas	TO DO																																		
TFG-31 Eliminar reseñas (cliente)	TO DO																																		
TFG-32 Ver reseñas (cliente)	TO DO																																		
TFG-33 Ver reseñas (administrador)	TO DO																																		
TFG-34 Eliminar reseñas (administrador)	TO DO																																		

Ilustración 9 y 10). En esta última, se observa un tablero con tres columnas principales: “To Do”, “In Progress” y “Done”, que representan el flujo de trabajo de las User Stories durante el proceso de desarrollo, de manera que se mueven de la columna “To Do” a “In Progress” cuando se inicia su trabajo y, finalmente, a “Done” una vez que se completan.

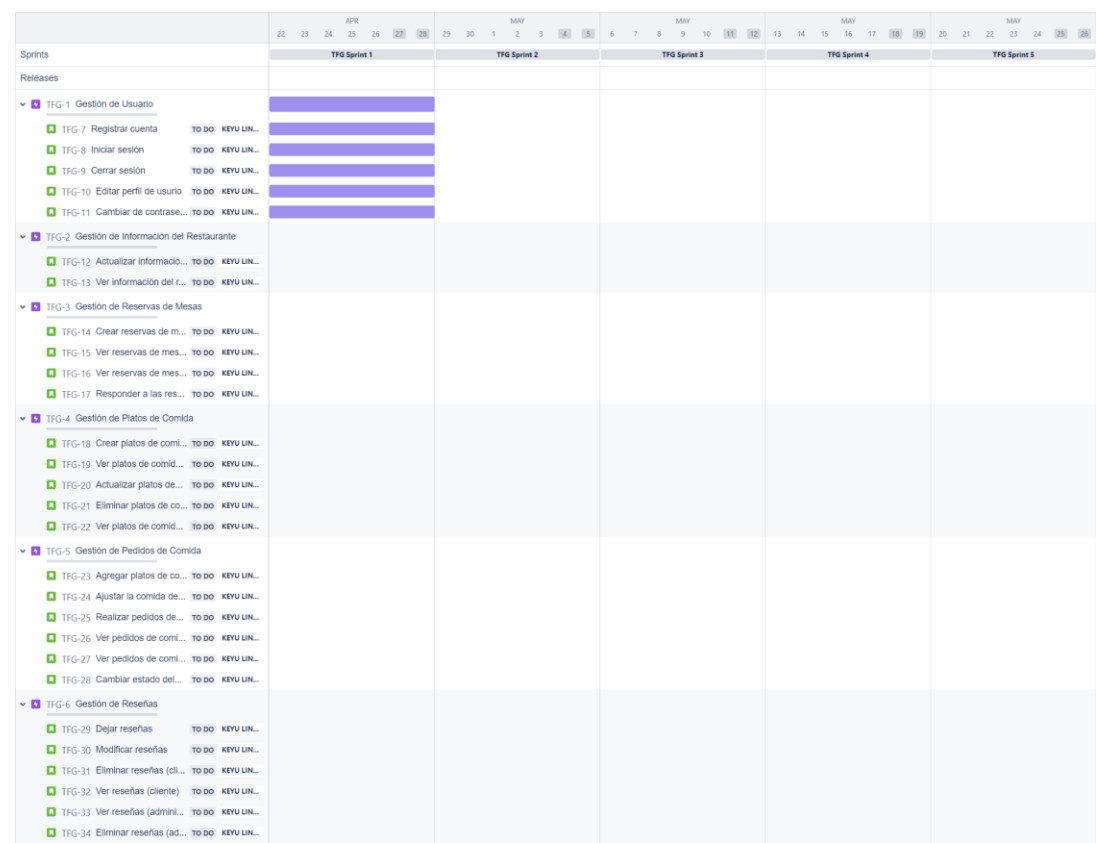


Ilustración 9: Planificación del Sprint 1.

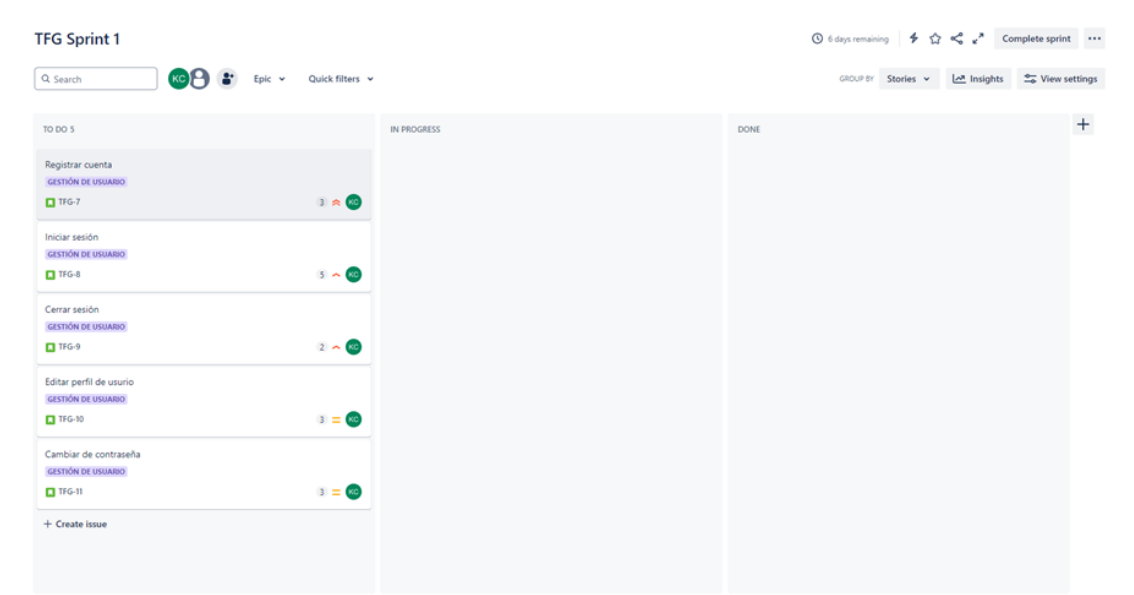


Ilustración 10: Tablero de Sprint 1 por hacer.

5.5.2 Diseño

5.5.2.1 Diseño Front-End

5.5.2.1.1 Diagrama de componentes y prototipos de interfaz de usuario

Para comprender mejor la interacción del usuario con la aplicación, se ha desarrollado una serie de diagramas de componentes de Angular con el fin de mostrar cómo se va de un componente a otro, junto con un conjunto de prototipos de interfaz del usuario. De esta manera, se facilitará la comprensión de las rutas y transiciones que experimentarán los usuarios. Cabe destacar que se ha tomado la decisión de no representar el flujo entre los diferentes componentes de la aplicación a través de la barra de navegación, excepto la conexión con HomeComponent, que se encuentra también en la barra de navegación. De esta forma, el diagrama se simplificará y será más entendible, ya que, en caso contrario, se tendría que representar todas las conexiones entre todos los componentes principales a través de la barra de navegación. Para este Sprint, se ha desarrollado los siguientes diagramas (véase Ilustración 11 y 12):



Ilustración 11: Diagrama de componentes para usuarios que no han iniciado sesión.

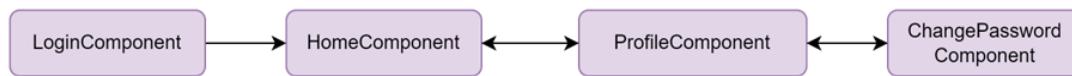


Ilustración 12: Diagrama de componentes para usuarios que sí han iniciado sesión del Sprint 1.

A continuación, se detallan los componentes y sus relaciones:

- **HomeComponent:** Página principal de la aplicación que sirve como punto de entrada para usuarios registrados y no registrados. En ella, aparecerá la información básica del restaurante (véase Ilustración 13, 14, 15 y 16). Desde este componente, los usuarios pueden ir a LoginComponent si aún no han iniciado sesión. Una vez iniciado sesión, los usuarios pueden acceder a ProfileComponent.

Dragon Eats

Inciar sesión

Bienvenidos

Reservar

Menú

Reseñas

Ilustración 13: HomeComponent 1.

Servimos



Comida

Descripción de
la comida



Comida

Descripción de
la comida



Comida

Descripción de
la comida



Comida

Descripción de
la comida

Ilustración 14: HomeComponent 2.



Ilustración 15: HomeComponent 3.

Información del restaurante



Ilustración 16: HomeComponent 4.

- **LoginComponent:** Componente que muestra un formulario de inicio de sesión para los usuarios de la aplicación. Desde este componente, los usuarios pueden ir a HomeComponent si aún no ha iniciado sesión y a RegisterComponent con el botón “Crear cuenta” (véase Ilustración 17). Una vez iniciado sesión, los usuarios accederán a HomeComponent y no podrán volver a LoginComponent hasta que hayan cerrado sesión.

Acceder

Acceder

Información

Crear cuenta

Ilustración 17: LoginComponent.

- **RegisterComponent:** Componente que muestra un formulario de registro para los usuarios de la aplicación (véase Ilustración 18). Desde este componente, los usuarios pueden ir a LoginComponent con el botón “Acceder”.

Crear cuenta

☐ Información

Crear cuenta

Información

Acceder

Ilustración 18: RegisterComponent.

- **ProfileComponent:** Componente donde los usuarios pueden ver y editar su información personal (véase Ilustración 19). Desde este componente, los usuarios pueden ir a HomeComponent y a ChangePasswordComponent con el botón “Cambiar contraseña”. Además, podrá cerrar sesión, de manera que vuelve a HomeComponent.

Perfil del usuario

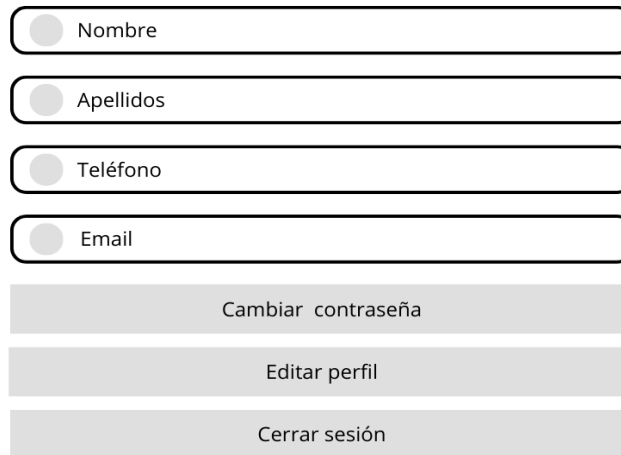


Ilustración 19: ProfileComponent. El formulario muestra cuatro campos de entrada con iconos de ojo: 'Nombre', 'Apellidos', 'Teléfono' y 'Email'. Debajo de los campos hay tres botones: 'Cambiar contraseña', 'Editar perfil' y 'Cerrar sesión'.

Ilustración 19: ProfileComponent.

- **ChangePasswordComponent:** Componente donde los usuarios pueden cambiar su contraseña actual (véase Ilustración 20). Desde este componente, los usuarios pueden ir a ProfileComponent.

Cambiar contraseña

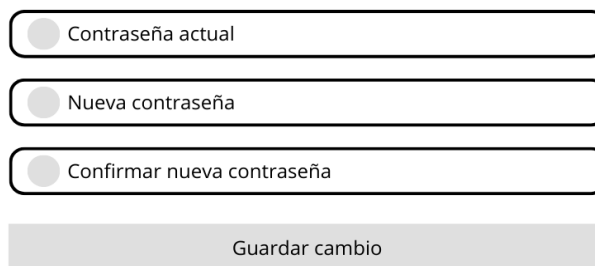


Ilustración 20: ChangePasswordComponent. El formulario muestra tres campos de entrada con iconos de ojo: 'Contraseña actual', 'Nueva contraseña' y 'Confirmar nueva contraseña'. Debajo de los campos hay un botón: 'Guardar cambio'.

Ilustración 20: ChangePasswordComponent.

5.5.2.2 Diseño Back-End

5.5.2.2.1 Diseño BBDD

Para comenzar con el diseño del Back-End, en primer lugar, es necesario diseñar la base de datos de la aplicación, ya que esto garantizará la organización eficiente de todos los datos de la aplicación. De esta manera, durante cada Sprint del proyecto, se dedicará tiempo para diseñar nuevas tablas para añadir a la base de datos. Por último, cabe destacar que una base de datos bien estructurado no solo facilita el almacenamiento y la recuperación de la información, sino que también mejora el rendimiento general del sistema y asegura la integridad de los datos.

En este primer Sprint, se creará la entidad “User” (véase Ilustración 21), que es fundamental para la mayoría de las funcionalidades de la aplicación:

- **User.** La entidad “User” representa a cada persona que interactúa con el sistema, ya sea como cliente o como administrador. Cada usuario tiene su propia información lo que permite su identificación y acceso a la aplicación. La entidad “User” contiene estos campos:
 - *id*: Identificador único que se asigna a cada usuario. Este campo sirve como clave primaria en la base de datos (entero largo, autoincremental).
 - *email*: Correo electrónico del usuario, debe ser única en todo el sistema (cadena de caracteres).
 - *firstname*: Nombre del usuario (cadena de caracteres).
 - *lastname*: Apellido del usuario (cadena de caracteres).
 - *password*: Contraseña del usuario para poder acceder a su cuenta (cadena de caracteres).
 - *phone*: Número de teléfono del usuario, debe ser única en todo el sistema (cadena de caracteres).
 - *role*: Rol del usuario dentro del sistema, define los permisos y el nivel de acceso que tiene cada usuario en las diferentes funcionalidades del sistema (cadena de caracteres).

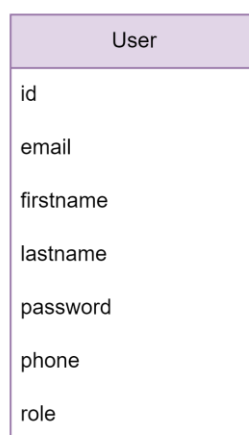


Ilustración 21: Diseño de BBDD del Sprint 1.

5.5.2.2.2 Diseño API

Junto al diseño de la base de datos, también es de suma importancia el diseño de las APIs REST (Representational State Transfer), ya que es fundamental para la comunicación del Front-End con el Back-End. De esta manera, un diseño de API bien estructurado asegura que todas las funcionalidades del sistema estén claramente definidas y accesibles, definiendo los métodos HTTP estándar (GET, POST, PUT, DELETE), los parámetros de entrada y salida, las cabeceras necesarias, y los posibles códigos de respuesta.

Al igual que el diseño de la base de datos, durante cada Sprint del proyecto, se realizará un diseño detallado de las APIs necesarias para implementar las funcionalidades requeridas. Esto garantizará que cada endpoint esté bien documentado, facilitando su posterior implementación y realización de las pruebas.

Para este Sprint, se han diseñado los siguientes endpoints:

URI	http://localhost:8080/api/v1/auth/register	
Método	POST	
Descripción	Crea un nuevo usuario en el sistema.	
Autenticación	Ninguno (Público)	
Cabecera	Content-Type: application/json	
Cuerpo de petición	<pre>{ "firstname": "Nombre", "lastname": "Apellido", "email": "email@example.com", "password": "Contraseña123!", "phone": "123456789" }</pre>	
Parámetros de consulta	Ninguno	
Código de respuesta	201: Created	El usuario se ha registrado con éxito.
	400: Bad Request	Parámetros no válidos.
	409: Conflict	Email y número de teléfono ya en uso.
	500: Internal Server Error	Error interno del servidor.

Tabla 30: POST – Register.

URI	http://localhost:8080/api/v1/auth/login	
Método	POST	
Descripción	Inicia sesión de un usuario existente en el sistema.	
Autenticación	Ninguno (Público)	

Cabecera	Content-Type: application/json	
Cuerpo de petición	<pre>{ "email": "email@example.com", "password": "Contraseña123!" }</pre>	
Parámetros de consulta	Ninguno	
Código de respuesta	200: OK	Autenticación de usuario válido, devolviendo un JWT al usuario.
	400: Bad Request	Parámetros no válidos.
	401: Unauthorized	Credenciales no válidas.
	404: Not Found	Usuario no encontrado.
	500: Internal Server Error	Error interno del servidor.

Tabla 31: POST – Login.

URI	http://localhost:8080/api/v1/users/me/profile	
Método	GET	
Descripción	Obtiene el perfil del usuario autenticado.	
Autenticación	Bearer Token (Rol cliente y administrador)	
Cabecera	Authorization: Bearer {token} Content-Type: application/json	
Cuerpo de petición	Ninguno	
Parámetros de consulta	Ninguno	
Código de respuesta	200: OK	La solicitud fue exitosa.
	404: Not Found	Usuario no encontrado.
	500: Internal Server Error	Error interno del servidor.

Tabla 32: GET – Profile.

URI	http://localhost:8080/api/v1/users/me/profile	
Método	PATCH	
Descripción	Actualiza el perfil de usuario autenticado.	
Autenticación	Bearer Token (Rol cliente y administrador)	
Cabecera	Authorization: Bearer {token} Content-Type: application/json	

Cuerpo de petición	<pre>{ "firstname": "Nombre", "lastname": "Apellido", "email": "email@example.com", "phone": "123456789" }</pre>	
Parámetros de consulta	Ninguno	
Código de respuesta	200: OK	Perfil actualizado con éxito.
	400: Bad Request	Parámetros no válidos.
	401: Unauthorized	Token de acceso no válido.
	403: Forbidden	Acceso denegado.
	404: Not Found	Usuario no encontrado.
	409: Conflict	Email y número de teléfono ya en uso.
	500: Internal Server Error	Error interno del servidor.

Tabla 33: PATCH - Profile

URI	http://localhost:8080/api/v1/users/me/password	
Método	PATCH	
Descripción	Actualiza la contraseña del usuario autenticado.	
Autenticación	Bearer Token (Rol cliente y administrador)	
Cabecera	Authorization: Bearer {token} Content-Type: application/json	
Cuerpo de petición	<pre>{ "oldPassword": "Contraseña123!", "newPassword": "Contraseña456!" }</pre>	
Parámetros de consulta	Ninguno	
Código de respuesta	200: OK	Contraseña actualizada con éxito.
	400: Bad Request	Parámetros no válidos.
	401: Unauthorized	Token de acceso no válido.
	403: Forbidden	Acceso denegado.
	404: Not Found	Usuario no encontrado.
	500: Internal Server Error	Error interno del servidor.

Tabla 34: PATCH - Password

5.5.3 Pruebas de sistema

Una vez completado con el desarrollo de este Sprint, realizar pruebas de sistema es fundamental para comprobar el correcto funcionamiento de la aplicación, así como identificar los posibles errores de software. De esta manera, no solo se asegura la calidad del software, sino que también permite una retroalimentación rápida para los clientes, garantizando que el producto final satisfaga sus necesidades.

Cada prueba de sistema tendrá un título, una pequeña descripción, un resultado esperado y un resultado real representado mediante imágenes. Para este Sprint, se ha llevado a cabo las siguientes pruebas de sistema:

- **PS1_US1 - Registro correcto**
 - Descripción: El usuario no registrado completa el formulario de registro de manera correcta.
 - Resultado esperado: El sistema registra al usuario en la base de datos, muestra un mensaje de que se ha registrado con éxito y luego redirige al usuario a la página de inicio de sesión.

Ilustración 22: Registro correcto 1.

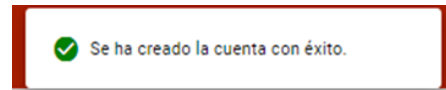


Ilustración 23: Registro correcto 2.

Ilustración 24: Registro correcto 3.

- **PS2_US1 - Registro incorrecto**

- Descripción: El usuario no registrado introduce los datos con el formato incorrecto o con un email o número de teléfono ya en uso.
- Resultado esperado: El sistema muestra al usuario los campos con formato incorrecta y en caso de que utilice un email o número ya en uso, se mostrará un mensaje de que ya está en uso. Además, deshabilitará el botón de crear cuenta.

The screenshot shows the 'Crear cuenta' form with the following fields and errors:

- Nombre***: Keyu
- Apellidos**: (empty)
- Teléfono***: 1234567890. Error message: "El número de teléfono debe tener 9 dígitos".
- Email***: keyu.cliente@gmail.com
- Contraseña***: KeyuCliente123. Error message: "La contraseña no cumple con los requisitos de seguridad".
- Confirmar contraseña***: (empty). Error message: "Las contraseñas no coinciden".
- ☐ Acepto los [términos y condiciones generales](#) y la [política de privacidad](#).
- Crear cuenta** button: Disabled (grey).
- ¿Ya tienes una cuenta?** link: (empty)
- Iniciar sesión** button: Active (red).

Ilustración 25: Registro incorrecto 1.

The screenshot shows the 'Crear cuenta' form with the following fields and errors:

- Nombre***: Keyu
- Apellidos**: (empty)
- Teléfono***: 123456789
- Email***: keyu.cliente@gmail.com. Error message: "El correo electrónico introducido ya está en uso".
- Contraseña***: KeyuCliente123!
- Confirmar contraseña***: KeyuCliente123!
- ☒ Acepto los [términos y condiciones generales](#) y la [política de privacidad](#).
- Crear cuenta** button: Disabled (grey).
- ¿Ya tienes una cuenta?** link: (empty)
- Iniciar sesión** button: Active (red).

Ilustración 26: Registro incorrecto 2.

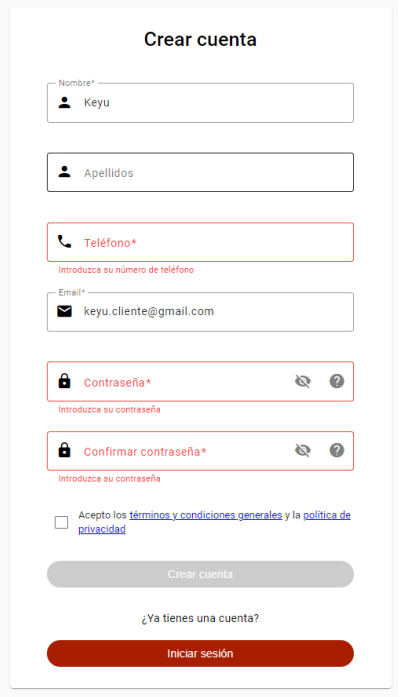
The screenshot shows the 'Crear cuenta' form with the following fields and errors:

- Nombre***: Keyu
- Apellidos**: (empty)
- Teléfono***: 123456789. Error message: "El número de teléfono introducido ya está en uso".
- Email***: keyu.cliente1@gmail.com
- Contraseña***: KeyuCliente123!
- Confirmar contraseña***: KeyuCliente123!
- ☒ Acepto los [términos y condiciones generales](#) y la [política de privacidad](#).
- Crear cuenta** button: Disabled (grey).
- ¿Ya tienes una cuenta?** link: (empty)
- Iniciar sesión** button: Active (red).

Ilustración 27: Registro incorrecto 3.

- **PS3_US1 - Registro incompleto**

- Descripción: El usuario no registrado le falta por rellenar algún campo obligatorio del registro.
- Resultado esperado: El sistema no registra al usuario, señala los campos que faltan por completar y deshabilitará el botón de crear cuenta.



Crear cuenta

Nombre*
Keyu

Apellidos

Teléfono*
Introduzca su número de teléfono

Email*
keyu.cliente@gmail.com

Contraseña*
Introduzca su contraseña

Confirmar contraseña*
Introduzca su contraseña

☐ Acepto los [términos y condiciones generales](#) y la [política de privacidad](#)

Crear cuenta

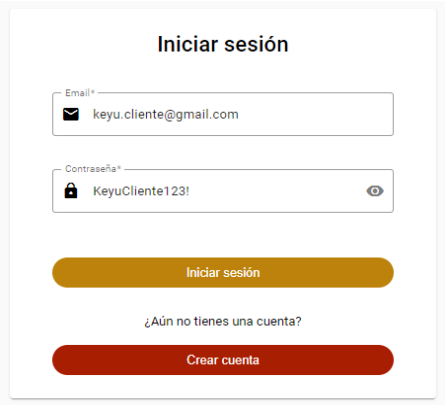
¿Ya tienes una cuenta?

Iniciar sesión

Ilustración 28: Registro incompleto.

- **PS4_US2 – Inicio de sesión correcto**

- Descripción: El cliente o el administrador introduce sus datos correctamente.
- Resultado esperado: El sistema permite al usuario acceder a su cuenta, muestra un mensaje de que se ha iniciado con éxito y luego redirige al usuario a la página inicial de la aplicación.



Iniciar sesión

Email*
keyu.cliente@gmail.com

Contraseña*
KeyuCliente123!

Iniciar sesión

¿Aún no tienes una cuenta?

Crear cuenta

Ilustración 29: Inicio de sesión correcto 1.

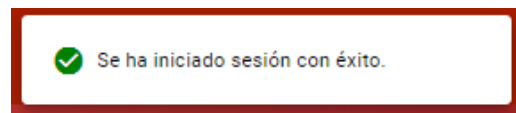


Ilustración 30: Inicio de sesión 2.



Ilustración 31: Inicio de sesión 3.

- **PS5_US2 – Inicio de sesión incorrecto**

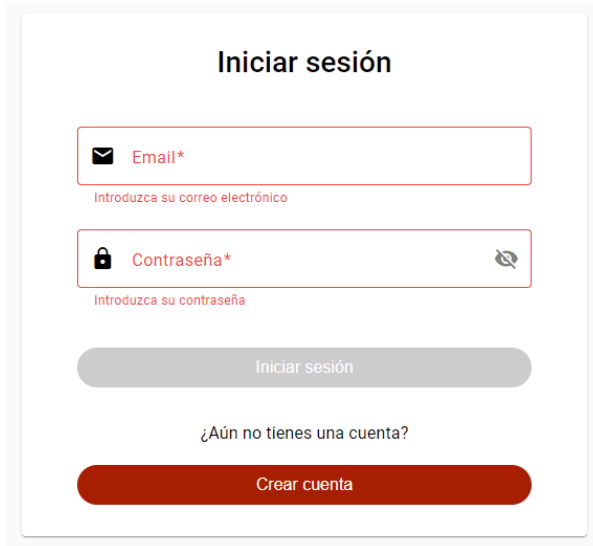
- Descripción: El cliente o el administrador introduce credenciales no válidas.
- Resultado esperado: El sistema muestra al usuario los campos con error y deshabilitará el botón de iniciar sesión.

Ilustración 32: Inicio de sesión incorrecto 1.

Ilustración 33: Inicio de sesión 2.

- **PS6_US2 – Inicio de sesión incompleto**

- Descripción: El cliente o el administrador no rellena todos los campos obligatorios del inicio de sesión.
- Resultado esperado: El sistema no permite el acceso al usuario, señala los campos que faltan por completar y deshabilitará el botón de iniciar sesión.

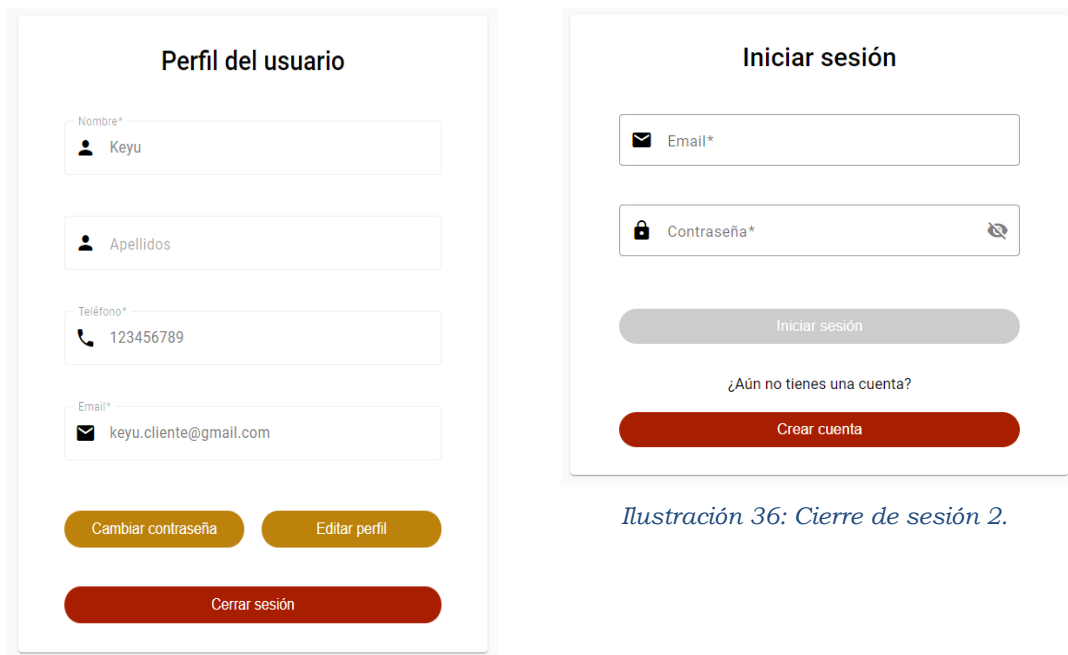


The screenshot shows a login form titled "Iniciar sesión". It contains two input fields: "Email*" and "Contraseña*". The "Email*" field has a red border and a red error message "Introduzca su correo electrónico" below it. The "Contraseña*" field also has a red border and a red error message "Introduzca su contraseña" below it. Below the fields is a grey button labeled "Iniciar sesión". Below the button is a link "¿Aún no tienes una cuenta?". At the bottom is a red button labeled "Crear cuenta".

Ilustración 34: Inicio de sesión incompleto.

- **PS7_US3- Cierre de sesión**

- Descripción: El cliente o el administrador cierra la sesión.
- Resultado esperado: El sistema redirige al usuario a la página de inicio de sesión.



The image contains two screenshots. The left screenshot shows a "Perfil del usuario" form with fields for "Nombre*" (Keyu), "Apellidos", "Teléfono*" (123456789), and "Email*" (keyu.cliente@gmail.com). Below the fields are buttons for "Cambiar contraseña", "Editar perfil", and "Cerrar sesión". The right screenshot shows the "Iniciar sesión" form, which is identical to the one in Illustration 34, with red borders and error messages on the input fields.

Ilustración 36: Cierre de sesión 2.

Ilustración 35: Cierre de sesión 1.

- **PS8_US4 – Edición de perfil correcto**

- Descripción: El cliente o el administrador modifica e introduce correctamente los cambios de perfil que quiere realizar.
- Resultado esperado: El sistema guarda los cambios realizados, actualiza la información del perfil del usuario en la base de datos y muestra un mensaje de que los cambios han sido guardados con éxito.

Perfil del usuario

Nombre*
Key

Apellidos

Teléfono*
123456789

Email*
keyu.cliente@gmail.com

Cambiar contraseña Editar perfil

Cerrar sesión

Ilustración 37: Edición de perfil correcto 1.

Perfil del usuario

Nombre*
Keyu

Apellidos

Teléfono*
123456789

Email*
keyu.cliente@gmail.com

Cambiar contraseña Guardar cambios

Cerrar sesión

Ilustración 38: Edición de perfil correcto 2.

Se ha guardado los cambios con éxito.

Perfil del usuario

Nombre*
Keyu

Apellidos

Teléfono*
123456789

Email*
keyu.cliente@gmail.com

Cambiar contraseña Editar perfil

Cerrar sesión

Ilustración 39: Edición de perfil correcto 3.

- **PS9_US4 – Edición de perfil incorrecto**

- Descripción: El cliente o el administrador introduce sus datos con el formato incorrecto.
- Resultado esperado: El sistema muestra los campos con error, no guarda los cambios realizados y deshabilitará el botón de guardar los cambios.

The screenshot shows a web form titled "Perfil del usuario". It contains four input fields: "Nombre*" (with "Keyu" entered), "Apellidos", "Teléfono*" (with "12345678" entered), and "Email*" (with "keyu.clientegmail.com" entered). The "Teléfono*" and "Email*" fields are highlighted with red borders, indicating validation errors. Below the "Teléfono*" field, a red message states "El número de teléfono debe tener 9 dígitos". Below the "Email*" field, a red message states "Introduzca un correo electrónico válido". At the bottom of the form, there are three buttons: "Cambiar contraseña" (yellow), "Guardar cambios" (gray and disabled), and "Cerrar sesión" (red).

Ilustración 40: Edición de perfil incorrecto.

- **PS10_US4 – Edición de perfil incompleto**

- Descripción: El cliente o el administrador no rellena todos los campos obligatorios en la edición del perfil.
- Resultado esperado: El sistema no guarda los cambios realizados, señala los campos que faltan por completar y deshabilitará el botón de guardar los cambios.

The screenshot shows the same "Perfil del usuario" form. The "Nombre*" field is filled with "Keyu". The "Apellidos" field is empty. The "Teléfono*" field is empty and has a red border with a red message below it: "Introduzca su número de teléfono". The "Email*" field is empty and has a red border with a red message below it: "Introduzca su correo electrónico". The "Guardar cambios" button is gray and disabled, while "Cambiar contraseña" and "Cerrar sesión" are active.

Ilustración 41: Edición de perfil incompleto.

- **PS11_US5 – Cambio de contraseña correcta**

- Descripción: El cliente o el administrador introduce sus datos correctamente.
- Resultado esperado: El sistema actualiza la contraseña del usuario y muestra un mensaje de que se ha cambiado con éxito.



Formulario de "Cambiar contraseña" con tres campos de entrada:

- Contraseña actual*: KeyuCiente123!
- Nueva contraseña*: KeyuCiente1234!
- Confirmar nueva contraseña*: KeyuCiente1234!

Debajo de los campos hay un botón rojo que dice "Cambiar contraseña".

Ilustración 42: Cambio de contraseña correcta 1.

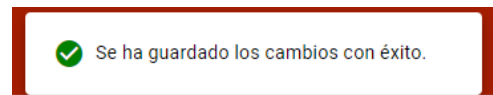


Ilustración 43: Cambio de contraseña 2.

- **PS12_US5 – Cambio de contraseña incorrecta**

- Descripción: El cliente o el administrador no introduce correctamente su contraseña actual, ya que la nueva contraseña no tiene el formato o que no concuerda con la confirmación de la contraseña.
- Resultado esperado: El sistema no actualiza la nueva contraseña y muestra un mensaje de que la contraseña antigua es incorrecta, la nueva contraseña no cumple con los requisitos o que hay un error de concordancia. Además, deshabilitará el botón de cambiar de contraseña.

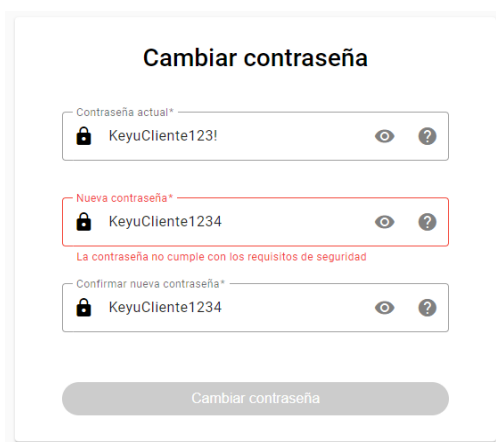


Formulario de "Cambiar contraseña" con tres campos de entrada:

- Contraseña actual*: KeyuCiente12! (marcado con un recuadro rojo)
- Nueva contraseña*: KeyuCiente1234!
- Confirmar nueva contraseña*: KeyuCiente1234!

Debajo del primer campo hay un mensaje de error: "La contraseña antigua no es correcta". El botón "Cambiar contraseña" está deshabilitado (gris).

Ilustración 44: Cambio de contraseña incorrecta 1.



Formulario de "Cambiar contraseña" con tres campos de entrada:

- Contraseña actual*: KeyuCiente123!
- Nueva contraseña*: KeyuCiente1234 (marcado con un recuadro rojo)
- Confirmar nueva contraseña*: KeyuCiente1234

Debajo del segundo campo hay un mensaje de error: "La contraseña no cumple con los requisitos de seguridad". El botón "Cambiar contraseña" está deshabilitado (gris).

Ilustración 45: Cambio de contraseña incorrecta 2.

The screenshot shows a web form titled "Cambiar contraseña". It contains three input fields, each with a lock icon, an eye icon, and a question mark icon. The first field is labeled "Contraseña actual*" and contains the text "KeyuCliente123!". The second field is labeled "Nueva contraseña*" and also contains "KeyuCliente123!". The third field is labeled "Confirmar nueva contraseña*" and contains "KeyuCliente123!". Below the third field, there is a red error message: "Las contraseñas no coinciden". At the bottom of the form is a grey button labeled "Cambiar contraseña".

Ilustración 46: Cambio de contraseña incorrecta 3.

- **PS13_US5 – Cambio de contraseña incompleta**
 - Descripción: El cliente o el administrador no rellena todos los campos obligatorios.
 - Resultado esperado: El sistema no guarda los cambios realizados, señala los campos que faltan por completar y deshabilitará el botón de cambiar de contraseña.

The screenshot shows the same "Cambiar contraseña" form. The first field, "Contraseña actual*", is empty and has a red error message below it: "Introduzca su contraseña actual". The second field, "Nueva contraseña*", is also empty and has a red error message below it: "Introduzca su nueva contraseña". The third field, "Confirmar nueva contraseña*", is empty and has a red error message below it: "Las contraseñas no coinciden". The grey button "Cambiar contraseña" is still present at the bottom.

Ilustración 47: Cambio de contraseña incompleta.

5.5.4 Revisión y Retrospectiva del Sprint

Como se ha explicado en el Capítulo 4, en las revisiones de los Sprints, se demuestra el incremento desarrollado a la parte interesada, que en este caso son los dueños del restaurante. Sin embargo, cabe mencionar que solo se documentará el trabajo completado durante cada Sprint, sin proporcionar ilustraciones de la aplicación. Esta decisión fue tomada con el objetivo de presentar de manera ordenada la aplicación final en el Capítulo 6.

Durante este Sprint, se ha logrado cumplir con todos los objetivos establecidos en la planificación del Sprint, implementando todas las funcionalidades de la gestión de usuarios, tales como el registro, inicio de sesión, cierre de sesión, edición de perfil y cambio de contraseña. Para lograrlo, se ha creado diagramas de componentes y prototipos y se ha documentado los diseños de la base de datos y de las APIs, además de realizar un conjunto de pruebas de sistema para asegurar el correcto funcionamiento de la aplicación. El feedback recibido por parte de los dueños del restaurante fue positivo y se identificaron áreas de mejora, como el tamaño de los formularios, que no será anotado en Jira para simplificar el proceso de desarrollo, aunque sí se corregirá para el siguiente Sprint. Por último, en Jira se ha movido todas las User Stories completadas a “Done” (véase Ilustración 48) y se proporciona el Product Backlog tras finalizar el Sprint 1 (véase Ilustración 49).

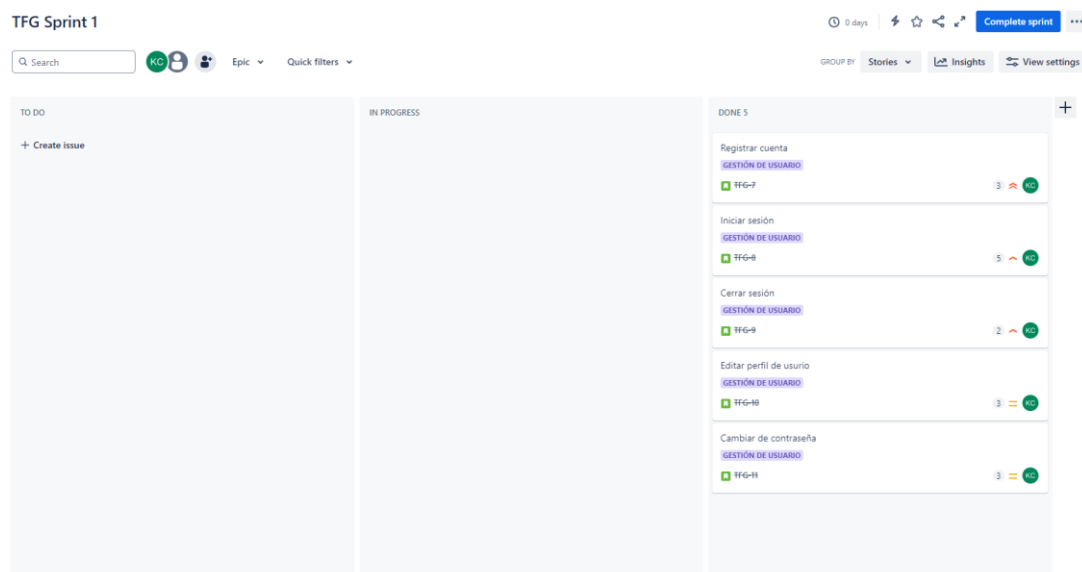


Ilustración 48: Tablero de Sprint 1 hecho.

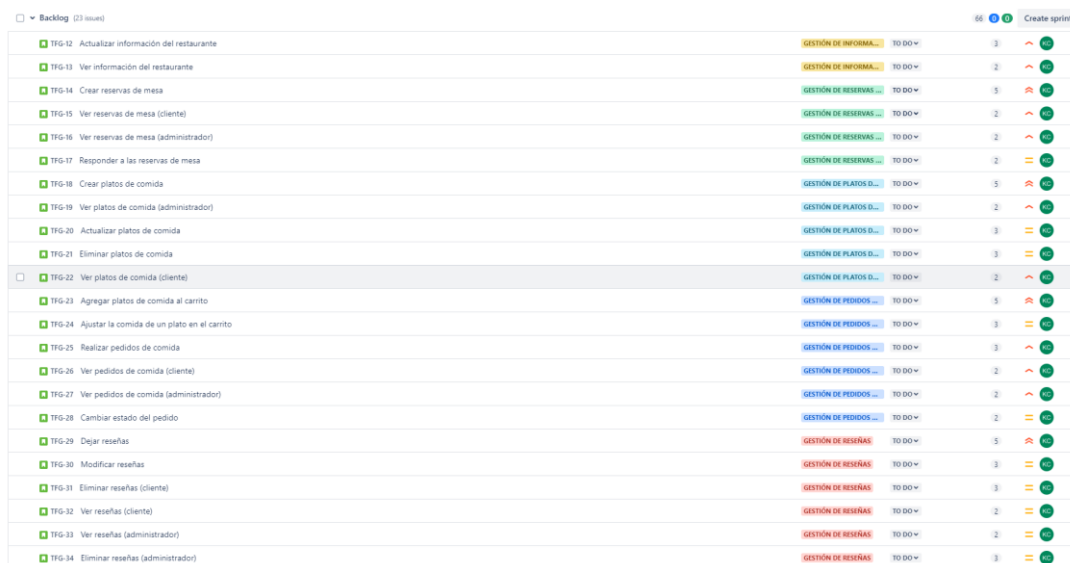


Ilustración 49: Product Backlog tras Sprint 1.

En cuanto a la retrospectiva del Sprint, se ha reflexionado sobre el proceso de desarrollo y se ha concluido que el enfoque actual no es muy efectivo, por tanto, se debe realizar algunos ajustes de tiempo para la realización de las distintas User Stories. A pesar de esto, se pudo completar el Sprint a tiempo. Cabe destacar que la comunicación con los dueños del restaurante es bastante fluida, por lo que no hubo retrasos de tiempo.

5.6 Sprint 2

5.6.1 Planificación del Sprint

Para este segundo Sprint, el **objetivo principal** consiste en implementar las funcionalidades de gestión de información del restaurante y gestión de reservas de mesas con el fin de agilizar este último proceso y mejorar la experiencia general del usuario. Para lograrlo, se han seleccionado las siguientes Features, junto con sus respectivas User Stories:

- FE2: Gestión de información del restaurante
 - US6: Actualizar información del restaurante
 - US7: Ver información del restaurante
- FE3: Gestión de reservas de mesas
 - US8: Crear reservas de mesa
 - US9: Ver reservas de mesa (cliente)
 - US10: Ver reservas de mesa (administrador)
 - US11: Responder a las reservas de mesa de mis clientes

Al igual que el Sprint anterior, se ha creado en Jira el Sprint Backlog correspondiente para este Sprint (véase Ilustración 50 y 51).

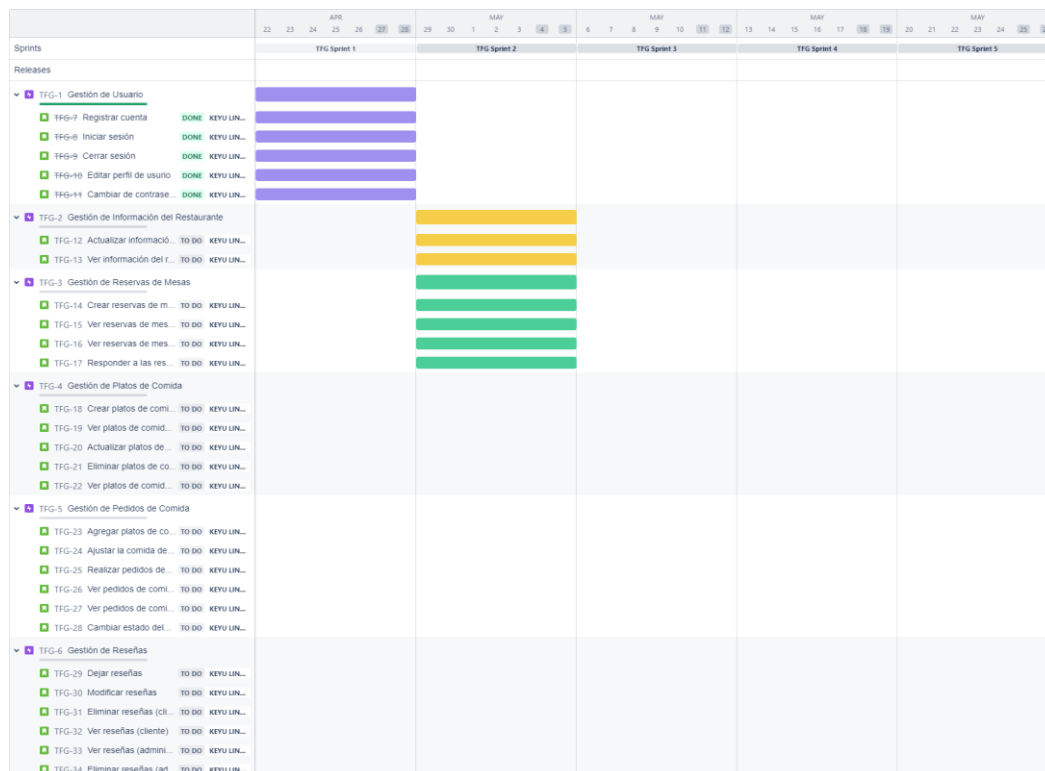


Ilustración 50: Planificación del Sprint 2.

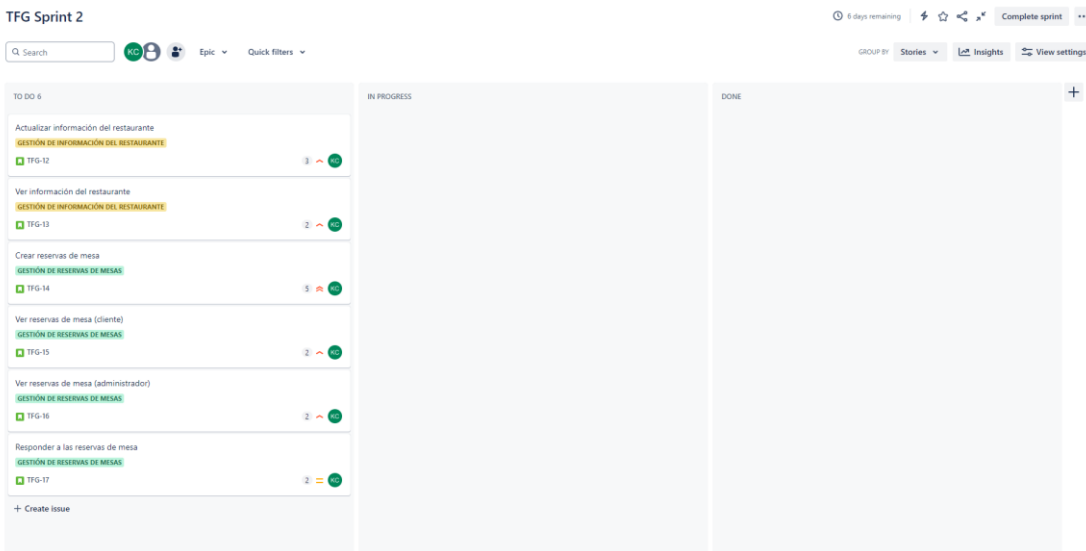


Ilustración 51: Tablero de Sprint 2 por hacer.

5.6.2 Diseño

5.6.2.1 Diseño Front-End

5.6.2.1.1 Diagrama de componentes y prototipos de interfaz de usuario

Para este Sprint, se ha desarrollado los siguientes diagramas de componentes (véase Ilustración 52 y 53):

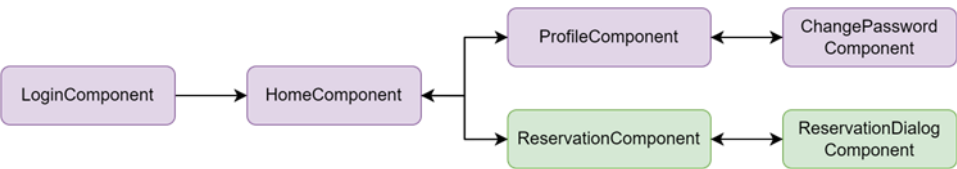


Ilustración 52: Diagrama de componentes para los clientes del Sprint 2.

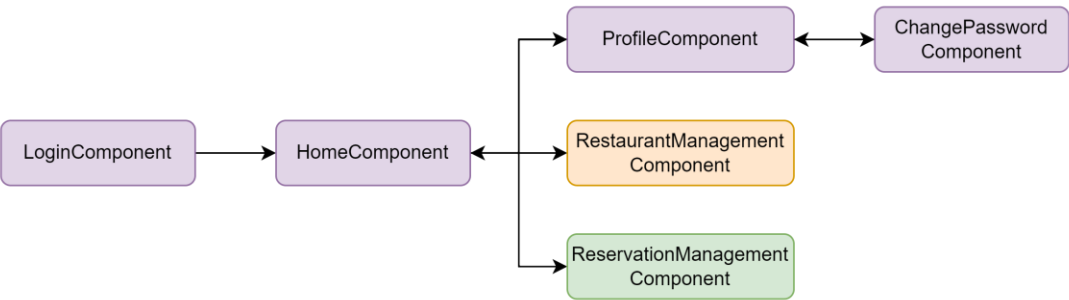


Ilustración 53: Diagrama de componentes para el administrador del Sprint 2.

A continuación, se detallan los componentes y sus relaciones:

- **RestaurantManagementComponent:** Componente donde el administrador puede modificar los datos del restaurante (véase Ilustración 54). Desde este componente, el administrador puede ir a HomeComponent.

Información del restaurante

☐ Nombre del restaurante
☐ Sobre nosotros
☐ Teléfono
☐ Email
☐ Horario
☐ Dirección

Editar información del restaurante

Ilustración 54: RestaurantManagementComponent.

- **ReservationComponent:** Componente donde los clientes pueden visualizar su lista de reservas (véase Ilustración 55). Desde este componente, los clientes pueden ir a HomeComponent o ReservationDialogComponent para realizar una reserva.

Mis reservas

Hacer reserva

Fecha y hora de la reserva	Número de personas	Estado

Ilustración 55: ReservationComponent.

- **ReservationDialogComponent:** Componente donde los clientes pueden realizar una reserva (véase Ilustración 56). Desde este componente, tras crear o cancelar la reserva, los clientes pueden volver a ReservationComponent.

Hacer reserva

Número de personas

Fecha Hora

Reservar Cancelar

Ilustración 56: ReservationDialogComponent.

- **ReservationManagementComponent:** Componente donde el administrador puede gestionar todas las reservas del sistema (véase Ilustración 57). Desde este componente puede ir a HomeComponent.

Lista de reservas

Fecha y hora	Número de personas	Nombre del cliente	Correo del cliente	Teléfono del cliente	Estado	Acciones

Ilustración 57: ReservationManagementComponent.

5.6.2.2 Diseño Back-End

5.6.2.2.1 Diseño BBDD

Para este Sprint, se creará la entidad “Restaurant” y “Reservation” (véase Ilustración 58):

- **Restaurant.** La entidad “Restaurant” representa la información del restaurante en el sistema. La entidad “Restaurant” contiene estos campos:
 - *id*: Identificador único que se asigna a cada registro del restaurante. En este caso solo habrá un único restaurante. Este campo sirve como clave primaria en la base de datos (entero largo, autoincremental).
 - *address*: Dirección del restaurante (cadena de caracteres).
 - *email*: Correo electrónico del restaurante (cadena de caracteres).
 - *name*: Nombre del restaurante (cadena de caracteres).
 - *phone*: Número de teléfono del restaurante (cadena de caracteres).
 - *schedule*: Horario del restaurante (cadena de caracteres).
 - *about_us*: Descripción sobre el restaurante (cadena de caracteres).

- **Reservation.** La entidad “Reservation” representa una reserva realizada por un cliente en el sistema. Cada usuario realiza su propia reserva que contiene una información detallada. La entidad “Reservation” contiene estos campos:
 - *id*: Identificador único que se asigna a cada reserva (entero largo, autoincremental)
 - *date_time*: Fecha y hora de la reserva (fecha-hora).
 - *people_count*: Número de personas que asistirán para la reserva (entero).
 - *user_id*: Identificador único del usuario que realizó la reserva. Este campo es una clave foránea (entero largo).
 - *status*: Estado actual de la reserva, indica “pendiente”, “aceptado” o “rechazado” (cadena de caracteres).

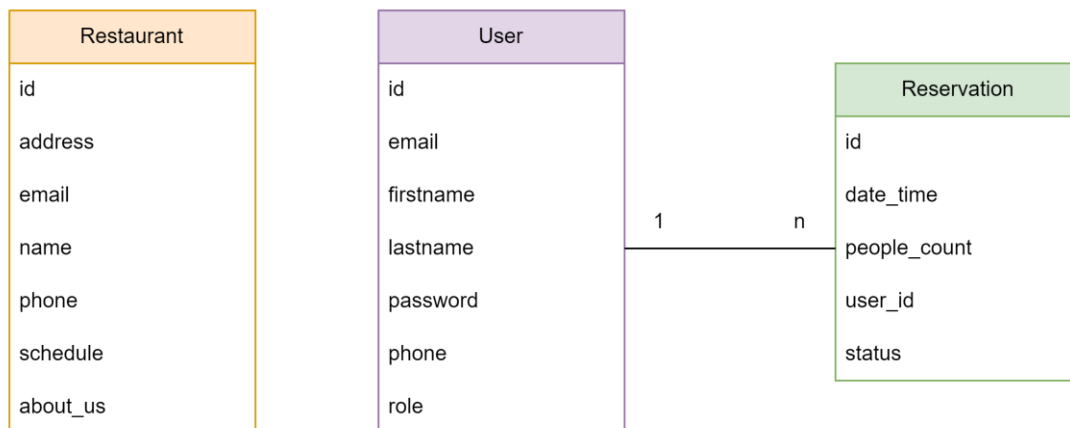


Ilustración 58: Diseño de BBDD del Sprint 2.

5.6.2.2.2 Diseño API

Para este Sprint, se han diseñado los siguientes endpoints:

URI	http://localhost:8080/api/v1/restaurant/info	
Método	GET	
Descripción	Obtiene la información actualizada del restaurante.	
Autenticación	Ninguno (Público)	
Cabecera	Authorization: Bearer {token} Content-Type: application/json	
Cuerpo de petición	Ninguno	
Parámetros de consulta	Ninguno	
Código de respuesta	200: OK	Solicitud exitosa, recibe la información del restaurante.
	401: Unauthorized	Token de acceso no válido.
	403: Forbidden	Acceso denegado.
	500: Internal Server Error	Error interno del servidor.

Tabla 35: GET – RestaurantInfo.

URI	http://localhost:8080/api/v1/restaurant/info	
Método	PUT	
Descripción	Actualiza la información del restaurante.	
Autenticación	Bearer Token (Rol administrador)	
Cabecera	Content-Type: application/json	
Cuerpo de petición	<pre>{ "name": "Nuevo nombre del restaurante", "aboutUs": "Nueva descripción sobre nosotros", "email": "nuevo.email@restaurante.com", "phone": "999999999", "schedule": "Abierto de 9:00 a 21:00", "address": "Nueva dirección, Ciudad, País" }</pre>	
Parámetros de consulta	Ninguno	
Código de respuesta	200: OK	Información del restaurante actualizada con éxito.

	400: Bad Request	Parámetros no válidos.
	401: Unauthorized	Token de acceso no válido.
	403: Forbidden	Acceso denegado.
	500: Internal Server Error	Error interno del servidor.

Tabla 36: PUT – RestaurantInfo.

URI	http://localhost:8080/api/v1/customers/me/reservations	
Método	POST	
Descripción	Crea una nueva reserva para el cliente.	
Autenticación	Bearer Token (Rol cliente)	
Cabecera	Authorization: Bearer {token} Content-Type: application/json	
Cuerpo de petición	{ “peopleCount”: 10, “dateTime”: “2024-12-31T18:30:00” }	
Parámetros de consulta	Ninguno	
Código de respuesta	201: Created	Se ha reservado con éxito.
	400: Bad Request	Parámetros no válidos
	401: Unauthorized	Token de acceso no válido
	403: Forbidden	Acceso denegado.
	404: Not Found	Usuario no encontrado.
	500: Internal Server Error	Error interno del servidor.

Tabla 37: POST – Reservations.

URI	http://localhost:8080/api/v1/customers/me/reservations	
Método	GET	
Descripción	Obtiene las reservas del cliente.	
Autenticación	Bearer Token (Rol cliente)	
Cabecera	Authorization: Bearer {token} Content-Type: application/json	
Cuerpo de petición	Ninguno	
Parámetros de consulta	Ninguno	
	200: OK	Solicitud completada con éxito.

Código de respuesta	401: Unauthorized	Token de acceso no válido
	403: Forbidden	Acceso denegado.
	404: Not Found	Usuario no encontrado.
	500: Internal Server Error	Error interno del servidor.

Tabla 38: GET – MyReservations.

URI	http://localhost:8080/api/v1/reservations	
Método	GET	
Descripción	Obtiene todas las reservas.	
Autenticación	Bearer Token (Rol administrador)	
Cabecera	Authorization: Bearer {token} Content-Type: application/json	
Cuerpo de petición	Ninguno	
Parámetros de consulta	Ninguno	
Código de respuesta	200: OK	Solicitud completada con éxito.
	401: Unauthorized	Token de acceso no válido
	403: Forbidden	Acceso denegado.
	500: Internal Server Error	Error interno del servidor.

Tabla 39: GET – AllReservations.

URI	http://localhost:8080/api/v1/reservations/{idReservation}	
Método	PATCH	
Descripción	Actualiza el estado de una reserva en concreto.	
Autenticación	Bearer Token (Rol administrador)	
Cabecera	Authorization: Bearer {token} Content-Type: application/json	
Cuerpo de petición	{ "status": "ACCEPTED" }	
Parámetros de consulta	Ninguno	
Código de respuesta	200: OK	Se actualizó el estado de la reserva con éxito.
	400: Bad Request	Parámetros no válidos
	401: Unauthorized	Token de acceso no válido

	403: Forbidden	Acceso denegado.
	404: Not Found	Reserva no encontrada.
	500: Internal Server Error	Error interno del servidor.

Tabla 40: PATCH – Reservations.

5.6.3 Pruebas de sistema

Para este Sprint, se ha llevado a cabo las siguientes pruebas de sistema:

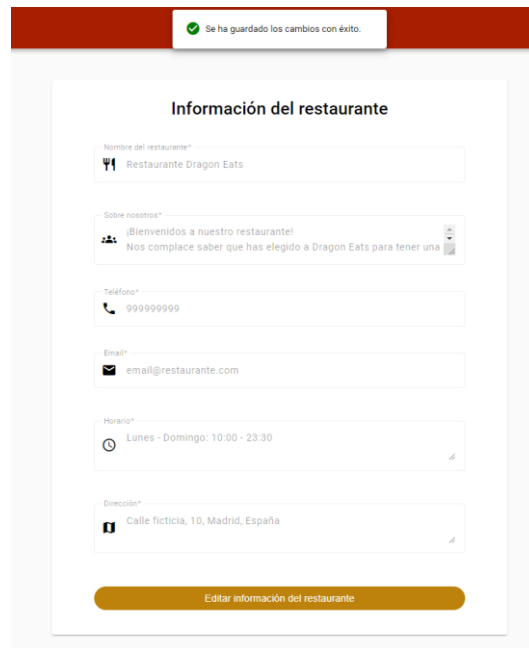
- **PS14_US6 – Actualización de la información del restaurante correcto**
 - Descripción: El administrador rellena correctamente el formulario de la información del restaurante.
 - Resultado esperado: El sistema guarda los cambios realizados, actualiza la información del restaurante en la base de datos y muestra un mensaje de que los cambios han sido guardados con éxito.

The screenshot shows a web form titled "Información del restaurante". It contains several input fields with placeholder text: "Nombre del restaurante*" (Restaurant), "Sobre nosotros*" (About us) with a dropdown menu, "Teléfono*" (Phone), "Email*", "Horario*" (Hours), and "Dirección*" (Address). At the bottom, there is a button labeled "Editar información del restaurante".

Ilustración 59: Actualización info correcto 1.

The screenshot shows the same web form as in Illustration 59, but with specific data entered: "Nombre del restaurante*" is "Restaurante Dragon Eats", "Sobre nosotros*" is "¡Bienvenidos a nuestro restaurante! Nos complace saber que has elegido a Dragon Eats para tener una", "Teléfono*" is "999999999", "Email*" is "email@restaurante.com", "Horario*" is "Lunes - Domingo: 10:00 - 23:30", and "Dirección*" is "Calle ficticia, 10, Madrid, España". At the bottom, there is a button labeled "Guardar cambios".

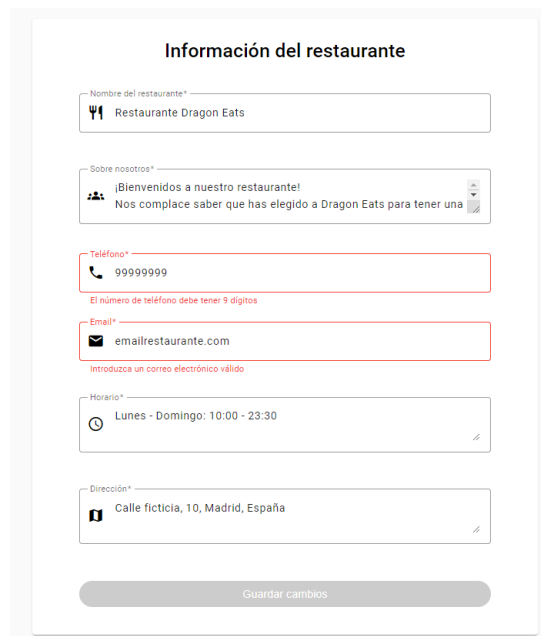
Ilustración 60: Actualización info correcto 2.



A screenshot of a web application interface for updating restaurant information. At the top, a red banner contains a green checkmark and the text "Se ha guardado los cambios con éxito." Below this, the form is titled "Información del restaurante". It contains several input fields: "Nombre del restaurante*" with the value "Restaurante Dragon Eats", "Sobre nosotros*" with a placeholder text "¡Bienvenidos a nuestro restaurante! Nos complace saber que has elegido a Dragon Eats para tener una", "Teléfono*" with the value "999999999", "Email*" with the value "email@restaurante.com", "Horario*" with the value "Lunes - Domingo: 10:00 - 23:30", and "Dirección*" with the value "Calle ficticia, 10, Madrid, España". At the bottom of the form is a yellow button labeled "Editar información del restaurante".

Ilustración 61: Actualización info correcto 3.

- **PS15_US6 – Actualización de la información del restaurante incorrecto**
 - Descripción: El administrador introduce datos con el formato incorrecto.
 - Resultado esperado: El sistema muestra los campos con formato incorrecto y deshabilitará el botón de guardar los cambios.



A screenshot of the same "Información del restaurante" form, but with validation errors. The "Teléfono*" field has a red border and a red error message below it: "El número de teléfono debe tener 9 dígitos". The "Email*" field also has a red border and a red error message below it: "Introduzca un correo electrónico válido". The "Guardar cambios" button at the bottom is now greyed out, indicating it is disabled due to the validation errors.

Ilustración 63: Actualización info incompleto.

- **PS16_US6 – Actualización de la información del restaurante incompleto**
 - Descripción: El administrador no rellena todos los campos obligatorios.
 - Resultado esperado: El sistema señala los campos que faltan por completar y deshabilitará el botón de guardar los cambios de la información del restaurante.

El formulario, titulado 'Información del restaurante', contiene los siguientes campos:

- Nombre del restaurante*:** Un campo de texto con un ícono de tenedor y cuchara a la izquierda. Debajo, un mensaje de error en rojo indica: 'Introduzca el nombre de su restaurante'.
- Sobre nosotros*:** Un campo de texto con un ícono de tres personas a la izquierda. El contenido actual es: '¡Bienvenidos a nuestro restaurante! Nos complace saber que has elegido a Dragon Eats para tener una...'
- Teléfono*:** Un campo de texto con un ícono de teléfono a la izquierda. Debajo, un mensaje de error en rojo indica: 'Introduzca un número de teléfono de su restaurante'.
- Email*:** Un campo de texto con un ícono de correo electrónico a la izquierda. Debajo, un mensaje de error en rojo indica: 'Introduzca un correo electrónico de su restaurante'.
- Horario*:** Un campo de texto con un ícono de reloj a la izquierda. El contenido actual es: 'Lunes - Domingo: 10:00 - 23:30'.
- Dirección*:** Un campo de texto con un ícono de ubicación a la izquierda. El contenido actual es: 'Calle ficticia, 10, Madrid, España'.

Debajo de los campos, hay un botón gris con el texto 'Guardar cambios'.

Ilustración 63: Actualización info incompleto.

- **PS17_US7 – Consulta de la información del restaurante**
 - Descripción: El usuario de la aplicación (no registrado, cliente o administrador) visualiza la información del restaurante.
 - Resultado esperado: El sistema muestra la información del restaurante al usuario.



Ilustración 64: Consulta info del restaurante.

- **PS18_US8 – Creación de reservas de mesa correcta**

- Descripción: El cliente introduce los datos de reserva correctamente.
- Resultado esperado: El sistema crea la reserva, actualiza la lista de reservas de dicho cliente, muestra un mensaje de que se ha reservado con éxito y le envía un correo electrónico.



Formulario "Hacer Reserva" con los siguientes campos:

- Número de Personas*: 5
- Fecha*: 6/3/2024
- Hora*: 14:30
- Botones: Reservar, Cancelar

Ilustración 65: Creación de mesa correcta 1.

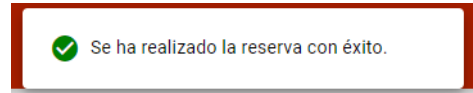


Ilustración 66: Creación de mesa correcta 2.

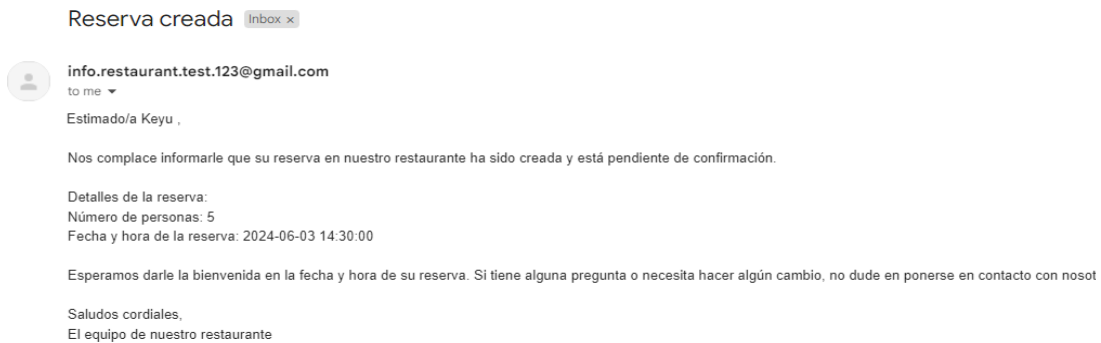


Ilustración 67: Creación de mesa correcta 3.

- **PS19_US8 – Creación de reservas de mesa incompleta**

- Descripción: El cliente no rellena todos los campos obligatorios de la reserva.
- Resultado esperado: El sistema no realiza la reserva, señala los campos que faltan por completar y deshabilitará el botón de reservar.

Hacer Reserva

Número de Personas*

1

Fecha*

Introduzca la fecha de reserva

Hora*

Introduzca la hora de reserva

Reservar

Cancelar

Ilustración 68: Creación de mesa incompleta.

- **PS20_US9 – Consulta de reservas de mesa (cliente)**
 - Descripción: El cliente visualiza las reservas que ha realizado.
 - Resultado esperado: El sistema muestra una lista de reservas que el cliente ha realizado.

Mis Reservas

Hacer reserva

ID	Fecha y hora de la reserva	Número de personas	Estado
2	2024-06-03 14:30:00	5	PENDIENTE
1	2024-06-02 21:00:00	2	PENDIENTE

Ilustración 69: Consulta de reservas de mesa (cliente).

- **PS21_US10 – Consulta de las reservas de mesa (administrador)**
 - Descripción: El administrador visualiza las reservas de todos sus clientes.
 - Resultado esperado: El sistema muestra al administrador todas las reservas de sus clientes.

Lista de reservas

ID	Fecha y hora	Número de personas	Nombre del cliente	Correo del cliente	Teléfono del cliente	Estado	Acciones
2	2024-06-03 14:30:00	5	Keyu	keyu.cliente@gmail.com	123456789	PENDIENTE	AceptarRechazar
1	2024-06-02 21:00:00	2	Keyu	keyu.cliente@gmail.com	123456789	PENDIENTE	AceptarRechazar

Ilustración 70: Consulta de reservas de mesa (administrador).

- **PS22_US11 – Respuesta a las reservas de mesa de los clientes**
 - Descripción: El administrador acepta o rechaza la reserva de los clientes.
 - Resultado esperado: El sistema actualiza el estado de la reserva y envía un correo electrónico al cliente.

Lista de reservas

ID	Fecha y hora	Número de personas	Nombre del cliente	Correo del cliente	Teléfono del cliente	Estado	Acciones
2	2024-06-03 14:30:00	5	Keyu	keyu.cliente@gmail.com	123456789	ACEPTADO	
1	2024-06-02 21:00:00	2	Keyu	keyu.cliente@gmail.com	123456789	PENDIENTE	<button>Aceptar</button> <button>Rechazar</button>

Ilustración 71: Respuesta a las reservas de mesa de los clientes 1.

Reserva aceptada Inbox x

 info.restaurant.test.123@gmail.com
to me ▾

Estimado/a Keyu ,

Nos complace informarle que su reserva en nuestro restaurante ha sido aceptada.

Detalles de la reserva:
Número de personas: 2
Fecha y hora de la reserva: 2024-06-05 19:00:00

Esperamos darle la bienvenida en la fecha y hora de su reserva. Si tiene alguna pregunta o necesita hacer algún cambio, no dude en ponerse en contacto con nosotros.

Saludos cordiales,
El equipo de nuestro restaurante

Ilustración 72: Respuesta a las reservas de mesa de los clientes 2.

Lista de reservas

ID	Fecha y hora	Número de personas	Nombre del cliente	Correo del cliente	Teléfono del cliente	Estado	Acciones
2	2024-06-03 14:30:00	5	Keyu	keyu.cliente@gmail.com	123456789	ACEPTADO	
1	2024-06-02 21:00:00	2	Keyu	keyu.cliente@gmail.com	123456789	RECHAZADO	

Ilustración 73: Respuesta a las reservas de mesa de los clientes 3

Reserva rechazada Inbox x

 info.restaurant.test.123@gmail.com
to me ▾

Estimado/a Keyu ,

Lamentamos informarle que su reserva en nuestro restaurante ha sido rechazada debido a la falta de disponibilidad.

Detalles de la reserva:
Número de personas: 5
Fecha y hora de la reserva: 2024-06-03 14:30:00

Nos disculpamos por cualquier inconveniente que esto pueda causarle y esperamos tener la oportunidad de servirle en otra ocasión.

Saludos cordiales,
El equipo de nuestro restaurante

Ilustración 74: Respuesta a las reservas de mesa de los clientes 4.

5.6.4 Revisión y Retrospectiva del Sprint

Durante este Sprint, se ha logrado cumplir con todos los objetivos establecidos en la planificación del Sprint, implementando todas las funcionalidades de la gestión de información del restaurante (actualización y consulta de la información del restaurante) y de la gestión de reservas de mesas (creación, consulta y actualización de las reservas). Para lograrlo, se ha creado diagramas de componentes y prototipos y se ha documentado los diseños de la base de datos y de las APIs, además de realizar un conjunto de pruebas de sistema para asegurar el correcto funcionamiento de la aplicación. El feedback recibido por parte de los dueños del restaurante fue positivo y se identificaron áreas de mejora, como el bloqueo de la aplicación hasta que se envía el correo electrónico, que no será anotado en Jira para simplificar el proceso de desarrollo, aunque sí se corregirá para el siguiente Sprint. Por último, en Jira se ha movido todas las User Stories completadas a “Done” (véase Ilustración 75) y se proporciona el Product Backlog tras finalizar el Sprint 2 (véase Ilustración 76).

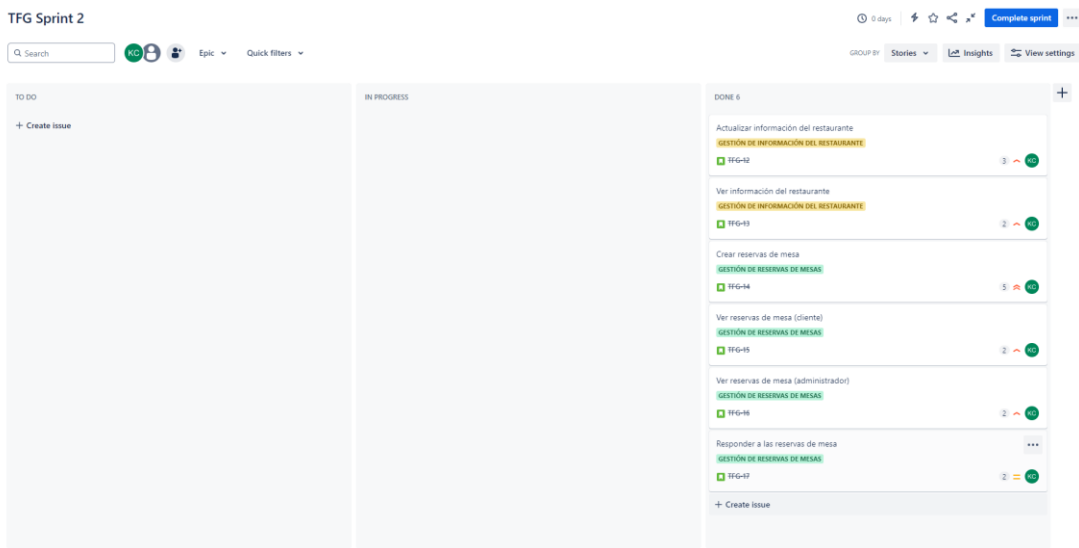


Ilustración 75: Tablero de Sprint 2 hecho.

Backlog (17 issues)

TFG-18	Crear platos de comida	GESTIÓN DE PLATOS D...	TO DO	5	100%
TFG-19	Ver platos de comida (administrador)	GESTIÓN DE PLATOS D...	TO DO	2	100%
TFG-20	Actualizar platos de comida	GESTIÓN DE PLATOS D...	TO DO	3	100%
TFG-21	Eliminar platos de comida	GESTIÓN DE PLATOS D...	TO DO	3	100%
TFG-22	Ver platos de comida (cliente)	GESTIÓN DE PLATOS D...	TO DO	2	100%
TFG-23	Agregar platos de comida al carrito	GESTIÓN DE PEDIDOS	TO DO	5	100%
TFG-24	Ajustar la comida de un plato en el carrito	GESTIÓN DE PEDIDOS	TO DO	3	100%
TFG-25	Realizar pedidos de comida	GESTIÓN DE PEDIDOS	TO DO	3	100%
TFG-26	Ver pedidos de comida (cliente)	GESTIÓN DE PEDIDOS	TO DO	2	100%
TFG-27	Ver pedidos de comida (administrador)	GESTIÓN DE PEDIDOS	TO DO	2	100%
TFG-28	Cambiar estado del pedido	GESTIÓN DE PEDIDOS	TO DO	2	100%
TFG-29	Dejar reseñas	GESTIÓN DE RESERVAS	TO DO	5	100%
TFG-30	Modificar reseñas	GESTIÓN DE RESERVAS	TO DO	3	100%
TFG-31	Eliminar reseñas (cliente)	GESTIÓN DE RESERVAS	TO DO	3	100%
TFG-32	Ver reseñas (cliente)	GESTIÓN DE RESERVAS	TO DO	2	100%
TFG-33	Ver reseñas (administrador)	GESTIÓN DE RESERVAS	TO DO	2	100%
TFG-34	Eliminar reseñas (administrador)	GESTIÓN DE RESERVAS	TO DO	3	100%

Ilustración 76: Product Backlog tras Sprint 2.

En cuanto a la retrospectiva, se ha reflexionado sobre el proceso de desarrollo y se ha concluido que el enfoque actual es efectivo en general, pero se puede realizar mejoras en la priorización de las tareas.

5.7 Sprint 3

5.7.1 Planificación del Sprint

El **objetivo principal** de este tercer Sprint es implementar la funcionalidad de gestión de platos de comida con el fin de ayudar a los administradores en la creación y edición del menú del restaurante, permitiendo a los clientes conocer una gran variedad de platos antes de pedir comida a domicilio o probarla en el establecimiento. Para lograr este objetivo, se ha seleccionado la siguiente Feature, junto con sus respectivas User Stories:

- FE4: Gestión de platos de comida
 - US12: Crear platos de comida
 - US13: Ver platos de comida (administrador)
 - US14: Actualizar platos de comida
 - US15: Eliminar platos de comida
 - US16: Ver platos de comida (cliente)

Al igual que en los Sprints anteriores, se ha creado en Jira el Sprint Backlog correspondiente para este Sprint (véase Ilustración 77 y 78).

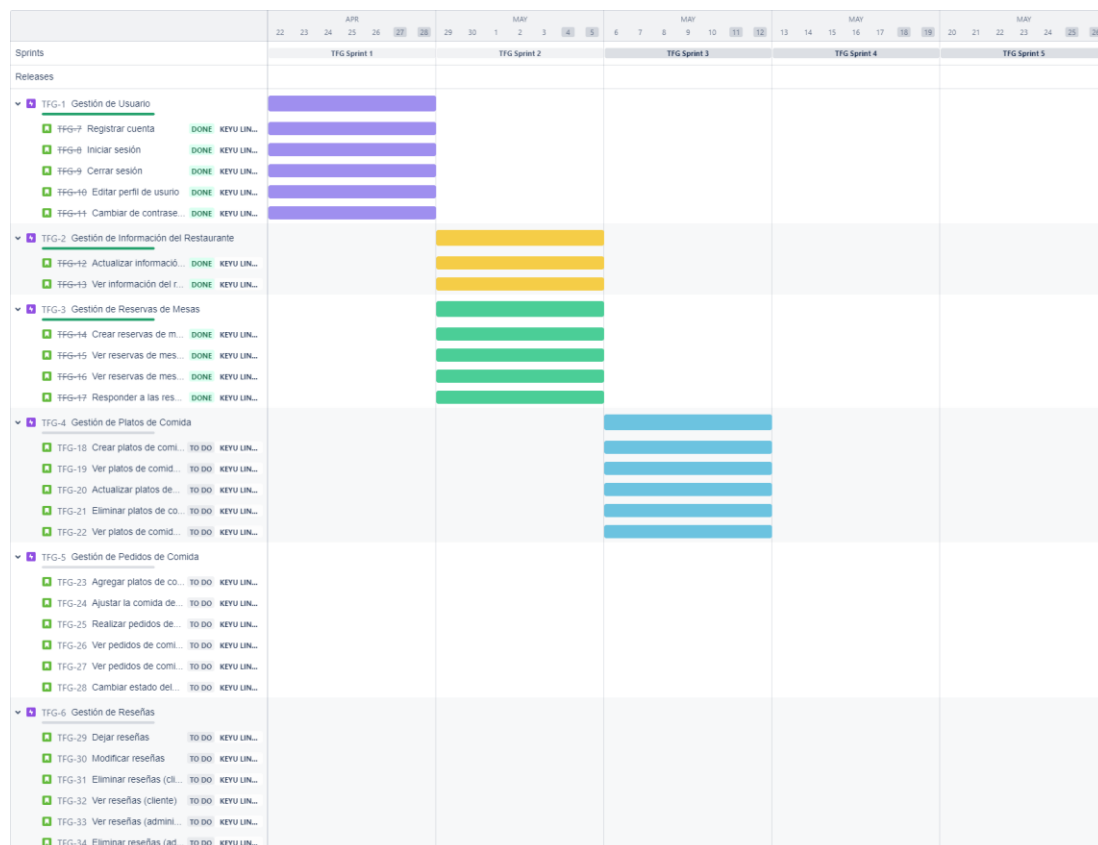


Ilustración 77: Planificación del Sprint 3.

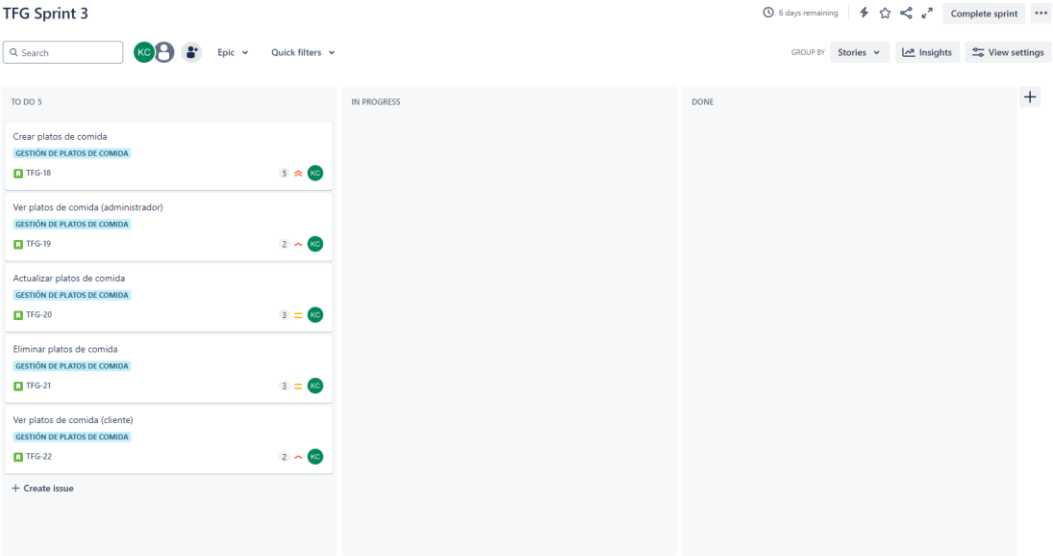


Ilustración 78: Planificación del Sprint 3.

5.7.2 Diseño

5.7.2.1 Diseño Front-End

5.7.2.1.1 Diagrama de componentes y prototipos de interfaz de usuario

Para este Sprint, se ha desarrollado los siguientes diagramas de componentes (véase Ilustración 79 y 80):

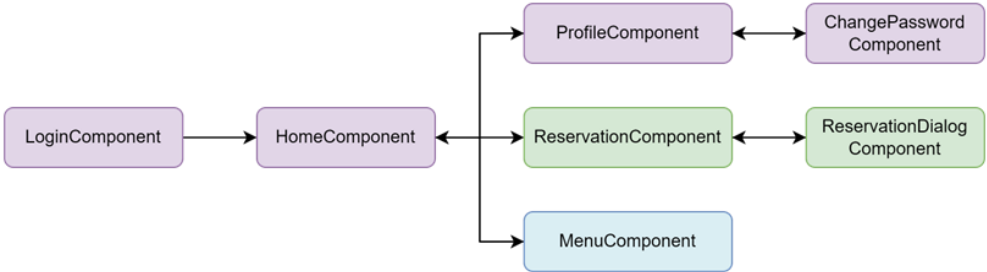


Ilustración 79: Diagrama de componentes para los clientes del Sprint 3.

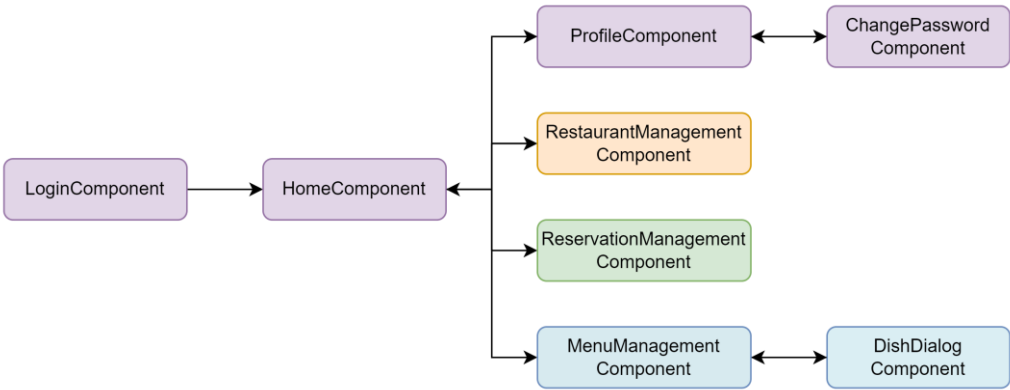


Ilustración 80: Diagrama de componentes para el administrador del Sprint 3.

A continuación, se detallan los componentes y sus relaciones:

- **MenuManagementComponent:** Componente donde el administrador puede gestionar todos los platos del menú (véase Ilustración 81). Desde este componente, el administrador puede ir a HomeComponent o a DishDialogComponent para crear o editar un plato de comida.

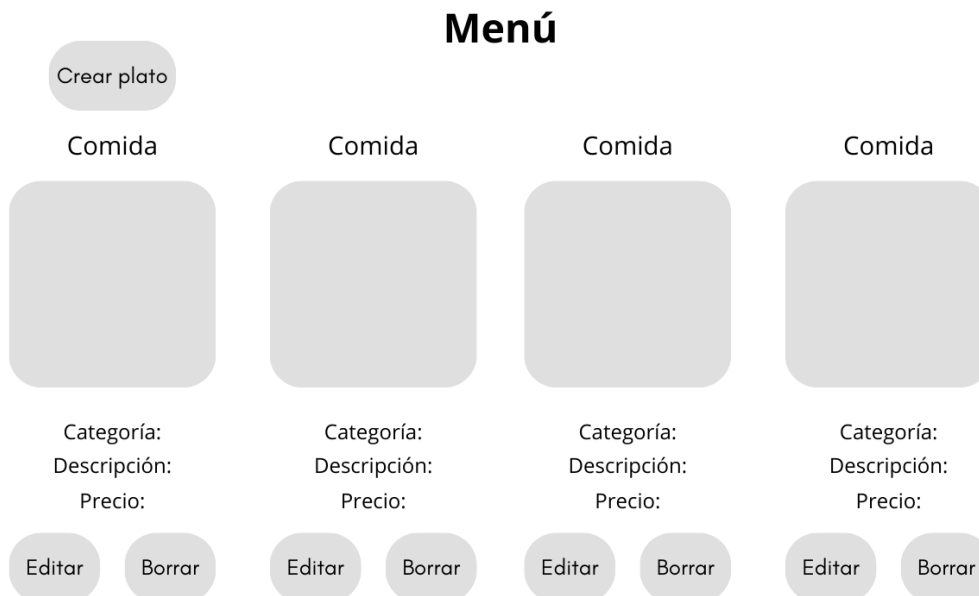


Ilustración 81: MenuManagementComponent.

- **DishDialogComponent:** Componente donde el administrador puede crear o editar un plato de comida (véase Ilustración 82). Desde este componente, el administrador puede ir a HomeComponent o a DishDialogComponent para crear o editar un palto de comida.



Ilustración 82: DishDialogComponent

- **MenuComponent:** Componente donde los clientes pueden visualizar y añadir al carrito los diferentes platos de comida (véase Ilustración 83). Desde este componente, los clientes pueden ir a HomeComponent.



Ilustración 83: MenuComponent.

5.7.2.2 Diseño Back-End

5.7.2.2.1 Diseño BBDD

Para este Sprint, se creará la entidad "Dish" (véase Ilustración 84):

- **Dish.** La entidad "Dish" representa los distintos platos ofrecidos en el menú del restaurante. Cada plato tiene su propia información, lo que facilita la consulta por parte de los clientes. La entidad "Dish" contiene estos campos:
 - *id*: Identificador único que se asigna a cada plato. Este campo sirve como clave primaria en la base de datos (entero largo, autoincremental).
 - *price*: Precio del plato (real).
 - *category*: Categoría del plato (cadena de caracteres).
 - *description*: Descripción del plato (cadena de caracteres).
 - *image_url*: URL de una imagen del plato (cadena de caracteres).
 - *name*: Nombre del plato (cadena de caracteres).

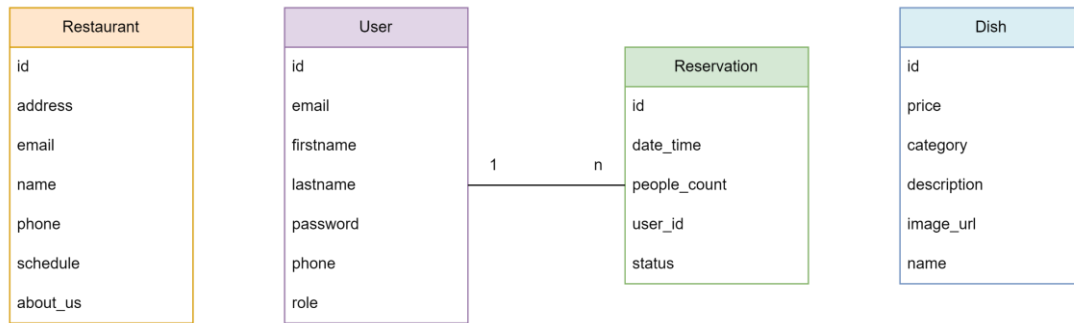


Ilustración 84: Diseño de BBDD del Sprint 3.

5.7.2.2.2 Diseño API

Para este Sprint, se han diseñado los siguientes endpoints:

URI	http://localhost:8080/api/v1/dishes	
Método	POST	
Descripción	Crea un nuevo plato en el sistema.	
Autenticación	Bearer Token (Rol administrador)	
Cabecera	Authorization: Bearer {token} Content-Type: application/json	
Cuerpo de petición	<pre> { "name": "Nombre del plato", "category": "Categoría del plato", "description": "Descripción del plato", "price": "Precio del plato", "imageUrl": "URL de la imagen del plato" } </pre>	
Parámetros de consulta	Ninguno	
Código de respuesta	201: Created	Se ha creado un plato con éxito.
	400: Bad Request	Parámetros no válidos.
	401: Unauthorized	Token de acceso no válido.
	403: Forbidden	Acceso denegado.
	500: Internal Server Error	Error interno del servidor.

Tabla 41: POST – Dishes.

URI	http://localhost:8080/api/v1/dishes/{idDish}	
Método	PATCH	
Descripción	Actualiza la información de un plato existente en el sistema.	
Autenticación	Bearer Token (Rol administrador)	
Cabecera	Authorization: Bearer {token} Content-Type: application/json	
Cuerpo de petición	<pre>{ "name": "Nombre del plato", "category": "Categoría del plato", "description": "Descripción del plato", "price": "Precio del plato", "imageUrl": "URL de la imagen del plato" }</pre>	
Parámetros de consulta	Ninguno	
Código de respuesta	200: OK	Se ha actualizado el plato con éxito.
	400: Bad Request	Parámetros no válidos.
	401: Unauthorized	Token de acceso no válido.
	403: Forbidden	Acceso denegado.
	404: Not Found	Plato no encontrado.
	500: Internal Server Error	Error interno del servidor.

Tabla 42: PATCH – Dishes.

URI	http://localhost:8080/api/v1/dishes	
Método	GET	
Descripción	Obtiene una lista de todos los platos existentes en el sistema.	
Autenticación	Bearer Token (Rol administrador)	
Cabecera	Authorization: Bearer {token} Content-Type: application/json	
Cuerpo de petición	Ninguno	
Parámetros de consulta	Ninguno	
Código de respuesta	200: OK	Se obtiene la lista de platos con éxito.
	401: Unauthorized	Token de acceso de válido.

	403: Forbidden	Acceso denegado.
	500: Internal Server Error	Error interno del servidor.

Tabla 43: GET – Dishes.

URI	http://localhost:8080/api/v1/dishes/{idDish}	
Método	DELETE	
Descripción	Elimina un plato en concreto.	
Autenticación	Bearer Token (Rol administrador)	
Cabecera	Authorization: Bearer {token} Content-Type: application/json	
Cuerpo de petición	Ninguno	
Parámetros de consulta	Ninguno	
Código de respuesta	200: OK	Se elimina el plato con éxito.
	401: Unauthorized	Token de acceso no válido.
	403: Forbidden	Acceso denegado.
	404: Not Found	Plato no encontrado.
	500: Internal Server Error	Error interno del servidor.

Tabla 44: DELETE – Dishes.

URI	http://localhost:8080/api/v1/images	
Método	POST	
Descripción	Sube una nueva imagen al sistema.	
Autenticación	Bearer Token (Rol cliente o administrador)	
Cabecera	Authorization: Bearer {token} Content-Type: application/json	
Cuerpo de petición	Form-data: Key: multipartFile Value: comida.png	
Parámetros de consulta	Ninguno	
Código de respuesta	201: Created	Se ha subido la imagen con éxito.
	400: Bad Request	Parámetros no válidos.
	401: Unauthorized	Token de acceso no válido.
	403: Forbidden	Acceso denegado.
	500: Internal Server Error	Error interno del servidor.

Tabla 45: POST – Images.

URI	http://localhost:8080/api/v1/images/{nameImage}	
Método	GET	
Descripción	Obtiene una imagen en concreto del sistema.	
Autenticación	Bearer Token (Rol cliente o administrador)	
Cabecera	Authorization: Bearer {token} Content-Type: application/json	
Cuerpo de petición	Ninguno	
Parámetros de consulta	Ninguno	
Código de respuesta	200: OK	Se ha obtenido la imagen con éxito.
	401: Unauthorized	Token de acceso no válido.
	403: Forbidden	Acceso denegado.
	404: Not Found	Imagen no encontrado.
	500: Internal Server Error	Error interno del servidor.

Tabla 46: GET – Images.

5.7.3 Pruebas de sistema

Para este Sprint, se ha llevado a cabo las siguientes pruebas de sistema:

- **PS23_US12 – Creación de platos de comida correcta**
 - Descripción: El administrador rellena los datos de plato de comida de manera correcta.
 - Resultado esperado: El sistema crea el plato de comida, actualiza la lista de platos de comida y muestra un mensaje de que se ha creado el plato con éxito.

Ilustración 85: Creación de platos de comida correcta 1.

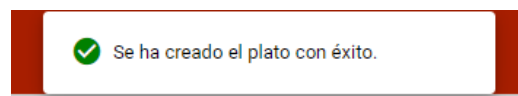


Ilustración 86: Creación de platos de comida correcta 2.

- **PS24_US12 – Creación de platos de comida incorrecta**

- Descripción: El administrador introduce un número negativo en el precio del plato.
- Resultado esperado: El sistema muestra al administrador un mensaje de error de que el precio no puede ser negativo y deshabilitará el botón de crear plato.
-

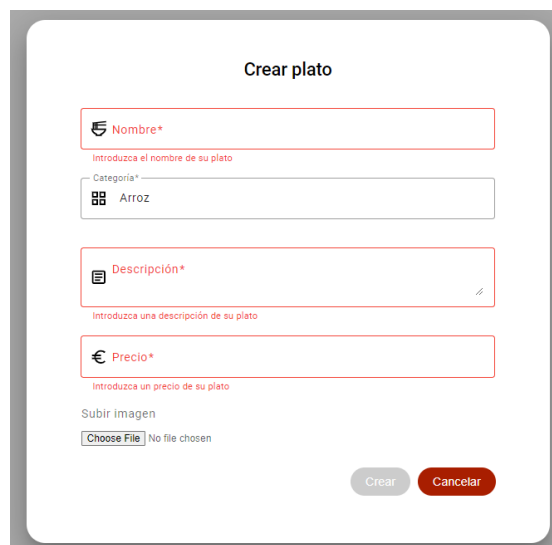


The screenshot shows a web form titled "Crear plato". It contains several input fields: "Nombre*" with the value "Tempura de langostino", "Categoría*" with the value "Entremeses", and "Descripción*" with the value "Langostinos rebozados acompañados de salsa mayonesa.". The "Precio*" field is highlighted with a red border and contains the value "€ -7". Below this field, a red error message reads "El precio no puede ser negativo". At the bottom, there is a "Subir imagen" section with a "Choose File" button and the filename "tempura-de-langostino.jpeg". Two buttons, "Crear" (disabled) and "Cancelar", are at the bottom right.

Ilustración 87: Creación de platos de comida incorrecta.

- **PS25_US12 – Creación de platos de comida incompleta**

- Descripción: El administrador le falta introducir algunos campos obligatorios para poder crear un plato de comida.
- Resultado esperado: El sistema muestra al administrador los campos obligatorios que debe introducir y deshabilitará el botón de crear plato.



The screenshot shows the "Crear plato" form with several fields highlighted in red to indicate they are required. The "Nombre*" field is empty with the placeholder "Introduzca el nombre de su plato". The "Categoría*" field contains "Arroz". The "Descripción*" field is empty with the placeholder "Introduzca una descripción de su plato". The "Precio*" field is empty with the placeholder "Introduzca un precio de su plato". Below these fields is the "Subir imagen" section with a "Choose File" button and the text "No file chosen". The "Crear" button is disabled, while the "Cancelar" button is active.

Ilustración 88: Creación de platos de comida incompleta.

- **PS26_US13 – Consulta de platos de comida (administrador)**
 - Descripción: El administrador visualiza la lista de platos de comida que ha creado para su menú.
 - Resultado esperado: El sistema muestra al administrador la lista de platos de comida que ha creado.

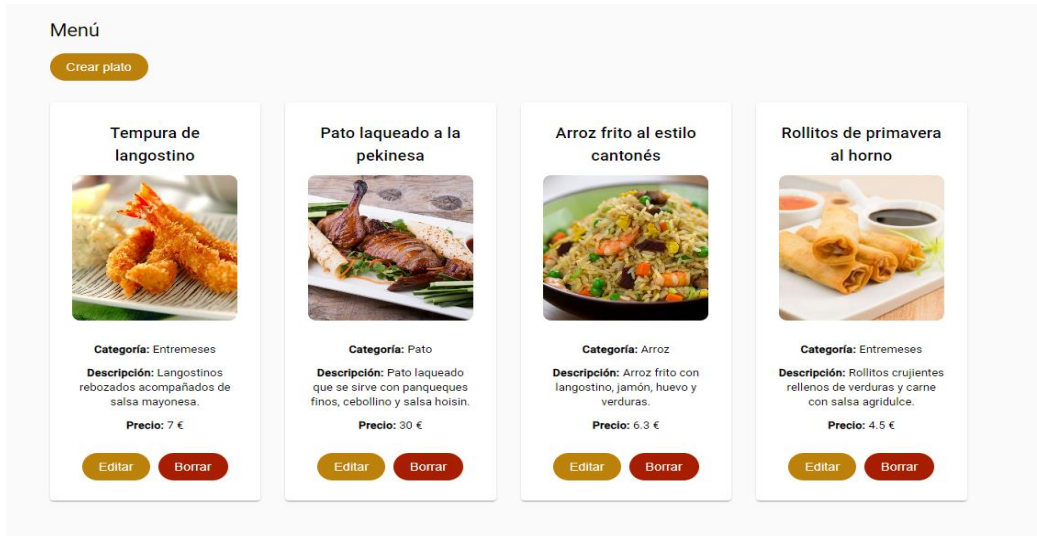


Ilustración 89: Consulta de platos de comida (administrador).

- **PS27_US14 – Actualización de platos de comida correcta**
 - Descripción: El administrador rellena los datos de plato de comida de manera correcta.
 - Resultado esperado: El sistema actualiza el plato de comida, muestra la lista de platos de comida actualizada y muestra un mensaje de que se ha actualizado el plato con éxito.

The screenshot shows a form titled 'Editar plato'. It has the following fields and elements:

- Nombre***: A text input field containing 'Rollitos de primavera al horno'.
- Categoría***: A dropdown menu showing 'Entremeses'.
- Descripción***: A text area containing 'Rollitos crujientes rellenos de verduras y carne con salsa agri dulce.'
- Precio***: A text input field with a Euro symbol and the value '2.5'.
- Subir imagen**: A section with a 'Choose File' button and the text 'No file chosen'.
- Buttons**: 'Guardar' (Save) and 'Cancelar' (Cancel) buttons at the bottom right.

Ilustración 90: Actualización de platos de comida correcta 1.

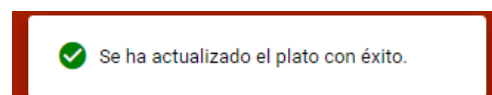


Ilustración 91: Actualización de platos de comida correcta 2.

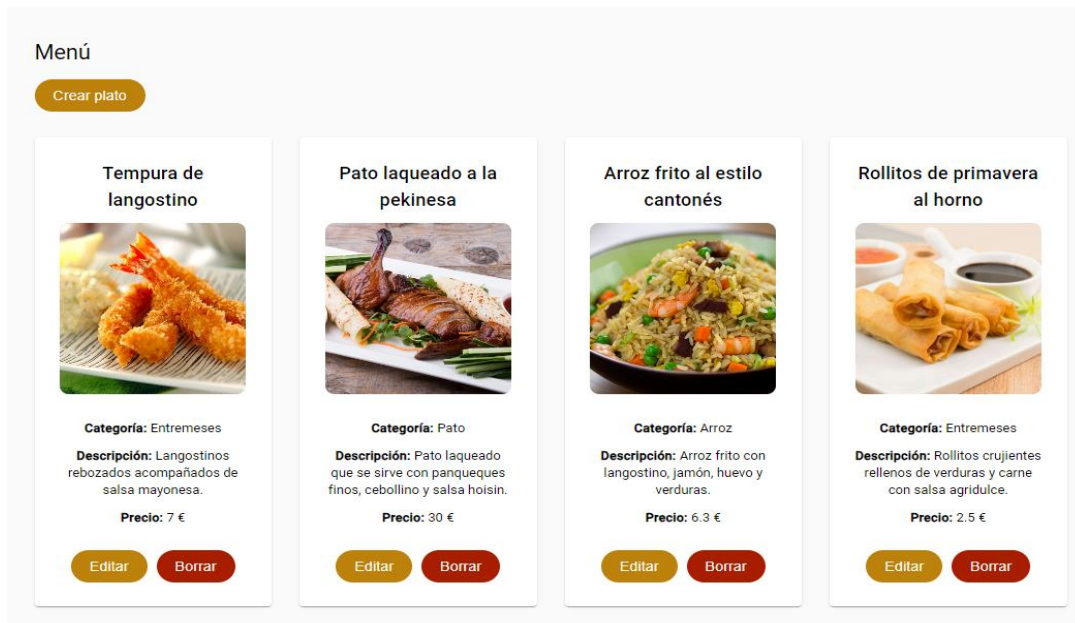


Ilustración 92: Actualización de platos de comida correcta 3.

- **PS28_US14 – Actualización de platos de comida incorrecta**
 - Descripción: El administrador introduce un número negativo en el precio del plato al intentar actualizarlo.
 - Resultado esperado: El sistema muestra al administrador un mensaje de error de que el precio no puede ser negativo y deshabilitará el botón de actualizar plato.

The screenshot shows the 'Editar plato' (Edit dish) form. The 'Nombre*' field contains 'Rollitos de primavera al horno', 'Categoría*' contains 'Entremeses', and 'Descripción*' contains 'Rollitos crujientes rellenos de verduras y carne con salsa agri dulce'. The 'Precio*' field contains '-4.5' and is highlighted with a red border. Below the field, a red error message reads 'El precio no puede ser negativo'. At the bottom, there is a 'Subir imagen' section with a 'Choose File' button and 'No file chosen' text, and two buttons: 'Guardar' (Save) and 'Cancelar' (Cancel).

Ilustración 93: Actualización de platos de comida incorrecta.

- **PS29_US14 – Actualización de platos de comida incompleta**
 - Descripción: El administrador le falta introducir algunos campos obligatorios para poder actualizar un plato de comida.
 - Resultado esperado: El sistema muestra al administrador los campos obligatorios que debe introducir y deshabilitará el botón de actualizar plato.

Editar plato

Nombre*
Rollitos de primavera al horno

Categoría*
Introduce la categoría de su plato

Descripción*
Introduce una descripción de su plato

Precio*
Introduce un precio de su plato

Subir imagen
Choose File No file chosen

Guardar Cancelar

Ilustración 94: Actualización de platos de comida incompleta.

- **PS30_US15 – Eliminación de platos de comida**

- Descripción: El administrador elimina un plato de comida.
- Resultado esperado: El sistema muestra un diálogo de confirmación y si el administrador acepta, entonces el sistema elimina el plato indicado y le mostrará un mensaje de que el plato se ha eliminado con éxito.

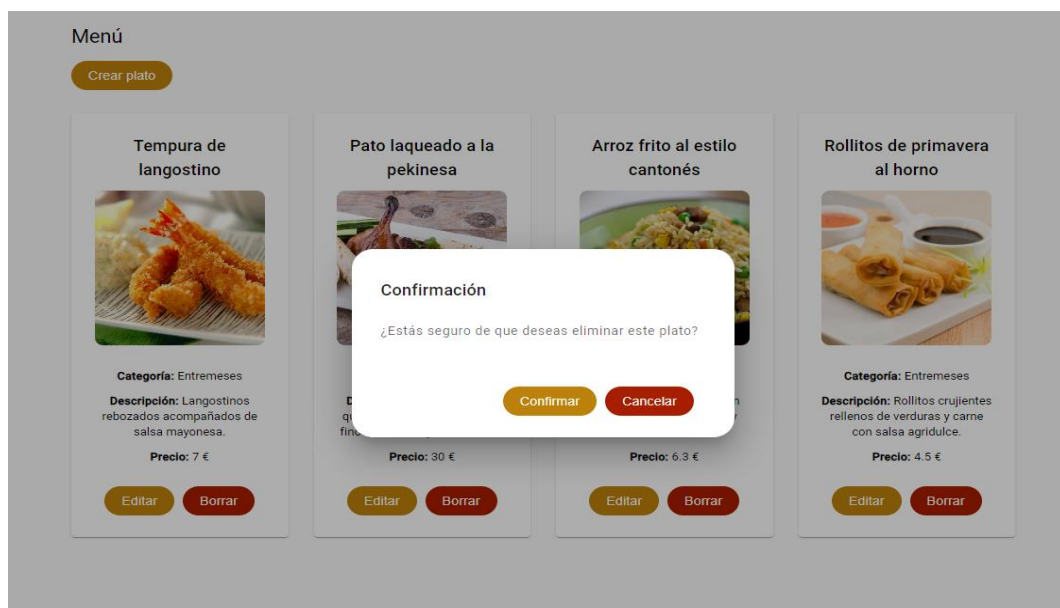


Ilustración 95: Eliminación de platos de comida 1.

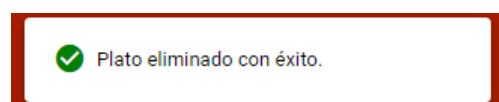


Ilustración 96: Eliminación de platos de comida 2.

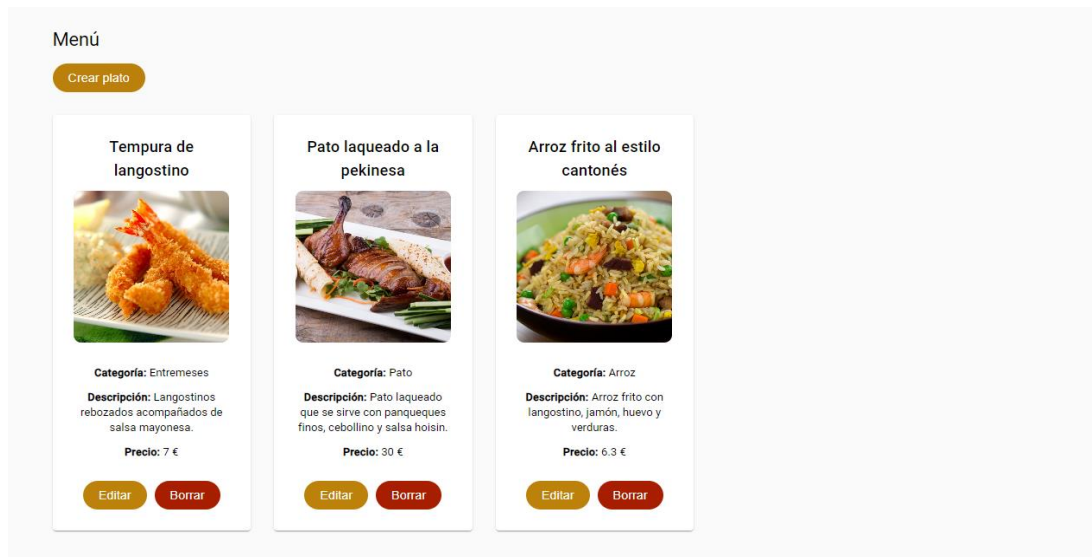


Ilustración 97: Eliminación de platos de comida 3.

- **PS31_US16 – Consulta de platos de comida (cliente)**
 - Descripción: El cliente visualiza la lista de platos de comida que ha creado el administrador.
 - Resultado esperado: El sistema muestra al cliente la lista de platos de comida que ha creado el administrador.

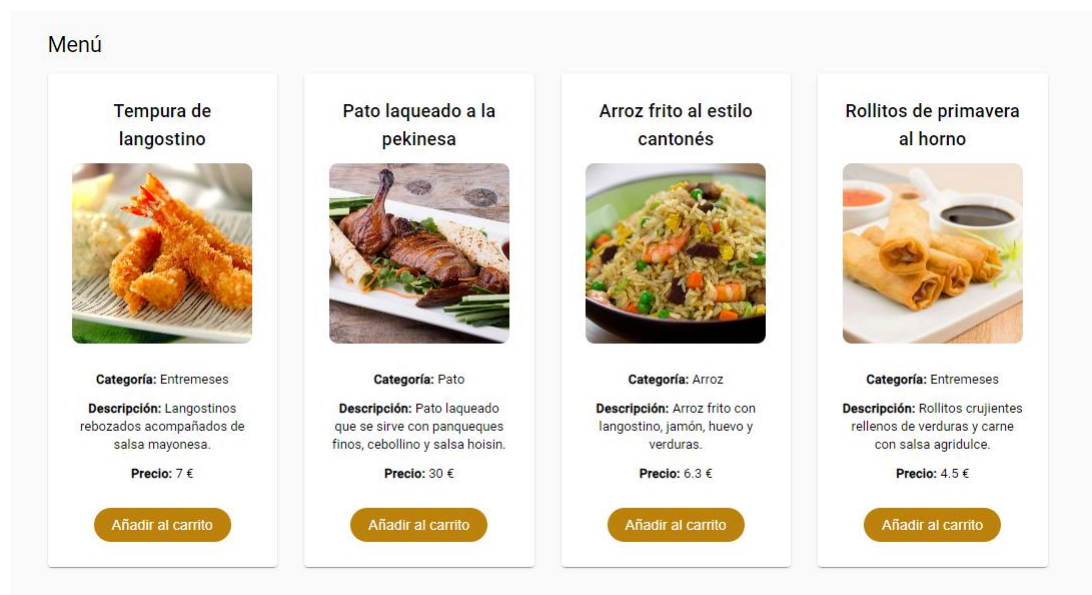


Ilustración 98: Consulta de platos de comida (cliente).

5.7.4 Revisión y Retrospectiva del Sprint

Durante este Sprint, se ha logrado cumplir con todos los objetivos establecidos en la planificación del Sprint, implementando todas las funcionalidades de la gestión de platos de comida (creación, edición, eliminación y consulta de platos). Para lograrlo, se ha creado diagramas de componentes y prototipos y se ha documentado los diseños de la base de datos y de las APIs, además de realizar

un conjunto de pruebas de sistema para asegurar el correcto funcionamiento de la aplicación. El feedback recibido por parte de los dueños del restaurante fue positivo y no se identificaron áreas de mejora. Por último, en Jira se ha movido todas las User Stories completadas a “Done” (véase Ilustración 99) y se proporciona el Product Backlog tras finalizar el Sprint 3 (véase Ilustración 100).

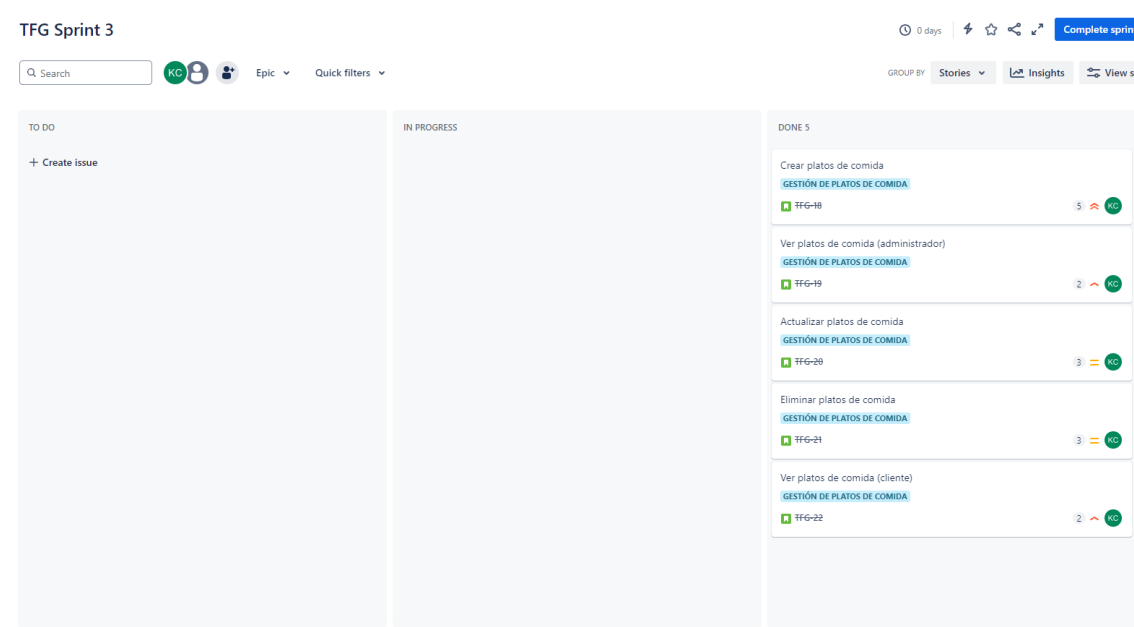


Ilustración 99: Tablero de Sprint 3 hecho.

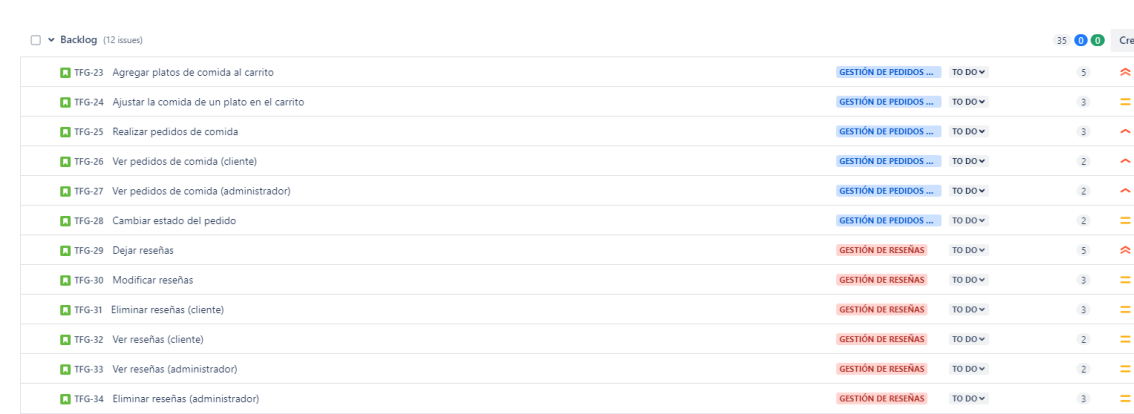


Ilustración 100: Product Backlog tras Sprint 3.

Al igual que el Sprint anterior, en cuanto a la retrospectiva del Sprint, se ha llegado a la conclusión de que el enfoque actual es efectivo en general, pero sigue habiendo espacio para mejoras de tiempo.

5.8 Sprint 4

5.8.1 Planificación del Sprint

El **objetivo principal** de este cuarto Sprint es implementar la funcionalidad de gestión de pedidos de comida con el fin de agilizar este proceso y permitir a los clientes pedir comida a domicilio. Para lograrlo, se ha seleccionado la siguiente Feature, junto con sus respectivas User Stories:

- FE4: Gestión de pedidos de comida
 - US17: Agregar platos de comida al carrito
 - US18: Ajustar la cantidad de un plato en el carrito
 - US19: Realizar pedidos de comida
 - US20: Ver pedidos de comida (cliente)
 - US21: Ver pedidos de comida (administrador)
 - US22: Cambiar estado del pedido

Al igual que en los Sprints anteriores, se ha creado en Jira el Sprint Backlog correspondiente para este Sprint (véase Ilustración 101 y 102).

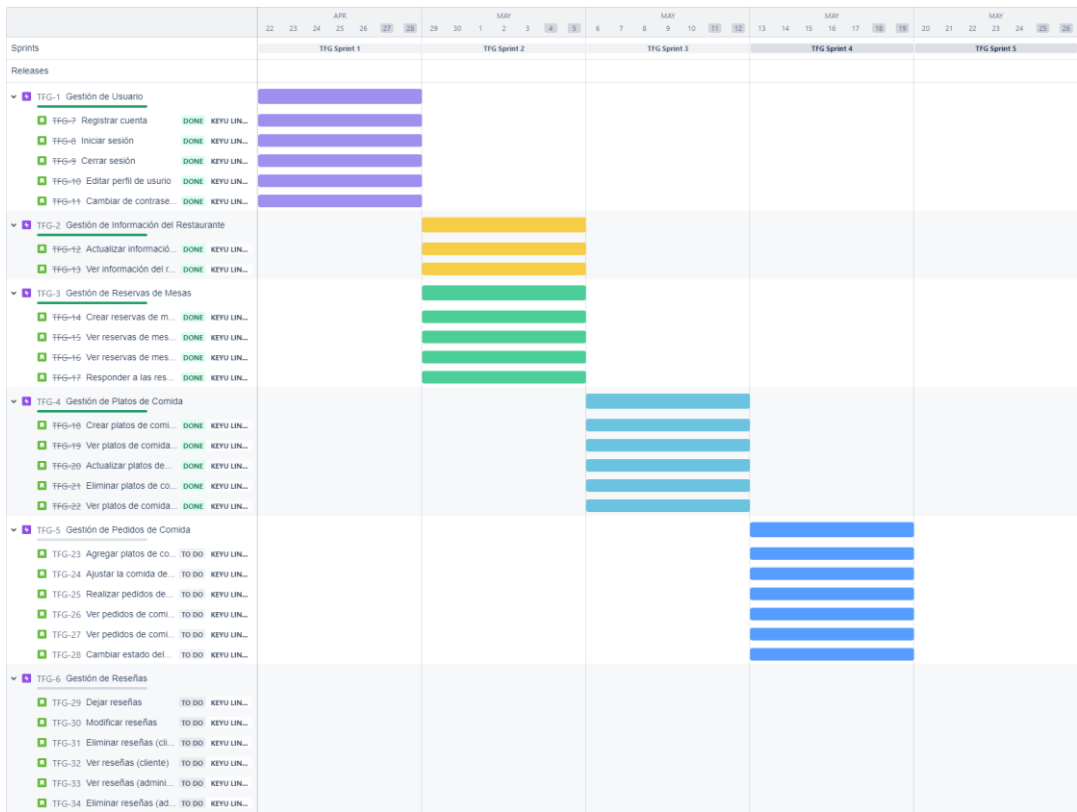


Ilustración 101: Planificación del Sprint 4.

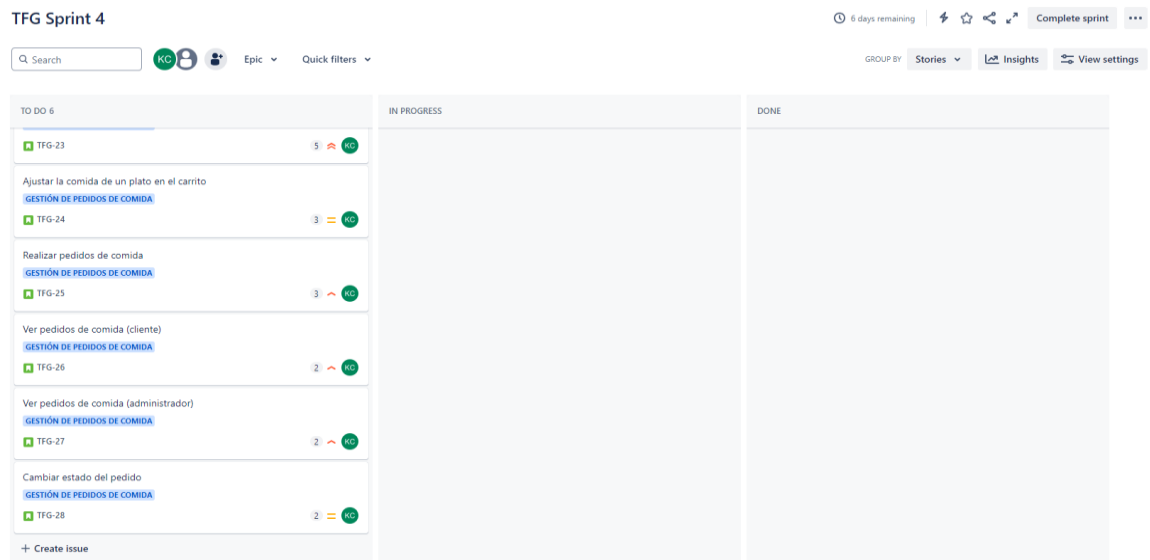


Ilustración 102: Tablero de Sprint 4 por hacer.

5.8.2 Diseño

5.8.2.1 Diseño Front-End

5.8.2.1.1 Diagrama de componentes y prototipos de interfaz de usuario

Para este Sprint, se ha desarrollado los siguientes diagramas de componentes (véase Ilustración 103 y 104):

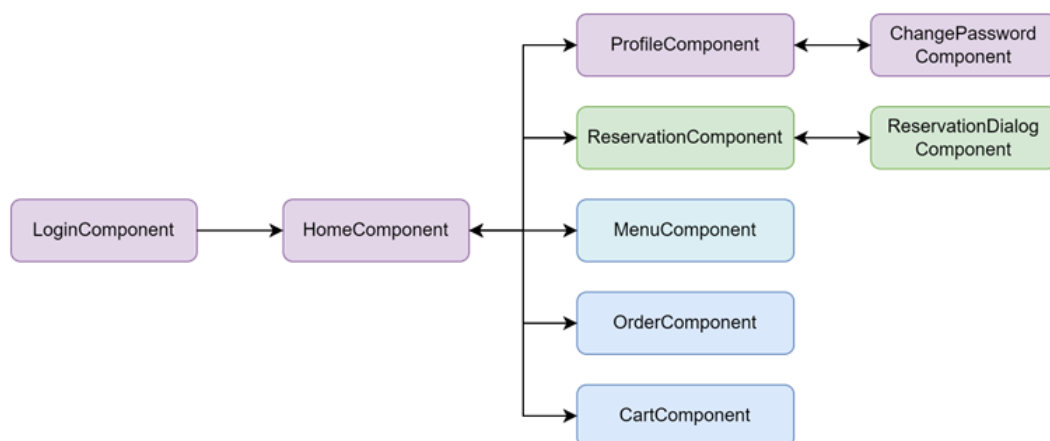


Ilustración 103: Diagrama de componentes para los clientes del Sprint 4.

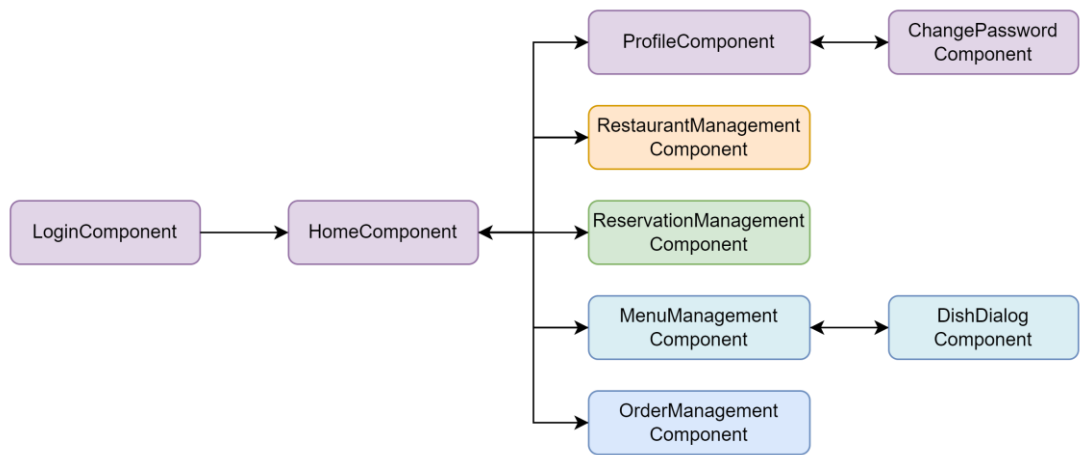


Ilustración 104: Diagrama de componentes para el administrador del Sprint 4.

- **CartComponent:** Componente donde los clientes pueden visualizar y modificar los diferentes elementos de su carrito y proceder con el pedido de comida (véase Ilustración 105, 106, 107). Desde este componente, los clientes pueden ir a HomeComponent.
-

Carrito de compras

● Mi carrito ————— ● Información extra ————— ● Resumen

Mi carrito

Foto	Nombre del plato	Cantidad	Precio	Eliminar

Total: Siguiente

Ilustración 105: CartComponent 1.

Carrito de compras

● Mi carrito ————— ● Información extra ————— ● Resumen

Información extra

Método de entrega

● Entrega a domicilio ● Recoger en restaurante

Dirección completa

● Dirección	● Número
● Piso	● Puerta
● Código postal	

Método de pago

● Efectivo ● Tarjeta de crédito ● Pago móvil

Volver

Siguiente

Ilustración 106: CartComponent 2.

Carrito de compras

● Mi carrito ————— ● Información extra ————— ● Resumen

Resumen del pedido

Foto	Nombre del plato	Cantidad	Precio

Resumen de la información extra

Método de entrega

● Entrega a domicilio ● Recoger en restaurante

Método de pago

● Efectivo ● Tarjeta de crédito ● Pago móvil

Volver

Realizar pedido

Ilustración 107: CartComponent 3.

- **OrderComponent:** Componente donde los clientes pueden visualizar los diferentes pedidos de comida que ha realizado (véase Ilustración 108). Desde este componente, los clientes pueden ir a HomeComponent.

Mis pedidos

Fecha y hora del pedido	Tipo de entrega	Pedido	Precio total	Método de pago	Estado

Ilustración 108: OrderComponent.

- **OrderManagementComponent:** Componente donde el administrador puede gestionar los diferentes pedidos de comida de sus clientes (véase Ilustración 109). Desde este componente, el administrador puede ir a HomeComponent.

Lista de pedidos

Fecha y hora del pedido	Tipo de entrega	Pedido	Precio total	Método de pago	Nombre del cliente	Correo del cliente	Teléfono del cliente	Estado	Acciones

Ilustración 109: OrderManagementComponent.

5.8.2.2 Diseño Back-End

5.8.2.2.1 Diseño BBDD

Para este Sprint, se creará la entidad "Cart", "Cart_item", "Order" y "Order_item" (véase Ilustración 110):

- **Cart.** La entidad "Cart" representa al carrito de compras de un usuario en un sistema de pedidos online. Cada usuario tiene su propio carrito lo que permite almacenar sus elecciones antes de realizar una compra. La entidad "Cart" contiene estos campos:
 - *id*: Identificador único que se asigna a cada carrito. Este campo sirve como clave primaria en la base de datos (entero largo, autoincremental).
 - *user_id*: Identificador único del usuario al que pertenece el carrito. Cada carrito está vinculado a un usuario en concreto. Este campo es una clave foránea (entero largo).
- **Cart_item.** La entidad "Cart_item" representa los elementos individuales de un carrito de compras. Cada uno de estos elementos se asocia a un plato y a un carrito. La entidad "Cart_item" contiene estos campos:
 - *id*: Identificador único que se asigna a cada elemento del carrito. Este campo sirve como clave primaria en la base de datos (entero largo, autoincremental).
 - *quantity*: Cantidad de unidades de un plato agregado por el usuario (entero).
 - *cart_id*: Identificador único del carrito al que pertenece este elemento. Este campo es una clave foránea (entero largo).
 - *dish_id*: Identificador único del plato agregado al carrito. Este campo es una clave foránea (entero largo).
- **Order.** La entidad "Order" representa un pedido realizado por un usuario. La entidad "Order" contiene estos campos:
 - *id*: Identificador único que se asigna a cada pedido. Este campo sirve como clave primaria en la base de datos (entero largo, autoincremental).
 - *user_id*: Identificador único del usuario que ha realizado el pedido. Este campo es una clave foránea (entero largo).
 - *total_price*: Precio total del pedido (real).
 - *date_time*: Fecha y hora en que se realizó el pedido (fecha-hora).
 - *address*: Dirección de entrega del pedido (cadena de caracteres).
 - *delivery_type*: Tipo de entrega del pedido (cadena de caracteres).
 - *payment_method*: Método de pago para el pedido (cadena de caracteres).
 - *status*: Estado actual del pedido (cadena de caracteres).

- **Order_item.** La entidad “Order_item” representa un elemento dentro de un pedido realizado por un cliente. Cada uno estos elementos se asocia a un plato solicitado por el cliente. La entidad “Order_item” contiene estos campos:
 - *id*: Identificador único que se asigna a cada elemento del pedido. Este campo sirve como clave primaria en la base de datos (entero largo, autoincremental).
 - *order_id*: Identificador único del pedido. Este campo es una clave foránea (entero largo).
 - *dish_id*: Identificador único del plato. Este campo es una clave foránea (entero largo).
 - *quantity*: Cantidad de unidades de un plato agregado por el usuario en el pedido (entero).

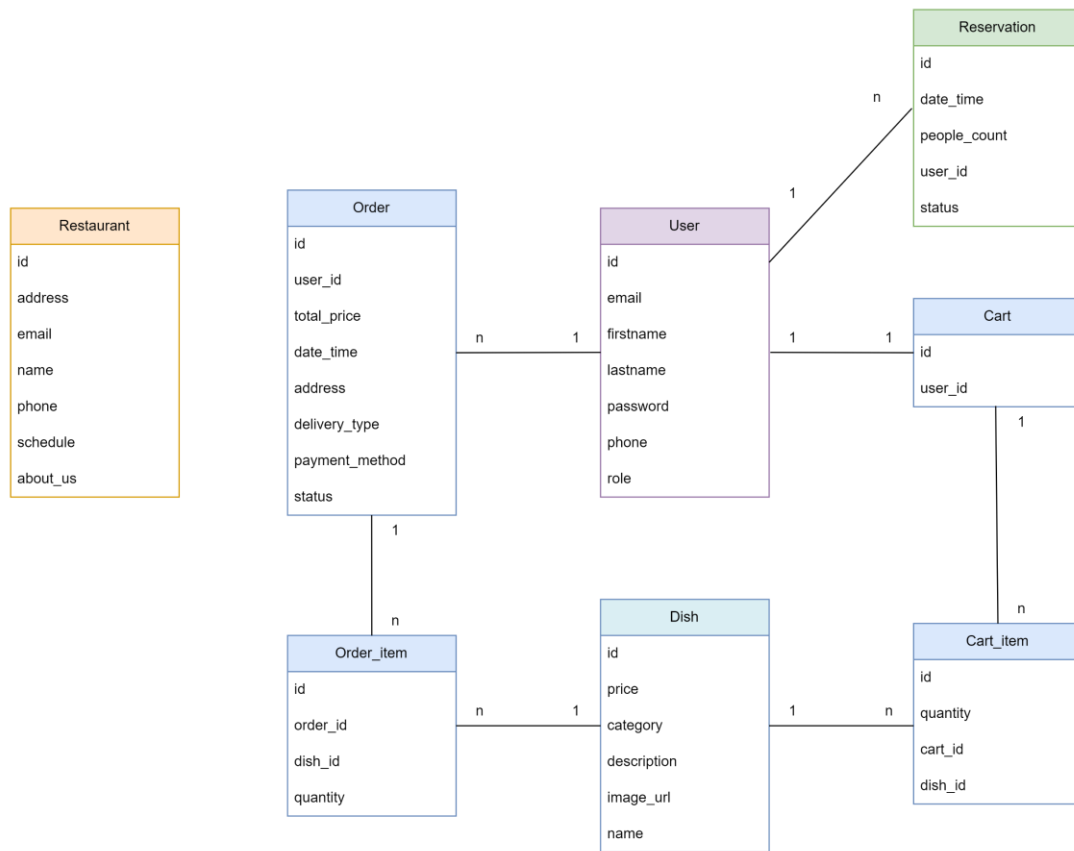


Ilustración 110: Diseño de BBDD del Sprint 4.

5.8.2.2.2 Diseño API

Para este Sprint, se han diseñado los siguientes endpoints:

URI	http://localhost:8080/api/v1/customers/me/cart
Método	POST
Descripción	Agrega un plato al carrito del cliente.
Autenticación	Bearer Token (Rol cliente)

Cabecera	Authorization: Bearer {token} Content-Type: application/json	
Cuerpo de petición	{ "dishId": 1, "quantity": 1 }	
Parámetros de consulta	Ninguno	
Código de respuesta	200: OK	Se ha agregado un plato al carrito del cliente con éxito.
	400: Bad Request	Parámetros no válidos.
	401: Unauthorized	Token de acceso no válido.
	403: Forbidden	Acceso denegado.
	404: Not Found	Usuario no encontrado.
	500: Internal Server Error	Error interno del servidor.

Tabla 47: POST – Cart.

URI	http://localhost:8080/api/v1/customers/me/cart	
Método	GET	
Descripción	Obtiene el carrito del cliente con una lista de platos de comida que se ha añadido anteriormente.	
Autenticación	Bearer Token (Rol cliente)	
Cabecera	Authorization: Bearer {token} Content-Type: application/json	
Cuerpo de petición	Ninguno	
Parámetros de consulta	Ninguno	
Código de respuesta	200: OK	Se ha obtenido la lista de elementos del carrito con éxito.
	401: Unauthorized	Token de acceso no válido.
	403: Forbidden	Acceso denegado.
	404: Not Found	Usuario no encontrado.
	500: Internal Server Error	Error interno del servidor.

Tabla 48: GET – Cart.

URI	http://localhost:8080/api/v1/customers/me/cart/{idCartItem}	
Método	PATCH	
Descripción	Actualiza la cantidad de un plato en el carrito del cliente.	
Autenticación	Bearer Token (Rol cliente)	
Cabecera	Authorization: Bearer {token} Content-Type: application/json	
Cuerpo de petición	{ "quantity": 3 }	
Parámetros de consulta	Ninguno	
Código de respuesta	200: OK	Se actualiza la cantidad de platos en el carrito con éxito.
	400: Bad Request	Parámetros no válidos.
	401: Unauthorized	Token de acceso no válido
	403: Forbidden	Acceso denegado.
	404: Not Found	Usuario no encontrado o elemento del carrito no encontrado.
	500: Internal Server Error	Error interno del servidor.

Tabla 49: PATCH – Cart.

URI	http://localhost:8080/api/v1/customers/me/cart/{idCartItem}	
Método	DELETE	
Descripción	Elimina un plato del carrito del cliente.	
Autenticación	Bearer Token (Rol cliente)	
Cabecera	Authorization: Bearer {token} Content-Type: application/json	
Cuerpo de petición	Ninguno	
Parámetros de consulta	Ninguno	
Código de respuesta	200: OK	Se ha eliminado el plato del carrito con éxito.
	401: Unauthorized	Token de acceso no válido.
	403: Forbidden	Acceso denegado.

	404: Not Found	Usuario no encontrado o elemento del carrito no encontrado.
	500: Internal Server Error	Error interno del servidor.

Tabla 50: DELETE – Cart.

URI	http://localhost:8080/api/v1/customers/me/orders	
Método	POST	
Descripción	Crea un nuevo pedido para el cliente.	
Autenticación	Bearer Token (Rol cliente)	
Cabecera	Authorization: Bearer {token} Content-Type: application/json	
Cuerpo de petición	<pre>{ "totalPrice": 59.99, "deliverytype": "homeDelivery", "address": "Dirección, Ciudad, País", "paymentMethod": "creditCard" }</pre>	
Parámetros de consulta	Ninguno	
Código de respuesta	201: Created	Se ha creado un nuevo pedido con éxito.
	400: Bad Request	Parámetros no válidos.
	401: Unauthorized	Token de acceso no válido.
	403: Forbidden	Acceso denegado.
	404: Not Found	Usuario no encontrado.
	500: Internal Server Error	Error interno del servidor.

Tabla 51: POST – Orders.

URI	http://localhost:8080/api/v1/customers/me/orders	
Método	GET	
Descripción	Obtiene los pedidos del cliente.	
Autenticación	Bearer Token (Rol cliente)	
Cabecera	Authorization: Bearer {token} Content-Type: application/json	
Cuerpo de petición	Ninguno	

Parámetros de consulta	Ninguno	
Código de respuesta	200: OK	Se obtiene los pedidos del cliente con éxito.
	401: Unauthorized	Token de acceso no válido.
	403: Forbidden	Acceso denegado.
	404: Not Found	Usuario no encontrado.
	500: Internal Server Error	Error interno del servidor.

Tabla 52: GET – MyOrders.

URI	http://localhost:8080/api/v1/orders	
Método	GET	
Descripción	Obtiene todos los pedidos.	
Autenticación	Bearer Token (Rol administrador)	
Cabecera	Authorization: Bearer {token} Content-Type: application/json	
Cuerpo de petición	Ninguno	
Parámetros de consulta	Ninguno	
Código de respuesta	200: OK	Se ha obtenido todos los pedidos con éxito.
	401: Unauthorized	Token de acceso no válido.
	403: Forbidden	Acceso denegado.
	500: Internal Server Error	Error interno del servidor.

Tabla 53: GET – AllOrders.

URI	http://localhost:8080/api/v1/orders/{idOrder}	
Método	PATCH	
Descripción	Actualiza el estado de un pedido.	
Autenticación	Bearer Token (Rol administrador)	
Cabecera	Authorization: Bearer {token} Content-Type: application/json	
Cuerpo de petición	{ "status": "PREPARING" }	
Parámetros de consulta	Ninguno	

Código de respuesta	200: OK	Se ha actualizado el estado de un pedido con éxito.
	400: Bad Request	Parámetros no válidos.
	401: Unauthorized	Token de acceso no válido.
	403: Forbidden	Acceso denegado.
	404: Not Found	El pedido no fue encontrado.
	500: Internal Server Error	Error interno del servidor.

Tabla 54: PATCH - Orders.

5.8.3 Prueba de sistema

Para este Sprint, se ha llevado a cabo las siguientes pruebas de sistema:

- **PS32_US17 – Adición de platos de comida al carrito**
 - Descripción: El cliente añade platos de comida al carrito desde el menú.
 - Resultado esperado: Si el cliente no ha añadido ningún plato al carrito, entonces el sistema le mostrará un mensaje de que el carrito se encuentra vacío. En caso contrario, el sistema creará el carrito con el plato añadido.

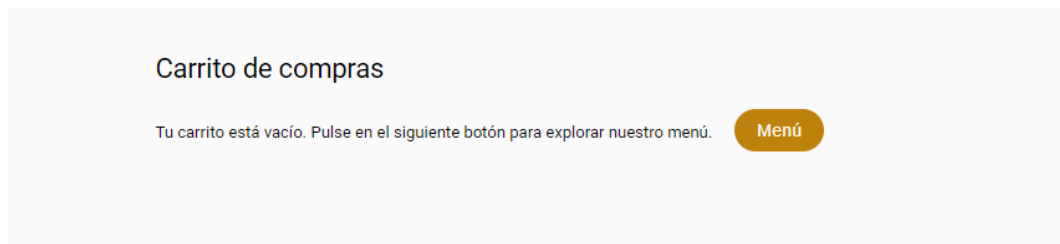


Ilustración 111: Adición de platos de comida al carrito 1.

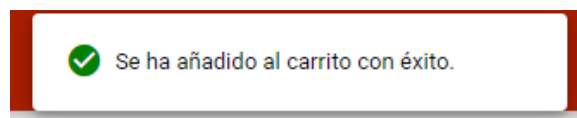


Ilustración 112: Adición de platos de comida al carrito 2:

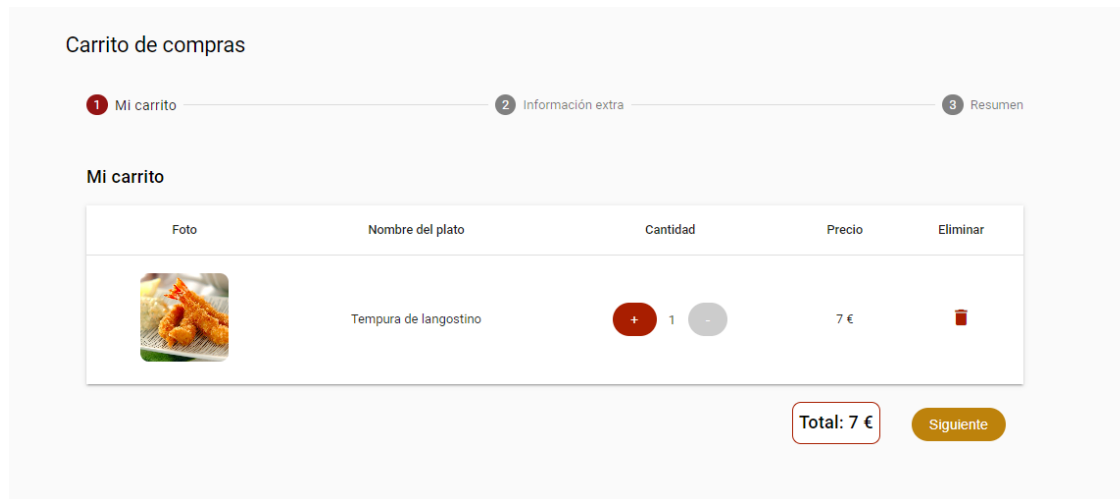


Ilustración 113: Adición de platos de comida al carrito 3.

- **PS33_US18 – Actualización de la cantidad de un plato en el carrito**
 - Descripción: El cliente modifica la cantidad de platos de comida añadidos anteriormente en el carrito.
 - Resultado esperado: El sistema permitirá al cliente aumentar o disminuir la cantidad de platos de comida, además de permitirle eliminar un plato de comida.

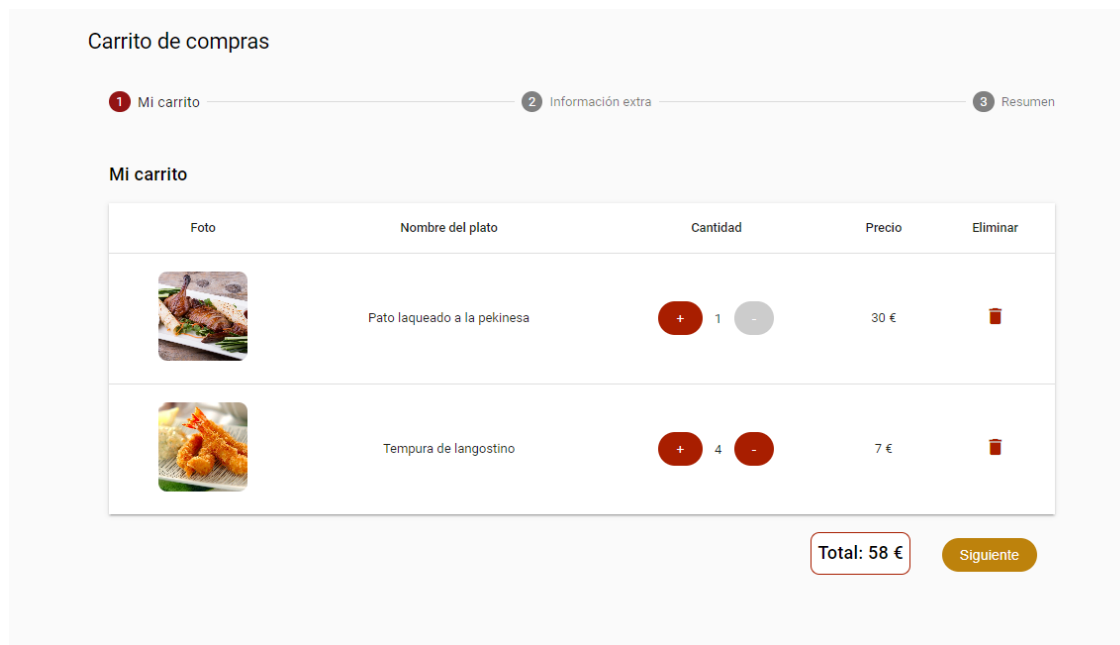


Ilustración 114: Actualización de la cantidad de un plato en el carrito 1.

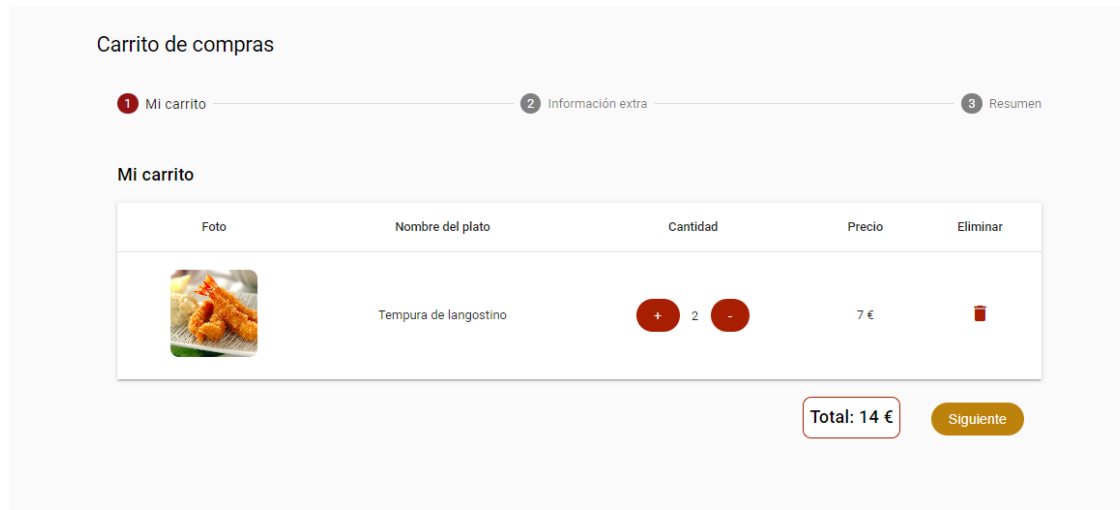


Ilustración 115: Actualización de la cantidad de un plato en el carrito 2.

- **PS34_US19 – Realización del pedido de comida correcta**
 - Descripción: Una vez que el cliente tenga el carrito deseado, rellena el formulario necesario para realizar un pedido de comida de manera correcta.
 - Resultado esperado: El sistema permitirá al cliente realizar el pedido, le redirigirá a la sección de pedidos y le mostrará un mensaje de que ha creado el pedido con éxito, además de mandar un correo electrónico al cliente.

Carrito de compras

1 Mi carrito 2 Información extra 3 Resumen

Información extra

Método de entrega

☒ Entrega a domicilio ☐ Recoger en restaurante

Dirección completa

Dirección* Número*

Piso* Puerta* Código postal*

Método de pago

☒ Efectivo ☐ Tarjeta de crédito ☐ Pago móvil

Volver Siguiente

Ilustración 116: Realización del pedido de comida correcta 1.

Carrito de compras

Mi carrito
Información extra
Resumen

Resumen de pedido

Foto	Nombre del plato	Cantidad	Precio
	Tempura de langostino	2	7 €

Total: 14 €

Resumen de la información extra

Método de entrega

☒ Entrega a domicilio
☐ Recoger en restaurante

Dirección completa

Dirección*

Recoletos

Número*

2

Piso*

2

Puerta*

A

Código postal*

28500

Método de pago

☒ Efectivo
☐ Tarjeta de crédito
☐ Pago móvil

Volver
Realizar pedido

Ilustración 117: Realización del pedido de comida correcta 2.

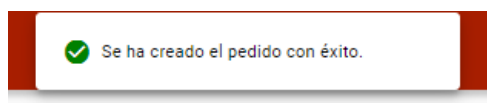


Ilustración 118: Realización del pedido de comida correcta 3.

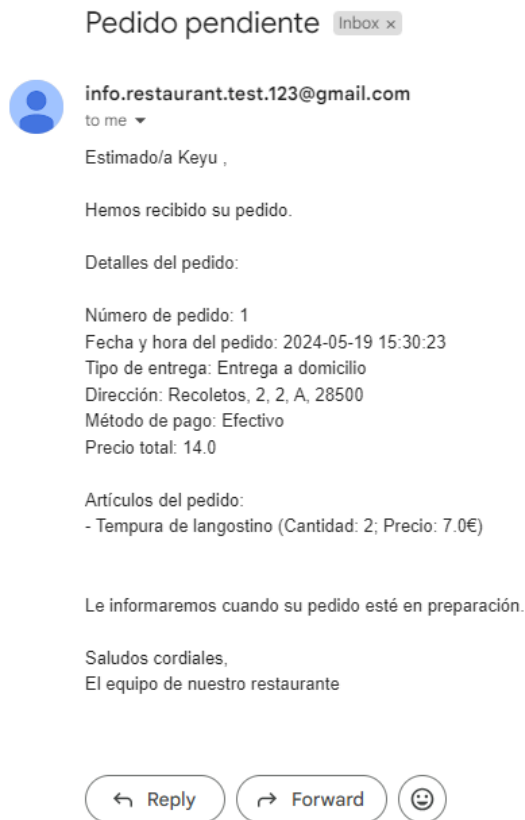


Ilustración 119: Realización del pedido de comida correcta 4.

Carrito de compras

Mi carrito

2 Información extra

3 Resumen

Información extra

Método de entrega

☐ Entrega a domicilio

☒ Recoger en restaurante

Dirección completa

Dirección*

Número*

Piso*

Puerta*

Código postal*

Método de pago

☐ Efectivo

☒ Tarjeta de crédito

☐ Pago móvil

Volver

Siguiente

Ilustración 120: Realización del pedido de comida correcta 5.

Carrito de compras

Mi carrito
Información extra
Resumen

Resumen de pedido

Foto	Nombre del plato	Cantidad	Precio
	Pato laqueado a la pekinesa	2	30 €
	Tempura de langostino	1	7 €

Total: 67 €

Resumen de la información extra

Método de entrega

☐ Entrega a domicilio
☒ Recoger en restaurante

Método de pago

☐ Efectivo
☒ Tarjeta de crédito
☐ Pago móvil

Volver
Realizar pedido

Ilustración 121: Realización del pedido de comida correcta 6.

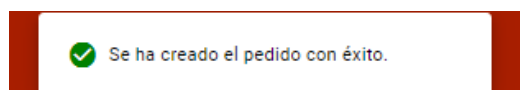


Ilustración 122: Realización del pedido de comida correcta 7.

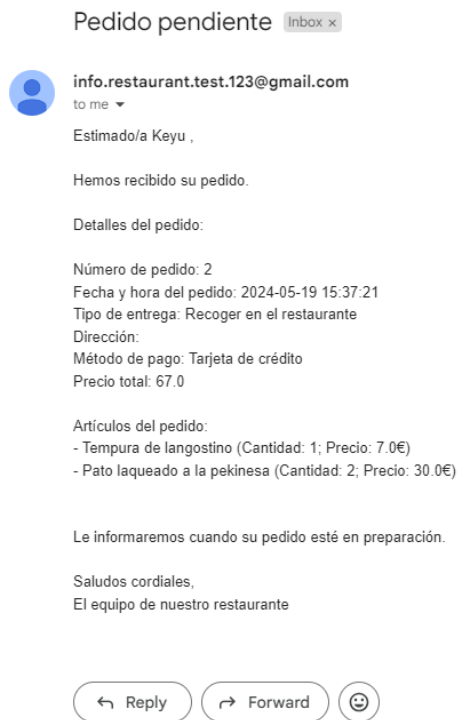


Ilustración 123: Realización del pedido de comida correcta 8.

- **PS35_US19 – Realización del pedido de comida incorrecta**
 - Descripción: Una vez que el cliente tenga el carrito deseado, rellena incorrectamente el campo de código postal del formulario de pedido.
 - Resultado esperado: El sistema mostrará al cliente el campo incorrecto y deshabilitará el botón de pasar al siguiente paso de la realización de pedido.

Carrito de compras

1 Mi carrito 2 Información extra 3 Resumen

Información extra

Método de entrega

☒ Entrega a domicilio ☐ Recoger en restaurante

Dirección completa

Dirección* Número*

Piso* Puerta* Código postal*

Un código postal tiene 5 dígitos

Método de pago

☒ Efectivo ☐ Tarjeta de crédito ☐ Pago móvil

Volver Siguiente

Ilustración 124: Realización del pedido de comida incorrecta.

- **PS36_US19 – Realización del pedido de comida incompleta**
 - Descripción: Una vez que el cliente tenga el carrito deseado, no rellena los campos obligatorios del formulario de pedido.
 - Resultado esperado: El sistema mostrará al cliente los campos obligatorios para poder realizar el pedido y deshabilitará el botón de pasar al siguiente paso de la realización de pedido.

Carrito de compras

1 Mi carrito 2 Información extra 3 Resumen

Información extra

Método de entrega

☒ Entrega a domicilio ☐ Recoger en restaurante

Dirección completa

Dirección*

Introduzca su dirección de domicilio

Número*

Introduzca su número

Piso*

Introduzca su piso

Puerta*

Introduzca su puerta

Código postal*

Introduzca su código postal

Método de pago

☒ Efectivo ☐ Tarjeta de crédito ☐ Pago móvil

Volver Siguiente

Ilustración 125: Realización del pedido de comida incompleta.

- **PS37_US20 – Consulta de pedidos de comida (cliente)**
 - Descripción: El cliente visualiza la lista de pedidos de comida que ha realizado.
 - Resultado esperado: El sistema muestra al cliente la lista de pedidos de comida que ha realizado.

Mis pedidos

ID	Fecha y hora del pedido	Tipo de entrega	Pedido	Precio total	Método de pago	Estado
2	2024-05-19 15:37:21	Recoger en el restaurante	1 x Tempura de langostino x 7 € 2 x Pato laqueado a la pekinesa x 30 €	67 €	Tarjeta de crédito	PENDIENTE
1	2024-05-19 15:30:23	Entrega a domicilio Recoletos, 2, 2, A, 28500	2 x Tempura de langostino x 7 €	14 €	Efectivo	PENDIENTE

Ilustración 126: Consulta de pedidos de comida (cliente).

- **PS38_US21 – Consulta de pedidos de comida (administrador)**
 - Descripción: El administrador visualiza la lista de pedidos de comida de sus clientes.
 - Resultado esperado: El sistema muestra al administrador la lista de pedidos de comida de sus clientes.

Lista de pedidos

ID	Fecha y hora del pedido	Tipo de entrega	Pedido	Precio total	Método de pago	Nombre del cliente	Correo del cliente	Teléfono del cliente	Estado	Acciones
2	2024-05-19 15:37:21	Recoger en el restaurante	1 x Tempura de langostino x 7 € 2 x Pato laqueado a la pelineasa x 30 €	67 €	Tarjeta de crédito	Keyu	keyu.cliente@gmail.com	123456789	PENDIENTE	<button>Preparando</button>
1	2024-05-19 15:30:23	Entrega a domicilio Recoletos, 2, 2, A, 28500	2 x Tempura de langostino x 7 €	14 €	Efectivo	Keyu	keyu.cliente@gmail.com	123456789	PENDIENTE	<button>Preparando</button>

Ilustración 127: Consulta de pedidos de comida (administrador).

- **PS39_US22 – Cambio de estado de pedidos de comida**
 - Descripción: El administrador cambia el estado del pedido que ha realizado el cliente.
 - Resultado esperado: El sistema actualiza el estado del pedido y envía un correo electrónico al cliente.

Lista de pedidos

ID	Fecha y hora del pedido	Tipo de entrega	Pedido	Precio total	Método de pago	Nombre del cliente	Correo del cliente	Teléfono del cliente	Estado	Acciones
2	2024-05-19 15:37:21	Recoger en el restaurante	1 x Tempura de langostino x 7 € 2 x Pato laqueado a la pelineasa x 30 €	67 €	Tarjeta de crédito	Keyu	keyu.cliente@gmail.com	123456789	PENDIENTE	<button>Preparando</button>
1	2024-05-19 15:30:23	Entrega a domicilio Recoletos, 2, 2, A, 28500	2 x Tempura de langostino x 7 €	14 €	Efectivo	Keyu	keyu.cliente@gmail.com	123456789	PREPARANDO	<button>En camino</button>

Ilustración 128: Cambio de estado de pedidos de comida 1.

Pedido en preparación

info.restaurant.test.123@gmail.com

to me

Estimado/a Keyu ,

Su pedido está en preparación.

Detalles del pedido:

Número de pedido: 1
Fecha y hora del pedido: 2024-05-19 15:30:23
Tipo de entrega: Entrega a domicilio
Dirección: Recoletos, 2, 2, A, 28500
Método de pago: Efectivo
Precio total: 14.0

Artículos del pedido:
- Tempura de langostino (Cantidad: 2; Precio: 7.0€)

Le informaremos cuando su pedido esté en camino.

Saludos cordiales,
El equipo de nuestro restaurante

Reply

Forward

Ilustración 129: Cambio de estado de pedidos de comida 2.

Lista de pedidos

ID	Fecha y hora del pedido	Tipo de entrega	Pedido	Precio total	Método de pago	Nombre del cliente	Correo del cliente	Teléfono del cliente	Estado	Acciones
2	2024-05-19 15:37:21	Recoger en el restaurante	1 x Tempura de langostino x 7 € 2 x Pato laqueado a la pelineña x 30 €	67 €	Tarjeta de crédito	Keyu	keyu.cliente@gmail.com	123456789	PENDIENTE	<button>Preparando</button>
1	2024-05-19 15:30:23	Entrega a domicilio Recoletos, 2, 2, A, 28500	2 x Tempura de langostino x 7 €	14 €	Efectivo	Keyu	keyu.cliente@gmail.com	123456789	EN CAMINO	<button>Entregado</button>

Ilustración 130: Cambio de estado de pedidos de comida 3.

Pedido en camino Inbox x



info.restaurant.test.123@gmail.com
to me ▾
Estimado/a Keyu ,

Su pedido está en camino.

Detalles del pedido:

Número de pedido: 1
Fecha y hora del pedido: 2024-05-19 15:30:23
Tipo de entrega: Entrega a domicilio
Dirección: Recoletos, 2, 2, A, 28500
Método de pago: Efectivo
Precio total: 14.0

Artículos del pedido:
- Tempura de langostino (Cantidad: 2; Precio: 7.0€)

Le informaremos cuando su pedido haya sido entregado.

Saludos cordiales,
El equipo de nuestro restaurante

 Reply

 Forward



Ilustración 131: Cambio de estado de pedidos de comida 4.

Lista de pedidos

ID	Fecha y hora del pedido	Tipo de entrega	Pedido	Precio total	Método de pago	Nombre del cliente	Correo del cliente	Teléfono del cliente	Estado	Acciones
2	2024-05-19 15:37:21	Recoger en el restaurante	1 x Tempura de langostino x 7 € 2 x Pato laqueado a la pelineña x 30 €	67 €	Tarjeta de crédito	Keyu	keyu.cliente@gmail.com	123456789	PENDIENTE	<button>Preparando</button>
1	2024-05-19 15:30:23	Entrega a domicilio Recoletos, 2, 2, A, 28500	2 x Tempura de langostino x 7 €	14 €	Efectivo	Keyu	keyu.cliente@gmail.com	123456789	ENTREGADO	

Ilustración 132: Cambio de estado de pedidos de comida 5.

Pedido entregado

info.restaurant.test.123@gmail.com

to me

Estimado/a Keyu ,

Su pedido ha sido entregado.

Detalles del pedido:

Número de pedido: 1

Fecha y hora del pedido: 2024-05-19 15:30:23

Tipo de entrega: Entrega a domicilio

Dirección: Recoletos, 2, 2, A, 28500

Método de pago: Efectivo

Precio total: 14.0

Artículos del pedido:

- Tempura de langostino (Cantidad: 2; Precio: 7.0€)

Esperamos que disfrute de su comida. Si tiene alguna pregunta, no dude en ponerse en contacto con nosotros.

Saludos cordiales,

El equipo de nuestro restaurante

Reply

Forward

Ilustración 133: Cambio de estado de pedidos de comida 6.

Lista de pedidos										
ID	Fecha y hora del pedido	Tipo de entrega	Pedido	Precio total	Método de pago	Nombre del cliente	Correo del cliente	Teléfono del cliente	Estado	Acciones
2	2024-05-19 15:37:21	Recoger en el restaurante	1 x Tempura de langostino x 7 € 2 x Pato laqueado a la pekinesa x 30 €	67 €	Tarjeta de crédito	Keyu	keyu.cliente@gmail.com	123456789	PREPARANDO	¡Listo para recoger!
1	2024-05-19 15:30:23	Entrega a domicilio Recoletos, 2, 2, A, 28500	2 x Tempura de langostino x 7 €	14 €	Efectivo	Keyu	keyu.cliente@gmail.com	123456789	ENTREGADO	

Ilustración 134: Cambio de estado de pedidos de comida 7.

Pedido en preparación

info.restaurant.test.123@gmail.com

to me

Estimado/a Keyu ,

Su pedido está en preparación.

Detalles del pedido:

Número de pedido: 2

Fecha y hora del pedido: 2024-05-19 15:37:21

Tipo de entrega: Recoger en el restaurante

Dirección:

Método de pago: Tarjeta de crédito

Precio total: 67.0

Artículos del pedido:

- Tempura de langostino (Cantidad: 1; Precio: 7.0€)

- Pato laqueado a la pekinesa (Cantidad: 2; Precio: 30.0€)

Le informaremos cuando su pedido esté lista para recoger.

Saludos cordiales,

El equipo de nuestro restaurante

Reply

Forward

Ilustración 135: Cambio de estado de pedidos de comida 8.

Lista de pedidos										
ID	Fecha y hora del pedido	Tipo de entrega	Pedido	Precio total	Método de pago	Nombre del cliente	Correo del cliente	Teléfono del cliente	Estado	Acciones
2	2024-05-19 15:37:21	Recoger en el restaurante	1 x Tempura de langostino x 7 € 2 x Pato laqueado a la pekinesa x 30 €	67 €	Tarjeta de crédito	Keyu	keyu.cliente@gmail.com	123456789	ENTREGADO	
1	2024-05-19 15:30:23	Entrega a domicilio Recóletos, 2, 2. A, 28500	2 x Tempura de langostino x 7 €	14 €	Efectivo	Keyu	keyu.cliente@gmail.com	123456789	ENTREGADO	

Ilustración 136: Cambio de estado de pedidos de comida 9.

Pedido listo para recoger

info.restaurant.test.123@gmail.com

to me

Estimado/a Keyu ,

Su pedido está listo para recoger.

Detalles del pedido:

Número de pedido: 2

Fecha y hora del pedido: 2024-05-19 15:37:21

Tipo de entrega: Recoger en el restaurante

Dirección:

Método de pago: Tarjeta de crédito

Precio total: 67.0

Artículos del pedido:

- Tempura de langostino (Cantidad: 1; Precio: 7.0€)

- Pato laqueado a la pekinesa (Cantidad: 2; Precio: 30.0€)

Si tiene alguna pregunta, no dude en ponerse en contacto con nosotros.

Saludos cordiales,

El equipo de nuestro restaurante

Reply

Forward

Ilustración 137: Cambio de estado de pedidos de comida 10.

5.8.4 Revisión y Retrospectiva del Sprint

Durante este Sprint, se ha logrado cumplir con todos los objetivos establecidos en la planificación del Sprint, implementando todas las funcionalidades de la gestión de pedidos de comida (añadir al carrito, ajustar la cantidad de los elementos del carrito, realizar pedidos, consultar los pedidos y cambiar el estado de pedidos). Para lograrlo, se ha creado diagramas de componentes y prototipos y se ha documentado los diseños de la base de datos y de las APIs, además de realizar un conjunto de pruebas de sistema para asegurar el correcto funcionamiento de la aplicación. El feedback recibido por parte de los dueños del restaurante fue positivo y no se identificaron áreas de mejora. Por último, en Jira se ha movido todas las User Stories completadas a “Done” (véase Ilustración 138) y se proporciona el Product Backlog tras finalizar el Sprint 4 (véase Ilustración 139).

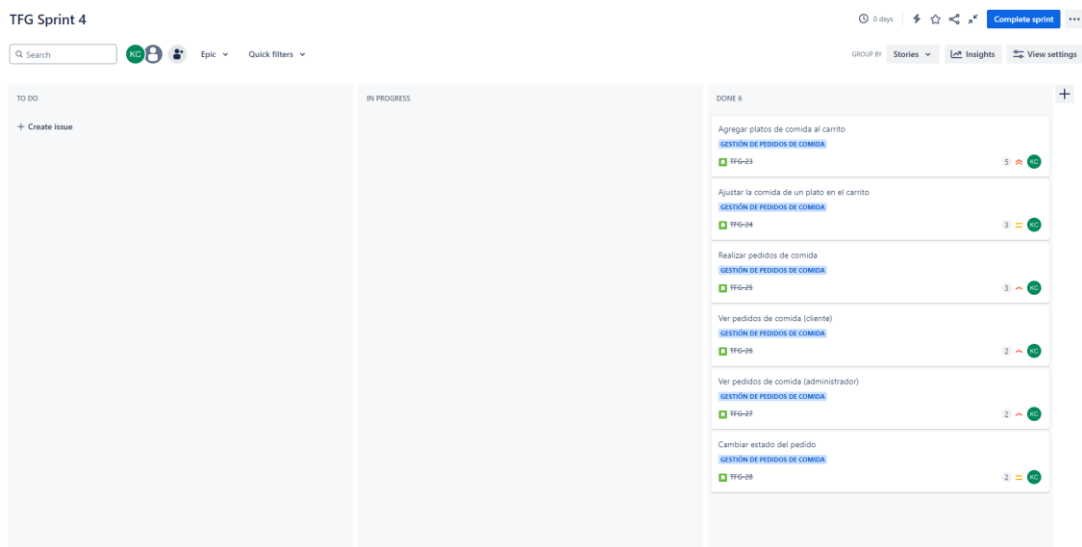


Ilustración 138: Tablero de Sprint 4 hecho.

Backlog (6 issues)				18	0	0	Create sprint
TFG-29	Dejar reseñas	GESTIÓN DE RESEÑAS	TO DO	5	0	0	
TFG-30	Modificar reseñas	GESTIÓN DE RESEÑAS	TO DO	3	0	0	
TFG-31	Eliminar reseñas (cliente)	GESTIÓN DE RESEÑAS	TO DO	3	0	0	
TFG-32	Ver reseñas (cliente)	GESTIÓN DE RESEÑAS	TO DO	2	0	0	
TFG-33	Ver reseñas (administrador)	GESTIÓN DE RESEÑAS	TO DO	2	0	0	
TFG-34	Eliminar reseñas (administrador)	GESTIÓN DE RESEÑAS	TO DO	3	0	0	

Ilustración 139: Product Backlog tras Sprint 4.

Al igual que el Sprint anterior, en cuanto a la retrospectiva del Sprint, se ha concluido que el enfoque actual es efectivo en general.

5.9 Sprint 5

5.9.1 Planificación del Sprint

El **objetivo principal** de este quinto Sprint consiste en implementar la funcionalidad de gestión de reseñas para que los propietarios del restaurante puedan mejorar sus servicios a través de los comentarios dejados por sus clientes. Para lograrlo, se ha seleccionado la siguiente Feature, junto con sus respectivas User Stories:

- FE6: Gestión de reseñas
 - US23: Dejar reseñas
 - US24: Modificar reseñas
 - US25: Eliminar reseñas
 - US26: Ver reseñas (cliente)
 - US27: Ver reseñas (administrador)

Al igual que en los Sprints anteriores, se ha creado en Jira el Sprint Backlog correspondiente para este Sprint (veáse Ilustración 140 y 141).

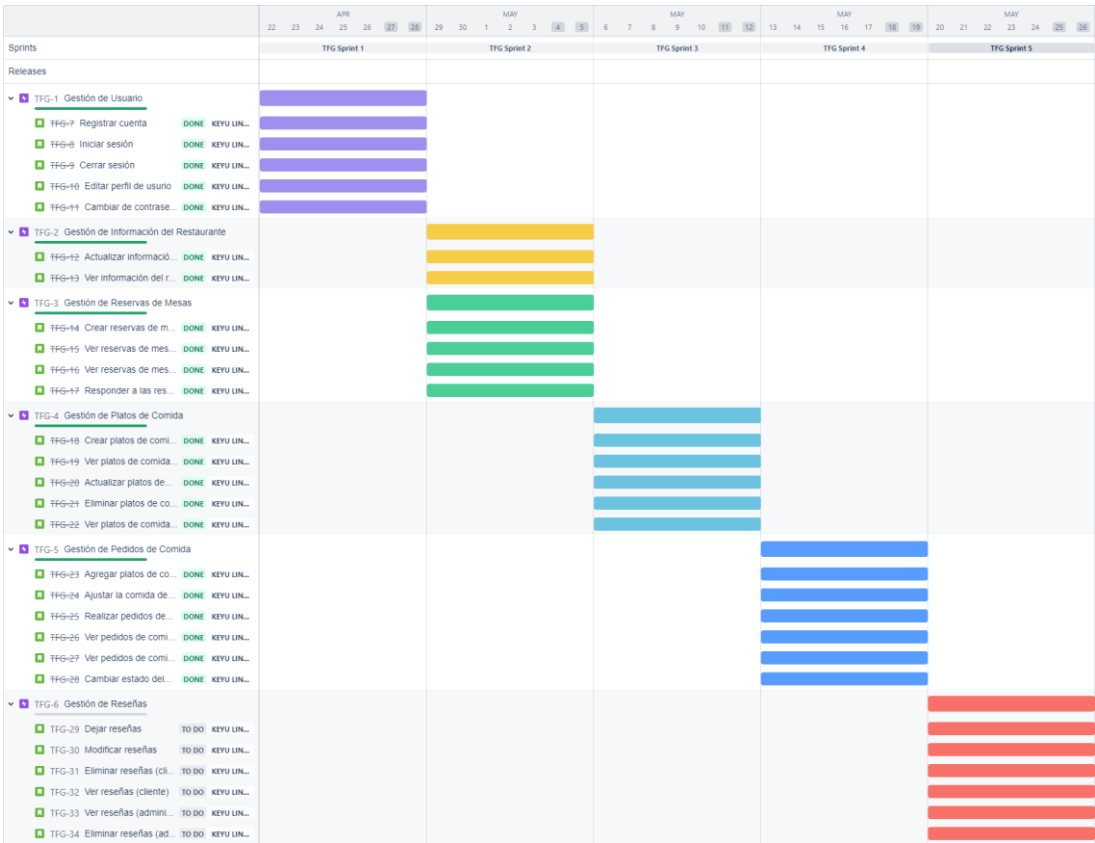


Ilustración 140: Planificación del Sprint 5.

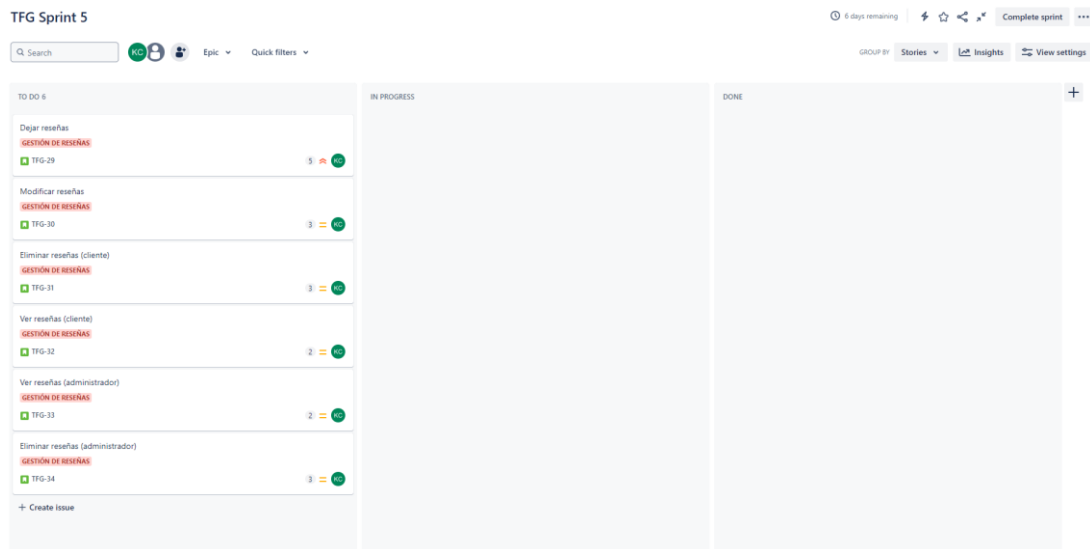


Ilustración 141: Tablero de Sprint 5 por hacer.

5.9.2 Diseño

5.9.2.1 Diseño Front-End

5.9.2.1.1 Diagrama de componentes y prototipos de interfaz de usuario

Para este Sprint, se ha desarrollado los siguientes diagramas de componentes (véase Ilustración 142 y 143):

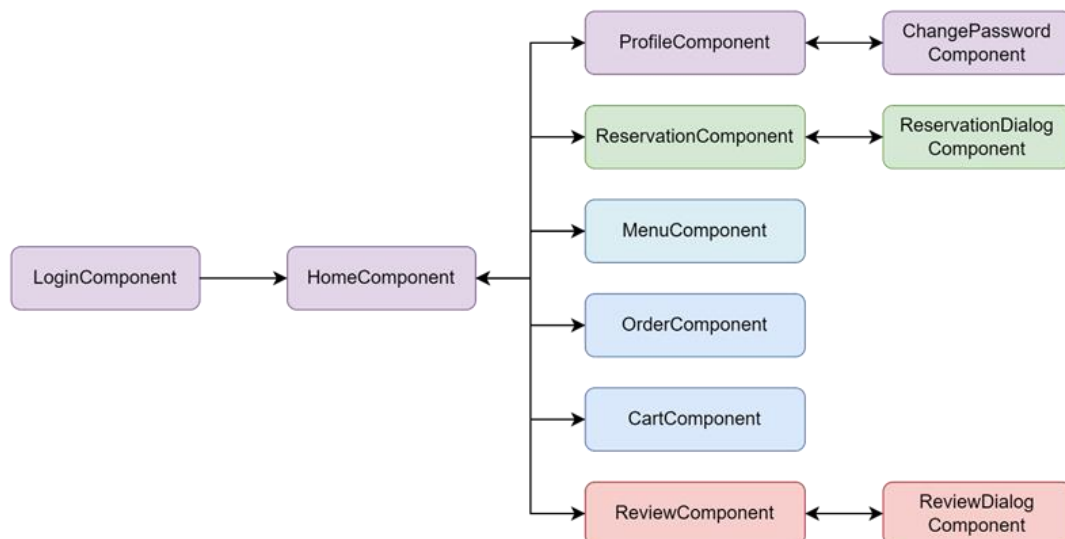


Ilustración 142: Diagrama de componentes para los clientes del Sprint 5.

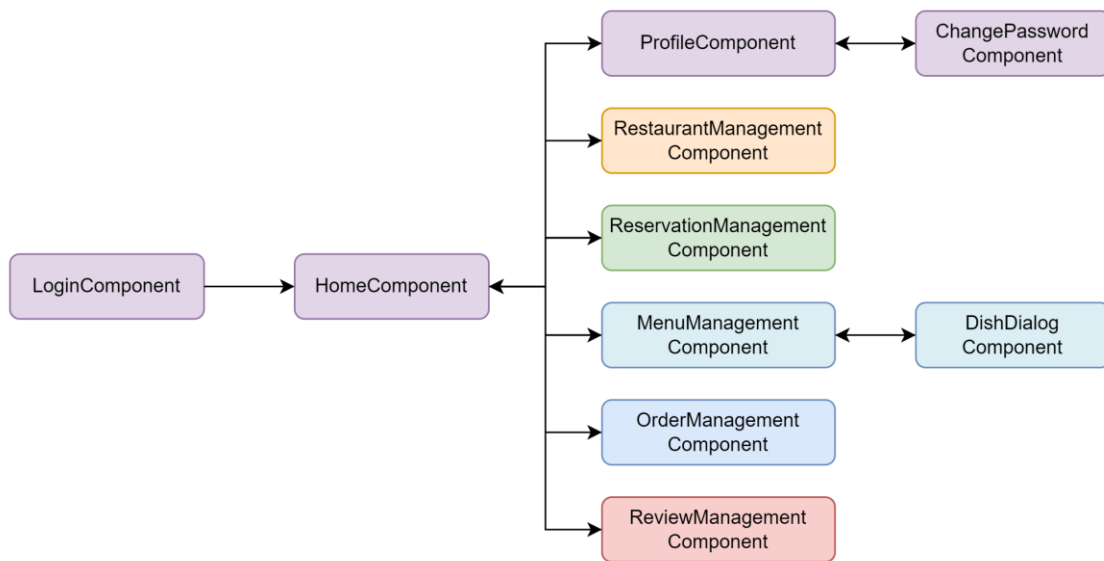


Ilustración 143: Diagrama de componentes para el administrador del Sprint 5.

- **ReviewComponent:** Componente donde los clientes pueden visualizar tanto las reseñas de otros clientes como sus reseñas (véase Ilustración X). Desde este componente, los clientes pueden ir a HomeComponent o a ReviewDialogComponent para crear o editar una reseña.

Mis reseñas

Reseñas

Mis reseñas

Crear reseña

Nombre

Valoración

Sobre

Contenido

Editar

Eliminar

Nombre

Valoración

Sobre

Contenido

Editar

Eliminar

Ilustración 144: ReviewComponent.

- **ReviewDialogComponent:** Componente donde los clientes pueden crear o editar una reseña. (véase Ilustración X). Desde este componente, los clientes pueden ir a ReviewComponent tras crear o editar una reseña.

Crear reseña

Reseña para: ☒ Restaurante ☐ Plato

Valoración: ★ ★ ★ ★ ★

Ilustración 145: ReviewDialogComponent.

- **ReviewManagementComponent:** Componente donde el administrador puede visualizar y eliminar las reseñas de sus clientes (véase Ilustración X). Desde este componente, el administrador puede ir a HomeComponent.
-

Reseñas

Nombre		Eliminar
Valoración		
Sobre		
Contenido		

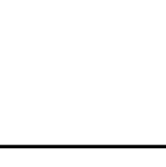
Nombre		Eliminar
Valoración		
Sobre		
Contenido		

Ilustración 146: ReviewManagementComponent.

5.9.2.2 Diseño Back-End

5.9.2.2.1 Diseño BBDD

Para este Sprint, se creará la entidad "Review":

- **Review.** La entidad "Review" representa las reseñas que dejan los usuarios sobre los platos del restaurante o del propio restaurante. Cada reseña contiene la opinión del cliente, una imagen opcional y una calificación. La entidad "Review" contiene estos campos:
 - *id*: Identificador único de la reseña. Este campo sirve como clave primaria en la base de datos (entero largo, autoincremental).
 - *content*: Contenido de la reseña escrito por el usuario (cadena de caracteres).
 - *image_url*: URL de una imagen asociada con la reseña (cadena de caracteres).
 - *rating*: Calificación que da el usuario al plato o al restaurante (entero).
 - *dish_id*: Identificador único del plato que es reseñado. Este campo es una clave foránea (entero).
 - *user_id*: Identificador único del usuario que ha escrito la reseña. Este campo es una clave foránea (entero).
 -

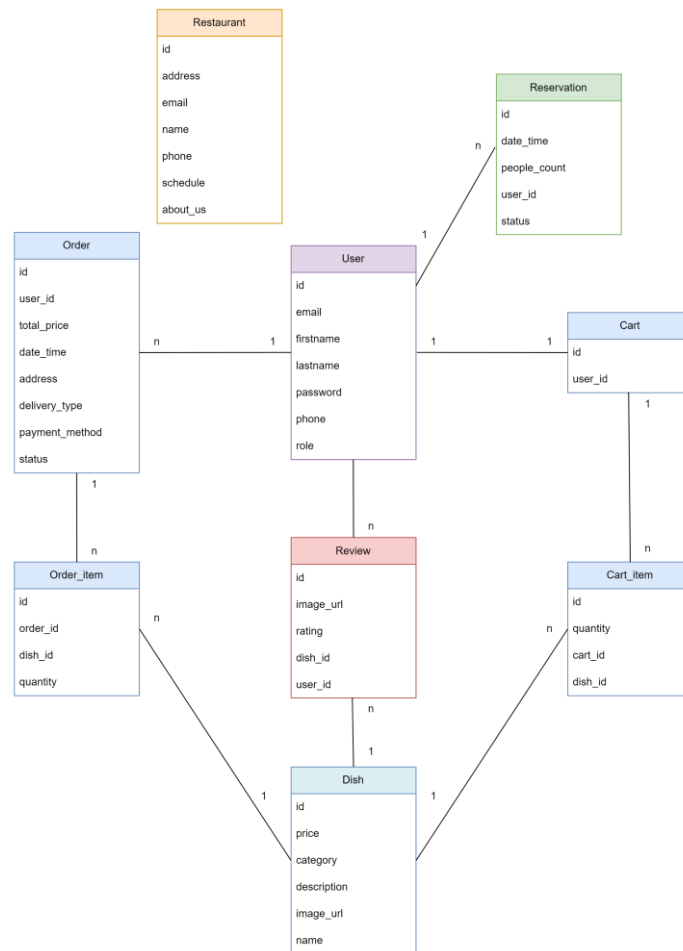


Ilustración 147: Diseño de BBDD del Sprint 5

5.9.2.2.2 Diseño API

Para este Sprint, se han diseñado los siguientes endpoints:

URI	http://localhost:8080/api/v1/customers/me/reviews	
Método	POST	
Descripción	Crea una reseña en el sistema.	
Autenticación	Bearer Token (Rol cliente)	
Cabecera	Authorization: Bearer {token} Content-Type: application/json	
Cuerpo de petición	<pre>{ "content": "Reseña", "rating": 5, "imageUrl": "Url de la imagen", "dishId": 1 }</pre>	
Parámetros de consulta	Ninguno	
Código de respuesta	201: Created	Se ha creado una reseña con éxito.
	400: Bad Request	Parámetros no válidos.
	401: Unauthorized	Token de acceso no válido.
	403: Forbidden	Acceso denegado.
	404: Not Found	Usuario no encontrado.
	500: Internal Server Error	Error interno del servidor.

Tabla 55: POST - Reviews.

URI	http://localhost:8080/api/v1/customers/me/reviews	
Método	GET	
Descripción	Obtiene las reseñas del cliente.	
Autenticación	Bearer Token (Rol cliente)	
Cabecera	Authorization: Bearer {token} Content-Type: application/json	
Cuerpo de petición	Ninguno	
Parámetros de consulta	Ninguno	
Código de respuesta	200: OK	Se ha obtenido las reseñas con éxito.

	401: Unauthorized	Token de acceso no válido.
	403: Forbidden	Acceso denegado.
	404: Not Found	Usuario no encontrado.
	500: Internal Server Error	Error interno del servidor.

Tabla 56: GET – MyReviews.

URI	http://localhost:8080/api/v1/reviews	
Método	GET	
Descripción	Obtiene la lista de todas las reseñas posibles.	
Autenticación	Bearer Token (Rol cliente y administrador)	
Cabecera	Authorization: Bearer {token} Content-Type: application/json	
Cuerpo de petición	Ninguno	
Parámetros de consulta	Ninguno	
Código de respuesta	200: OK	Se ha obtenido todas las reseñas del sistema con éxito.
	401: Unauthorized	Token de acceso no válido.
	403: Forbidden	Acceso denegado.
	500: Internal Server Error	Error interno del servidor.

Tabla 57: GET – AllReviews.

URI	http://localhost:8080/api/v1/customers/me/reviews/{idReview}	
Método	PATCH	
Descripción	Actualiza una reseña del cliente.	
Autenticación	Bearer Token (Rol cliente)	
Cabecera	Authorization: Bearer {token} Content-Type: application/json	
Cuerpo de petición	<pre>{ "content": "Reseña actualizada", "rating": 5 "imageUrl": "URL de la imagen", "dishId": 1 }</pre>	

Parámetros de consulta	Ninguno	
Código de respuesta	200: OK	Se ha actualizado la reseña con éxito.
	400: Bad Request	Parámetros no válidos.
	401: Unauthorized	Token de acceso no válido.
	403: Forbidden	Acceso denegado.
	404: Not Found	Usuario no encontrado o reseña no encontrada.
	500: Internal Server Error	Error interno del servidor.

Tabla 58: PATCH – Reviews.

URI	http://localhost:8080/api/v1/customers/me/reviews/{idReview}	
Método	DELETE	
Descripción	Elimina una reseña del cliente.	
Autenticación	Bearer Token (Rol cliente)	
Cabecera	Authorization: Bearer {token} Content-Type: application/json	
Cuerpo de petición	Ninguno	
Parámetros de consulta	Ninguno	
Código de respuesta	200: OK	Se ha eliminado la reseña del cliente con éxito
	401: Unauthorized	Token de acceso no válido.
	403: Forbidden	Acceso denegado.
	404: Not Found	Usuario no encontrado o reseña no encontrada.
	500: Internal Server Error	Error interno del servidor.

Tabla 59: DELETE – Reviews.

URI	http://localhost:8080/api/v1/reviews/{idReview}	
Método	DELETE	
Descripción	Elimina una reseña existente en el sistema.	
Autenticación	Bearer Token (Rol administrador)	
Cabecera	Authorization: Bearer {token}	

	Content-Type: application/json	
Cuerpo de petición	Ninguno	
Parámetros de consulta	Ninguno	
Código de respuesta	200: OK	Se ha eliminado la reseña del cliente con éxito
	401: Unauthorized	Token de acceso no válido.
	403: Forbidden	Acceso denegado.
	500: Internal Server Error	Error interno del servidor.

Tabla 60: DELETE – Reviews.

5.9.3 Pruebas de sistema

Para este Sprint, se ha llevado a cabo las siguientes pruebas de sistema:

- **PS40_US23 – Creación de reseñas correcta**
 - Descripción: El cliente rellena correctamente los campos necesarios para crear una reseña.
 - Resultado esperado: El sistema crea la reseña, actualiza la lista de reseñas y muestra un mensaje de que se ha creado una reseña con éxito.

Ilustración 148: Creación de reseñas correcta 1.

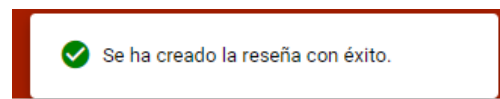


Ilustración 149: Creación de reseñas correcta 2.

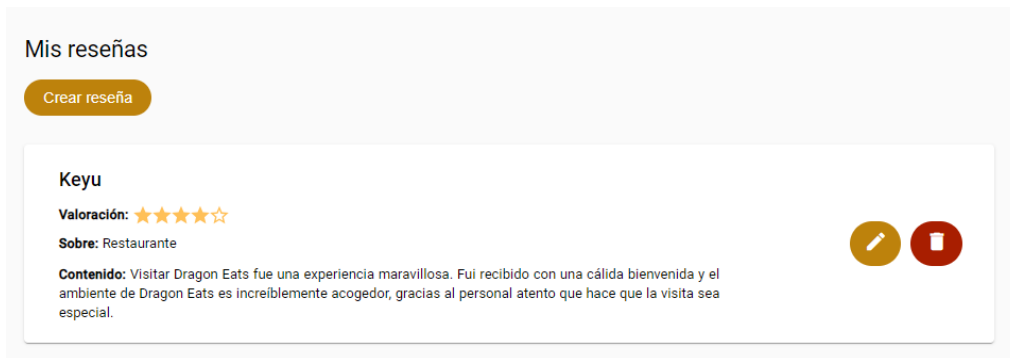


Ilustración 150: Creación de reseñas correcta 3.

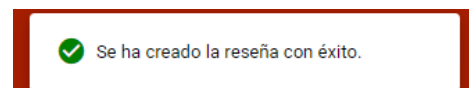
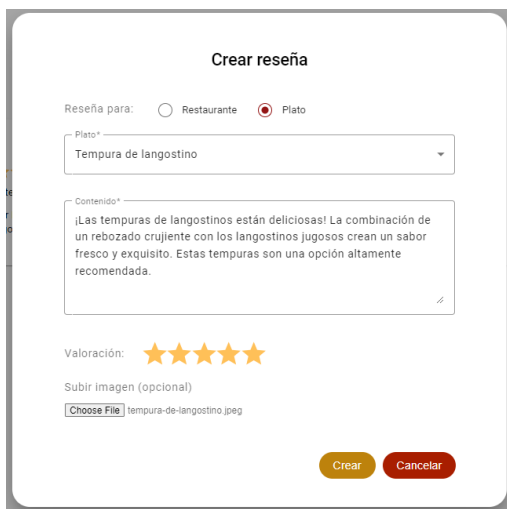


Ilustración 152: Creación de reseñas correcta 5.

Ilustración 151: Creación de reseñas correcta 4.

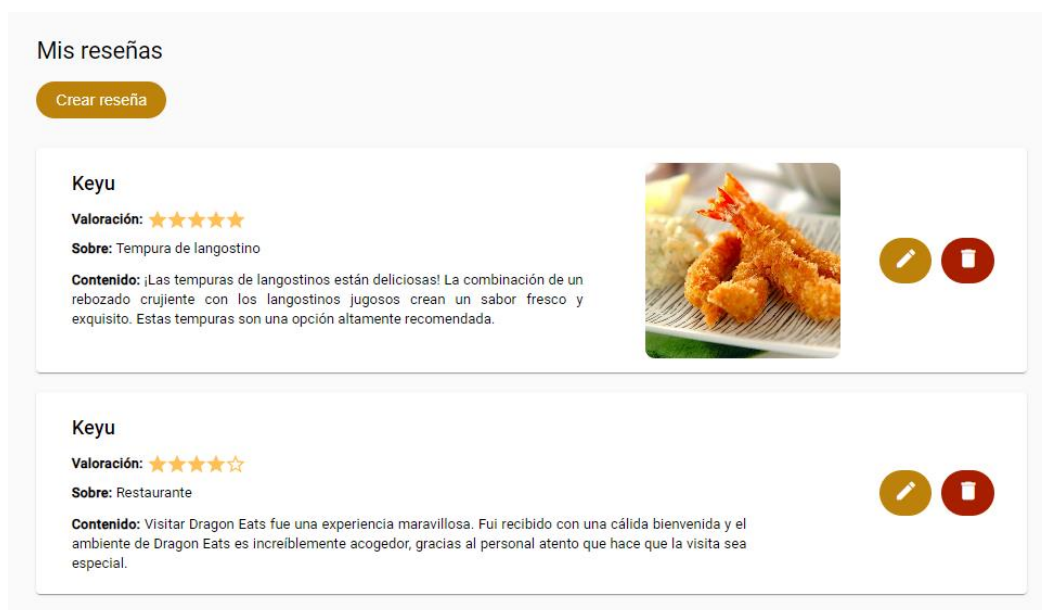


Ilustración 153: Creación de reseñas correcta 6.

- **PS41_US23 – Creación de reseñas incompleta**

- Descripción: El cliente no rellena todos los campos necesarios para crear una reseña.
- Resultado esperado: El sistema muestra al cliente los campos obligatorios que debe introducir y deshabilitará el botón de crear reseña.

Ilustración 154: Creación de reseñas incompleta.

- **PS42_US24 – Actualización de reseñas correcta**

- Descripción: El cliente rellena correctamente los campos necesarios para actualizar una reseña.
- Resultado esperado: El sistema actualiza la reseña, muestra la lista de reseñas y un mensaje de que se ha editado una reseña con éxito.

Ilustración 155: Actualización de reseñas correcta 1.

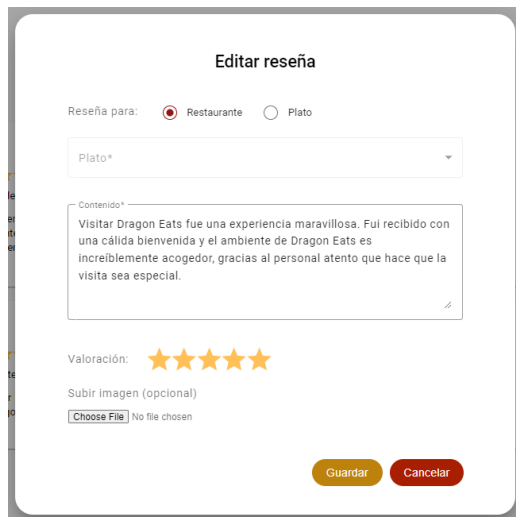


Ilustración 156: Actualización de reseñas correcta 2.

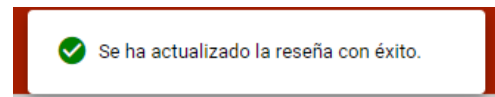


Ilustración 157: Actualización de reseñas correcta 3.

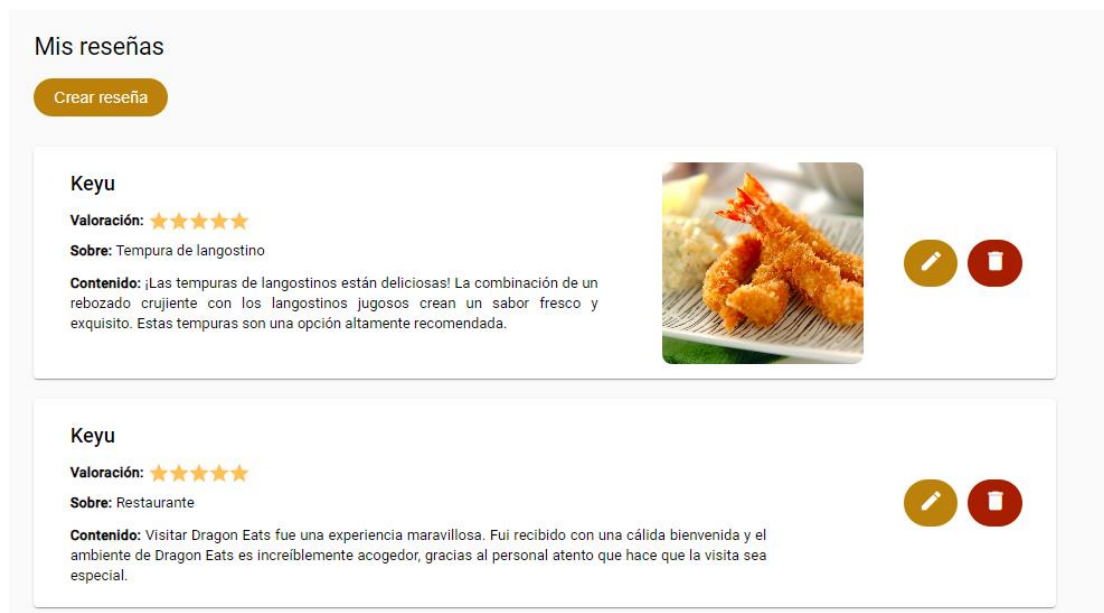


Ilustración 158: Actualización de reseñas correcta 4.

- **PS43_US24 – Actualización de reseñas incompleta**
 - Descripción: El cliente no rellena todos los campos necesarios para editar una reseña.
 - Resultado esperado: El sistema muestra al cliente los campos obligatorios que debe introducir y deshabilitará el botón de editar reseña.

Ilustración 159: Actualización de reseñas incompleta.

- **PS44_US25 – Eliminación de reseñas (cliente)**
 - Descripción: El cliente elimina su propia reseña al darle al botón de eliminar reseña.
 - Resultado esperado: El sistema muestra un diálogo de confirmación y si el cliente acepta, entonces el sistema elimina su reseña y le mostrará un mensaje de que la reseña se ha eliminado con éxito.

Mis reseñas

Crear reseña

Keyu

Valoración: ★★★★★

Sobre: Pato laqueado a la pekinsa

Contenido: Plato único y delicioso.

Keyu

Valoración: ★★★★★

Sobre: Tempura de langostino

Contenido: ¡Las tempuras de langostinos están deliciosas! La combinación de un rebozado crujiente con los langostinos jugosos crean un sabor fresco y exquisito. Estas tempuras son una opción altamente recomendada.

Keyu

Valoración: ★★★★★

Sobre: Restaurante

Contenido: Visitar Dragon Eats fue una experiencia maravillosa. Fui recibido con una cálida bienvenida y el ambiente de Dragon Eats es increíblemente acogedor, gracias al personal atento que hace que la visita sea especial.

Ilustración 160: Eliminación de reseñas (cliente) 1.

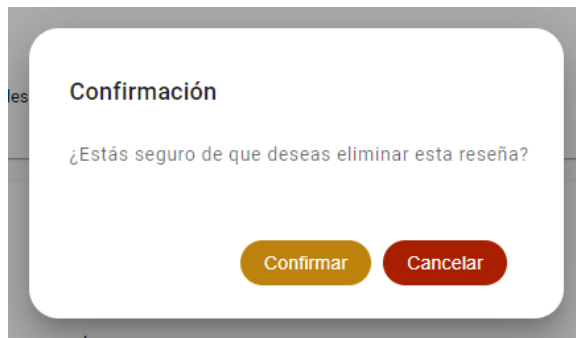


Ilustración 161: Eliminación de reseñas (cliente) 2.

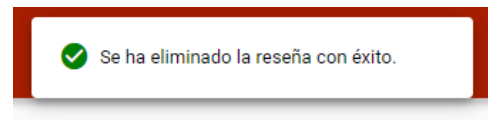


Ilustración 162: Eliminación de reseñas (cliente) 3.

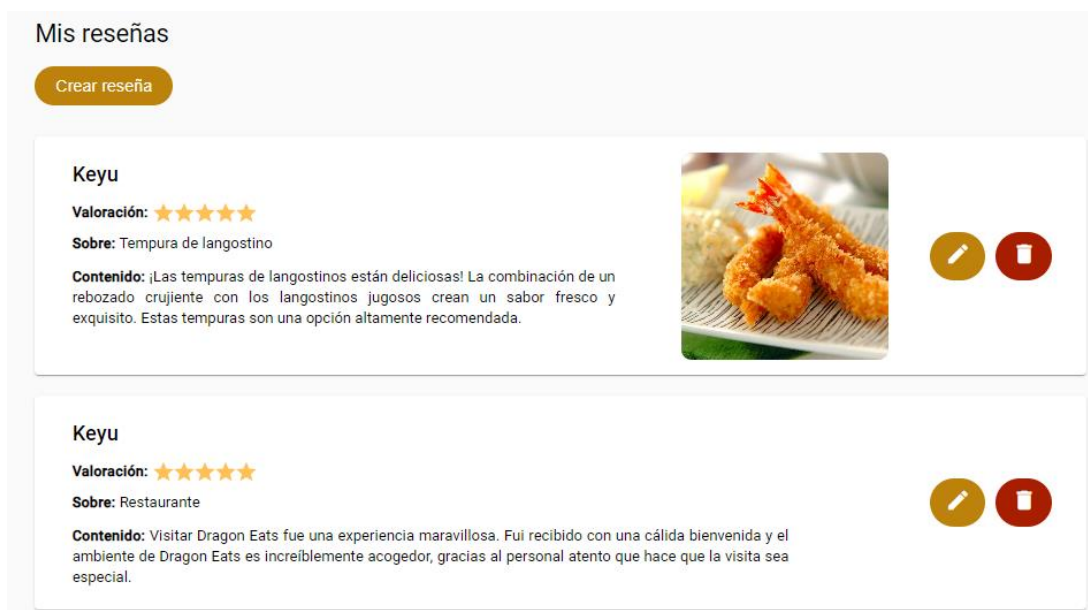


Ilustración 163: Eliminación de reseñas (cliente) 4.

- **PS45_US26 – Consulta de reseñas (cliente)**

- Descripción: El cliente visualiza su lista de reseñas y la lista de reseñas de otros clientes.
- Resultado esperado: El sistema muestra al cliente tanto su lista de reseñas como la lista de reseñas de otros clientes.

Mis reseñas

Crear reseña

Keyu

Valoración: ★★★★★

Sobre: Tempura de langostino

Contenido: ¡Las tempuras de langostinos están deliciosas! La combinación de un rebozado crujiente con los langostinos jugosos crean un sabor fresco y exquisito. Estas tempuras son una opción altamente recomendada.





Keyu

Valoración: ★★★★★

Sobre: Restaurante

Contenido: Visitar Dragon Eats fue una experiencia maravillosa. Fui recibido con una cálida bienvenida y el ambiente de Dragon Eats es increíblemente acogedor, gracias al personal atento que hace que la visita sea especial.



Ilustración 164: Consulta de reseñas (cliente) 1.

Reseñas

Jony Johnson

Valoración: ★☆☆☆☆

Sobre: Pato laqueado a la pekinesa

Contenido: Decepción de comida, ¡asqueroso!

Keyu

Valoración: ★★★★★

Sobre: Tempura de langostino

Contenido: ¡Las tempuras de langostinos están deliciosas! La combinación de un rebozado crujiente con los langostinos jugosos crean un sabor fresco y exquisito. Estas tempuras son una opción altamente recomendada.



Keyu

Valoración: ★★★★★

Sobre: Restaurante

Contenido: Visitar Dragon Eats fue una experiencia maravillosa. Fui recibido con una cálida bienvenida y el ambiente de Dragon Eats es increíblemente acogedor, gracias al personal atento que hace que la visita sea especial.

Jony Johnson

Valoración: ★☆☆☆☆

Sobre: Restaurante

Contenido: El servicio fue lento y desorganizado, lo que resultó en largos períodos de espera entre los platos. Además, el ambiente tenía una iluminación muy brillante y una música de fondo demasiado alta.

Ilustración 165: Consulta de reseñas (cliente) 2.

- **PS46_US27 – Consulta de reseñas (administrador)**

- Descripción: El administrador visualiza la lista de reseñas de todos sus clientes.
- Resultado esperado: El sistema muestra al administrador todas las reseñas de sus clientes.
-

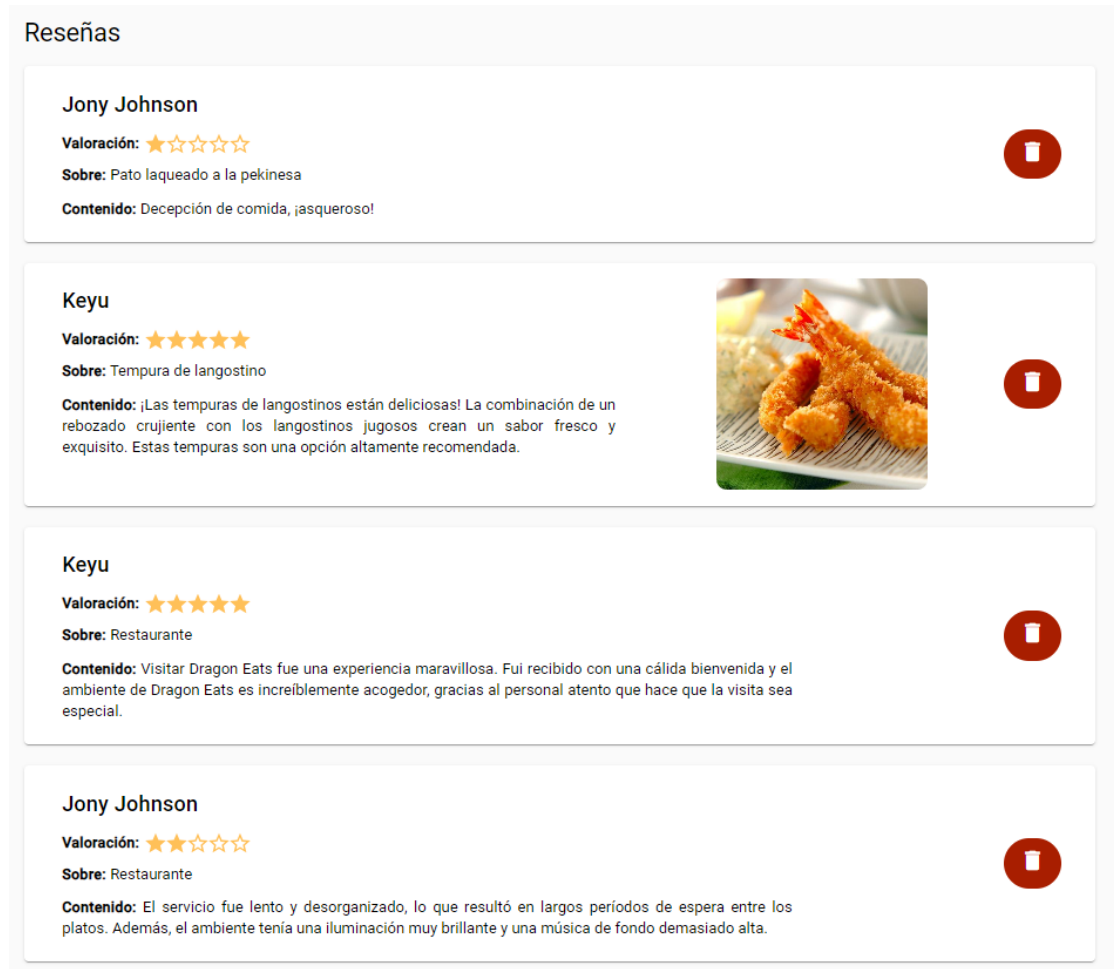


Ilustración 166: Consulta de reseñas (administrador).

- **PS47_US28 – Eliminación de reseñas (administrador)**

- Descripción: El administrador elimina una reseña de un cliente debido a su contenido irrespetuoso al botón de eliminar reseña.
- Resultado esperado: En caso de que el administrador intenta eliminar una reseña del cliente, entonces el sistema también le muestra un diálogo de confirmación y si acepta, entonces el sistema elimina la reseña del cliente y le mostrará un mensaje de que la reseña se ha eliminado con éxito. Tras esto se actualizará la lista de reseñas

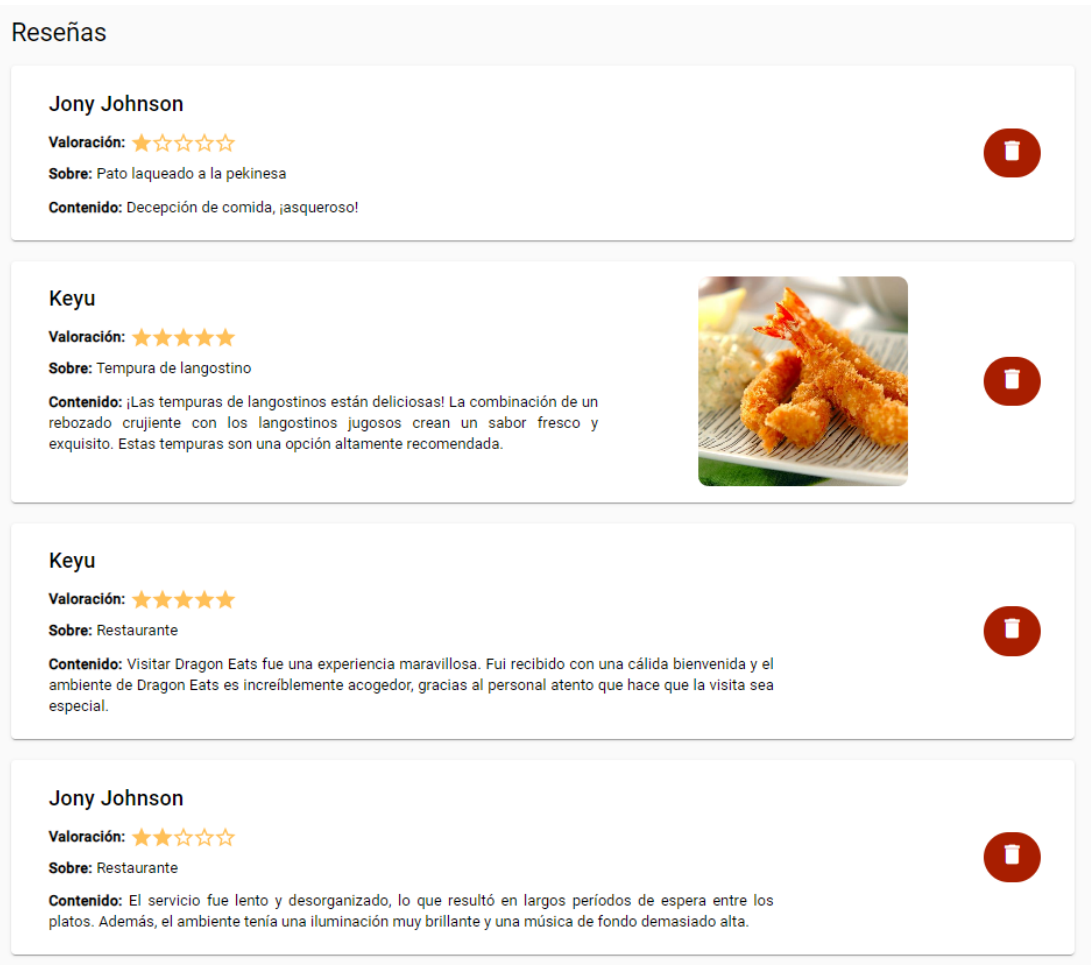


Ilustración 167: Eliminación de reseñas (administrador) 1.

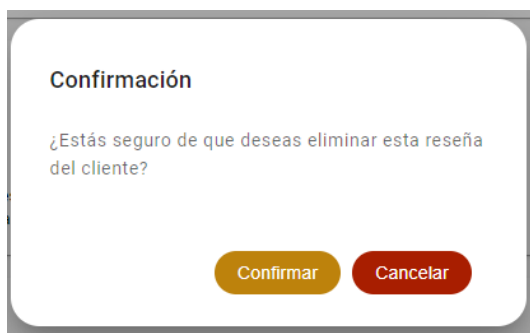


Ilustración 168: Eliminación de reseñas (administrador) 2.

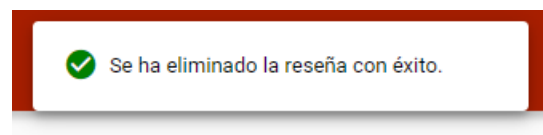


Ilustración 169: Eliminación de reseñas (administrador) 3.

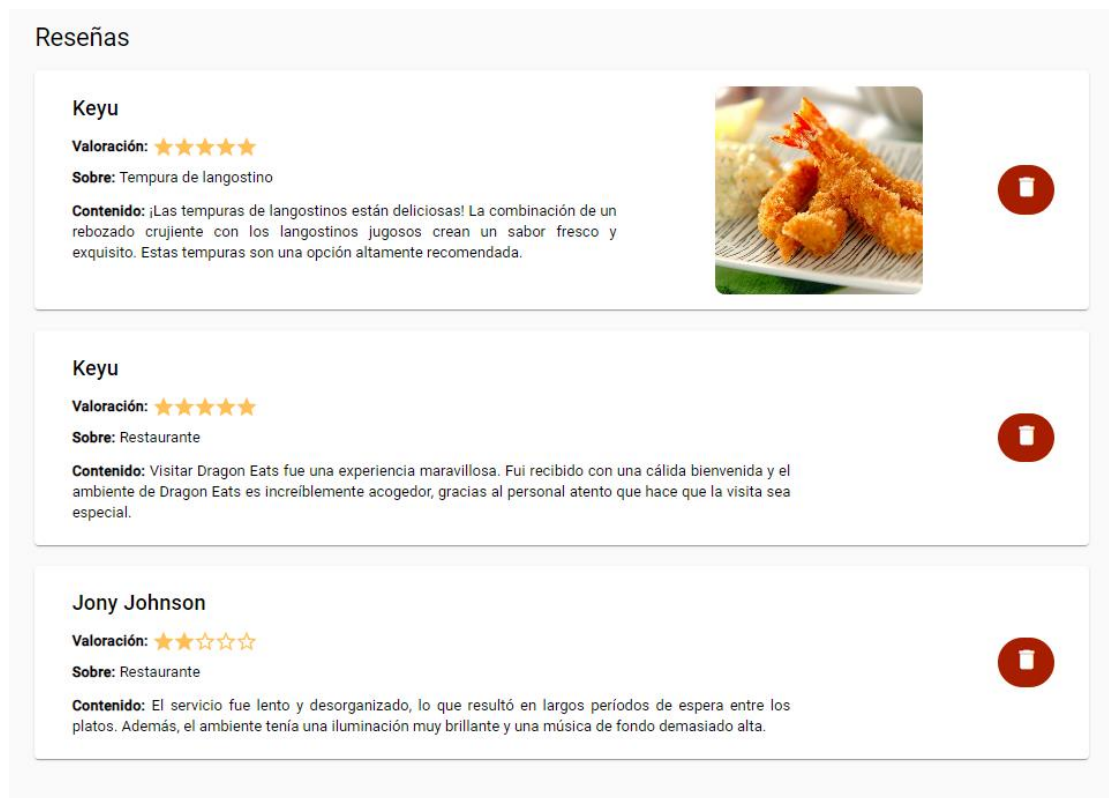


Ilustración 170: Eliminación de reseñas (administrador) 4.

5.9.4 Revisión y Retrospectiva del Sprint

Durante este Sprint, se ha logrado cumplir con todos los objetivos establecidos en la planificación del Sprint, implementando todas las funcionalidades de la gestión de reseñas (añadir, editar, eliminar y consultar reseñas). Para lograrlo, se ha creado diagramas de componentes y prototipos y se ha documentado los diseños de la base de datos y de las APIs, además de realizar un conjunto de pruebas de sistema para asegurar el correcto funcionamiento de la aplicación. El feedback recibido por parte de los dueños del restaurante fue positivo y no se identificaron áreas de mejora. Por último, en Jira se ha movido todas las User Stories completadas a “Done” (véase Ilustración 171).

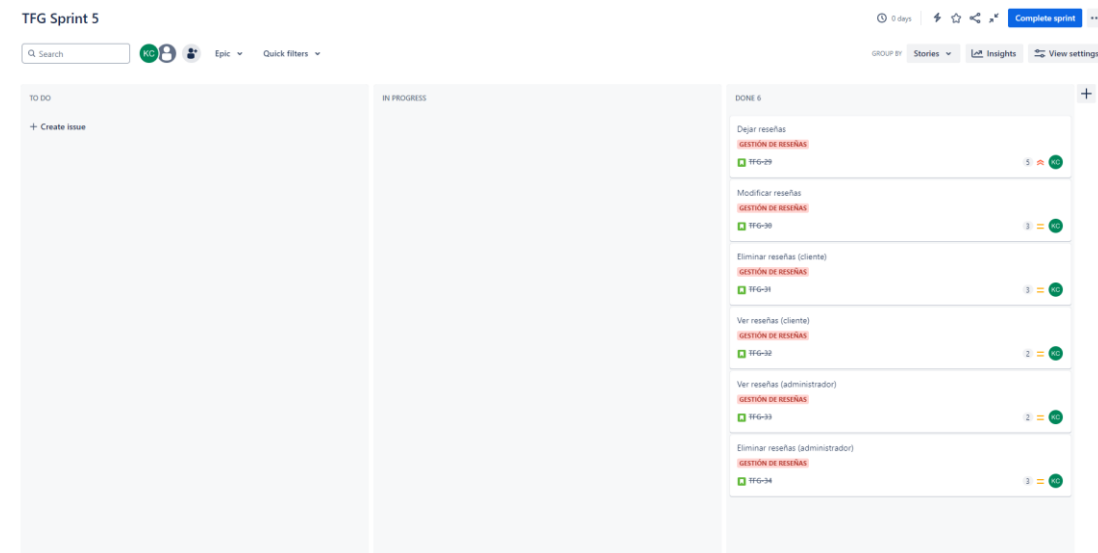


Ilustración 171: Tablero de Sprint 5 hecho.

Al igual que el Sprint anterior, se ha llegado a la conclusión de que el enfoque actual es efectivo.

6 Dragon Eats

6.1 Breve explicación del capítulo

En este capítulo se presentará brevemente todas las vistas de la aplicación web para el restaurante Dragon Eats (nombre ficticio de restaurante). Cabe mencionar que no se utilizará el nombre real del restaurante para preservar su confidencialidad.

6.2 Vistas de todos los usuarios antes de autenticarse



Ilustración 172: Página principal 1.



Ilustración 173: Página principal 2.



Sobre Nosotros

¡Bienvenidos a nuestro restaurante! Nos complace saber que has elegido a Dragon Eats para tener una vivencia gastronómica inolvidable. Aquí encontrarás una experiencia culinaria única y deliciosa, donde nuestra pasión por la comida se refleja en los platos de la más alta calidad. Además, en Dragon Eats abunda de platos tradicionales y clásicos con una gran variedad de opciones para todos los gustos. Por último, nos puedes encontrar en la Calle ficticia, 123, Madrid, España. ¡Te esperamos con los brazos abiertos!

Información del restaurante

 Dirección	 Horario	 Email	 Teléfono
Calle ficticia, 10, Madrid, España	Lunes - Domingo: 10:00 - 23:30	email@restaurante.com	999999999

Ilustración 174: Inicio 3.

 Dragon Eats

Iniciar sesión

Iniciar sesión

Email*

 keyu.cliente@gmail.com

Contraseña*





Iniciar sesión

¿Aún no tienes una cuenta?

Crear cuenta

Ilustración 175: Inicio de sesión.

Crear cuenta

Nombre*
Keyu

Apellidos*

Teléfono*
666111333

Email*
keyu.cliente@gmail.com

Contraseña*
KeyuCliente123!

Confirmar contraseña*

☒ Acepto los [términos y condiciones especiales](#) y la [política de privacidad](#)

Crear cuenta

¿Ya tienes una cuenta?

Iniciar sesión

Ilustración 176: Registro.

Términos y condiciones de uso

Bienvenido a Dragon Eats. Al acceder o al crear una cuenta en Dragon Eats, usted acepta cumplir con estos términos y condiciones:

Modificaciones de los términos

Podemos modificar estos términos, por lo tanto, usted debe revisar estas páginas con frecuencia. Cuando cambiamos los Términos y Condiciones, le notificaremos de que se han realizado cambios a través del correo facilitado.

Uso de la aplicación

Usted no debe utilizar la aplicación con fines negativos, ni para causar daño o con algún propósito no autorizado como colocar uno de nuestros productos, modificado o sin modificar, en un sitio web u otro medio.

Registro de usuario

Para realizar la mayoría de las funciones de la aplicación, será necesario registrar y crear una cuenta, por lo que usted deberá proporcionar una información precisa y completa. El usuario puede modificar posteriormente su información proporcionada en el registro.

Propiedad intelectual

Todos los contenidos, características y funcionalidades de la aplicación están protegidos por las leyes de derechos de autor y otras leyes de propiedad intelectual. Por lo que usted no puede declarar propiedad intelectual a ninguno de nuestros productos.

Limitación de responsabilidad

La aplicación no será responsable de ningún daño indirecto, consecuente o incidental relacionados o derivados de nuestras condiciones o servicios, incluyendo la pérdida de beneficios, datos o uso, incurridos por usted o cualquier tercero, que surge de su acceso o uso en la aplicación.

Ley aplicable y jurisdicción

Estos Términos y Condiciones se registrarán e interpretarán de acuerdo con las leyes de España. Cualquier

Aceptar

Ilustración 177: Términos y confciones de uso.

Política de privacidad

Dragon Eats se compromete a proteger tu privacidad. En los siguientes puntos se describe cómo recopilamos, usamos y protegemos su información personal cuando utiliza la aplicación:

Información que recopilamos

Podemos recopilar la información de registro como el nombre, la dirección de correo electrónico, el número de teléfono y la contraseña. También se recopila la información de uso como los datos de inicio de sesión, la dirección IP y tiempo de permanencia. Y además, la información de pago si se realizan compras a través de la aplicación.

Uso de la información

La información recopilada la utilizaremos para operar y mantener la aplicación, mejorarla y expandirla. También sirve para desarrollar nuevos productos, servicios y para poder comunicarnos con usted, proporcionándole actualizaciones y otras informaciones en relación con la aplicación.

Divulgación de la información

Podemos compartir su información personal con proveedores de servicios que ayudan a mantener la aplicación. Asimismo, se compartirá con la ley cuando esta lo requiera o en respuesta a procesos legales.

Retención de datos

Su información personal será retenida durante el tiempo que sea necesario para cumplir con los fines que se han mencionado anteriormente, incluyendo los fines de requisitos legales, contables o de informes.

Sus derechos

Usted tiene derecho a acceder, actualizar o solicitar la eliminación de su información personal. Para ejercer estos derechos, por favor póngase en contacto con nosotros a través del correo electrónico o número de teléfono proporcionado en la página principal.

Cambios a la política de privacidad

Esta política de privacidad puede actualizarse en cualquier momento, por lo que le notificaremos los cambios que se realizarán y también, le recomendamos que revise esta nueva política de privacidad para estar al tanto de los

Aceptar

Ilustración 178: Política de privacidad.



Ilustración 179: Página no encontrada.

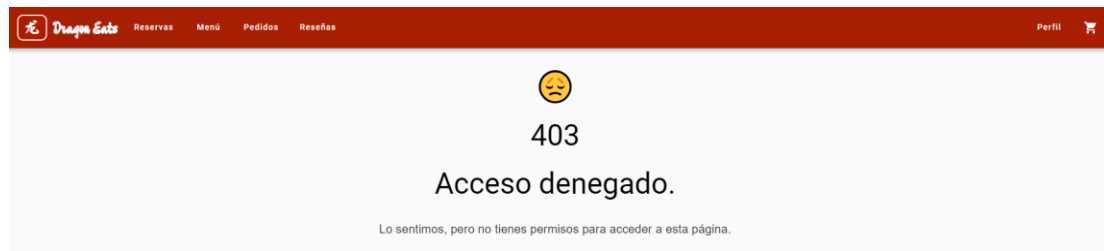


Ilustración 180: Acceso denegado.

6.3 Vistas de los clientes

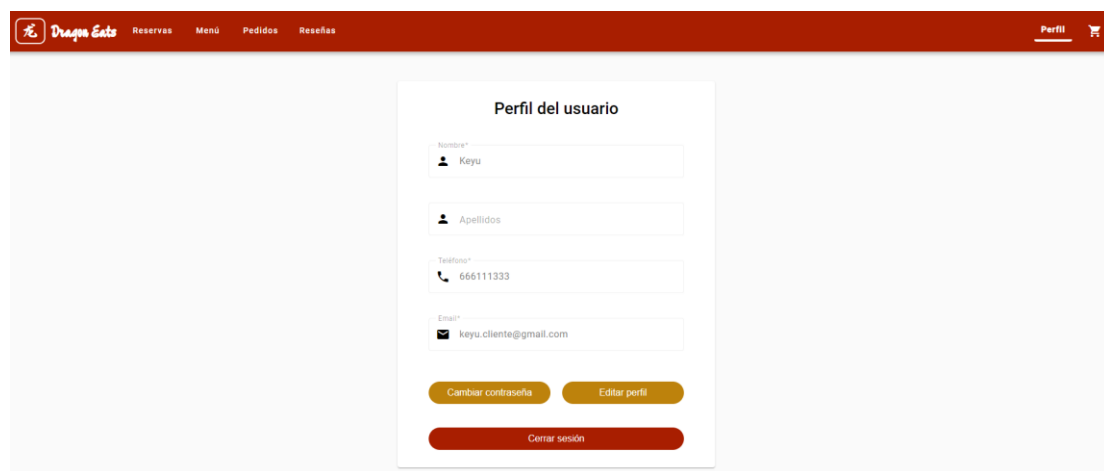


Ilustración 181: Cliente – Perfil.

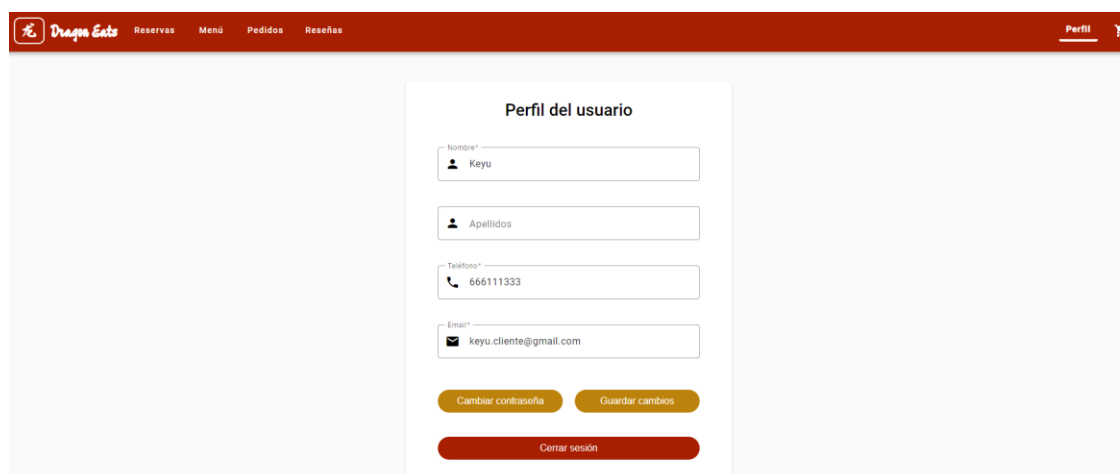


Ilustración 182: Cliente – Editar perfil.

Dragon Eats

ReservasMenúPedidosReseñas

Perfil

Cambiar contraseña

Contraseña actual*

Nueva contraseña*

Confirmar nueva contraseña*

Cambiar contraseña

Ilustración 183: Cliente – Cambiar contraseña.

Dragon Eats

ReservasMenúPedidosReseñas

Perfil

Mis Reservas

Hacer reserva

ID	Fecha y hora de la reserva	Número de personas	Estado
3	2024-05-31 22:00:00	20	PENDIENTE
2	2024-05-30 15:00:00	5	PENDIENTE
1	2024-05-27 20:00:00	10	PENDIENTE

Ilustración 184: Cliente - Reservas.

Dragon Eats

ReservasMenúPedidosReseñas

Perfil

Mis Reservas

Hacer reserva

Hacer Reserva

Número de Personas*

Fecha*

Hora*

Reservar

Cancelar

Ilustración 185: Cliente - Hacer reserva.

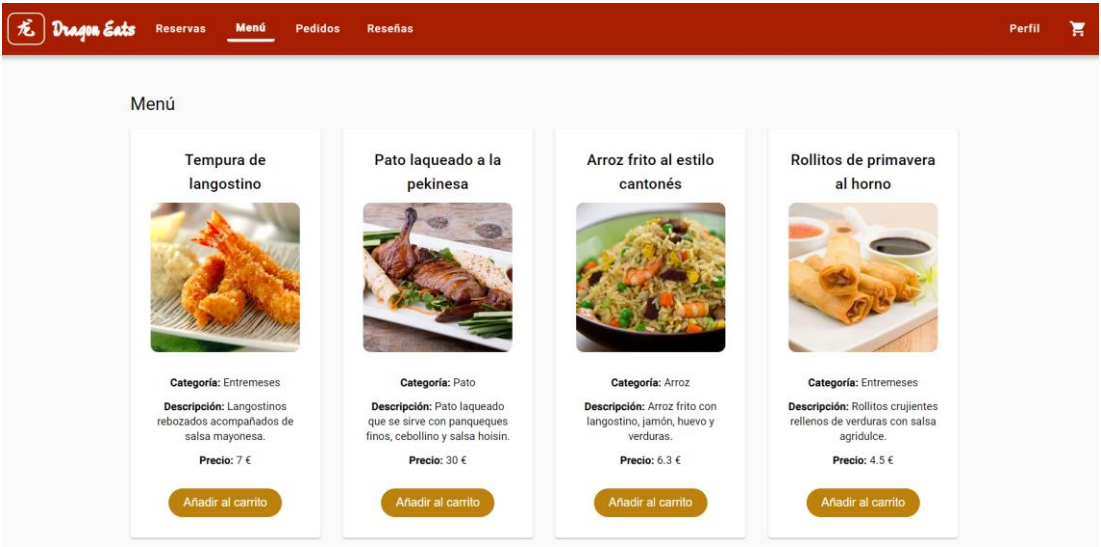


Ilustración 186: Cliente - Menú.

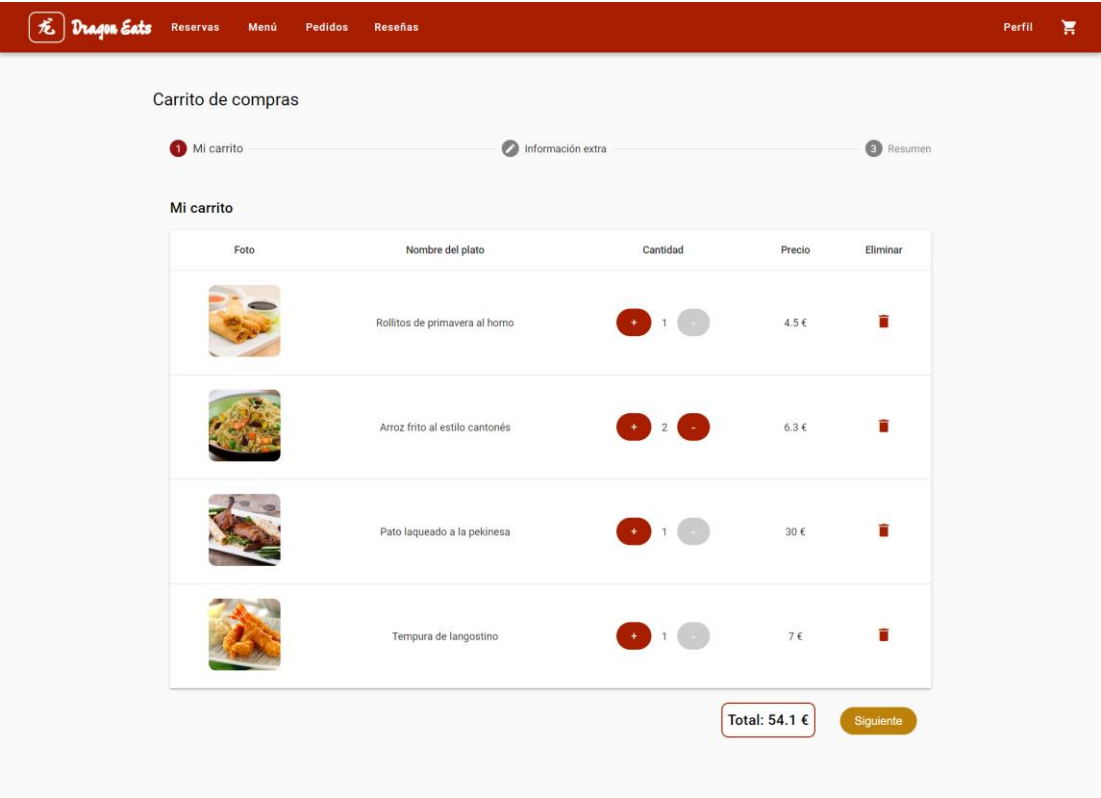


Ilustración 187: Cliente – Carrito 1.

The screenshot shows the 'Carrito de compras' (Shopping Cart) page for Dragon Eats. The progress bar indicates the current step is 'Información extra' (2 of 3). The 'Información extra' section contains the following fields and options:

- Método de entrega:** Radio buttons for 'Entrega a domicilio' (selected) and 'Recoger en restaurante'.
- Dirección completa:** A form with four fields:
 - Dirección*:** A text input field containing 'Recoletos'.
 - Número*:** A text input field containing '2'.
 - Piso*:** A text input field containing '2'.
 - Puerta*:** A text input field containing 'A'.
 - Código postal*:** A text input field containing '28500'.
- Método de pago:** Radio buttons for 'Efectivo' (selected), 'Tarjeta de crédito', and 'Pago móvil'.

At the bottom right of the form are two buttons: 'Volver' (red) and 'Siguiente' (yellow).

Ilustración 188: Cliente – Carrito 2.

The screenshot shows the 'Carrito de compras' (Shopping Cart) page for Dragon Eats. The progress bar indicates the current step is 'Información extra' (2 of 3). The 'Información extra' section contains the following fields and options:

- Método de entrega:** Radio buttons for 'Entrega a domicilio' and 'Recoger en restaurante' (selected).
- Dirección completa:** A form with four fields:
 - Dirección*:** A text input field.
 - Número*:** A text input field.
 - Piso*:** A text input field.
 - Puerta*:** A text input field.
 - Código postal*:** A text input field.
- Método de pago:** Radio buttons for 'Efectivo', 'Tarjeta de crédito' (selected), and 'Pago móvil'.

At the bottom right of the form are two buttons: 'Volver' (red) and 'Siguiente' (yellow).

Ilustración 189: Cliente – Carrito 3.

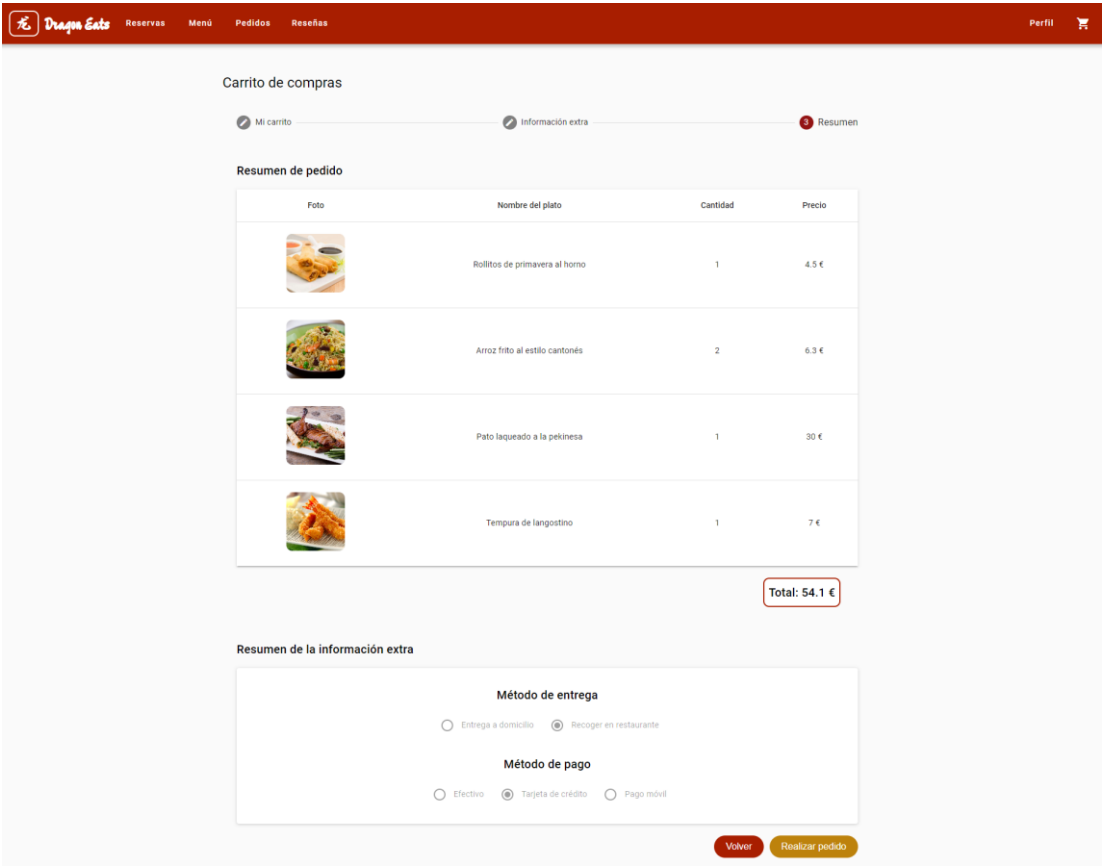


Ilustración 190: Cliente – Carrito 4



Ilustración 191: Cliente – Pedidos.

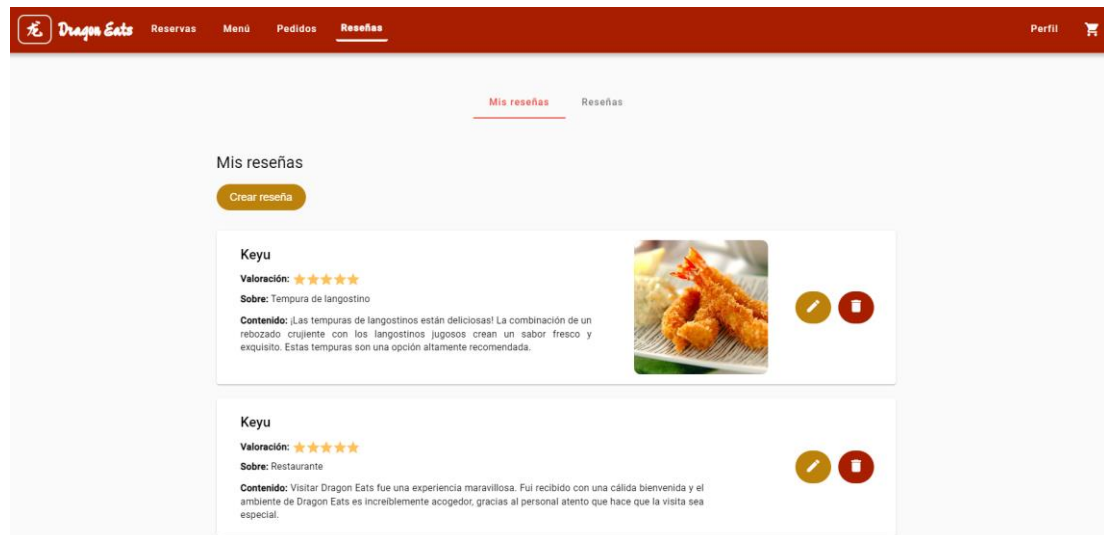


Ilustración 192: Cliente – Mis reseñas.

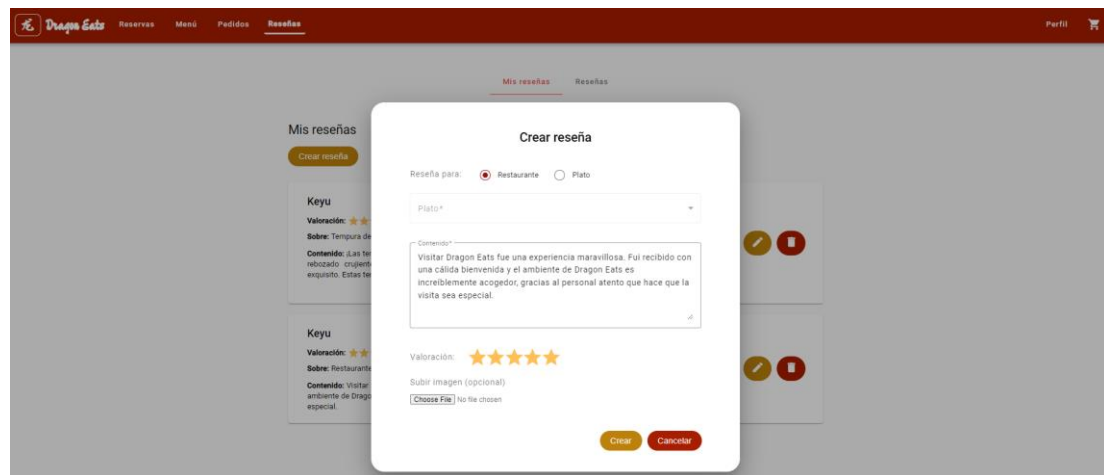


Ilustración 192: Cliente – Crear reseña.

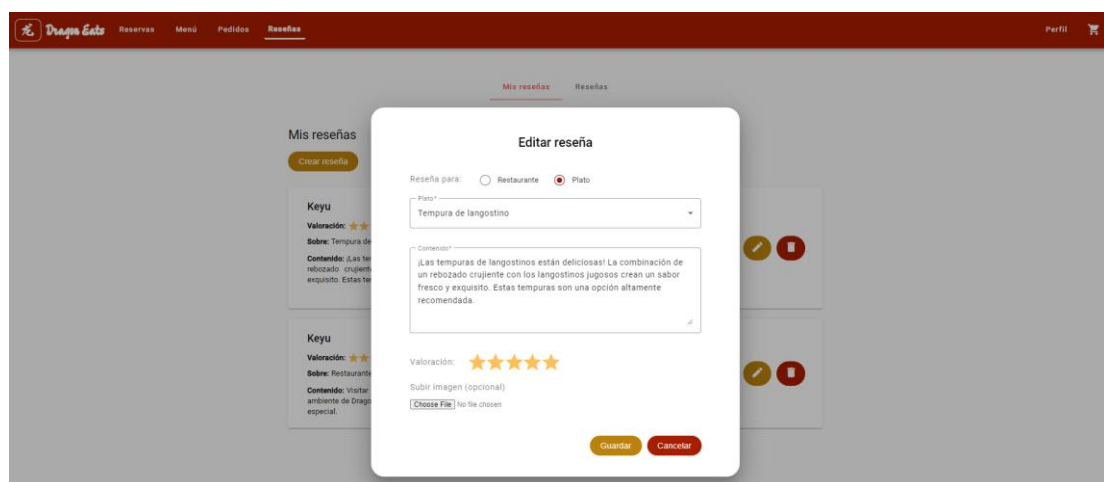


Ilustración 193: Cliente – Editar reseña.

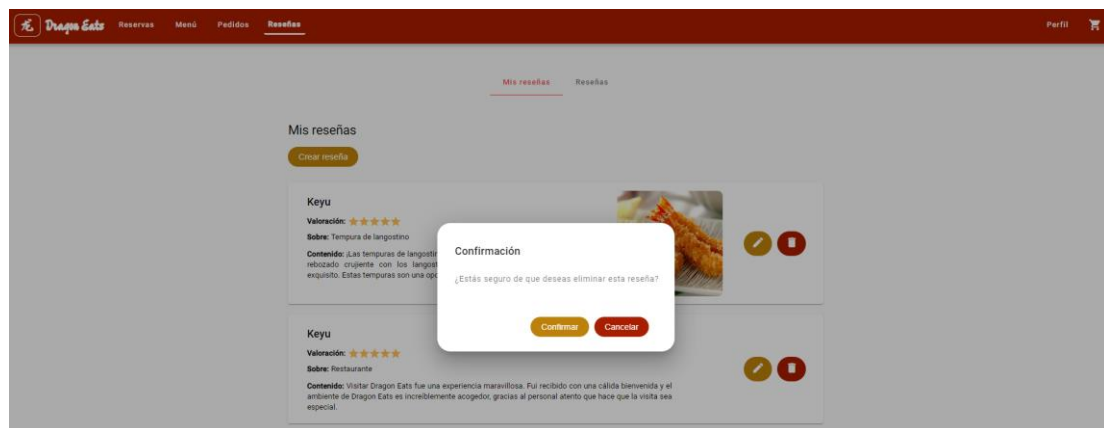


Ilustración 194: Cliente – Eliminar reseña.

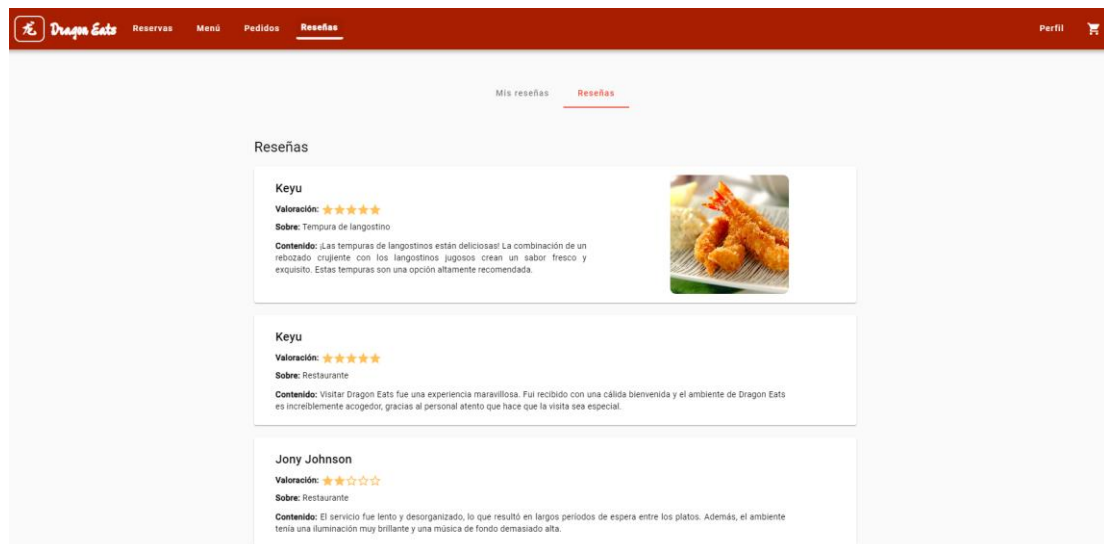


Ilustración 193: Cliente – Reseñas.

6.4 Vistas del administrador

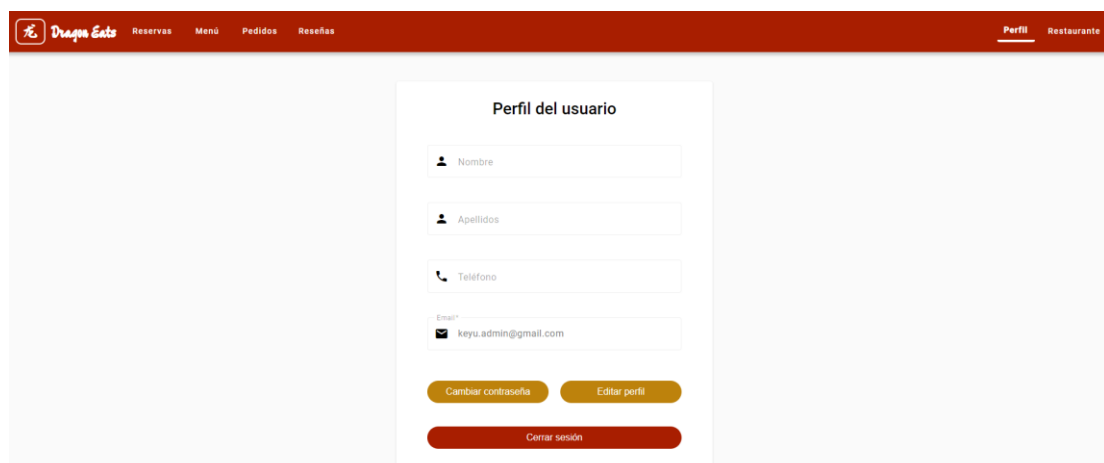


Ilustración 194: Administrador – Perfil.

Perfil del usuario

Nombre

Apellidos

Teléfono

Email*
keyu.admin@gmail.com

Cambiar contraseña Guardar cambios

Cerrar sesión

Ilustración 195: Administrador – Editar perfil.

Cambiar contraseña

Contraseña actual*
.....

Nueva contraseña*
.....

Confirmar nueva contraseña*
.....

Cambiar contraseña

Ilustración 196: Administrador – Cambiar contraseña.

Dragon Eats

ReservasMenúPedidosReseñas

PerfilRestaurante

Información del restaurante

Nombre del restaurante*

Restaurante Dragon Eats

Sobre nosotros*

¡Bienvenidos a nuestro restaurante!

Nos complace saber que has elegido a Dragon Eats para tener una

Teléfono*

999999999

Email*

email@restaurante.com

Horario*

Lunes - Domingo: 10:00 - 23:30

Dirección*

Calle ficticia, 10, Madrid, España

Editar información del restaurante

Ilustración 197: Administrador – Información del restaurante.

Dragon Eats

ReservasMenúPedidosReseñas

PerfilRestaurante

Lista de reservas

ID	Fecha y hora	Número de personas	Nombre del cliente	Correo del cliente	Teléfono del cliente	Estado	Acciones
4	2024-06-02 22:10:00	16	Jony Johnson	jony.johnson@gmail.com	234567890	PENDIENTE	AceptarRechazar
3	2024-05-31 22:00:00	20	Keyu	keyu.cliente@gmail.com	666111333	RECHAZADO	
2	2024-05-30 19:00:00	5	Keyu	keyu.cliente@gmail.com	666111333	ACEPTADO	
1	2024-05-27 20:00:00	10	Keyu	keyu.cliente@gmail.com	666111333	ACEPTADO	

Ilustración 198: Administrador – Reservas.

Dragon Eats

ReservasMenúPedidosReseñas

PerfilRestaurante

Menú

Crear plato

Tempura de langostino

Categoría: Entramejes

Descripción: Langostinos rebozados acompañados de salsa mayonesa.

Precio: 7 €

EditarBorrar

Pato laqueado a la pekinesa

Categoría: Pato

Descripción: Pato laqueado que se sirve con panqueques finos, cebollino y salsa hoisin.

Precio: 30 €

EditarBorrar

Arroz frito al estilo cantonés

Categoría: Arroz

Descripción: Arroz frito con langostino, jamón, huevo y verduras.

Precio: 6.3 €

EditarBorrar

Rollitos de primavera al horno

Categoría: Entramejes

Descripción: Rollitos crujientes rellenos de verduras con salsa agri-dulce.

Precio: 4.5 €

EditarBorrar

Ilustración 199: Administrador – Menú.

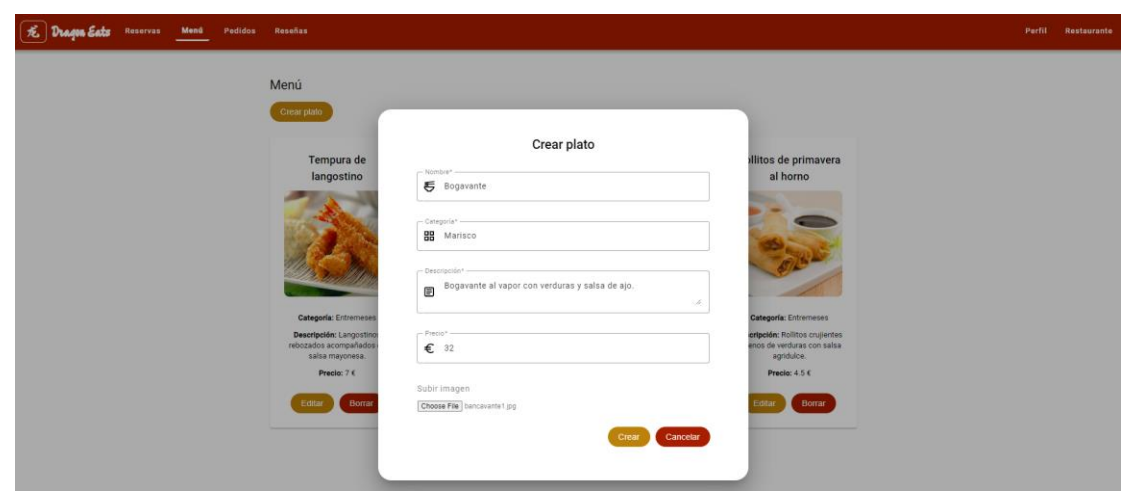


Ilustración 200: Administrador – Crear plato.

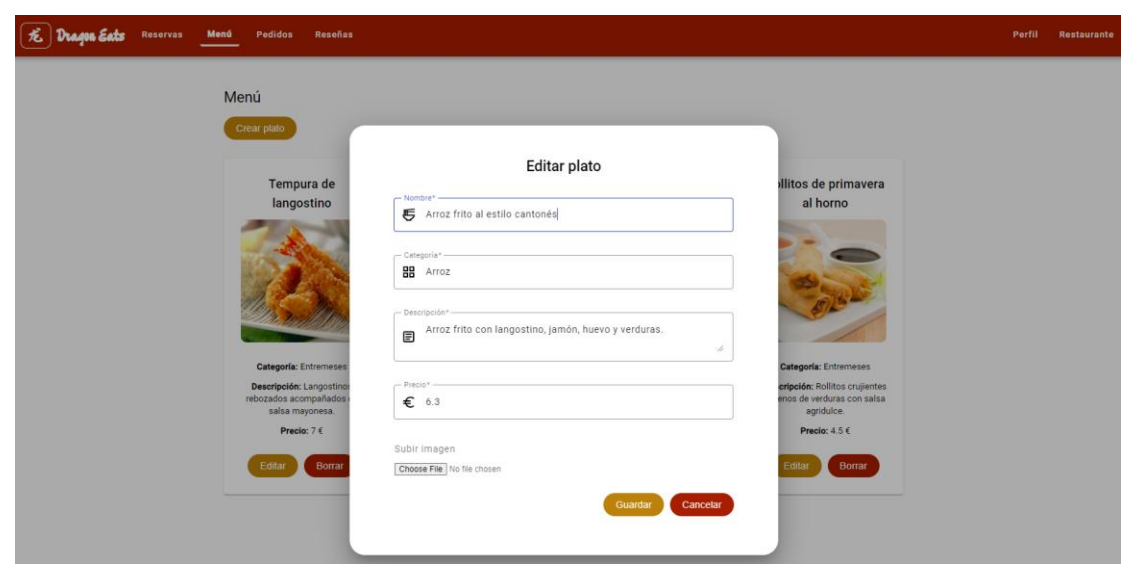


Ilustración 201: Administrador – Editar plato.

ID	Fecha y hora del pedido	Tipo de entrega	Pedido	Precio total	Método de pago	Nombre del cliente	Correo del cliente	Teléfono del cliente	Estado	Acciones
4	2024-05-19 15:37:21	Recoger en el restaurante	1 x Tempura de langostino x 7 € 1 x Plato laqueado a la pekinesa x 30 € 1 x Rollos de primavera al horno x 4.5 € 2 x Arroz frito al estilo cantonés x 6.3 €	54.1 €	Tarjeta de crédito	Kayu	kayu_cliente@gmail.com	666111333	PENDIENTE	<button>Preparando</button>
3	2024-05-19 15:37:21	Recoger en el restaurante	1 x Plato laqueado a la pekinesa x 30 €	30 €	Pago móvil	Jony Johnson	jony.johnson@gmail.com	234567890	PENDIENTE	<button>Preparando</button>
2	2024-05-19 15:37:21	Recoger en el restaurante	1 x Tempura de langostino x 7 € 2 x Plato laqueado a la pekinesa x 30 €	67 €	Tarjeta de crédito	Kayu	kayu_cliente@gmail.com	666111333	ENTREGADO	
1	2024-05-19 15:30:23	Entrega a domicilio Recoletos, 2, 2, A, 28002	2 x Tempura de langostino x 7 €	14 €	Efectivo	Kayu	kayu_cliente@gmail.com	666111333	ENTREGADO	

Ilustración 202: Administrador – Pedidos.

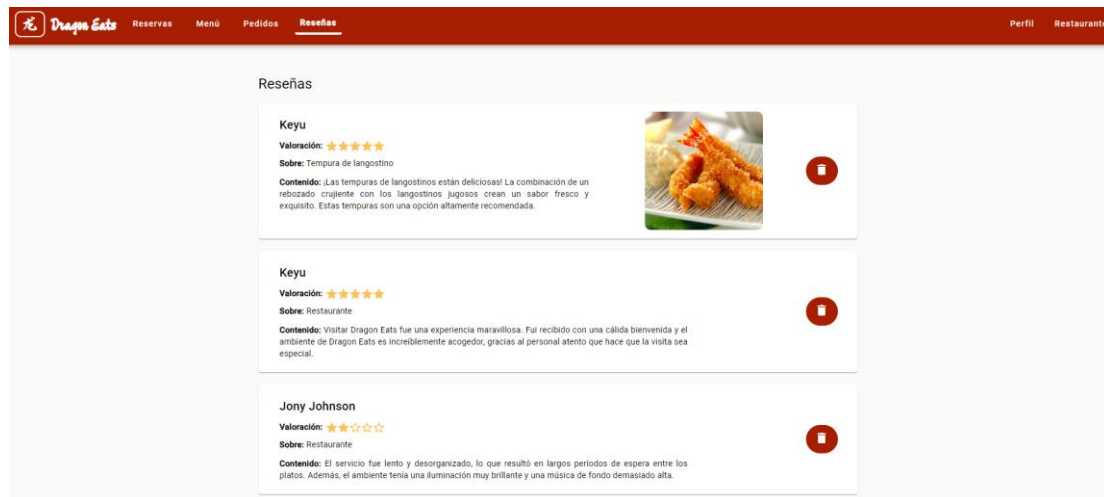


Ilustración 203: Administrador – Reseñas.

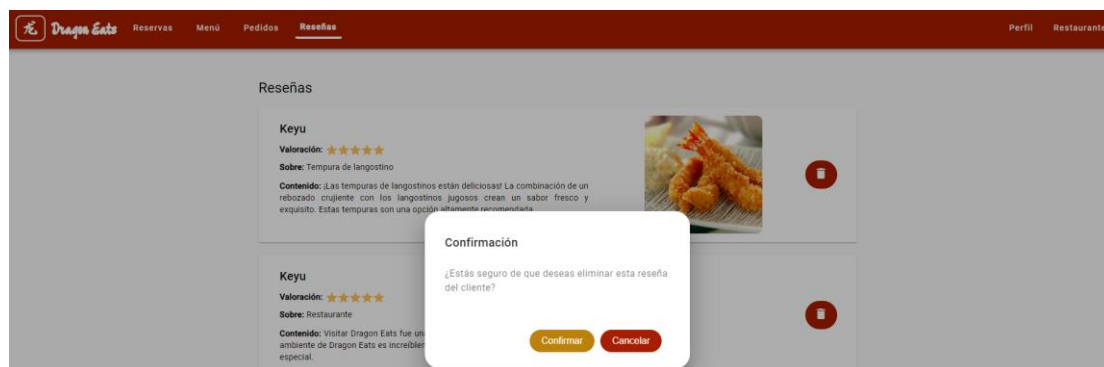


Ilustración 204: Administrador – Eliminar reseñas.

7 Resultados y conclusiones

7.1 Repaso general y resultados obtenidos

A pesar del aumento de la digitalización, bastantes establecimientos hosteleros siguen sin digitalizarse, lo que supone una mayor carga de trabajo para los propietarios y los empleados del restaurante, quienes deben manejar las tareas manualmente, como atender a las llamadas, gestionar las reservas o manejar los pedidos. Además de esto, generalmente las reseñas se encuentran en la web, pero, al no poseer una, se pierde visibilidad de estas valoraciones y las posibles oportunidades de mejora que puede haber para el restaurante. Sumado a lo anterior, actualmente los clientes prefieren interactuar de manera digital, por lo que la falta de digitalización implicará una pérdida de clientela. No obstante, aquellos que han adoptado tecnologías digitales a menudo dependen de plataformas externas, limitándoles realizar determinadas acciones como tener el control sobre la gestión de las reseñas o el diseño personalizado de un restaurante. Además de lo anterior, estas plataformas pueden aumentar indirectamente la competencia entre los distintos restaurantes registrados.

Como solución a estos problemas se ha propuesto una aplicación web integral personalizada para un restaurante en concreto, con el fin de facilitar los diversos servicios que ofrece el establecimiento a sus clientes. De este modo, además de dejar de depender de aplicaciones de terceros, así como la reducción de los posibles costes asociados a las comisiones, también proporcionará a los propietarios un mayor control administrativo del restaurante. Por otro lado, desde el punto de vista del cliente, esta aplicación proporciona una experiencia más sencilla y conveniente, ya que pueden realizar múltiples acciones en una misma plataforma sin la necesidad de acceder a diferentes aplicaciones, tales como reservar, hacer pedidos o dejar reseñas.

Para la construcción de esta aplicación web integral, se ha seguido la metodología Scrum, lo cual permitió una gestión ágil y eficiente del proyecto mediante Sprints y entregas incrementales. Además, se ha utilizado tecnologías modernas como Angular 17 para el desarrollo del Front-End, ofreciendo una interfaz de usuario intuitiva y responsiva, y Spring Boot 3 para el Back-End, ofreciendo un servidor robusto y escalable. En cuanto al sistema de gestión de bases de datos, se ha hecho uso de PostgreSQL 16 con el fin de garantizar una alta fiabilidad y rendimiento.

Para llevar a cabo el desarrollo de la aplicación, en primer lugar, se ha identificado los requisitos de la aplicación mediante historias de usuarios con el fin de cubrir todas las necesidades del restaurante, luego se ha diseñado tanto el Front-End como el Back-End, y su posterior integración y, finalmente, se ha realizado múltiples pruebas para verificar su correcto funcionamiento. Por último, cabe destacar que se ha ido documentando todo el ciclo de vida del desarrollo, desde el análisis y diseño hasta la implementación y pruebas, dejando como resultado una aplicación web integral con un sistema de identificación y autenticación, un sistema de gestión eficiente de pedidos y reservas, así como un sistema de reseñas y valoraciones. Con esto se ha podido alcanzar todos los subobjetivos planteados del Capítulo 1, excepto el objetivo opcional de desarrollar un sistema de recomendación mediante inteligencia artificial por falta de tiempo. A pesar de esto último, se ha logrado cumplir con el objetivo principal de este proyecto, que consistía en la documentación y el desarrollo de una aplicación web integral para facilitar los servicios de un restaurante con sus clientes.

7.2 Líneas futuras

A pesar de que esta aplicación web posee una gran variedad de funcionalidades, es importante seguir mejorando la aplicación con el objetivo de adaptarse en todo momento a las expectativas de los propietarios, de los clientes y del mercado, ya que la adición de nuevas características puede determinar el éxito del restaurante a largo plazo. De esta manera, no solo mejorará la gestión administrativa del restaurante, sino que también aumentará la fidelidad del cliente. En cuanto a las posibles ampliaciones de la aplicación, se encuentran los siguientes:

- Integración de un asistente virtual inteligente que ofrezca recomendaciones personalizadas de platos en base a las preferencias, las valoraciones y el historial de pedidos del cliente. Además de esto, este asistente virtual tendrá la capacidad de brindar soporte al cliente de manera rápida y en cualquier momento del día, aumentando así la satisfacción del cliente.
- Mejora del diseño responsivo de la aplicación para poder adaptarse a cualquier dispositivo electrónico, independientemente del tamaño de la pantalla, garantizando una experiencia óptima al usuario. Otra mejora consistiría en retocar la interfaz gráfica de la aplicación para hacerla más atractiva e intuitiva.
- Mejora del sistema de reservas con recordatorios por correo electrónico con el objetivo de reducir los posibles olvidos de los clientes. Además de esto, otras mejoras para el sistema de reservas sería la edición o cancelación de una reserva por parte del cliente, la visualización de un calendario de disponibilidad en tiempo real o la gestión automatizada de reservas.
- Mejora del sistema de pedidos de comida, permitiendo a los clientes realizar transacciones en línea con múltiples opciones de pago, como tarjeta de crédito, tarjeta de débito o PayPal.
- Mejora del sistema de reseñas, proporcionando una interacción directa entre los clientes y los propietarios del restaurante. Otra mejora consistiría en la moderación automática de las reseñas irrespetuosas o poco constructivas.
- Por último, la implementación de un sistema de promociones y cupones de descuento puede incentivar al cliente a seguir realizando pedidos en el restaurante, aumentando así su lealtad y compromiso con el negocio. De igual manera, el programa de fidelidad podrá recompensar a los clientes más fieles con regalos especiales gracias a un sistema de puntos acumulables en cada pedido.

7.3 Análisis de impacto

En este apartado se realizará un análisis del impacto potencial que puede tener la aplicación web en diferentes contextos:

- **Personal.** Se explicará más adelante como último apartado del capítulo.
- **Empresarial.** A nivel empresarial, esta aplicación tiene el potencial de ser una herramienta muy útil para los propietarios del restaurante, permitiéndoles gestionar de manera eficaz los múltiples servicios que ofrece su establecimiento, ya que, al digitalizar los procesos de pedidos, reservas y reseñas, se minimizará la carga de trabajo manual, lo que resulta en una mayor eficiencia operativa. Además de esto, la aplicación puede ayudar al restaurante a establecer su propia marca en el mercado,

mejorando así su presencia online y su capacidad de atraer nuevos clientes.

- **Social.** A nivel social, esta aplicación podría ser el punto de encuentro entre diferentes comensales al disponer de un sistema de reseñas y valoraciones, fomentando así una mayor transparencia y confianza entre los clientes y el restaurante. Además de esto, esta aplicación mejora la accesibilidad de los clientes, permitiéndoles realizar pedidos y reservas en línea de manera rápida desde cualquier dispositivo.
- **Económico.** Relacionado con el contexto empresarial, a nivel económico, la aplicación puede incrementar significativamente los ingresos del restaurante al mejorar la eficiencia operativa del establecimiento y al aumentar la satisfacción del cliente. Además, gracias a la presencia en línea del restaurante, se podrá alcanzar un público más amplio, lo que puede resultar en un aumento de ventas. Por último, al no depender de plataformas de terceros, se podrá reducir los costes asociados a las comisiones.
- **Medioambiental.** Desde una perspectiva medioambiental, la digitalización de los procesos puede contribuir a la reducción del uso de papel y otros recursos físicos. Además, una gestión más eficiente de reservas y pedidos puede ayudar a minimizar el desperdicio de alimentos, ya que permite una mejor planificación y previsión de la demanda.
- **Cultural.** A nivel cultural, la aplicación puede influir en la forma en que los restaurantes interactúan con sus clientes, permitiendo a estos últimos explorar y apreciar la cocina culinaria del restaurante.

A pesar de todos estos beneficios esperados, la aplicación podría contribuir a la reducción de interacciones personales directas entre los clientes y el personal del restaurante, lo cual podría afectar el ambiente tradicional que algunos comensales valoran.

Una vez analizado el impacto que puede tener la aplicación web en diversos contextos, a continuación, se va a explicar cómo esta solución tecnológica puede ayudar a alcanzar los Objetivos de Desarrollo Sostenible (ODS) propuestos por las Naciones Unidas [83]:

- **ODS 8: Trabajo decente y crecimiento económico.** Esta aplicación web trata de ayudar a los propietarios del restaurante a optimizar la gestión de su negocio, aumentando así su productividad y promoviendo la sostenibilidad del negocio. También puede ayudar a fomentar el crecimiento económico del restaurante, de manera que puede invertir las ganancias en crear nuevos empleos y mejorar sus servicios [84].
- **ODS 9: Industria, innovación e infraestructura.** Esta aplicación web, al contar con un sistema de reservas y pedidos en línea, elimina la necesidad de asistir físicamente al restaurante o de realizar llamadas telefónicas, de manera que se promueve la innovación digital y la accesibilidad, lo que contribuye a una industria más moderna y eficiente, impulsando así la transformación digital en el sector de la restauración [85].
- **ODS 12: Producción y consumo responsables.** Esta aplicación web, al contar con un sistema de reservas y pedidos en línea, puede contribuir a la gestión eficiente de los recursos y a la reducción del desperdicio de alimentos, ya que, al permitir una mejor planificación de la demanda, se minimiza el riesgo de sobreproducción. Además, la digitalización de los procesos reduce el uso de recursos físicos, como el papel, promoviendo así un consumo y producción más sostenibles [86].

7.4 Conclusión personal

Para terminar con la memoria, quería realizar una pequeña reflexión personal sobre la importancia y el impacto que ha tenido el Trabajo de Fin de Grado en mi desarrollo académico y, muy posiblemente, en mi futuro profesional. A pesar de que este proyecto ha sido un gran desafío para mí, me ha permitido poner en práctica la gran mayoría de las habilidades que he adquirido durante toda la carrera, así como aprender nuevas tecnologías de gran importancia en el mundo laboral.

En primer lugar, este proyecto me ha permitido comprender que el desarrollo de software es un proceso de mejora continua del producto que va más allá de la simple codificación, pasando por múltiples etapas hasta que llega a manos del cliente. De esta manera, la aplicación final no solo cumple con los requisitos técnicos, sino que también satisface las necesidades y expectativas del cliente, que en este caso son tanto los dueños como los comensales del restaurante. Como resultado, se ha podido crear una solución innovadora para la digitalización en el ámbito de la restauración, facilitando la gestión administrativa del restaurante y mejorando la experiencia de los comensales.

En segundo lugar, durante la realización del proyecto, me he encontrado con diversas dificultades. Entre ellas, destaco la redacción de la propia memoria, ya que me resultaba difícil expresar de manera clara y concisa la información que había investigado. Además de esto, también me ha resultado desafiante el diseño de la aplicación, ya que inicialmente no tenía experiencia en Angular, por lo que tuve que esforzarme bastante para aprender esta tecnología Front-End. Asimismo, con respecto al Back-End, aunque tenía una base más sólida, también tuve que dedicar tiempo a aprender Spring Boot. Por último, también me resultó difícil realizar el diseño de las APIs e implementar la seguridad en el sistema. A pesar de todos estos desafíos encontrados, fue una valiosa oportunidad para aprender nuevas habilidades, que tendrá un gran impacto en mi futuro.

En tercer lugar, con respecto a la metodología seleccionada, fue interesante adoptar la metodología Scrum, ya que me ha permitido gestionar este proyecto de software de manera eficiente y eficaz, proporcionando una planificación clara del desarrollo. De esta manera, se ha podido ir mejorando poco a poco la aplicación, adaptándose a nuevas necesidades del cliente. Además, este enfoque me ayudó a comprender la dinámica del mundo laboral de las grandes empresas, ya que suelen utilizar este tipo de metodología para trabajar en equipo. Sumado a esto, al aprender tecnologías como PostgreSQL, Spring Boot y Angular, muy demandadas en estas grandes empresas, me ha permitido prepararme mejor para mi futuro laboral.

En resumen, el Trabajo de Fin de Grado ha sido una magnífica oportunidad para reforzar mis conocimientos y aplicar las habilidades que he aprendido durante toda mi carrera, convirtiéndose en una experiencia muy valiosa para mi futura carrera profesional.

8 Bibliografía

- [1] We Are Social Spain, (2024), “Digital 2024” [Online]. Available: <https://wearesocial.com/es/blog/2024/01/digital-2024/>
- [2] Instituto Nacional de Estadística INE, (2023, Noviembre 28), “Encuesta sobre Equipamiento y Uso de Tecnologías de Información y Comunicación en los Hogares” [Online]. Available: https://www.ine.es/prensa/tich_2023.pdf
- [3] Confederación Empresarial de Turismo de Extremadura CETEX, (2022), “Anuario 2022 Hostelería España” [Online]. Available: <https://www.cetex.es/wp-content/uploads/2022/12/ANUARIO-HOSTELERIA-2022.pdf>
- [4] 7 Caníbales, (2022, Octubre 26), “El futuro de los restaurantes es digital” [Online]. Available: <https://www.7canibales.com/actualidad/restaurante-digitalizacion/>
- [5] Hostelería Digital, (2023, Diciembre 19), “La hostelería cierra 2023 con un crecimiento anual entre un 5 y 10% pero con pérdida de rentabilidad” [Online]. Available: <https://www.hosteleriadigital.es/2023/12/19/la-hosteleria-cierra-2023-con-un-crecimiento-anual-entre-un-5-y-10-pero-con-perdida-de-rentabilidad/>
- [6] EY, (2021), “Estudio Digitalización Hostelería” [Online]. Available: <https://www.segittur.es/wp-content/uploads/2021/09/ESTUDIO-DIGITALIZACION-HOSTELERIA-EN-ESPANA.pdf>
- [7] BBC Innovation y Delectatech, (2022, Noviembre 28), “Estado de la Digitalización de la Hostelería en España” [Online]. Available: <https://foodradar.app/reports/estado-de-la-digitalizacion-de-la-hosteleria-en-espana>
- [8] Just Eat, “Comida a domicilio y para llevar” [Online]. Available: <https://www.just-eat.es/>
- [9] Uber Eats, “La Entrega de los Pedidos está Cerca de ti” [Online]. Available: <https://www.ubereats.com/es>
- [10] Glovo, “Food Delivery and more” [Online]. Available: <https://glovoapp.com/>
- [11] OpenTable, “Encuentra la mesa adecuada para cada ocasión” [Online]. Available: <https://www.opentable.es/>

[12] TheFork, “Descubre y Reserva el mejor Restaurante” [Online]. Available: <https://www.thefork.es/>

[13] Tripadvisor, “Build a Trip in Minutes” [Online]. Available: <https://www.tripadvisor.com/>

[14] Google, “Mapas y Direcciones” [Online]. Available: <https://www.google.es/maps/preview>

[15] Salesforce España, “¿Qué es la CRM? - Gestión de las relaciones con los clientes” [Online]. Available: <https://www.salesforce.com/es/learning-centre/crm/what-is-crm/>

[16] Salesforce, “CRM número uno del mundo” [Online]. Available: <https://www.salesforce.com/es/>

[17] Bloo Media, “Inbound Marketing: Qué es, Fases y Por Qué Puede Ayudarte” [Online]. Available: <https://bloo.media/blog/inbound-marketing/#Que-es-el-Inbound-Marketing>

[18] HubSpot, “Crece mejor con HubSpot” [Online]. Available: <https://www.hubspot.es/>

[19] Hostinger, (14 Diciembre 2023), “¿Qué es un CMS? Definición, Funciones y Ejemplos” [Online]. Available: <https://www.hostinger.es/tutoriales/que-es-un-cms>

[20] MDN Web Docs, (24 Julio 2023), “HTML: Lenguaje de Etiquetas de Hipertexto” [Online]. Available: <https://developer.mozilla.org/es/docs/Web/HTML>

[21] Markdown.es, “La Guía Definitiva en Español” [Online]. Available: <https://markdown.es/>

[22] Ivan de Souza, (26 Febrero 2020), “Jetpack: cómo funciona este plugin de Wordpress y sus ventajas” [Online]. Available: <https://rockcontent.com/es/blog/jetpack/>

[23] André Mounsinho, (3 Junio 2020), “SEO: la guía completa para que conquistes la cima de Google en el 2022” [Online]. Available: <https://rockcontent.com/es/blog/que-es-seo/>

- [24] Johanna Rodriguez, (20 Enero 2023), “Qué es WooCommerce, para qué sirve y ventajas” [Online]. Available: <https://blog.hubspot.es/sales/que-es-woocommerce>
- [25] Rosie Greaves, (30 Octubre 2023), “Wix vs Squarespace vs WordPress (2023): La batalla de los creadores de sitios web” [Online]. Available: <https://ecommerce-platforms.com/es/articles/wix-vs-squarespace-vs-wordpress>
- [26] WordPress.com, “WordPress.com: crea un sitio, vende tus productos, escribe un blog y muchas cosas” [Online]. Available: <https://wordpress.com/es/>
- [27] Wix, “Solicitud: sistema CMS” [Online]. Available: <https://support.wix.com/es/article/solicitud-sistema-crm>
- [28] Wix, “Crear Página Web gratis” [Online]. Available: <https://es.wix.com/>
- [29] Squarespace, “Creador de páginas web — Tu web en minutos” [Online]. Available: <https://es.squarespace.com/>
- [30] Monouso, “Las mejores empresas de delivery para tu restaurante” [Online]. Available: <https://blog.monouso.es/mejores-empresas-delivery-restaurant/>
- [31] Tecseris, (6 Mayo 2024), “Ventajas y desventajas de un CRM” [Online]. Available: <https://tecseris.com/blog/ventajas-y-desventajas-de-un-crm/>
- [32] Rafael Marín, (5 Octubre 2018), “CMS vs Framework. Diferencias, ventajas y desventajas” [Online]. Available: <https://www.inesem.es/revistadigital/informatica-y-tics/cms-vs-framework-ventajas-desventajas/>
- [33] Hunabku, “Diseño web a la medida vs plataforma CMS, ¿Cuál elegir?” [Online]. Available: <https://hunabku.mx/disenio-web-a-la-medida-vs-plataforma-cms-cual-es-mejor>
- [34] Isidro Vivas, (14 Febrero 2022), “Desarrollo web a medida vs CMS populares (Joomla, Wordpress, Drupal)” [Online]. Available: <https://www.latevaweb.com/desarrollo-a-medida-vs-estandar-cms>

[35] David Ospina, (3 Enero 2023), “Diseño Web a la medida o plantillas: Ventajas y desventajas” [Online]. Available: <https://www.marketerosagencia.com/blog/disenio-web/sitio-web-medida-vs-plantilla/>

[36] Abhishek Tiwari, (19 Agosto 2023), “The History and Evolution of Web Development: From HTML to the Modern Web” [Online]. Available: <https://medium.com/@Abhishek-Tiwari/the-history-and-evolution-of-web-development-from-html-to-the-modern-web-982e3f90e891>

[37] GeeksforGeeks, (18 Abril 2023), “Frontend vs Backend” [Online]. Available: <https://www.geeksforgeeks.org/frontend-vs-backend/>

[38] GeeksforGeeks, (15 Abril 2024), “CSS Tutorial” [Online]. Available: <https://www.geeksforgeeks.org/css-tutorial/>

[39] María Coppola, (12 Junio 2023), “Qué es JavaScript, para qué sirve y cómo funciona” [Online]. Available: <https://blog.hubspot.es/website/que-es-javascript#que-es>

[40] jQuery, “jQuery: The Write Less, Do More, JavaScript Library” [Online]. Available: <https://jquery.com/>

[41] React, “React: The Write Less, Do More, JavaScript Library” [Online]. Available: <https://react.dev/>

[42] TypeScript, “TypeScript: JavaScript With Syntax For Types” [Online]. Available: <https://www.typescriptlang.org/>

[43] Angular, “Angular: One framework for Mobile & desktop” [Online]. Available: <https://angular.io/>

[44] Vue.js, “Vue.js-The Progressive JavaScript Framework” [Online]. Available: <https://vuejs.org/>

[45] Node.js, “Node.js-Run JavaScript Everywhere” [Online]. Available: <https://nodejs.org/en>

[46] Express.js, “Express.js: Un marco de aplicación web minimalista y flexible de Node.js” [Online]. Available: <https://expressjs.com/>

[47] Python, “Python is a programming language that lets you work quickly and integrate systems more effectively.” [Online]. Available: <https://www.python.org/>

[48] Django, “The web framework for perfectionists with deadlines” [Online]. Available: <https://www.djangoproject.com/>

[49] Java, “Get Java for desktop applications” [Online]. Available: <https://www.java.com/en/>

[50] Paul Bakker, “Spring makes Java simple” [Online]. Available: <https://spring.io/>

[51] Microsoft, “C# | Lenguaje de programación moderno y de código abierto para .NET” [Online]. Available: <https://dotnet.microsoft.com/es-es/languages/csharp>

[52] Microsoft, (23 Mayo 2024), “Introduction to .NET - .NET” [Online]. Available: https://learn.microsoft.com/en-us/dotnet/core/introduction?WT.mc_id=dotnet-35129-website

[53] PHP, “PHP: ¿Qué es PHP? – Manual” [Online]. Available: <https://www.php.net/manual/es/intro-what-is.php>

[54] Symfony.es, “¿Qué es Symfony?” [Online]. Available: <https://symfony.es/pagina/que-es-symfony/>

[55] Oracle, “¿Qué es una base de datos? | Oracle España” [Online]. Available: <https://www.oracle.com/es/database/what-is-database/>

[56] Astera, (3 Octubre 2023), “SQL vs NoSQL: 5 diferencias principales” [Online]. Available: <https://www.astera.com/es/knowledge-center/sql-vs-nosql/>

[57] PostgreSQL, “PostgreSQL: The world's most advanced open source database” [Online]. Available: <https://www.postgresql.org/>

[58] MySQL, “The world's most popular open source database” [Online]. Available: <https://www.mysql.com/>

[59] SQLite, “SQLite Home Page” [Online]. Available: <https://www.sqlite.org/>

[60] MongoDB, “MongoDB: The Developer Data Platform” [Online]. Available: <https://www.mongodb.com/>

[61] Google, “Cloud Firestore” [Online]. Available: <https://firebase.google.com/docs/firestore?hl=es-419>

[62] Redis, “Redis - The Real-time Data Platform” [Online]. Available: <https://redis.io/>

[63] Stack Overflow, (2023), “Stack Overflow Developer Survey 2023” [Online]. Available: <https://survey.stackoverflow.co/2023/#technology-most-popular-technologies>

[64] Atlassian, “Qué es el control de versiones | Atlassian Git Tutorial” [Online], Available: <https://www.atlassian.com/es/git/tutorials/what-is-version-control>

[65] Git, “Git everything is local” [Online], Available: <https://git-scm.com/>

[66] Atlassian, “Qué es Git | Atlassian Git Tutorial” [Online], Available: <https://www.atlassian.com/es/git/tutorials/what-is-git>

[67] Subversion, “Apache Subversion” [Online], Available: <https://subversion.apache.org>

[68] Perforce, “What Is Subversion? SVN Overview” [Online], Available: <https://www.perforce.com/blog/vcs/what-svn>

[69] AWS, “¿Qué es el ciclo de vida del desarrollo de software (SDLC)?” [Online], Available: <https://aws.amazon.com/es/what-is/sdlc/>

[70] IONOS, “El modelo en cascada en el desarrollo de software” [Online], Available: <https://www.ionos.es/digitalguide/paginas-web/desarrollo-web/el-modelo-en-cascada/>

[71] Anna Pérez, “Características y fases del modelo incremental” [Online], Available: <https://www.obsbusiness.school/blog/caracteristicas-y-fases-del-modelo-incremental>

[72] IONOS, “Modelo en espiral: el modelo para la gestión de riesgos en el desarrollo de software” [Online], Available: <https://www.ionos.es/startupguide/productividad/modelo-en-espiral/>

[73] Computer Weekly, “Desarrollo de software ágil: ¿Qué es y cómo implementarlo?” [Online], Available: <https://www.computerweekly.com/es/definicion/Desarrollo-de-software-agil-o-Agile>

[74] Julia Martins, (15 Febrero 2024), “Scrum: conceptos clave y cómo se aplica en la gestión de proyectos” [Online], Available: <https://asana.com/es/resources/what-is-scrum>

[75] Proyectos Ágiles, “Historia de Scrum” [Online], Available: <https://proyectosagiles.org/historia-de-scrum/>

[76] Agustín Varela, (19 Febrero 2021), “Historia y evolución de Scrum” [Online], Available: <https://www.linkedin.com/pulse/historia-y-evoluci%C3%B3n-de-scrum-agustin-varela/?originalSubdomain=es>

[77] Paul Bultmann, (9 Junio 2023), “The History of Scrum” [Online], Available: <https://www.agile42.com/en/blog/scrum-history>

[78] AWS, “¿En qué consiste Scrum? - Explicación sobre la metodología Scrum” [Online], Available: <https://aws.amazon.com/es/what-is/scrum/>

[79] Claire Drumond, “¿Qué es scrum? [+ Cómo empezar] | Atlassian” [Online], Available: <https://www.atlassian.com/es/agile/scrum>

[80] Atlassian, “Jira | Software de seguimiento de proyectos e incidencias” [Online], Available: <https://www.atlassian.com/es/software/jira>

[81] Arizbé Ken, (27 Noviembre 2023), “Arquitectura de software: ¿Qué es y qué tipos hay?” [Online], Available: <https://www.gluo.mx/blog/arquitectura-de-software-que-es-y-que-tipos-hay>

[82] Miguel Angel Alvarez, (20 Septiembre 2023), “Qué es MVC” [Online], Available: <https://desarrolloweb.com/articulos/que-es-mvc.html>

[83] Naciones Unidas, “Desarrollo Sostenible” [Online], Available: <https://www.un.org/sustainabledevelopment/es/>

[84] Naciones Unidas, “Crecimiento económico - Desarrollo Sostenible” [Online], Available: <https://www.un.org/sustainabledevelopment/es/economic-growth/>

[85] Naciones Unidas, "Infraestructura - Desarrollo Sostenible" [Online], Available: [https://www.un.org/sustainabledevelopment/es/infrastructure/ç](https://www.un.org/sustainabledevelopment/es/infrastructure/)

[86] Naciones Unidas, "Consumo y producción sostenibles - Desarrollo Sostenible" [Online], Available: <https://www.un.org/sustainabledevelopment/es/sustainable-consumption-product ion/>

Este documento esta firmado por



Firmante	CN=tfgm.fi.upm.es, OU=CCFI, O=ETS Ingenieros Informaticos - UPM, C=ES
Fecha/Hora	Mon Jun 03 23:52:43 CEST 2024
Emisor del Certificado	EMAILADDRESS=camanager@etsiinf.upm.es, CN=CA ETS Ingenieros Informaticos, O=ETS Ingenieros Informaticos - UPM, C=ES
Numero de Serie	561
Metodo	urn:adobe.com:Adobe.PPKLite:adbe.pkcs7.sha1 (Adobe Signature)